

MAI-Thinking-1: 构建一台爬山机器

The Microsoft AI Team 1

目录

1 引言	4
2 预训练	8
2.1 模型架构	9
2.2 模型消融方法论	11
2.2.1 规模阶梯	11
2.2.2 效率增益	11
2.2.3 模型消融示例	12
2.3 评测方法论	13
2.3.1 公共评测集净化	14
2.3.2 准确率与 NLL 评测的比较	14
2.4 预训练数据	16
2.4.1 数据来源、许可与治理	17
2.4.2 HTML 提取	17
2.4.3 去重	18
2.4.4 数据过滤与分类	19
2.4.5 数据消融实验	19
2.5 选择训练数据配比	20
2.5.1 优化面临的挑战	21
2.5.2 基于预测的方法	21
2.5.3 最终配比选择	23
2.5.4 训练中期的数据配比	25
2.6 训练方案	26
2.6.1 训练阶段	26
2.6.2 超参数	26
2.6.3 数值精度	27
2.6.4 训练损失曲线	28
2.7 评估与同期模型对比	28
2.8 YOLO: 大规模分布式训练	32
2.8.1 系统概览	32
2.8.2 确定性与正确性	34
2.8.3 容错	35
2.8.4 与模型架构协同优化性能	35
3 强化学习攀登	37

3.1	强化学习配方	38
3.1.1	RL 目标	38
3.1.2	奖励设计	40
3.1.3	采样策略	41
3.1.4	自蒸馏	42
3.1.5	超参数	43
3.2	STEM 攀登	45
3.2.1	数据管线	46
3.2.2	竞赛编程数据	47
3.2.3	去重与去污染	47
3.3	智能体攀登	47
3.3.1	软件工程	49
3.3.2	通用工具使用	51
3.4	有益性与安全攀登	52
3.4.1	奖励	52
3.4.2	指令遵循与可引导性	53
3.4.3	安全	54
3.4.4	诚实性	55
3.4.5	风格	56
3.5	将多种能力整合进单一模型	56
3.6	强化学习基础设施	57
3.6.1	控制器	57
3.6.2	问题工作节点	57
3.6.3	推演工作节点	58
3.6.4	路由与推理	58
3.6.5	权重迁移	59
4	评测	60
4.1	基准评测	60
4.2	人工并排评测	61
4.3	内部安全评测	62
5	安全红队测试	63
5.1	内部红队测试	63
5.2	独立红队测试	65
6	集群环境	66
6.1	训练集群	66
6.2	训练稳定性、确定性与有效产出指标	66
6.2.1	MFU 与有效产出	67
6.3	推理效率与模型部署	67

6.4 可持续性社区优先的 AI 基础设施倡议	67
7 结论与未来方向	68
8 引用格式	92
9 预训练数据管线细节	92
9.1 网页 HTML	92
9.2 网页 PDF	94
9.3 图书与期刊	94
9.4 公开 GitHub	94
10 长上下文扩展	95
10.1 数据	96
10.2 评测设置	96
10.3 渐进式上下文长度扩展	96
10.4 适应速度	97
10.5 最终配方	97
11 推理轨迹的演进	102
11.1 STEM 思维链	102
11.2 弱模型摘录：猜测极小值点，捏造根	103
11.3 强模型摘录：逐一推导每个候选值并检查定义域	103
11.4 弱模型摘录：错误的线性化	103
11.5 强模型摘录：识别不变量	104
11.6 弱模型摘录：把空格子标为“极大”且从不复核	104
11.7 强模型摘录：质疑自己的逆命题并用小例子检验	104
11.8 智能体思维链	104
11.9 强模型摘录：审查问题需求并运行单元测试以验证工作	105
12 SWE 智能体工具 Schema	107
13 指令遵循数据的约束分类体系	109
14 G 构建 SWE 环境的基础设施	109
15 H STEM 评测设置	110
15.1 数学	110
15.2 科学	111
15.3 竞赛编程	111
16 I 智能体编码评测	113
17 J 安全评测	113

17.1 内部评测细节	113
17.1.1 方法论	113
17.1.2 数据收集与采样	114
18 K 通用能力评测	115
18.1 知识	115
18.2 指令遵循	115
18.3 长上下文	116
18.4 安全	116
18.5 诚实性	117
18.6 健康	117
18.7 工具调用	117
19 L 集群环境细节	117
19.1 硬件与计算集群	118
19.2 集群就绪与认证	118
19.3 调度、编排与控制平面	119
19.4 可观测性、遥测与集群监控	120

摘要

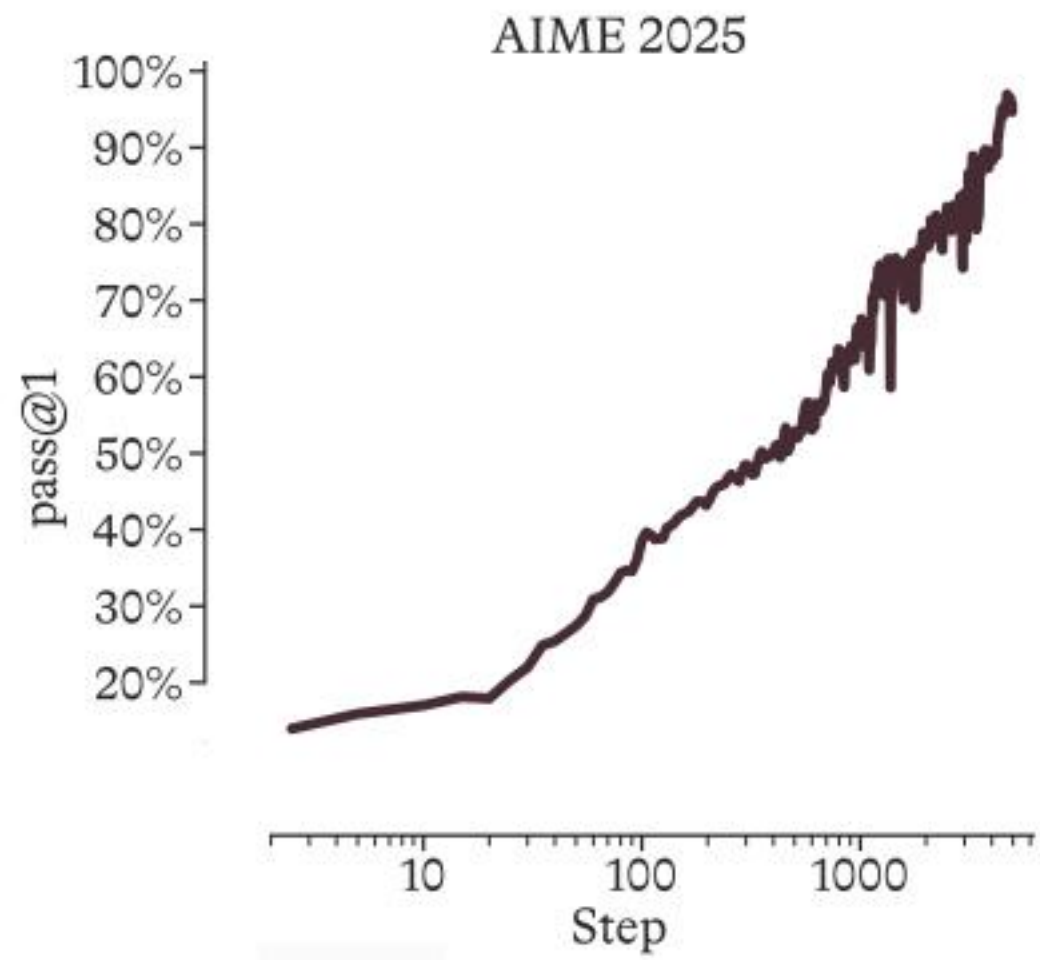
AI 的进步并非由单一模型驱动，而在于能够持续改进当前模型的水平。要做到这一点，需要将模型开发视为一个系统级优化问题，其解决方案是构建一台用于快速改进的爬山机器 (hill-climbing machine)。我们的流程包括一个以规模化为核心的预训练建模决策框架，以及一套稳健的强化学习配方与基础设施，能够维持长期、对数线性 (log-linear) 的性能提升。首个采用我们流程开发的模型是 MAI-Thinking-1，一个 35B 激活参数 / 1T 总参数的 MoE 模型，在 STEM 推理与编程任务上位居同类规模模型中的最强之列（例如，SWE-Bench Pro 上 52.8%，AIME 2025 上 97.0%，LiveCodeBench v6 上 87.7%）。MAI-Thinking-1 完全从零训练，仅使用干净、企业级的数据，不依赖任何第三方模型的蒸馏。在本技术报告中，我们提供了对 MAI-Thinking-1 开发过程的深入剖析。通过分享我们的技术细节与经验教训，我们希望推动一种透明、以科学为导向的方法，促进 AI 的进一步发展。

1 引言

我们介绍 MAI-Thinking-1，一个由微软 AI (MAI) 从零开发的高性能推理模型，在 STEM 推理与编程任务上与同规模模型具有竞争力。我们的预训练聚焦于一种简单的规模化方法，强调以经验驱动的迭代改进来优化架构与数据；同时，我们的强化学习 (RL) 框架针对在数千步内持续的对数线性 (log-linear) 爬升进行了优化 (图 1)。

我们遵循三条主要设计原则来开发模型：第一，能力应当被习得，而非被继承。虽然通过蒸馏模仿而来的智能获取速度更快，但它缺乏长期、持久爬升所必需的可引导性与稳健性。第二，简单性才是可持续的。我们偏好简单、可扩展的配方；干净、可信的数据；以及透明的、共同支撑从零爬升的基础设施。第三，科学严谨，不

走捷径。每一项决策都必须能够通过数据驱动的梯级、消融实验与评估加以验证，从而揭示通往顶点的可靠路径。¹



¹Correspondence should be sent to mai-technical-report@microsoft.com. 引用格式见附录 A。

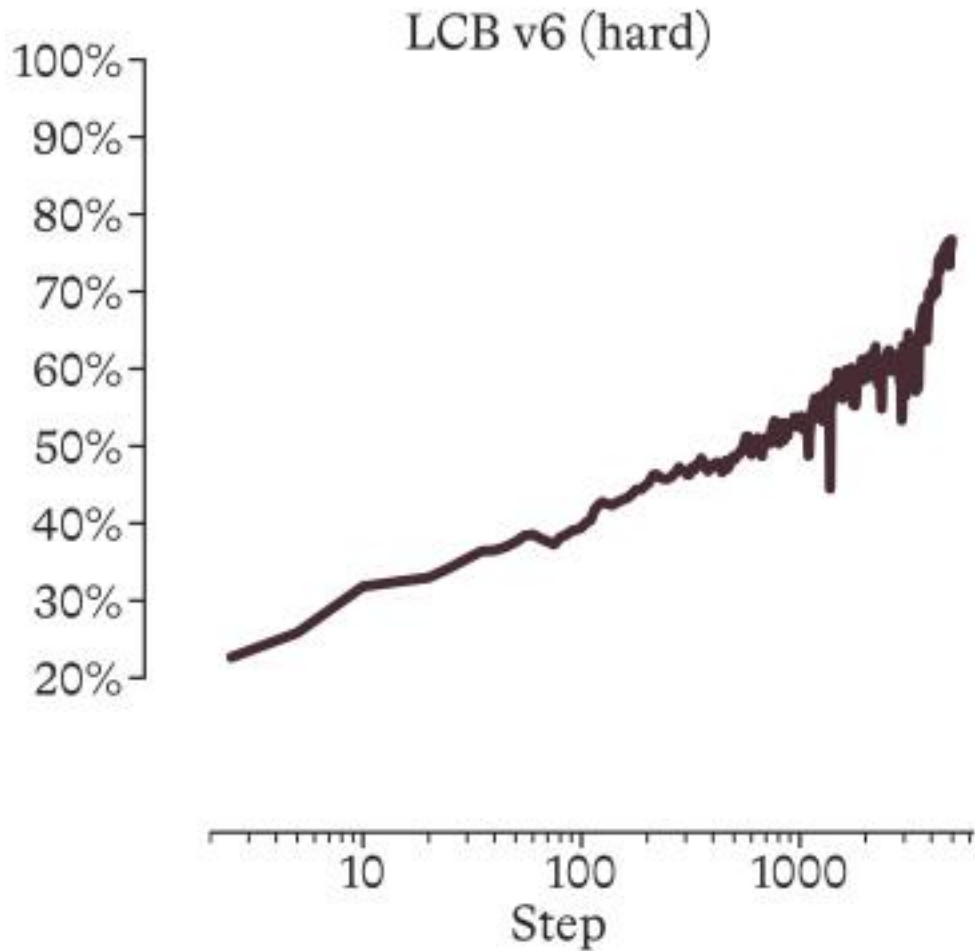
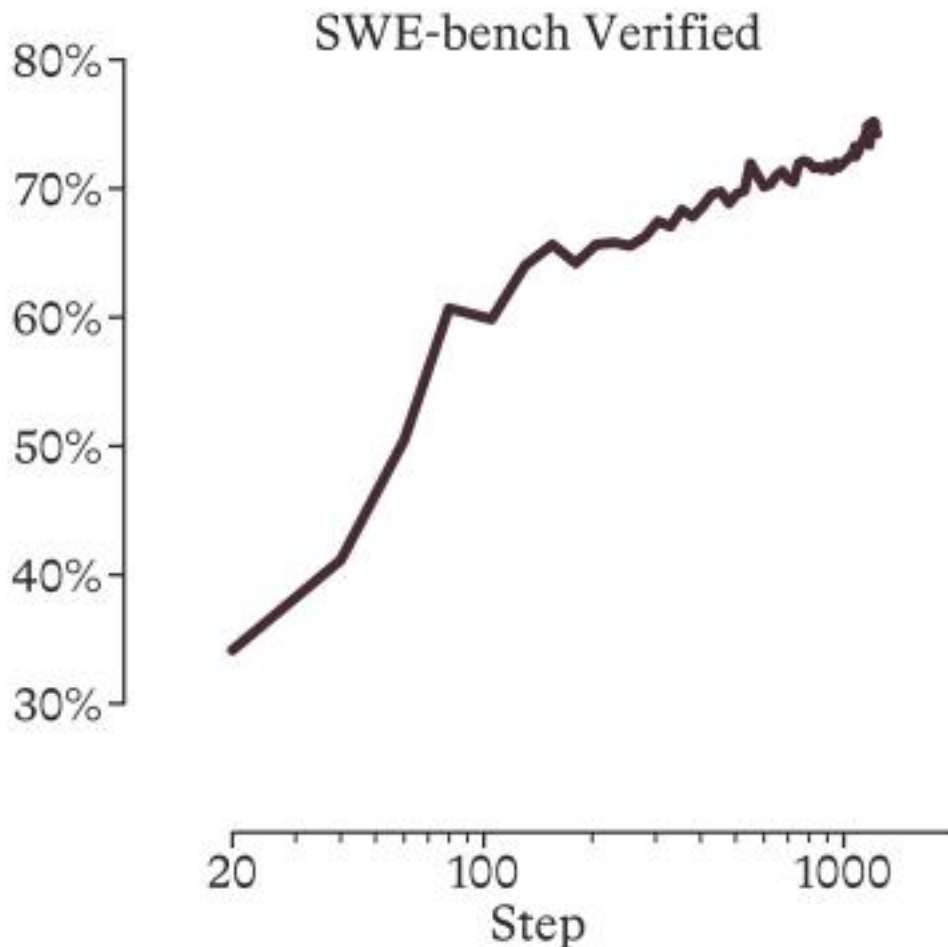


图 1. MAI-Thinking-1 在强化学习期间的性能。STEM 爬升（左与中）：我们以 STEM 为重点的爬升过程中的评估性能，评测使用 AIME 25 和 LiveCodeBench v6 的困难子集（2025 年 1 月之后）。对两条曲线，我们展示相邻 3 个检查点的 pass@1 平均值。智能体爬升（右）：在我们以代码为主的智能体爬升期间，SWE-bench Verified 上的 pass@1 性能。



MAI-Thinking-1 在 30T tokens 上完成预训练，这些 tokens 提取自公开可用与授权的人类生成数据的混合，涵盖网页数据、公开 GitHub 代码、书籍、学术论文、新闻、多语言文本和领域特定材料。每个数据源都在内部从始至终完成处理。我们选择在预训练中不使用任何由语言模型生成的合成数据，并努力避免与移除所收集数据源中的 AI 生成内容。在预训练中，我们不使用任何开源训练数据集，并从训练数据中去除常见的机器学习数据库（decontaminate）。在中期训练（mid-training）中，我们进一步强化 STEM、数学和编程能力，为推理 RL 爬升奠定坚实基础。中期训练之后，MAI-Thinking-1 的最大上下文长度达到 256K。预训练和中期训练赋予我们的基础模型广泛的预测能力与知识，但它们并未规定模型应如何表现、如何解决长程任务，或如何分配推理时的计算资源。

在强化学习（RL）爬升阶段，我们教会模型推理与回应。模型学会利用思维链（CoT）应对任务特定的反馈、使用外部工具与环境交互，以及遵循人类偏好与安全信号。我们的 RL 爬升从零开始，在没有任何先验推理轨迹接触的情况下学习推理。稳健的 RL 配方、自蒸馏（self distillation）与基础设施改进，使我们能够将 RL 运行持续数千步。利用这一配方，我们训练了三个领域专用的专家模型：一个用于 STEM 推理，一个用于智能体编程与工具使用，一个用于有用性与安全性。

我们将 MAI-Thinking-1 的性能与一系列开放权重与封闭权重的前沿模型，在公开基准和人类并排对比评估

中进行比较。这些评估涵盖广泛的领域，凸显了模型在不同领域中的通用性：STEM、智能体编程、知识、指令遵循、长上下文、安全、健康与工具调用。MAI-Thinking-1 在 SWE-Bench Pro 上取得 52.8%，在 AIME 2025 上取得 97.0%，在 AIME 2026 上取得 94.5%，在 LiveCodeBench v6 上取得 87.7%，并在广泛的基准上与 Sonnet 4.6 具有竞争力。

开发 MAI-Thinking-1 时的一个关键技术挑战，是在安全性（要求模型拒绝用户请求）与有用性（要求模型遵从用户）之间取得平衡。一个安全的模型需要提供有帮助的回应，同时符合我们的安全政策与标准。我们开发内部安全基准来锚定进展，并将有用性与安全性训练纳入 RL 爬升。作为负责任部署的一部分，我们在整个模型开发过程中持续进行红队测试（red-team），在发布前发现并修复漏洞。

MAI-Thinking-1 是首个使用我们的爬山机器开发的模型：它整合了数据管线构建、训练基础设施、强化学习环境、奖励、评估套件与安全测试，将模型开发转化为针对特定领域的经验优化循环。爬山机器使我们在推进 AI 的同时，将进展自下而上地锚定于人类需求之上。

2 预训练

MAI-Thinking-1 的基础模型 MAI-Base-1 是一个 35B-active / 1T-total 的稀疏 MoE 模型，从零开始在 Azure 平台内由 Microsoft 运营的集群上的 8K GB200 GPU 上预训练，使用自研分布式训练基础设施（第 2.8 节）。我们的训练语料完全由内部构建，来源包括公开可获取的数据与获取的数据，不含任何蒸馏数据（第 2.4 节）。基于该语料，我们进行了 30 万亿 token 的主预训练阶段，随后是总计 3.55 万亿 token 的中期训练阶段（第 2.6 节）。

我们在预训练中的目标是产出一个在广泛领域内都具有能力的基础模型，且其能力源于人类知识，而非对现有 AI 模型的模仿。我们建立了一个消融协议，通过沿着一条由规模递增的预训练模型构成的“阶梯”测量效率增益（第 2.2 节），仔细考察我们模型决策的扩展性质。为支持这些消融实验，我们开发了一套预训练指标（第 2.3 节），包括覆盖广泛学科、任务和数据分布的留出负对数似然（NLL）评估。我们报告 MAI-Base-1 与同期基础模型的性能，以说明这些评估在未经后训练（post-training）的模型上的应用（第 2.7 节）。

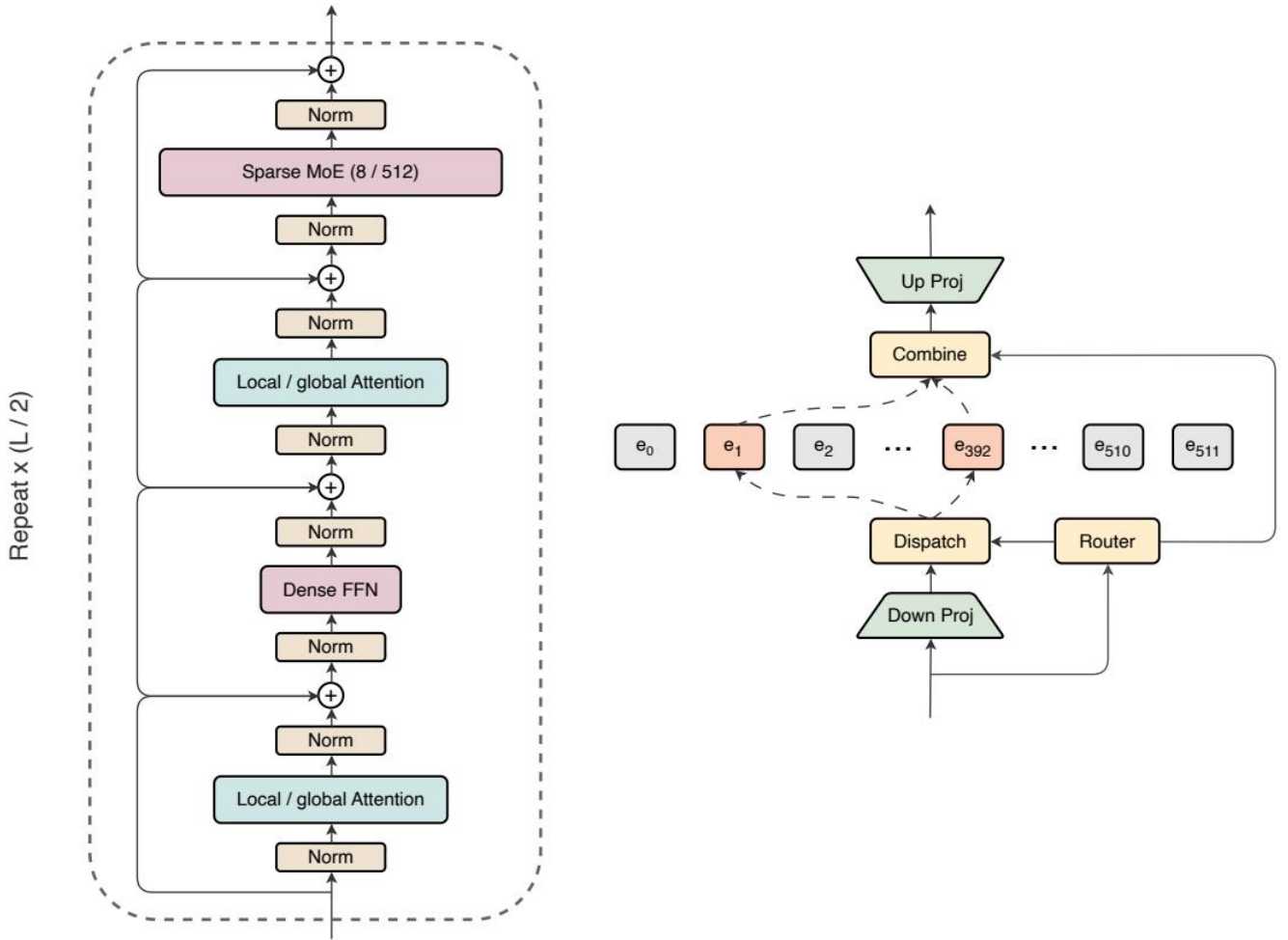


图 2. MAI-Base-1 架构总览。左：Transformer 主体的整体布局，我们将高稀疏度 MoE 层与小型稠密 FFN 交错，并将全局注意力与局部注意力交错。右：MoE 层，在压缩潜在空间中每个 token 激活 512 个专家中的 8 个。

2.1 模型架构

MAI-Base-1 的模型架构在设计上充分利用了规模在我们的基础设施和硬件上的优势。我们使用仅解码器 (decoder-only) Transformer (Vaswani et al., 2017)，包含周期性交替的局部与全局注意力层，以及稠密与 MoE 前馈块。架构的高层总览如图 2 所示。在每一层中，我们在残差相加之前的输入和输出处都使用 RMSNorm (Zhang and Sennrich, 2019) (Gemma Team et al., 2025)。模型中不使用任何偏置 (bias)，并且输入与输出 embedding 权重共享 (tied)。虽然我们观察到自研训练的 tokenizer 能带来一些模型性能提升，但我们选择了词汇量为 200,019 的 o200k_base tokenizer (OpenAI, 2022; Hurst et al., 2024)，以简化与现有自研工具和工作流的集成。我们架构设计中的一个关键考量是与底层 GPU 基础设施的协同优化，以确保预训练和强化学习 (RL) 都能高效扩展。

注意力机制。我们遵循 Gemma 3 (Gemma Team et al., 2025) 的周期性注意力设计，将 5 个局部注意力层

(Beltagy et al., 2020) 与 1 个全局注意力层配对，这显著降低了训练的注意力计算成本以及推理期间的 KV 缓存大小。局部注意力层使用旋转位置编码 (RoPE) (Su et al., 2023)，滑动窗口大小为 512，基准频率为 10,000。全局注意力层不使用任何位置编码 (Kazemnejad et al., 2023)，其性能与 RoPE 相当，同时更高效。我们使用分组查询注意力 (GQA)，含 8 个 KV 头 (Ainslie et al., 2023)，每头维度为 128。我们对查询和键应用 RMSNorm (Roy et al., 2020; Henry et al., 2020; Dehghani et al., 2023; Wortsman et al., 2023)。这些标准组件的使用使我们能够在长上下文上利用 FlashAttention-4 (Zadouri et al., 2026) 和 Ulysses 风格上下文并行 (Jacobs et al., 2023) 实现高效训练。

前馈网络与 MoE。对于每个块中的前馈层，我们在 MoE 层与稠密前馈网络 (FFN) 之间交替。我们发现，将高稀疏度层与零稀疏度 (稠密) 层配对的这种模式，其扩展性与均衡稀疏度分配——即全程使用中稀疏度 MoE——相当，而在 iso-active 与 iso-total 参数量设置下墙钟时间更高效 (第 2.2.3 节)。我们还发现，全层 MoE 布局更依赖共享专家，而向交错布局添加共享专家几乎无益。我们的第一个前馈层是稠密的 (DeepSeek-AI et al., 2025a; Kimi Team et al., 2026; Dai et al., 2024)，并且我们对稠密和 MoE 前馈层都使用 SwiGLU (Shazeer, 2020)。

我们采用 LatentMoE 设计 (NVIDIA et al., 2025)，在 all-to-all 调度之前应用共享降维投影。在 all-to-all 聚合之后，潜在表示被投影回其原始维度。重要的是，路由决策基于原始表示，每个压缩表示通过 softmax 门控路由到 512 个专家中的 8 个。

我们使用全局批次负载均衡损失 (Qiu et al., 2025; Yang et al., 2025)，其中经验专家频率通过对同一全局批次内的数据并行 worker 和微批次进行聚合获得。我们发现聚合策略比负载均衡损失类型重要得多。只要保证全局聚合，我们使用的 GShard 式损失 (Lepikhin et al., 2020) 表现与无辅助损失变体 (Wang et al., 2024a) 相似。

值得注意的是，我们观察到关于最优负载均衡方法的实验结果可能取决于专家容量。在有限容量下，即使精心设计 token 丢弃策略以避免细微的因果泄漏，并且在充裕的容量预算下保持较低的丢弃率，所得结论仍可能与无丢弃设置下得出的结论不同。因此我们最终采用完全无丢弃 (droppless) 的 MoE 实现 (详见第 2.8.1 节)，它支持 all-to-all 通信的可变消息大小，并保证在高失衡情况下内存使用有界。

表 1. MAI-Base-1 架构家族中各模型规模的配置。(Expert-)FFN 大小指第一个线性层的输出大小，由于 SwiGLU，它是第二个线性层输入大小的 2 倍。MAI-Base-1 的配置与 L78 略有不同，以提升训练与推理效率。

模型	激活参数	总参数	层数	隐藏维度	FFN	降维投影	专家 FFN	Top-k/专家数	KV/Q
L12	365M	3.9B	12	1024	2048	512	1536	8/512	8/16
L18	760M	13B	18	1536	3072	768	2304	8/512	8/16
L24	1.5B	30B	24	2048	4096	1024	3072	8/512	8/24
L30	2.6B	58B	30	2560	5120	1280	3840	8/512	8/32
L36	4.0B	100B	36	3072	6144	1536	4608	8/512	8/32
L42	6.1B	159B	42	3584	7168	1792	5376	8/512	8/40
L66	21.7B	615B	66	5632	11264	2816	8448	8/512	8/64
L78	35.6B	1015B	78	6656	13312	3328	9984	8/512	8/80
MAI-Base-1	34.7B	962B	78	6656	13312	3072	10240	8/512	8/80

2.2 模型消融方法论

下面我们描述用于评估预训练中不同架构与数据相关设计选择的实验设置。一个关键的考量是规模的作用：随着计算预算增加，模型创新或新数据集带来的表面收益往往会减弱。因此，在将改动纳入我们的基线之前，我们必须在不同规模下收集充分的支撑证据。在接下来的小节中，我们介绍使我们能够以有原则且可复现的方式执行这一过程的主要组件：规模阶梯与效率增益。

2.2.1 规模阶梯

所有消融实验都明确以扩展性质为核心关注点。为此，我们利用规模阶梯：对于任何消融实验，我们以恒定的每（激活）参数 token 数（TPP）训练不同规模的模型，并将扩展曲线与我们的基线进行比较。TPP 的选择取决于实验的性质。例如，大多数架构消融实验在 Chinchilla 最优区域（Hoffmann et al., 2022）附近以 100-200 TPP 进行，而我们的主运行通常在更高的 500-1,000 TPP 下过度训练，以得到相对更紧凑、适合重度推理需求的模型。消融实验使用最新的阶梯进行，被接受的改动成为下一版基线阶梯配置的一部分，如此往复。

对于第 2.1 节所述的架构家族，一个模型配置由单一参数完全确定：层数 L 。对于我们所使用的 5:1 局部与全局注意力之比， L 必须是 6 的倍数，以便最后一个注意力层应用全局注意力。隐藏维度 D 则由 $D = L \times \frac{256}{3}$ 确定，以保持阶梯上相同的纵横比。这一纵横比选择与一些近期模型（OpenAI, 2025; Liu et al., 2024; Gemma Team, 2024）类似，并且由于 L 始终是 6 的倍数，在我们的设置中会得到对硬件更友好的 D 值。查询头数量设为 L ，并向上取整到最近的 16 的倍数，以便于使用张量并行进行推理服务。每个稠密前馈层将隐藏维度扩展 2 倍。对于 LatentMoE 层，我们使用 $2\times$ 的压缩因子，并在每个专家内部将压缩表示扩展 $3\times$ 倍。

完整规模阶梯的细节见表 1。MAI-Base-1 的最终架构有 78 层。虽然它基于阶梯中的 L78，我们略微调整了其专家输入大小以适配 DeepEP（Zhao et al., 2025a），DeepEP 要求 all-to-all 集合通信的隐藏维度可被 512 整除。

表 2. 与 MAI-Base-1 的交错布局（8/512 MoE + 稠密 FFN）相比，两种全层 MoE 变体的效率增益。以 EG_{FLOPs} 衡量时两种变体都接近基线，但在 EG_{Time} 上明显落后。

评估类别	全层 MoE (8/384)		全层 MoE (7+1 共享/384)	
	EG_{FLOPs}	EG_{Time}	EG_{FLOPs}	EG_{Time}
General	0.88	0.69	0.99	0.78
STEM	0.93	0.73	1.04	0.82
Math	0.94	0.74	1.06	0.84
Code	0.94	0.74	1.03	0.82
Multilingual	0.94	0.75	1.05	0.82
Weighted Average	0.94	0.73	1.03	0.82

2.2.2 效率增益

在我们的架构与扩展讨论中，效率增益（EG）衡量的是：基线模型需要多高的训练成本才能达到给定候选模型达到的评估指标（Kaplan et al., 2020; Hoffmann et al., 2022）。大多数情况下，评估指标是基于我们的预训练评估套件（第 2.3 节）定义的聚合评估损失。我们首先将缩放定律拟合到基线阶梯上，

$$L = f(C) = AC^{-\alpha} + E \quad (1)$$

其中 C 表示训练成本，如给定基础设施下的 FLOPs 或时间。 A 是控制可约损失项整体幅度的缩放系数， α 是控制损失随计算量增加而下降速度的缩放指数， E 是不可约损失。对于以成本 C' 达到 L' 的候选运行，我们计算达到相同损失所需的基线成本预算 $f^{-1}(L')$ ，并将 EG 定义为

$$\text{EG} = \frac{f^{-1}(L')}{C'} \quad (2)$$

例如， $\text{EG} = 1.3$ 意味着基线模型需要高出 30% 的成本才能达到候选模型的损失 (Liu et al., 2025a; Dey et al., 2023)。我们还在不同的成本预算下进行消融实验，以获得一系列 EG vs. cost (EG 随成本变化) 曲线，这有助于我们找到随规模增加而持续存在的改进。

我们最常用 FLOPs 来定义成本 C ，这有意不考虑训练中可实现的墙钟效率。成熟架构的实现往往随时间积累了大量的性能优化，这可能使它们看起来比更新的变体更强。通过将指标与 FLOPs 利用率 (MFU) (Chowdhery et al., 2023) 解耦，我们可以考察在对其实现投入同等优化努力的情况下，不同架构之间的相对表现。然而，在考虑到我们现有技术栈确实关心硬件效率的情况下，我们使用上述时间定义作为成本 C 。在这些情况下，我们明确将效率增益称为 EG_{Time} 。

2.2.3 模型消融示例

我们在日常消融实验中广泛使用阶梯与 EG 比较。下面我们展示两组实验，说明 EG 如何在不同评估类别间变化，以及 EG 如何扩展以反映与我们基础设施的协同设计因素。

示例 1: 稀疏度分配。表 2 将我们的交错布局 (高稀疏度 MoE + 稠密 FFN) 与更常见的全程使用中稀疏度 MoE 层的设计进行比较。实验在稀疏度受控的设置下进行，其中所有模型家族共享相似的总参数与激活参数之比；这是通过固定 top-k (8, 或 7+1 带一个共享专家 (Dai et al., 2024; Qwen et al., 2025))，同时调整全层 MoE 家族的专家扩展率和专家数量实现的。我们报告沿两个成本维度定义的 EG: FLOPs 和训练时间，在第 2.2.1 节所述的阶梯模型 L12-30 上测量。虽然全层 MoE 结合共享专家在 FLOPs 上达到了 1.03 的 EG，但一旦考虑训练 MFU，交错范式提供了更好的权衡，这反映在 $\text{EG}_{\text{Time}} < 1$

Top 8 / 256 专家 Top 8 / 512 专家 Top 8 / 1024 专家

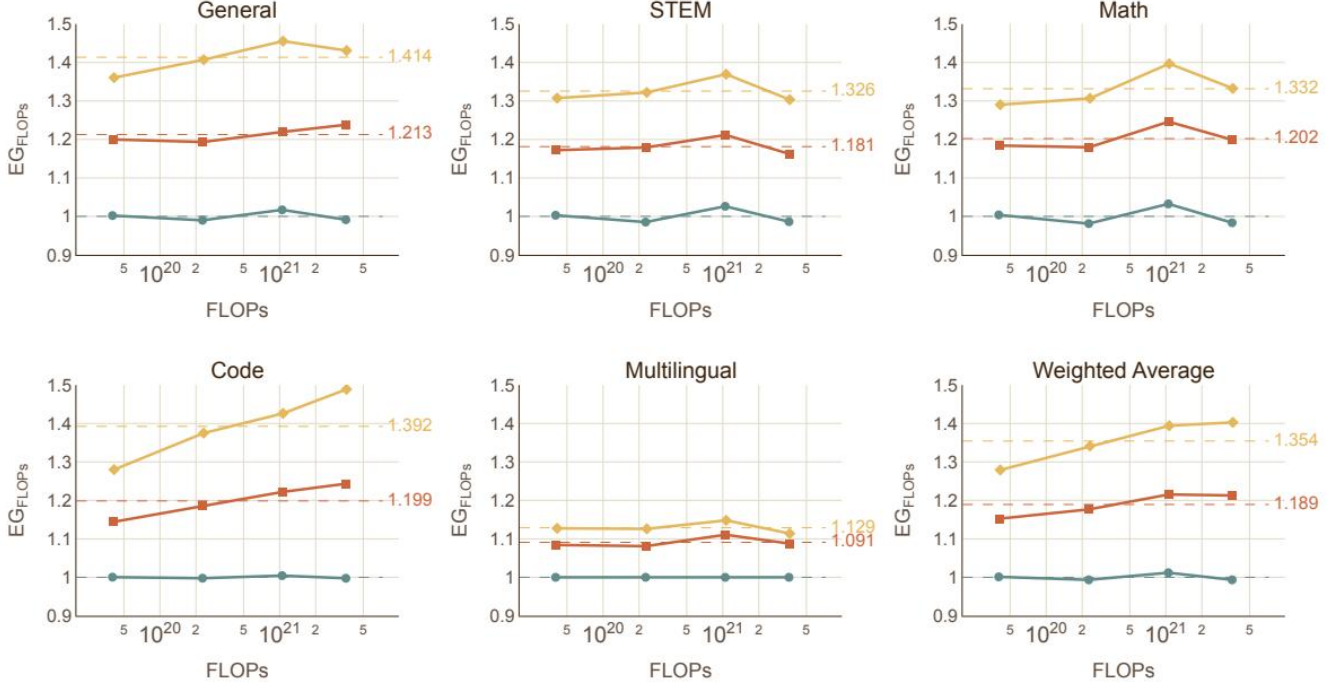


图 3. 随模型家族稀疏度增加，各评估任务类别上的预训练效率增益（第 2.2.2 节）。结果为针对每个稀疏度水平训练一条模型阶梯（第 2.2.1 节）获得。由于每个数据点都与缩放定律（式 1）进行比较，基线原始数据点的 EG 可能不严格等于 1。

示例 2：稀疏度扩展。可扩展的架构设计应在稀疏度增加时带来健康的改进。图 3 表明我们的架构家族具备这一特性，随着专家数量从 256 增加到 1024（top-k = 8，MoE 稀疏度水平从 32× 到 128×），效率增益保持一致。对于 MAI-Base-1，我们选择 top-8/512 配置以平衡模型质量与训练和推理效率，并将稀疏度扩展的进一步探索留给未来工作。

2.3 评测方法论

我们使用一组专注于 NLL（负对数似然）任务的内部评测基准，用于实验与进度跟踪。它们来自不同的来源：供应商创建、与训练完全隔离的数据；互联网上不存在的内部来源；以及我们小心地将其从训练数据中移除的网络来源。我们的评测基准示例列于表 3，并在第 2.3.2 节讨论采用 NLL 评测基准的动机。我们将在第 2.7 节报告预训练后的评测结果。

我们总共拥有近 40 个此类评测基准，分为 5 个不同的类别：代码（Code）、STEM、数学（Math）²、通用知识（General Knowledge）和多语言（Multilingual）。当我们需要将这些基准聚合为单一数值，以直接进行 EG 比较（第 2.2.2 节）或对模型排序时，我们使用以下公式：

$$\text{Target} = 0.5 \times \text{Coding} + 0.175 \times \text{STEM} + 0.175 \times \text{Math} + 0.1 \times \text{General knowledge} + 0.05 \times \text{Multilingual} \quad (3)$$

这些显式权重使我们能够固化关于各评测基准相对优先级与重要性的决策。我们对代码和推理任务下游性能的重视，反映在预训练中我们为这些类别分配的更高权重上。

²我们有意上调数学的权重，并将其与更宽泛的 STEM 类别区别对待。

为计算式 3 中各类别得分，我们首先使用固定的内部参考模型作为基线对原始 NLL 结果进行归一化，然后对该类别中所有评测基准取均匀平均。所有被评测模型使用相同的 tokenizer，因此 NLL 值可直接比较。请注意，我们选择将数学提升为独立类别，而 STEM 则聚焦于科学、技术与工程。

表 3. 我们 NLL 评测套件中的数据示例类型。所有示例均经过仔细去重，确保与任何训练数据不重叠。

类别	评测数据	来源
代码	Microsoft 代码与 pull request	内部私有项目
	人机协作编码会话	对先前模型的内部测试
STEM	研究生级别 STEM 问题的完整解答	由供应商定制
数学	高等数学问题的完整解答	由供应商定制
	在线社区讨论	公共网络论坛
通用知识	人机交互	对先前模型的内部测试
	Pyramidal trivia（金字塔式问答）	与训练数据去重后的公开数据库
	高难度冷知识问答	由供应商定制
多语言	多语言人机交互	对先前模型的内部测试

2.3.1 公共评测集净化

公开可用的评测基准在指导我们许多模型的开发方面发挥了重要作用。近年来，随着 AI 模型重要性的提升，评测数据集泄漏到训练数据来源中的情况也在增加。这在 GitHub 上尤为常见，许多 AI 研究人员和开发者会在 GitHub 上分享他们的工作。许多常见评测以其原始形式发布在 GitHub 上 (Austin et al., 2021)，几乎每个基准都会在那里被复制多次，通常附带解答、模型生成结果等内容。许多公共评测源自我们用于训练的部分在线来源，但经过了额外的整理流程（例如 (Wei et al., 2024a; Hendrycks et al., 2020)）。随着我们训练集的规模与覆盖面不断增长，这种重叠的影响变得显著。与先前工作类似 (Godey et al., 2025)，我们在实验中观察到，这种污染可能导致反直觉的结果，例如名义上旨在提升编码性能的数据同时也带来了长尾通用知识评测的提升。

我们采用了多种方法来检测并移除训练数据中泄漏的评测样本。所有来自 huggingface.co 及其镜像域名的数据均被移除，并对我们所有训练来源应用通用的 20-gram 模糊去重，相似度阈值为 80%。我们发现这些方法能准确移除大量泄漏样本，但它们并不完美。

鉴于这一点，我们开发了仅供内部使用的私有基准，我们确信这些基准在互联网上的其他地方无法找到。

2.3.2 准确率与 NLL 评测的比较

我们将内部专有评测用于日常开发。这些评测由 MAI 团队成员策划，其中一部分由聘用领域专家的数据供应商定制，另一部分则收集自 Microsoft 全球组织内的领域专家。

对于预训练，我们选择聚焦于采用 NLL 评分方法的评测，而非选择题、生成式或其他形式。衡量模型能正确回答多少问题的评测在跨模型比较以及与人类基准对比时很方便，但它们存在下述若干操作层面的挑战。

评测的效率与成本。许多前沿基准需要思维链推理 (Phan et al., 2025; Rein et al., 2023)、工具使用或其他形式的生成。这种自回归生成既昂贵又耗时。训练框架与配置往往并非为适合此类评测的快速自回归推理而优

化。使用更高效的推理框架可以加速这些评测，但会引入额外的复杂度以及对架构变体的支持需求。采用开放式或自然语言答案的评测通常需要裁判模型 (judge model)，进一步增加成本。相比之下，NLL 评测与预训练具有相同的基本下一 token 预测目标，可以非常高效地运行。通过降低每次评测在时间与算力上的成本，我们可以运行更全面的评测，并确保每次实验之间的一致性。

对混杂因素的敏感性。评测可能对预训练模型行为中看似微小的变化高度敏感。例如，MMLU (Hendrycks et al., 2020)、MMLU-Pro (Wang et al., 2024b) 及相关基准 (Yue et al., 2024, 2025; Singh et al., 2025b) 对评估预训练知识很有价值，但其选择题格式隐含地假设模型已具备解读和作答选择题的能力——这种能力在预训练评估中通常只有在规模出乎意料地大时才会出现 (Du et al., 2024)。尽管这些任务有时采用基于 A、B、C、D 各选项 logits 的 NLL 评分方案，但许多问题需要的不仅仅是事实回忆，自由形式生成往往是一种更忠实的评估协议。

类似地，MATH (Hendrycks et al., 2021) 要求严格遵守格式约定，例如将答案包裹在 $\square$ 中；MBPP (Austin et al., 2021) 则使模型暴露于共享 prompt 中的 Unix 风格换行符 (‘\n’) 与问题陈述中 Windows 风格换行符 (‘\r\n’) 之间的不一致。预训练数据中微小的格式差异——尤其是跨来源各不相同的情况——会显著影响模型对这些不一致的响应方式。我们在许多实验中观察到了此类交互，这使结果解读变得复杂，也增加了新老研究人员误读结果的可能性。

NLL 评测对此类问题更具鲁棒性。在每一步预测中，模型都以真实前缀为条件。这种教师强制 (teacher-forcing) 设置限制了微小误差不断累积的程度。我们的结论与先前工作一致，即基于损失或困惑度的指标能够在带有格式选择的任务上提供更高信噪比的模型开发反馈 (Heineman et al., 2025; Du et al., 2024)。

构建的复杂性。构建一个高质量的问答 (Q&A) 评测需要大量且精心协调的努力。通常需要多轮数据收集、难度校准、去重和质量控制，评测才能达到可接受的标准，且每个阶段都必须与领域专家密切协作完成。设计全新问题本身就是一项不平凡挑战，尤其是在领域知识的前沿。

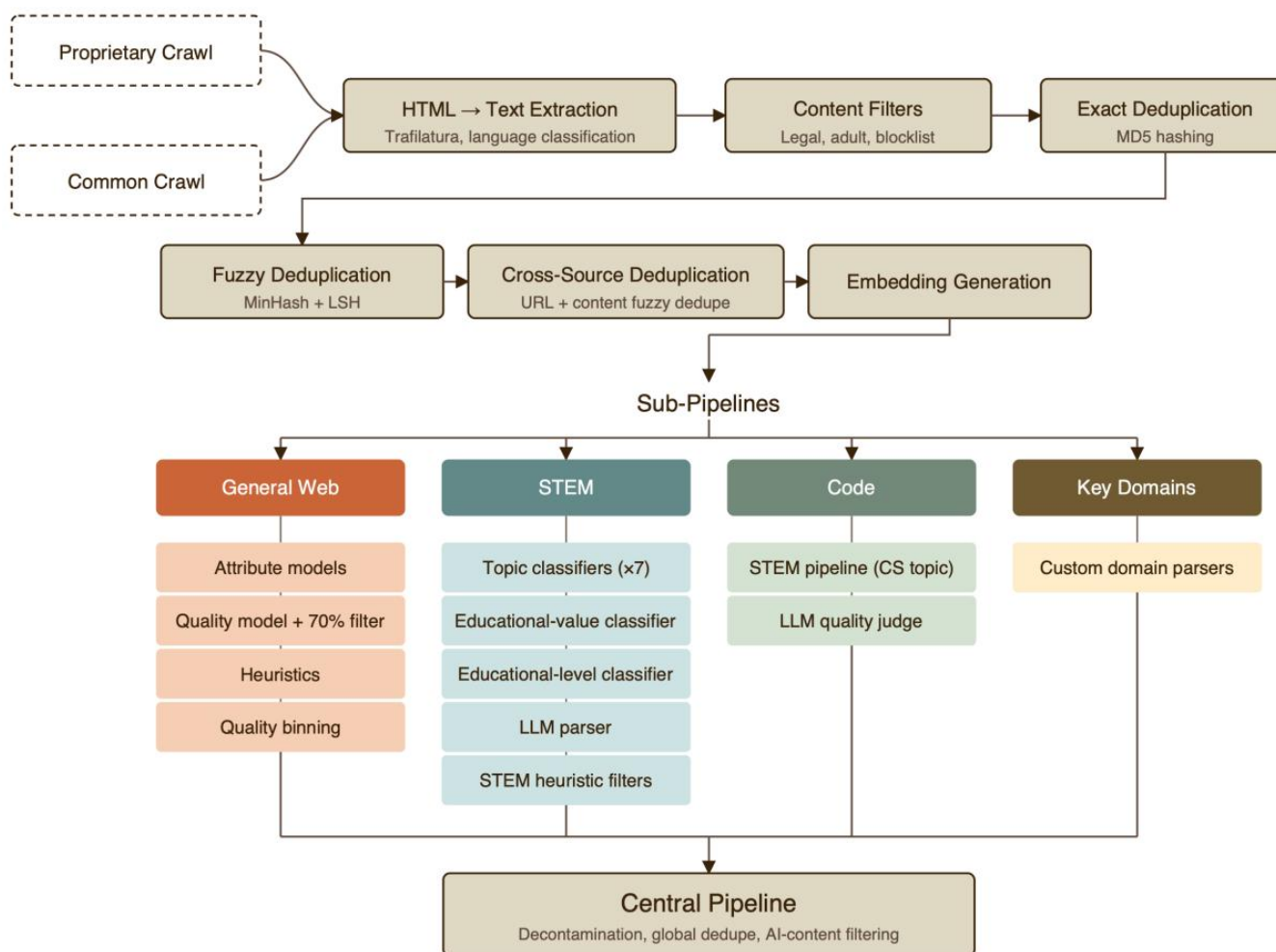


图 4. 处理 HTML 预训练数据的流水线。我们从自研爬虫抓取和 Common Crawl 出发，先进行 HTML 提取，再依据各种启发式规则进行过滤。随后执行精确与模糊去重、跨来源去重以及 embedding 生成。这些数据随后由不同的流水线进一步处理；各流水线的具体细节见附录 B。

相比之下，基于 NLL 的评测的进入门槛要低得多。任何与主题相关的内容都可以作为初始语料，后续改进——例如施加额外结构、剪枝到更有代表性的样本，或纳入更高级的材料——会直接转化为评测质量的渐进提升。

2.4 预训练数据

MAI-Base-1 在公开可获取与获得许可的人类生成数据的混合语料上训练。我们汇集了涵盖网页、公开 GitHub 代码、图书、学术论文、新闻、多语言文本以及领域特定材料的高质量、多样化预训练数据。我们选择不在预训练中使用语言模型生成的合成数据，并努力避免和移除所收集数据来源中的 AI 生成内容（附录 B）。

我们所有的预训练数据均在内部处理，源自我们的几类基础来源：网络上的 HTML 与 PDF、图书与期刊，以及公开 GitHub 代码。我们不使用任何开源训练数据集，并将常见机器学习仓库排除在我们的网络数据之外，例如 huggingface.co 及类似网站。我们反复发现，对数据质量（而不仅仅是数量）进行仔细迭代至关重要——

这不仅影响预训练结果，也影响推理基准上的下游表现。以下小节描述我们在处理各主要数据来源时采用的通用技术。每个来源数据流水线的具体细节与完整技术说明见附录 B。我们网络数据处理流水线的总览如图 4 所示。

表 4. 各训练数据来源的知识截止日期。

来源族系	知识截止日期
网页 HTML	September 2025
网页 PDF	December 2025
公开 GitHub 代码	June 2025
图书与期刊	March 2026

2.4.1 数据来源、许可与治理

我们数据中公开可获取的部分按照适用的使用条款和网络控制的行业标准进行收集。对于网络数据，我们使用尊重 Robots 排除协议（robots.txt）及相关 meta 标签和 HTML 控制的自研爬虫，使站点所有者能够管理其站点内容的访问与使用方式。我们还排除了违反 Microsoft 负责任 AI 政策或出现在美国贸易代表办公室（USTR）不良市场名单上的来源。

我们的训练语料还包括通过商业协议从第三方提供商处获取的数据集。第三方数据集需经过尽职调查流程，以评估数据保真度、所有权和适用使用权，并受保密条款与条件的约束。各来源的知识截止日期见表 4。

除用户明确选择加入或相关协议涵盖且尊重用户退出选择的情况外，我们未将私有客户数据或 Microsoft 产品与服务的数据用于预训练。训练之前，我们的整个语料都要经过 PII 风险与安全过滤；这些步骤的细节见附录 B。出于隐私、法律、安全与竞争方面的原因，我们不会披露完整的数据集清单或数据提供商。

2.4.2 HTML 提取

我们很大一部分数据来自网络，其中可训练内容需要从原始 HTML 中提取。实现这一目标的方式有很多，所得质量差异往往相当细微。我们最感兴趣的部分内容——数学、代码等——在网络上的表示方式并不一致，许多现成方法最终会丢弃大量此类关键数据，同时保留周围的文本。表格和其他结构化内容也常常相当重要，但保留这些数据需要一些技巧。

我们不对所有来源使用单一的提取方法；相反，我们根据每个域名的结构和可靠性应用不同的方法：

- 针对特定来源的结构化解析器用于 HTML 或 XML 等标准化格式，基于 schema 的解析能够准确转换为文本表示。
- 使用 BeautifulSoup (Richardson, 2026) 等工具实现的手工构造提取器，应用于结构一致但通用启发式规则处理不佳的域名，或内容价值与体量足以支撑额外工程投入的域名。
- LLM 与基于 agent 的处理用于需要定向提取、归一化或超出确定性规则能力的语义过滤的域名 (Mahabadi et al., 2025)。在所有采用此方法的场景中，LLM 只能选择保留或删除原始来源文本中的内容，无法向输出中添加额外的合成内容。

- 当进一步处理有可能丢弃重要信息时，我们直接对原始内容进行训练。例如，Wikipedia 特有的标记语言 wikitext (MediaWiki, 2026) 相比 HTML 缺乏成熟的工具链，而 infobox 等元素并不总能被现有解析器忠实保留。尽管 wikitext 的冗余度约为 3×，我们发现直接训练完整的原始标记比依赖剥离简化后的转换效果更好。对于 Wikipedia 这类高价值、低体量的来源，这种取舍是值得的。

2.4.3 去重

我们这种规模与稀疏度的模型拥有非凡的知识获取与回忆能力。虽然这种能力是其作为通用系统发挥有效性的核心，但也增加了对训练数据的记忆与过拟合风险 (Lee et al., 2022; Abbas et al., 2023)。当记忆发生时，模型无法泛化底层概念，导致在其他领域性能下降。此类过拟合的一个主要驱动因素是训练语料中的重复。

大规模网络数据包含大量冗余：许多文档互为近似重复，仅存在细微的格式差异，或在多个站点间被复制。若不进行仔细过滤，模型可能会接触到大量相同内容。随着模型容量的增加，这种冗余的影响变得更加显著。特别是，预测扩展行为对有效唯一 token 的数量敏感：当语料新颖性降低时，较大模型会表现出退化的扩展行为，因为它们更早地耗尽了新信息的供给。一旦数据集中的唯一内容饱和，额外的训练 token 几乎不再带来收益，从而限制了进一步的改进。在许多实验中，我们一致观察到，严格去重和移除冗余数据能同时改善预训练指标和下游推理基准。

鉴于限制冗余的重要性，我们的流水线包含多个去重阶段，每个阶段针对不同类别的冗余。我们的所有数据——无论来源如何——都会接受其中一种或多种去重处理。各来源（网络 HTML 与 PDF、图书与期刊、公开 GitHub 代码）的具体处理方式见附录 B 中的完整技术细节。这里我们介绍所采用的各种技术：

- 样板内容去除。网页 HTML 常包含重复的样板文本。利用文档内和跨文档的行出现统计，我们移除页眉、导航栏、页脚、侧边栏以及解析产物引入的冗余段落。
- 精确重复。相同内容可能因跨网站转载或因系统级故障造成的重复快照而多次出现。我们移除所有字节级与哈希级的精确重复。
- 模糊重复。我们应用基于 MinHash 局部敏感哈希 (LSH) 的模糊去重 (Broder, 1997)，相似度阈值为 0.8，遵循 Smith et al. (2022) 的配置。
- 模板化网页。许多网站由共享模板生成页面，仅存在少量词汇变化（例如带原始算术表的“calculator”网页）。我们将每个页面骨架化为其最频繁出现的 token，并对这些模板执行模糊去重，以消除大批近似相同的页面。
- 语义重复。独立产出的文档由于共享上下文或规范的问题表述仍可能高度相似。这在代码数据集中尤为常见，众所周知的编程练习会在作业、考试、面试和竞赛中反复出现。尽管是独立编写的，解答往往趋于一致（例如二叉搜索树的标准遍历）。此类聚类在自然数据集中大量存在，移除它们能提升模型的鲁棒性和任务多样性 (Sorscher et al., 2023; Abbas et al., 2023; Tirumala et al., 2023)。我们使用 Qwen3-Embedding-0.6B (Yang et al., 2025) 模型识别语义相似的文档，并仅保留每个聚类中有限数量的代表样本。

我们既在每条处理流水线的早期阶段应用这些去重技术，也会在训练前作为最终步骤跨数据集再次应用。历史上，我们通过显式过滤器管理训练来源之间的重叠——例如，从所有网络爬取数据中排除 Wikipedia 域名。然而，随着我们来源的多样性与覆盖面不断增加，以及处理流水线变得更加复杂，跨数据集意外重叠的机会也在增长。

为解决这一问题，我们使用全局丢弃顺序（drop-order）在数据集上执行跨数据集去重。当检测到模糊或语义重复时，该实例仅保留在其出现的数据集中排名最高者中，并从所有丢弃顺序优先级更低的数据集中移除。因此，在解读实验结果和评估单个数据集的贡献或价值时，丢弃顺序的选择至关重要。对某个数据集的修改可能引入新的重叠，即使在未添加任何新内容的情况下，也会将数据从一个来源实际转移到另一个来源。

2.4.4 数据过滤与分类

下游处理的一个核心目标是将庞大而异构的原始语料转化为既有利于预训练、又可受控地用于数据混合优化（第 2.5 节）的数据。在各来源中，我们首先移除不太可能对训练产生正向贡献的数据，包括垃圾内容、受限或政策敏感内容，以及其他来源特定的噪声。随后，我们将剩余数据归类为可解释的桶，例如质量分层、语言组、主题类别、教育价值、教育水平、来源类型和领域特定子语料。

由于不同的语料展现出不同的信号与失败模式，精确的过滤与分类策略因来源而异。因此，我们应用并组合了以下几类技术：

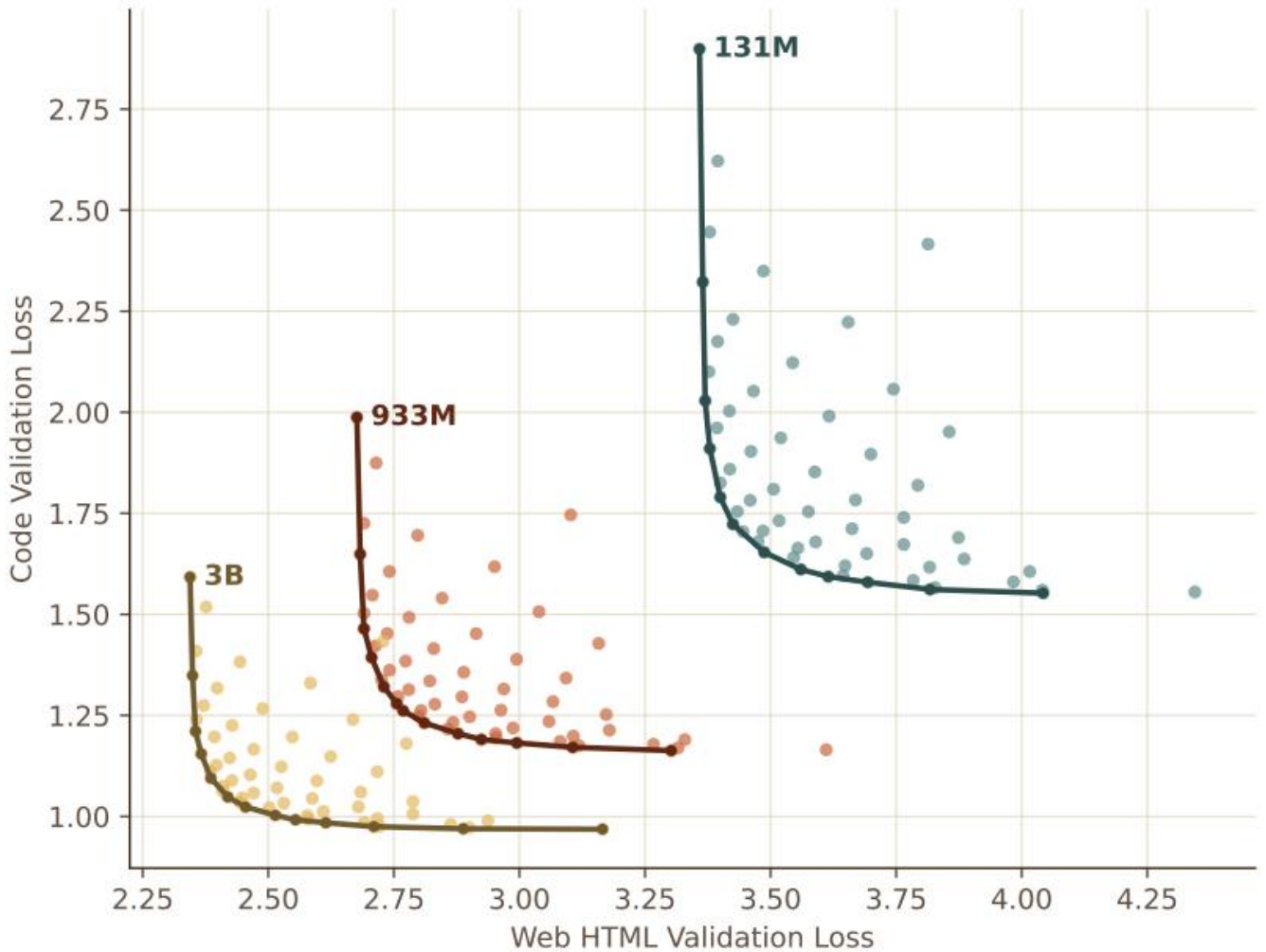
- 利用元数据信号，例如域名、文件名、仓库元数据、PDF 创建者与生产者字段，以及文档级元数据。
- 来源特定的启发式规则，例如网络文本质量过滤器、OCR 伪影过滤器、面向 STEM 内容的数学感知过滤器，以及针对生成代码的基于路径或内容的过滤器。
- 学习式分类器，包括 fastText 式分类器（Joulin et al., 2016）和基于 embedding 的模型，用于语言、主题、教育价值、教育水平、质量及其他语义属性。
- 提示式 LLM，选择性地用于更高价值或更模糊的决策，例如章节级提取、质量评判和细微的主题标注。
- 人工探索与标注，用于识别失败模式、验证过滤精度、审计高影响来源类别，并为分类器和 LLM 裁判构建训练数据。

过滤与分类共同将处理后的来源语料转化为结构化的数据桶集合。过滤通过移除低价值或有问题的内容提升数据质量，而分类则将剩余语料组织为可用于数据混合优化的可解释维度。这一结构支持在来源族系、质量分层、主题、教育水平、语言和来源特定格式之间进行受控消融。各主要来源的进一步实现细节见附录 B。

2.4.5 数据消融实验

每个处理后的预训练数据集都要经过严格的消融。我们采用两种数据消融方法：单来源消融和扩展阶梯消融。单来源消融是一种隔离方法，用于量化特定数据集的边际效用，并将其贡献归因于基于 NLL 的评测。我们将目标数据在混合中的权重上调至 50%，并从零开始训练。对于扩展阶梯消融，我们使用第 2.2.1 节中描述的扩展阶梯，在全量混合中消融该数据，以预测某项数据整理决策在所有其他数据集的背景下、于最终目标规模和 token 范围内将表现如何。阶梯消融中执行数据源降采样，以模拟目标模型的多轮次（multi-epoch）特性。

图 5. 数据混合前沿的示例说明。我们在 61 种 Web HTML、代码（Code）及其他数据的多样化混合配置上，以相同 TPP 在 3 种不同模型规模下从零训练了 183 个模型。实线表示 Web HTML 与代码的验证损失性能前沿。位于该 Pareto 前沿上的模型是这两个组分占主导的混合配置，而偏离前沿的模型则包含更大比例的“其他”（other）子集。



2.5 选择训练数据配比

训练期间采样的数据源及其相对比例会显著影响最终模型。在算力预算固定、语料库包含数百个异构数据源的情况下，我们必须确定如何为它们分配权重。

为此，我们首先定义一个目标函数。如图 5 所示，给定一组目标能力（如编码或通用知识），可以得到多个帕累托高效（Pareto-efficient）配比，在其中一个能力无法在不损害另一能力的前提下得到提升。因此，我们必须为这些不同能力分配相对重要性。具体来说，我们选择最小化式 3，即对一系列留出（held-out）NLL 评测的聚合。

优化数据配比需要在感兴趣领域、数据质量、可用的唯一 token 数量，以及一个数据集在收益递减或过拟合之前最多可重复的次数之间进行权衡。

2.5.1 优化面临的挑战

数据配比的选择对最终模型有重大影响，因此我们希望以有原则的、结构化的方式来解决这一问题。一个有效的技术必须应对以下几项挑战：

- 效用的定义。我们如何具体定义优化所针对的目标？我们的总体目标是开发在真实世界中有效的模型，但对每个候选数据配比都运行一次完整的训练后处理（post-training）流程，代价将高得难以承受。
- 巨大的搜索空间。随着数据集数量增长到数百个，数据配比的空间变得极大，穷举搜索在计算上不可行。
- 与规模相关的影响。某个数据源可能对较小规模的模型特别有用，而另一个数据源在大规模下更有用（Ye et al., 2025）。
- 跨数据集交互。数据集之间在不同方面既互补又可替代。一个数据集的效用无法脱离配比中的其余部分独立衡量。
- 多轮次影响。数据配比无法孤立评估：训练长度很重要（Chen et al., 2026a）。虽然很有吸引力，但对高质量但规模小的数据集赋予过高权重，在训练长度较长时会导致收益递减，因为该数据集最终会被消费多次。在最坏的情况下，这可能导致过拟合。
- 计算成本。训练大量采用不同数据配比的模型很快就会变得昂贵，尤其是在试图探索极高维搜索空间时。

2.5.2 基于预测的方法

一类数据选择方法是以显著更小的规模训练大量模型，然后基于这些模型的评测结果构建预测模型。从这些小模型上的拟合函数出发，可以尝试估计全局最优的数据配比，或至少为搜索提供指导。这类方法包括 RegMix（Liu et al., 2025b）等。这些技术的一个关键特征是，通过依赖小模型，它们能够实现快速迭代。

我们尝试了这些方法的变体：基于一个基线配比采样数据配比，并训练了几千个小模型，每个模型的激活参数在 760M 到 4B 之间（按表 1 为 L12-L36 模型）——Web / Code 数据前沿的示例如图 5 所示。对每个模型，我们计算完整的 NLL 评测套件，考察不同数据子集与不同评测之间的相关性——包括聚合评测与单项评测——示例见图 5 和图 7。最终，我们按照 2.3 节中的式 3 为每个配比给出单一得分，并试图对其进行全局优化（最小化）。

实践中尺度依赖的一个实例。在数据配比实验期间，我们发现了一个挑战我们核心缩放假设之一的经验结论：即两个数据配比的相对排序——哪个配比能带来更好的评测表现——会随着算力的增加而保持不变。这一假设通常被非正式地称为排序不变性假设（rank invariance hypothesis）（Liu et al., 2025b），它之所以有吸引力，是因为它表明廉价的小规模实验可以可靠地指导大规模运行的决策。然而，我们发现情况并非总是如此。

在早前的一代模型中，我们试图衡量编码与 STEM 强化学习对预训练数据配比的敏感度。我们使用不同的配比优化方法和不同的评测权重，产出了两个候选配比：code-heavy-mix（约 50% 代码，预计在 Coding 类别上表现尤为突出）和 stem-heavy-mix（显著上调 STEM 数据，以针对 STEM 和 Math 类别）。我们先进行了中间规模的验证，随后在每种配比上训练了约 20T tokens 的 23B 激活参数模型（按表 1 大致相当于 L66）。除训练数据源的权重外，其余均未改动。

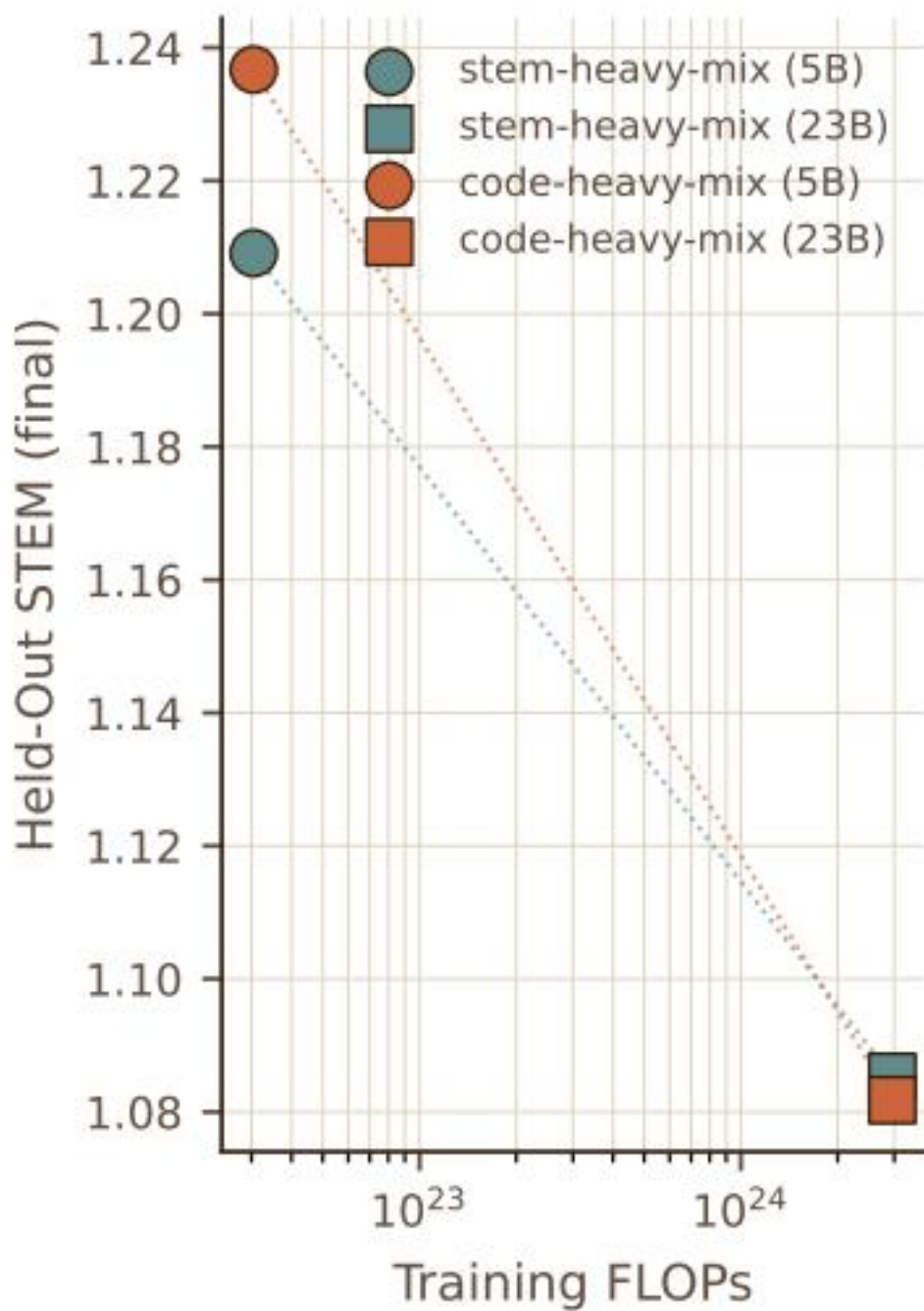
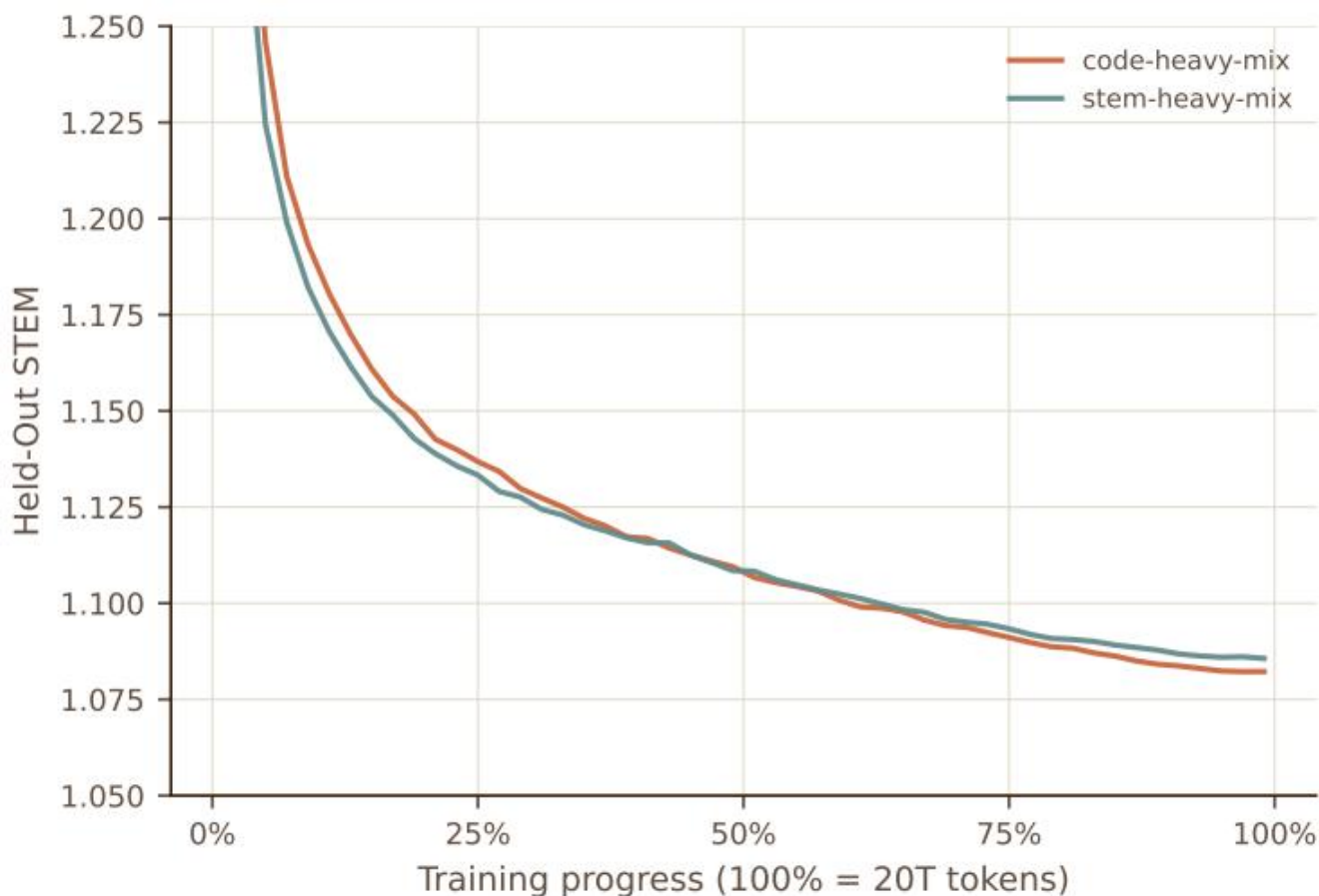


图 6. 数据配比缩放中的排序非不变性。(左): 在较小规模下, stem-heavy-mix 在留出 STEM NLL 评测上表现更好, 但在更大 (23B) 规模下 code-heavy-mix 表现更好。(右): 两种 23B 激活参数模型在不同数据配比下整个训练过程中 STEM 评测的 NLL。



结果如图 6 所示。与小规模结果一致，训练早期 stem-heavy-mix 在 NLL STEM 评测上优于 code-heavy-mix。然而，STEM 评测曲线在训练中途发生交叉，code-heavy-mix 最终反超 stem-heavy-mix。这与“数据配比的相对排名会随训练算力扩大而保持不变”的假设相矛盾。

通过检查各数据源的训练与验证损失趋势，我们识别出两个数据源：它们包含高质量的 STEM 内容，但比其他数据集有更多模糊重复，且内容多样性较低。这两个数据源在 stem-heavy-mix 中的权重为 11.8%，而在 code-heavy-mix 中仅为 0.3%。我们的假设是：这些数据源对较小模型非常有用，但其多样性不足在更大规模下本质上帮助有限。

基于这一结果，除了候选数据配比在固定实验规模下的表现，我们还更加重视其缩放性能。此外，在后续的数据配比研究中，我们通过缩放阶梯方法极其仔细地关注了数据配比的缩放特性。

2.5.3 最终配比选择

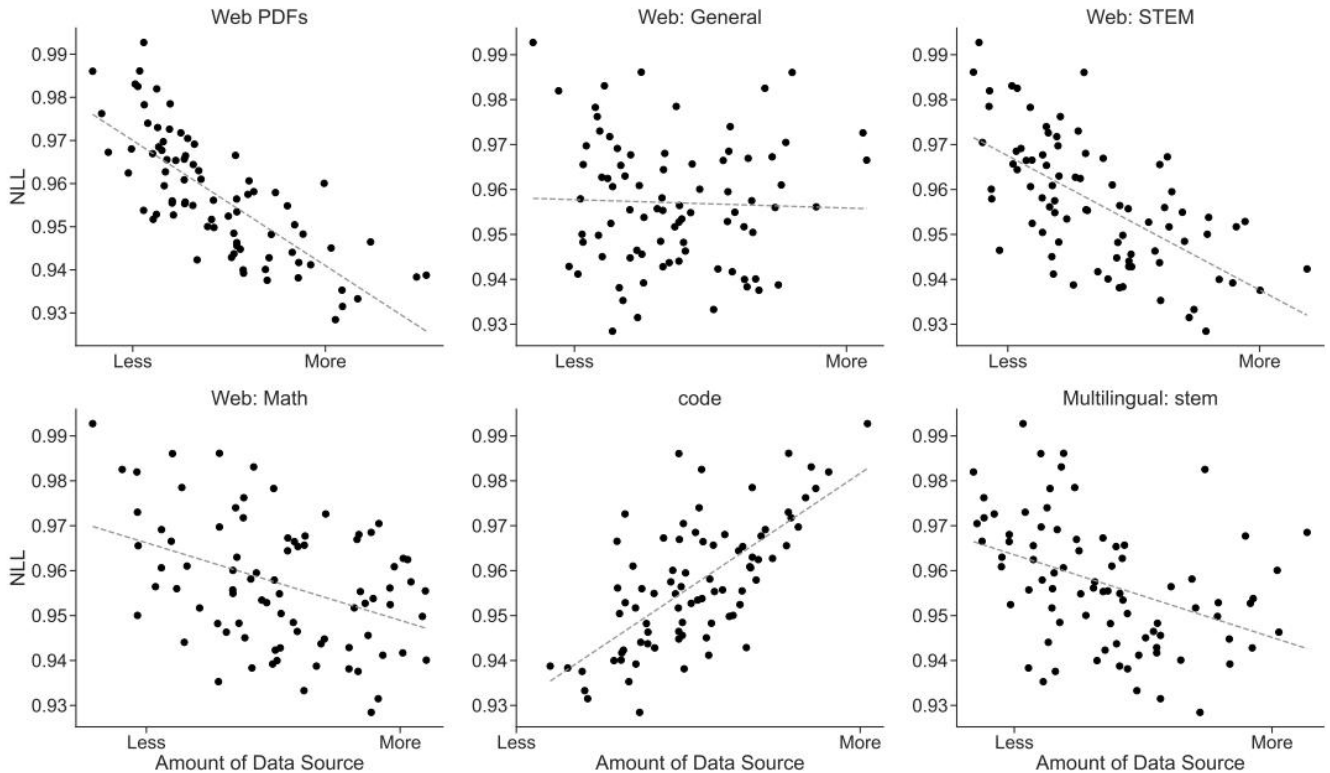
由于数据集数量众多，且我们希望在不同规模下对其进行验证，我们采用了一种分层方法来改进此前的数据配比。我们将数据划分为约 10 个类别，如代码、STEM、PDF 和通用网页。然后交替进行局部与全局优化：

- 局部搜索。我们保持所有高层类别固定，在单个子集内局部调整权重——例如，在代码文件、PR 和 commit 之间调整权重，同时保持通用网页、STEM 等权重不变。

- 全局搜索。我们保持每个高层分箱的相对构成固定，调整不同子集之间的相对权重（例如，上调 STEM 和 PDF，降低所有其他组件的权重）。图 7 展示了这一过程。

研究生水平物理 NLL

图 7. 对于一套由第三方创建的研究生水平物理评测集，我们展示了不同数据集的相对贡献（NLL 越低越好）。我们采样不同的数据配比，分别训练并评估所得模型，然后考察最终 NLL 与配比数据构成之间的相关性。我们的 PDF 数据集、从网页中提取的数学与 STEM 数据呈现出明显的正向趋势。通用网页语料基本为中性，而在配比中增加更多代码（而非其他数据）似乎对物理并无帮助。



在整个搜索过程中，我们将任一数据集的重复次数上限设为 8，以防止过拟合和反复循环同一数据带来的收益递减（Chen et al., 2026a）。

在进行了多轮全局与局部数据优化的迭代之后，我们选出几个最小化 NLL 目标函数的候选配比。我们以约 $2.8\times$ 倍于全局混配所用算力的规模训练了这些模型，并选出最佳候选。在对数据配比流程进行这些调整后，我们在这一放大验证步骤中看到了良好的缩放表现：最优候选不再随规模变化。

表 5 总结了最终构成。每个源家族包含多个数据集，各数据集的轮数各不相同。某些数据集可能同时属于多个类别（例如一个法语的 STEM 数据集），在表中我们按自上而下的顺序将每个数据集归入第一个匹配的行。在最终配比中，我们得到了约 3000 亿高质量数学 tokens，其平均采样倍数为 $5.28\times$ ，是所有源家族中最高的。不过，训练配比的主体是代码数据，达到 16.4T 训练 tokens，平均略多于 2 轮。网页文本和 PDF 数据平均各被看到不到一次（ $0.55\times$ 和 $0.53\times$ ），这意味着即使在整个 30T-token 的训练中，这些数据源的全部可用语料也未被用完。需要说明的是，对于 PDF 和网页文本，其中的数学与 STEM 部分被计入相应的学科类别。我

们的多语言语料是被降采样最激进的：可用 8.1T 唯一 tokens，但最终配比仅消耗 0.5T (0.06×) ——同样需要说明，特定领域的多语言数据已被包含在不同的类别中。

表 5. MAI-Base-1 预训练数据构成。唯一 tokens (Unique tokens) 为各源家族去重后的 token 数；训练 tokens (Training tokens) 为整个训练过程中从该家族消耗的 token 数；平均轮数 (Avg. Epochs) 为训练 tokens 与唯一 tokens 之比，大于 1 表示重复采样。

数据源家族	唯一 tokens (T)	训练 tokens (T)	配比百分比 (%)	平均轮数
代码	7.4	16.4	54.6	2.22×
STEM	2.2	4.7	15.8	2.17×
数学	0.3	1.6	5.4	5.28×
书籍与期刊	0.6	0.9	3.1	1.65×
PDF	2.7	1.4	4.7	0.53×
网页文本	8.1	4.5	14.9	0.55×
多语言（其他）	8.1	0.5	1.6	0.06×
总计	29.2	30.0	100.0	1.03×

我们仍在持续迭代数据配比目标、构成该目标的基于 NLL 的评测，以及用于提出新候选配比的算法，因为每个组件都对最终数据配比有重大影响。理解端到端的缩放行为——以及找出在预训练期间最可靠地预测下游性能的指标——仍是一个基础性的开放挑战，我们认为其中仍有很大的提升空间。

2.5.4 训练中期的数据配比

预训练之后，MAI-Base-1 按 2.6.1 节的描述分两个连续阶段进行训练中期 (mid-training) 训练。训练中期数据完全来自预训练语料 (2.4 节)，不引入任何新的或合成的数据源。我们通过对现有配比进行过滤和重新加权来整理出质量更高的子集，然后以更长的序列长度重新打包结果，用于上下文扩展。

总体而言，我们进一步将配比向 STEM、数学和代码倾斜，为推理 RL 的爬升奠定坚实基础 (Wang et al., 2025b)。在预训练配比的基础上，我们将 STEM/数学数据的比例提高到 35%，代码保持在 55%，其余 10% 分配给背景数据源。在每个类别内部，我们根据单源微退火 (microanneals) (Team OLMo et al., 2024) 局部调整权重，优化与预训练配比搜索相同的基于 NLL 的目标，并针对上下文扩展加入长上下文 NLL 任务 (附录 C.2)。此外，我们还对部分选定的 STEM 和代码数据源应用了质量过滤，具体如下。

STEM 推理数据。为提升推理任务的表现，我们主要使用 (Essential AI et al., 2025) 提出的内容质量分类法对预训练 PDF 语料中的文档进行过滤。我们定义了一个 Bloom Analyze 启发式规则：保留技术正确性高、推理深度至少为中等、且 Bloom 认知加工水平达到或高于 Analyze (分析) 层的文档 (Anderson and Krathwohl, 2001)。同时去除存在提取痕迹或缺少主要内容的文档。该过滤器保留具有结构化推理和分析的文档，而非简单的事实陈述。

代码数据。训练中期的代码数据还要经过两个额外步骤。首先，我们按质量分档 (quality-bin) 进行文件扩展名过滤：仓库根据仓库级质量指标分为三个层级。例如，常见的网页资源 (HTML、CSS、SVG) 在顶级仓库中会保留，因为它们通常属于较大的前端项目；但在较低层级仓库中会被移除，因为这些仓库中它们主要是低

质量的独立页面。其次，我们在预训练已有的仓库级格式之外引入文件级文档格式，将仓库理解和单个文件理解视为互补任务，这提升了内部文件局部 NLL 基准上的泛化表现。

记忆感知的轮次上限。除了质量过滤，我们还在训练中限制数据源的暴露量，以避免对预训练结束时已被高度记忆的数据源过拟合。我们通过每个数据源的验证代理指标来估计这一点：两个预训练 checkpoint 之间验证损失的改善中，来自以接近确定性预测出的 tokens 的比例 ($NLL < 0.01$)。该比例越高，说明 NLL 的显著降低来自记忆化或高度重复的结构。在训练中期，我们对这类数据源设定更严格的轮次上限，同时允许数值较小的数据源获得相应更多的暴露。

长上下文数据配比。训练中期分两个阶段将 MAI-Base-1 的上下文窗口扩展到 64k 再到 256k，详见附录 C。两个阶段都在更长的序列长度下重新打包数据，而不修改配比权重。这样做可以最大程度地减少分布偏移，同时显著减少现有高质量长上下文文档（如教学或学术文本）被截断的情况。

2.6 训练方案

MAI-Base-1 分三个阶段训练，序列长度逐步增加：预训练、训练中期阶段 1 和训练中期阶段 2。下面介绍各阶段的训练方案。

2.6.1 训练阶段

表 6 列出了从 2.5.3 节所述数据配比中采样的 token 数、上下文长度以及各阶段使用的 GPU 数量。预训练和训练中期 1 使用专家并行度 64 搭配 ZeRO-2，训练中期阶段 2 启用 ZeRO-3 (FSDP)。训练中期阶段使用上下文并行。训练使用我们自研的大规模训练框架 (2.8 节)。

表 6. MAI-Base-1 各阶段的训练规格。

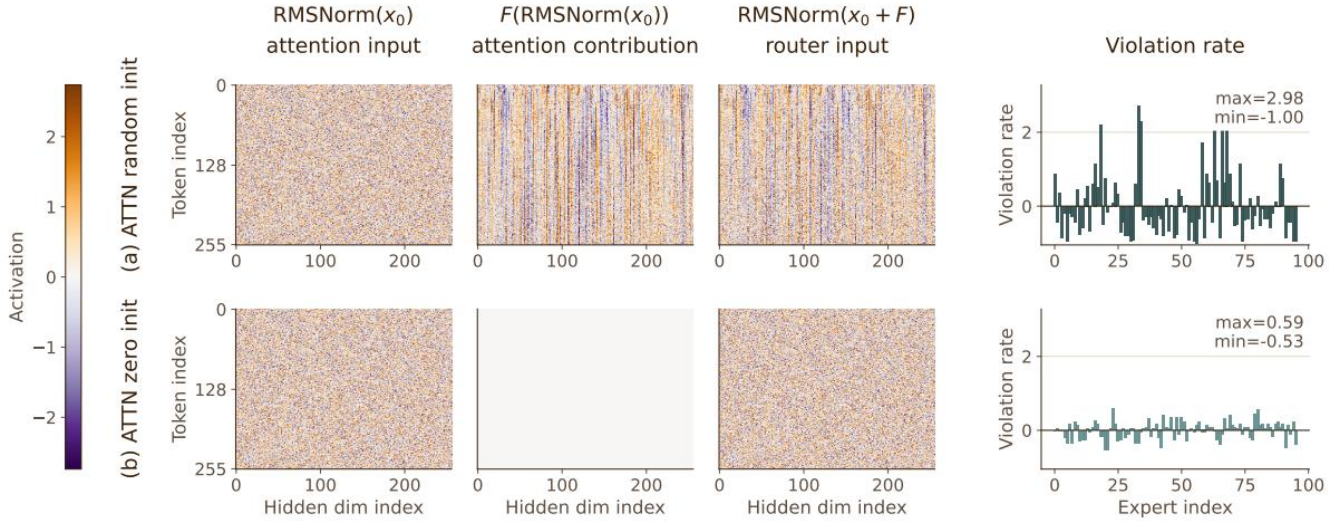
阶段	Tokens	上下文长度	GB200 GPU 数
预训练	30 T	16,384	8,192
训练中期 1	3.4T	65,536	8,192
训练中期 2	150 B	262,144	4,096

2.6.2 超参数

MAI-Base-1 使用 AdamW 优化 (Loshchilov and Hutter, 2017) ($\beta_1 = 0.95, \beta_2 = 0.925, \epsilon = 10^{-8}$)，权重衰减恒为 0.1，其中注意力权重 (0.01) 和嵌入权重 (0.005) 使用较小的权重衰减，以避免对受益较少的参数过度正则化。全局梯度范数裁剪到 1.0。学习率经过约 12B tokens 的线性预热后，按余弦调度从峰值 2×10^{-4} 衰减到最小值 2×10^{-5} ——最终与峰值的比值为 0.1 $\times$ ，而不是更常见的 0.01 $\times$ ，因为我们发现学习率衰减更少可以改善 RL 之后的结果。我们在每层输出上、残差相加之前应用 0.15 的 dropout。虽然据我们所知高 dropout 并非标准做法，但我们发现它与权重衰减一起提供了互补的正则化效果，并且观察到它能提升缩放阶梯评测任务上的表现。默认情况下，模型权重从均值为零、标准差为 0.02 的正态分布初始化，残差相加之前的输出投影权重按模型中残差连接总数的平方根的倒数进行缩放 (Radford et al., 2019)。

图 8. (a) 随机初始化时，注意力层最初产生坍缩的表示——本质上是在序列维度上执行因果平均池化。注意力贡献中不同行之间存在高相关性，这会降低 token 表示的多样性，并导致后续 MoE 层高度不平衡。(b) 将

注意力层的输出初始化为零后，MoE 层中的 token 分布更均衡。专家的违反率（violation rate）是其与各专家平均 token 数的偏差比例。



所有阶段使用 134M tokens 的全局 batch size。训练中期阶段继承预训练的超参数设置，除非另有说明。训练中期阶段 1 使用余弦衰减学习率调度，峰值学习率为 2×10^{-5} ，最低为 1×10^{-5} ，无预热。训练中期阶段 2 使用恒定学习率 1×10^{-6} ，同样无预热。

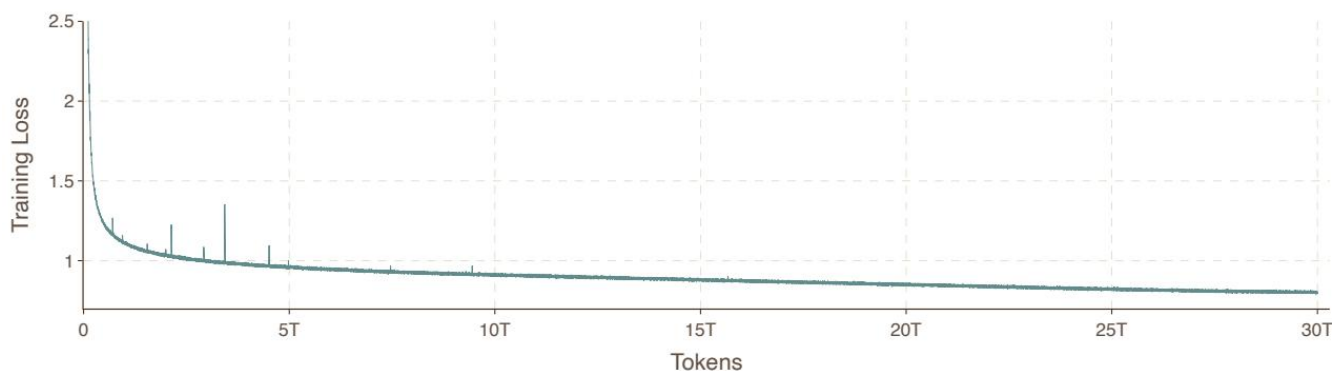
注意力初始化。初始化时，注意力 softmax 接近均匀分布，实际上等同于在因果约束下对 tokens 进行平均池化，如图 8 所示。这种行为降低了 token 表示的多样性，并可能导致后续 MoE 层出现高度不平衡的路由，且随深度增加而加剧。为解决这一问题，我们将注意力输出初始化为零，具体做法是将输出 RMSNorm 的增益 (gains) 设为零。这意味着模型初始表现为一堆应用于单个 token 的前馈层，而注意力层捕获的跨 token 交互会在训练过程中逐渐生效。我们确认，该方法显著降低初始峰值不平衡，从而带来了 EG 测量的质量提升（见 2.2.2 节）。

2.6.3 数值精度

模型训练中，权重和激活的默认数据类型为 BF16。前向传播中的 GEMM 使用 FP8 E4M3，数据梯度使用 FP8 E5M2，权重梯度的计算使用 BF16，并采用 FP32 梯度累积 (Micikevicius et al., 2017, 2022)。所有 FP8 运算使用延迟缩放，历史窗口为 1024 步的绝对值最大值。

我们在许多对数值敏感的位置使用 FP32。在激活方面，包括所有 softmax 之前的激活（注意力分数、MoE router logits 和最终输出 logits）、MoE combine，以及从输入嵌入到输出的整个残差流。在参数方面，包括嵌入权重、RMSNorm 权重和 router 权重。优化器中的主参数和动量缓冲区 (buffers) 保持 FP32，所有 AdamW 计算也是如此。数据并行 all-reduce 和微批次梯度累积缓冲区以 FP32 运行。MoE 路由和最终词表权重 GEMM 也均保持 FP32。

图 9. MAI-Base-1 (35B 激活 / 1T 总参数) 在 30T tokens 上的预训练损失曲线，未做平滑处理。损失在几次早期尖峰后迅速恢复。整个训练过程中没有跳过任何训练批次，也没有对训练配置进行任何手动干预。



我们尽可能将类型转换与相邻运算（如 RMSNorm）融合。当梯度从高精度流向低精度时，应用随机舍入（stochastic rounding）（Zamirai et al., 2020; Ozkara et al., 2025）。这是因为层内计算精度低于残差流的精度，因此反向传播中需要对梯度进行降精度转换。

2.6.4 训练损失曲线

MAI-Base-1 的训练损失曲线见图 9，未做平滑处理。训练早期出现了几次尖峰，但损失值每次都迅速恢复，我们没有干预便继续了训练。通过运行时日志，我们发现这些尖峰主要影响代码数据集，并与无丢弃（dropless）路由下的高专家不平衡相关：即 tokens 过度集中在少数几个专家中。

2.7 评估与同期模型对比

在本节中，我们将 MAI-Base-1 与现有的同期基础模型进行对比，使用我们在开发中（如第 2.3 节所讨论）所用的同一组 NLL 评估的一个子集。其中一个是基于我们内部训练栈（第 2.8 节）的源代码；其余则由厂商委托提供，因此我们有信心它们不在任何模型的训练集中。我们没有使用表 3 中描述的多语言基准，这些基准来自我们已从自身训练中排除的公开来源，因此无法与这些模型进行可靠的比较。

在本对比中，我们聚焦于预训练基础模型，并报告跨分词器不变的每字节比特数（bits-per-byte, BPB）数值。大多数模型 API 无法进行 NLL 评估，因此我们聚焦于托管在 huggingface.co 上的开放权重模型：DeepSeek v3.2 (DeepSeek-AI et al., 2025b) ³、DeepSeek v4 Pro (DeepSeek-AI, 2026) ⁴、Kimi-K2 (Kimi Team et al., 2026) ⁵ 和 Gemma4-31B (Gemma Team, 2026) ⁶。此外，我们还纳入了我们上一代的一个同样以 30T 数据训练的 23B 模型，以便量化我们的进展。

结果汇总于图 10。MAI-Base-1 在活跃参数数量相近的模型中展现出明显优势，在所有四项留出任务上都取得了更低的每字节比特数。DeepSeek-v4 取得了最佳结果，其活跃参数为 MAI-Base-1 的 1.4 $\times$ ，总参数为 1.6 $\times$ 。我们的

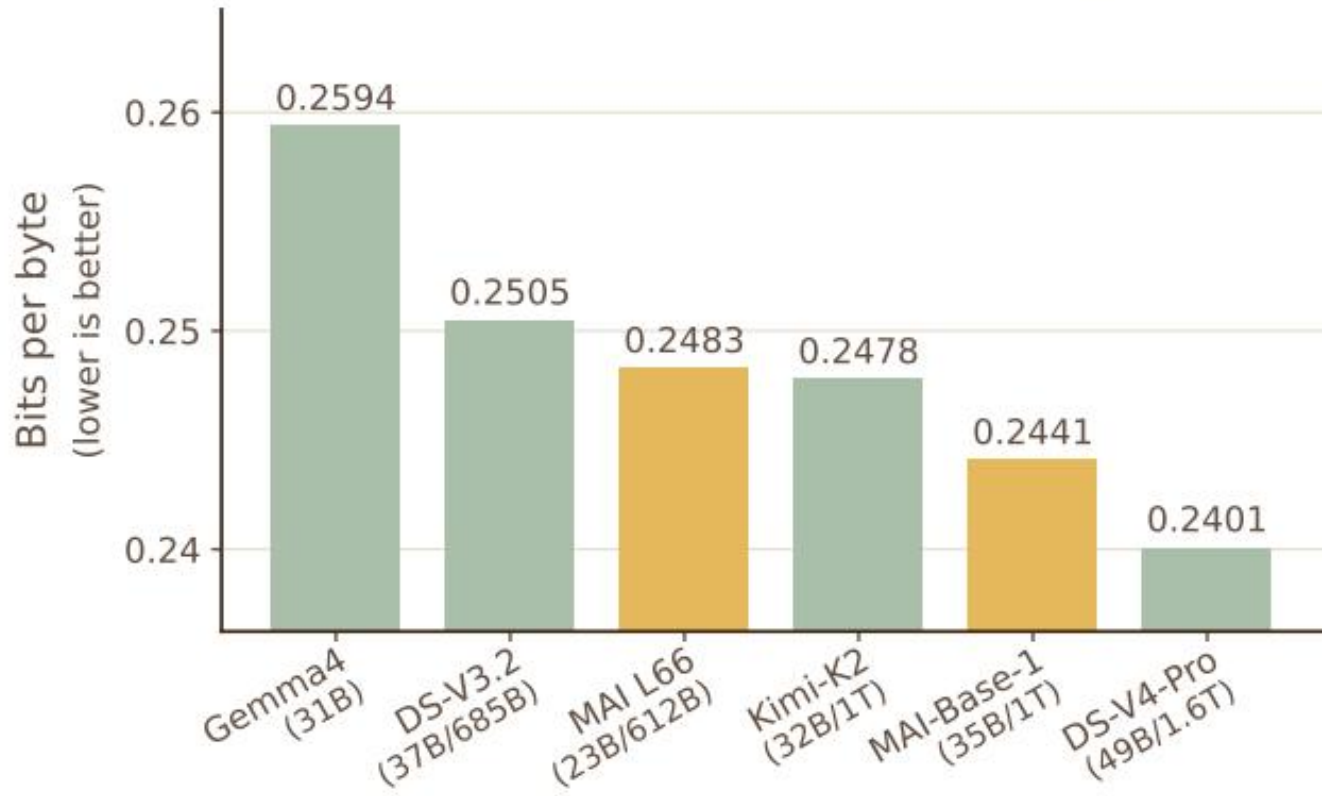
³deepseek-ai/DeepSeek-V3.2-Exp-Base

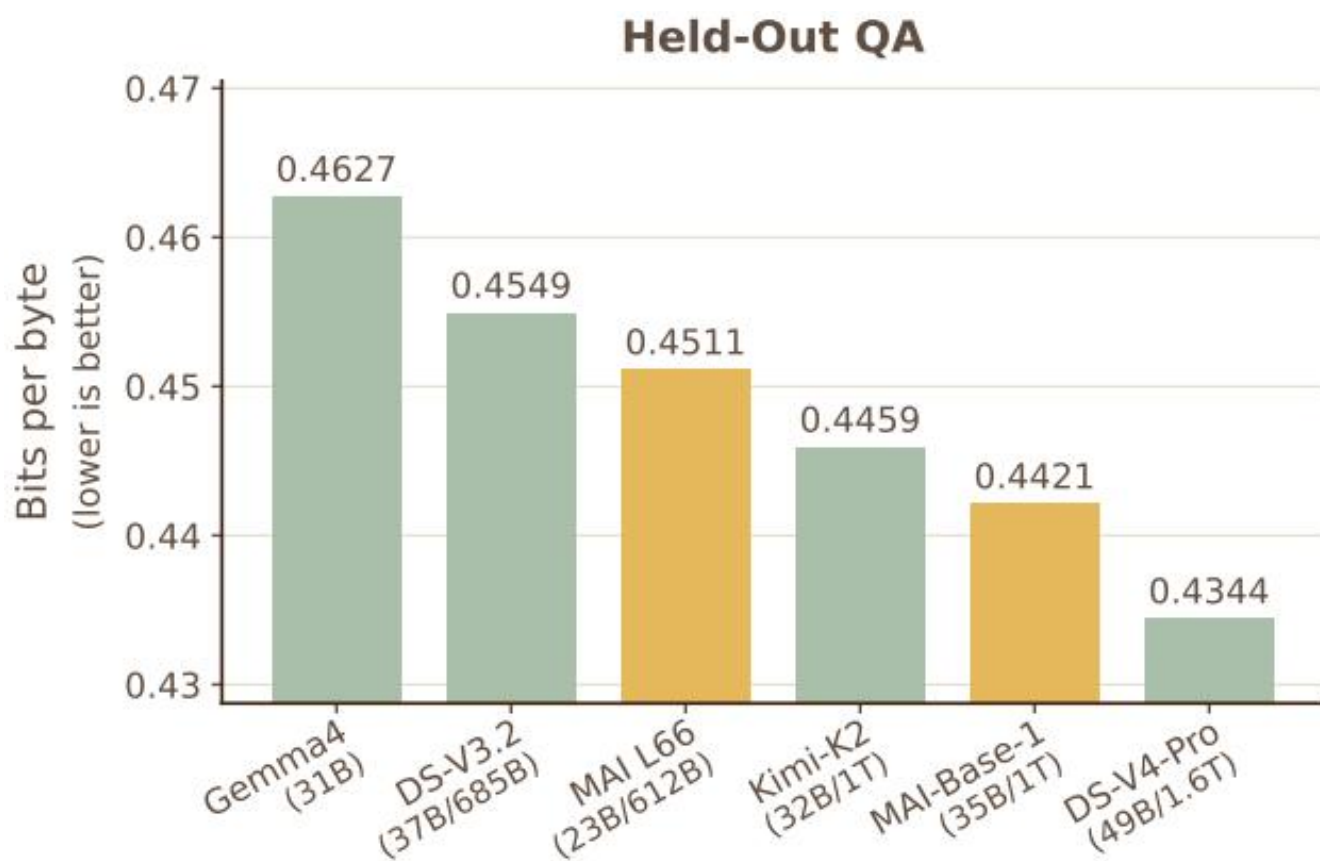
⁴deepseek-ai/DeepSeek-V4-Pro

⁵moonshotai/Kimi-K2-Base

⁶google/gemma-4-31B

Held-Out Code (MAI's YOLO repo)





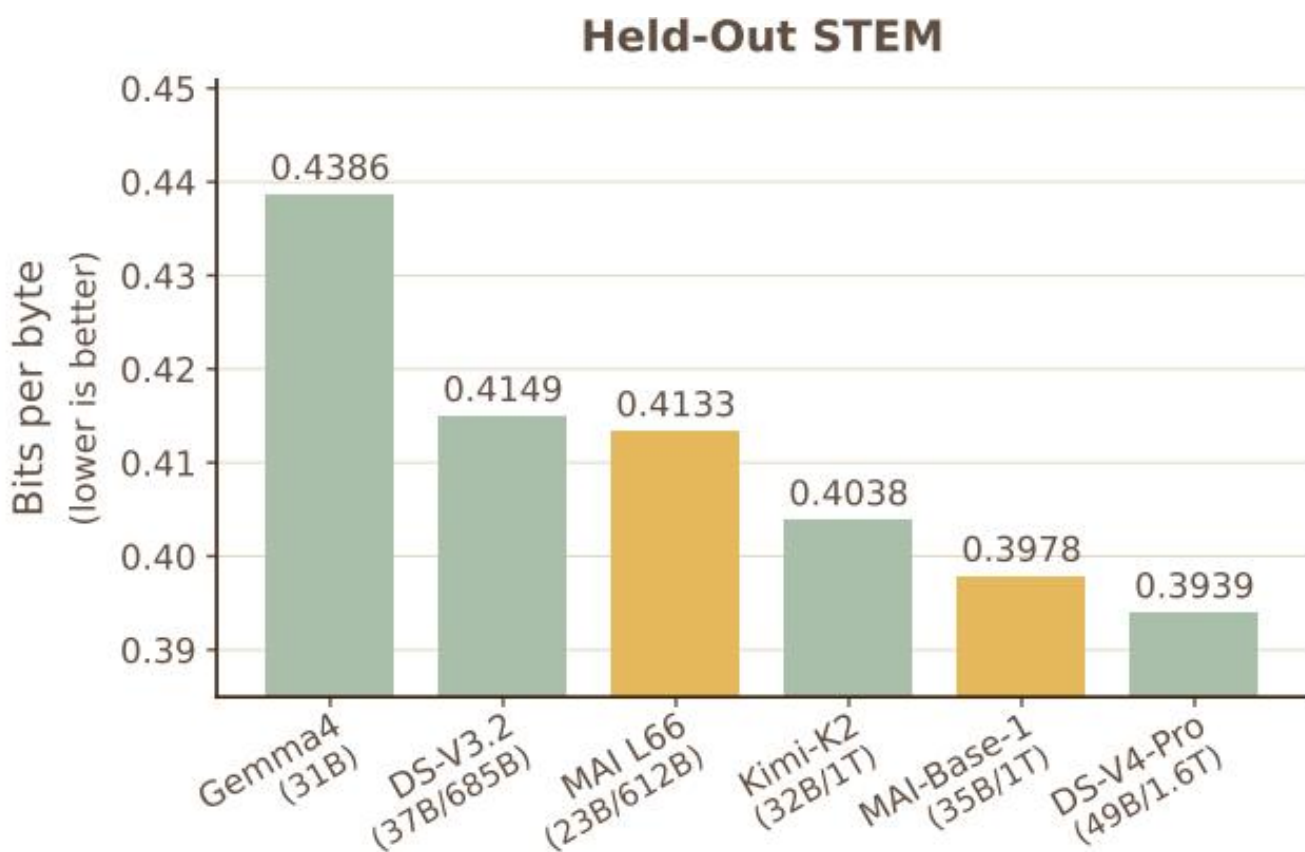
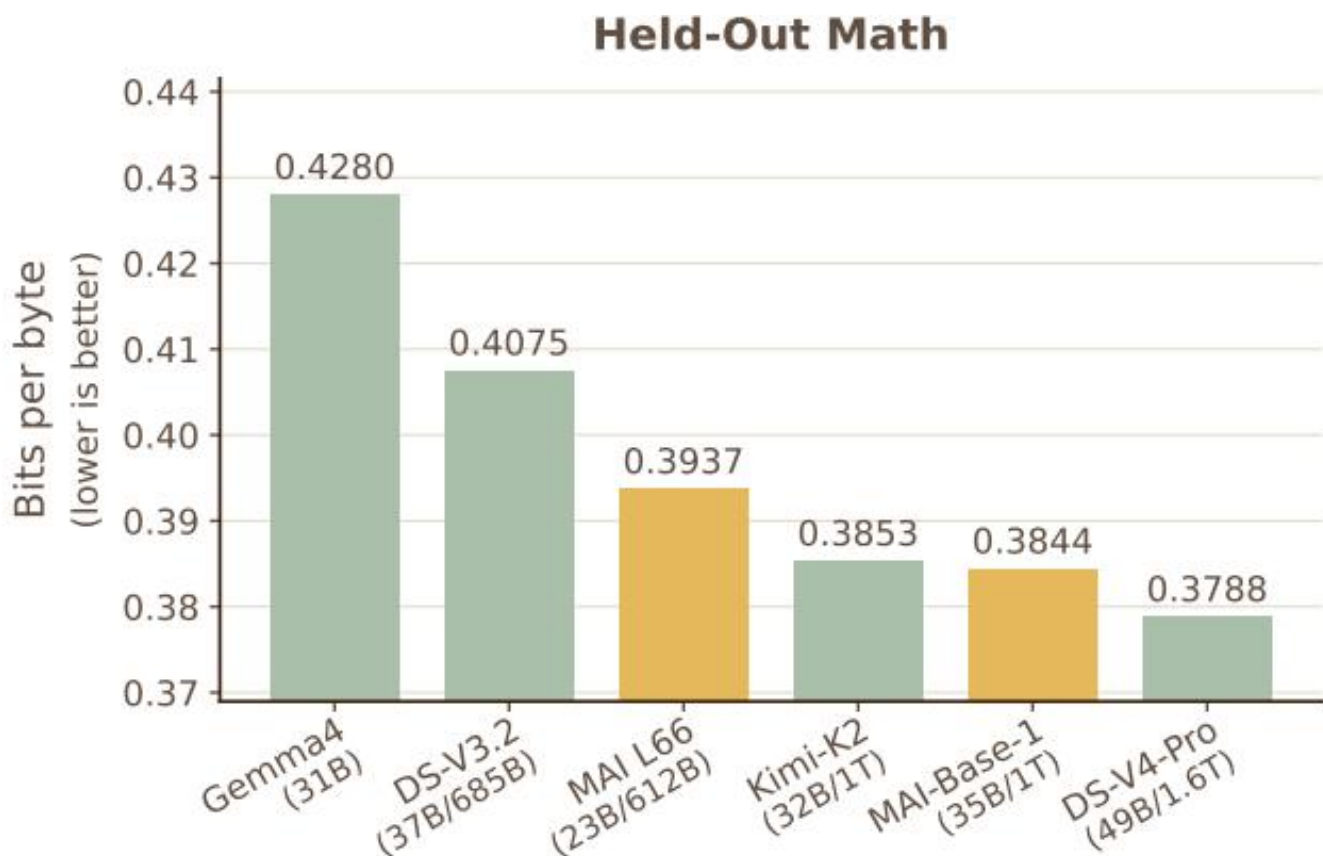


图 10. 基础预训练模型在四项留出评估任务上的每字节比特数 (BPB) 对比 (越低越好)。MAI 模型以黄色标示。我们还额外提供了一个数据点：23B (L66, 来自我们 ladder 2.2.1 的旧版本)，这是我们此前用与 MAI-Base-1 相同数量的 token (30T) 训练的版本。所使用的内部留出评估任务见第 2.7 节。



我们的 23B 模型仅使用其 62% 的活跃参数和相近的总参数量，就超越了 DeepSeek v3.2 基础模型。

2.8 YOLO：大规模分布式训练

You Only Launch Once (YOLO) 是 MAI 自研的大规模训练框架，贯穿于 MAI-Thinking-1 的整个开发周期。YOLO 构建于 PyTorch (Paszke et al., 2019) 之上，位于第 L.3 节所述的调度与运行时栈内部。

YOLO 旨在支持大规模模型训练的所有阶段，包括预训练、中期训练、监督微调，以及强化学习的训练部分。它实现了核心训练循环，包括我们的模型定义、分片、优化器、数据加载和检查点保存。我们从零构建 YOLO，以对整个训练栈进行端到端控制：从内核到并行再到调度。YOLO 借鉴了 Megatron-Core (Yan et al., 2026)、DeepSpeed (Rasley et al., 2020) 和 TorchTitan (Liang et al., 2024) 等大规模训练框架的经验，并与我们其余的计算基础设施和模型训练协同设计、深度集成。这使得我们能够在计算环境和模型架构演进过程中，同等优先地考虑训练速度、有效吞吐 (goodput)、确定性、灵活性和开发者敏捷性。以下章节描述 YOLO 的主要组件，以及它们如何按照我们的设计原则实现优化。

2.8.1 系统概览

YOLO 由一组构建模块组成，用于实现 Transformer 模型，以及训练循环和数据加载器、检查点、日志与可观测性等辅助组件。

内核 (Kernels)。为了在大规模下高效支持数值配方 (第 2.6.3 节)，YOLO 包含一套用 Triton、CUDA、

CuteDSL 和 CUTLASS 编写的自定义内核。其中包括针对前向和数据梯度计算优化、并支持 FP8 延迟缩放的 FP8 GEMM 内核；面向 MoE 的分组 GEMM，可在每个 GPU 承载多个专家时提升效率——我们的预训练运行中通常为 8 个（512 个专家，专家并行度 64）——并支持所有混合精度执行模式，包括 FP8 延迟缩放；以及融合量化内核，它对反向传播所需的原始和转置张量同时量化，同时应用缩放因子 swizzling 以提高内存效率，并在可能的情况下将紧邻 RMSNorm 层的转换融合到归一化内核中，以减少开销。此外，非分组量化内核支持集群启动控制（Cluster Launch Control, CLC），这是 Blackwell 特有的用于动态内核负载均衡的功能。

张量分片（Tensor sharding）。根据模型大小和单次训练运行的集群基础设施，YOLO 使用不同形式的并行组合。为了对模型中各张量的分片方式进行细粒度控制，YOLO 实现了一套自定义分片标注，类似于 JAX (Bradbury et al., 2018) 或 PyTorch 的 DTensor (Paszke et al., 2019)。分片标注描述了张量如何在 n 维笛卡尔网格上划分或复制，并可用于按分片类型对张量分组。每个张量所划分到的底层网格位于 PyTorch 通信器之上，该通信器为相应张量暴露集合通信和点对点通信原语。我们刻意让标注保持纯描述性，即它们不会自动在计算图中插入通信原语，以避免产生意外的同步点。

YOLO 的分布式张量使我们能够独立定义模型中每个参数的分片方式。这允许我们在模型内部灵活地进行张量并行。例如，我们对嵌入、损失和注意力权重使用张量并行，但对 MLP 和 MoE 权重不使用。另一个例子是，我们可以在属于 MoE 层或非 MoE 层的张量之间使用不同程度的数据并行（称为并行折叠（parallel folding）(Yan et al., 2026)）。

并行（Parallelism）。基于分布式张量，YOLO 实现了所有常用的并行形式：数据并行、张量与上下文/序列并行、专家并行和流水线并行 (Huang et al., 2019; Qi et al., 2023)。

- 数据并行。YOLO 使用从零构建的自定义版本 ZeRO 阶段 1-3 (Rajbhandari et al., 2020)，以适配我们的优化器和性能优化。该实现按参数的唯一分片、数据类型和可配置的 ZeRO 单元对参数分组，并将所有参数和梯度合并到连续的缓冲区（buffers）。这最大限度地减少了梯度归约和 ZeRO 1-3 所需的集合通信总数。此外，我们的 ZeRO 实现将参数拆分为多维块，使其与分布式检查点的切片保持兼容，并避免在执行集合通信前后重新排序。无论处于哪个 ZeRO 阶段，YOLO 的 ZeRO 实现始终以分片形式存储参数。这简化了分布式 AdamW 的实现，因为优化器状态也始终自动分片。ZeRO 各阶段的主要区别在于收集参数或梯度的缓冲区何时以及多久被清除。例如，对于 ZeRO-1，参数在第一次前向传播期间全量收集（all-gather），直到最后一个微批次后才清除；而对于 ZeRO-3 (FSDP)，参数在使用后立即清除。

- YOLO 中的张量并行通过列并行和行并行 GEMM 实现，其中激活和权重张量都在相同数量的 rank 上进行分片（通常称为序列并行）(Shoeybi et al., 2019; Korthikanti et al., 2023)。超出权重分片程度的任何额外激活张量分片，在注意力层中视为上下文并行，在其他情况下视为数据并行。

- 上下文并行。YOLO 使用 Ulysses 风格的上下文并行 (Jacobs et al., 2023)，其中激活张量通过并行矩阵转置（all-to-all）重新划分，将序列维度的分片转移到注意力头（隐藏）维度。对于 MAI-Base-1，上下文并行仅用于长上下文中期训练，而张量/序列并行用于所有训练阶段。

- 专家并行。最初，YOLO 的专家并行实现类似于 GShard (Lepikhin et al., 2020)，使用静态张量形状和预定义容量因子的 all-to-all 通信。在后续运行中，我们切换到了无丢弃（droppless）MoE 实现，通过可变大小的 all-to-all 通信将每个 token 路由到其选定的专家。所有预训练运行都在 NVLink 域内使用专家并行（张量并行度为 1），并在跨机架间使用数据并行。

激活检查点与卸载 (Activation checkpointing and offloading)。当使用 ZeRO 数据并行训练 MAI-Base-1 时, 激活占用了大部分可用的高带宽设备内存 (HBM)。为了减少内存占用并支持在单个 NVLink 域内进行专家并行, YOLO 同时支持激活检查点以及将激活卸载到主机内存。与 Megatron-Core 等框架类似, 激活检查点可以按不同粒度配置, 从重算一个或多个完整 Transformer 层, 到层内中间激活的细粒度重算 (例如重算激活函数的输出)。在 MAI-Base-1 的所有训练阶段中, 我们结合使用 Transformer 层检查点和细粒度重算。由于存储检查点激活所需的内存量随 Transformer 层数线性增长, YOLO 还实现了激活卸载。激活卸载与检查点互补, 且开销极小, 因为设备到主机的传输在专用 CUDA 流上使用锁页主机内存异步执行。

专家混合 (Mixture of experts)。我们的 MoE 层在研究灵活性与训练效率之间取得平衡。它同时支持无丢弃和容量受限两种模式、多种负载均衡策略 (包括全局批次负载均衡损失 (Qiu et al., 2025; Yang et al., 2025) 和无损失负载均衡 (Wang et al., 2024a))、可配置的 top-k 评分、细粒度遥测, 以及用于 RL 采样的路由器回放。对于专家并行, 我们将本地专家划分为可配置大小的组, 并在各组之间流水线化调度 → 计算 → 收集各阶段, 使得除第一次调度和最后一次收集之外的所有阶段都与专家计算重叠; 剩余暴露的通信与全局负载均衡计算以及 (可选的) 共享专家相重叠。为防止无丢弃模式中因不均衡导致的内存波动、碎片化和 OOM, 我们支持静态内存无丢弃模式, 每组运行多轮容量受限的调度 → 计算 → 收集, 每轮最多处理固定容量的 token。在反向传播中, 我们执行每专家每轮的细粒度重算, 以避免存储不均衡的激活。在性能方面, 我们使用低开销地支持所有数值配方的自定义分组 GEMM 和量化内核, 以及自定义 CuTe DSL 对称内存内核, 用于通过 NVLink 进行设备发起的、可变大小的、高吞吐 all-to-all 通信。

2.8.2 确定性与正确性

我们强制训练中的确定性, 原因包括科学可复现性、开发者敏捷性和系统健康调试, 即使以降低训练效率 (如 MFU) 为代价。对于固定的硬件拓扑、模型配置和软件版本, 我们确保预训练过程达到逐位可复现。简而言之, 使用相同硬件资源和配置的两次训练运行将产生逐位一致的模型。确定性需要在训练栈的多个层面进行仔细处理:

- 数据管线。对于给定的数据并行 worker 配置, 训练微批次的顺序是固定的。虽然这些 worker 会异步预取批次以隐藏 I/O 延迟, 我们确保待处理批次的队列在多次运行和重启之间保持一致。
- 训练检查点。我们存储所有有状态数据, 包括模型权重、优化器状态、FP8 缩放历史、数据加载器进度和随机数生成器。
- GPU 内核。浮点累加不可结合, 因此可复现的归约需要控制值的合并顺序。我们的训练栈按规定顺序执行确定性累加, 而不是依赖无序的 GPU 原子操作 (其累加顺序取决于调度器)。例如, RMSNorm 反向传播使用两阶段分块归约累加梯度: 第一个内核物化每个分块的部分和, 一个最终化内核按固定顺序归约这些部分和。另一个例子是 MoE 路由的 top-k 选择: 我们的内核使用稳定排序以避免非确定性的平局打破。
- 网络集合通信。与 GPU 内核内的浮点归约类似, all-reduce 和 reduce-scatter 等集合通信必须是确定性的。我们禁用 NVLink SHARP (Graham et al., 2020), 以确保集合操作确定, 代价是性能下降。我们还强制每次运行采用一致的 NCCL 拓扑, 使机架内归约始终使用相同的累加顺序。

我们还利用确定性来建立对 YOLO 演进过程中数值正确性的信心。我们的测试跟踪 “golden” 模型配置, 并与所有训练指标 (如损失、梯度范数和权重张量校验和) 进行精确性对比。这并不意味着我们禁止任何会导致不同训练结果的数值改动; 当我们做出影响浮点行为 (例如升级系统软件或 GPU 内核) 的改动时, 出现变化

是预期之中的。这为我们提供了软件历史中预期数值变化的明确记录。

2.8.3 容错

大规模训练必须容忍频繁的硬件故障，并将对训练效率的影响降至最低。YOLO 的容错方法结合了分布式检查点和向热备（hot standby）的快速故障转移，以最大限度地减少故障期间的训练进度损失和停机时间。

分布式检查点格式（Distributed checkpoint format）。我们基于 PyTorch 的分布式检查点库（DCP）构建，该库对跨 rank 的复制状态进行去重，并支持加载时的张量重分片。我们重写了 DCP 的元数据创建和序列化路径，以降低 CPU 内存开销、减少 Python 垃圾回收压力，并加速序列化/反序列化。总体而言，这些优化将 CPU 内存开销和检查点保存时间都降低了 $10\times$ 以上。在我们的实现中，原库仅保留了公共 API 和将张量分片映射到存储的核心逻辑——即保存计划（save plan）。我们还预先计算 DCP 保存计划并缓存在持久化存储中，将该昂贵计算从关键路径中移除。

异步检查点（Asynchronous checkpointing）。为最大限度地减少保存检查点时的训练停机，我们首先在训练进程中直接将检查点张量从设备复制到主机，然后在训练继续的同时将剩余工作卸载到一个单独的检查点进程。检查点进程通过前端网络将数据写入持久化存储，并处理跨 rank 协调以原子地提交检查点。为支撑我们的规模，我们用构建在共享键值存储之上的自定义协调层替换了标准 CPU 集合通信后端。我们还采用机会式准入控制机制，轮询进行中的检查点是否完成，并尽快启动下一个检查点，同时确保任意时刻最多只有一个检查点在进行中。这种方法与细致的主机内存管理相结合，使我们能够频繁保存检查点，同时将对训练 MFU 的影响降至最低。

存储基础设施（Storage infrastructure）。我们的训练数据中心拥有本地 Azure Blob Storage（Calder et al., 2011），提供低延迟、高带宽的访问。这使我们能够直接依赖检查点实现容错，而无需中间节点本地缓存或跨 rank 状态复制。为避免检查点加载时大规模出现存储热点（许多 rank 需要相同的复制状态），我们仅从存储加载一次复制状态，并通过 NCCL 广播，消除了可能压垮单个存储服务器的单 blob 扇入热点。

快速任务恢复（Rapid job recovery）。我们通过使用 Ray（Moritz et al., 2018）actor 配合热备实现任务内重启，从而获得快速恢复，省去了 pod 重建的开销。在 GB200 系统上，我们在 actor 重启期间显式清除操作系统缓存，以防止因 GPU 持有上次运行的残留内存导致内存不足错误而引发的崩溃循环。最后，我们通过将重算步骤与历史损失指标对比，在重启后验证确定性。

2.8.4 与模型架构协同优化性能

MAI-Base-1 的最终架构是跨越模型五个主要世代（v1–v5）的持续协同设计过程的成果。每一世代追求两个互补的目标。首先，我们通过改变模型设计、路由策略和训练配方，力求提升架构的效率增益与建模能力。其次，我们通过识别性能瓶颈并引入新的基础设施、内核、通信和内存管理优化，持续提升训练效率。由于架构改动改变了模型的计算特性，它们常常引入新的系统瓶颈，需要在训练栈中做出相应的改进。

我们描述模型的架构与基础设施演进，以突出我们构建大规模模型的方法。每次演进中跟踪的两个指标是：

- 模型 FLOP 利用率（Model FLOP Utilization, MFU）（Chowdhery et al., 2023），定义为 $\text{FLOP}/(t_{\text{step}} \cdot \text{FLOP}_{\text{spec}})$ 用于量化实际训练效率。这里 FLOP 表示所有前导阶计算内核的浮点运算总数，包括 GEMM 和注意力，但不包括 RMSNorm 和激活函数等内存受限操作。 t_{step} 是端到端训练步时间，包括数据加载、前向和反向传播、通信以及优化器更新。尽管训练混合使用 FP8、BF16 和 FP32 运算，我

们使用 NVIDIA 报告的 FP16/BF16 张量核心吞吐量来归一化 MFU ($\text{FLOPsspec} = 2.5 \times 10^{15}$ FLOPS 每 GB200 GPU，用于稠密运算)。激活检查点引入的运算不计入 FLOP 计数。

- 效率增益 (Efficiency Gain): 定义见第 2.2.2 节，用于刻画在达到给定质量时模型效率的理论提升。

表 7 汇总了各模型世代间的关键架构改动。图 11 同时报告了新架构配合上一世代优化栈运行时的初始 MFU，以及应用针对各版本的性能改进后达到的最终 MFU。在 MAI-Base-1 的开发过程中，引入了超过二十项基础设施和内核优化，使每一世代在模型规模和架构复杂性大幅增加的情况下仍能维持 20% 以上的 MFU。世代 v0 和 v1 是在 H100 系统上训练的早期原型，因此未包含在图 11 中。

从版本 2 开始，所有主要训练运行都在 GB200 NVL72 集群上进行。MAI-Base-1 的版本 2 (23B 活跃参数) 是首个在 GB200 NVL72 集群上训练的模型版本，共使用 4,096 块 GPU。为提高系统鲁棒性，我们每机架使用 64 块 GPU (NVL64)，预留冗余容量以容忍节点故障和不健康设备。版本 2 采用了比早期版本更深更窄的设计，同时保持 23B 活跃参数预算，因此隐藏维度相对较小。为最大化 GEMM 效率，我们选择了专家并行 (EP) 度为 64、张量并行 (TP) 度为 1，确保所有专家 all-to-all (A2A) 通信都保持在 NVL64 域内。该配置实现了 18% 的初始 MFU。我们使用跨机架 InfiniBand (IB) 进行数据并行 (DP) 通信 (如参数 all-gather 和梯度 reduce-scatter)。我们在后续版本中保持了相同的设置。

表 7. MAI-Base-1 开发过程中的架构演进。这里报告的是用于大规模基础设施测试的目标架构，而非 ladder 模型。

模型版本	v2	v3	v4	v5 (MAI-Base-1)
激活/总参数量	23B/600B	23B/600B	23B/611B	35B/1T
层数	54	54	66	78
隐藏维度	4352	4352	5632	6656
FFN	34816	34816	11264	13312
Down-proj 维度			2560	3072
专家 FFN	17408	17408	9216	10240
专家容量因子	2	∞	∞	∞
Top-k/专家数	4/192	4/192	8/512	8/512

性能剖析揭示了三个主要瓶颈：DP 通信、确定性注意力内核和专家路由开销。启用 GPU Direct RDMA 支持改善了通信与计算的重叠，并将端到端训练时间缩短了约 1.1 $\times$ 。由于 FlashAttention-2 (FA2) 的确定性模式在 GB200 上效率低下，且 FA4 不可用，我们开发了一个自定义块稀疏注意力后端，将步时间额外缩短了 1.06 $\times$ 。鉴于单微批次训练配置，我们采用 ZeRO-2 优化器分片，与 ZeRO-1 相比，在不引入额外通信的情况下降低了梯度内存消耗。这使我们能够禁用额外的激活重算，并进一步提升 1.03 $\times$ 的性能。最后，我们将一个内存受限的 PyTorch 专家编码内核（仅利用 10% 的可用 HBM 带宽）替换为自定义 Triton 实现，达到了约 80% 的 HBM 利用率，并将端到端性能额外提升了 1.03 $\times$ 。综合起来，这些优化将 MFU 从 18% 提升到 22%。

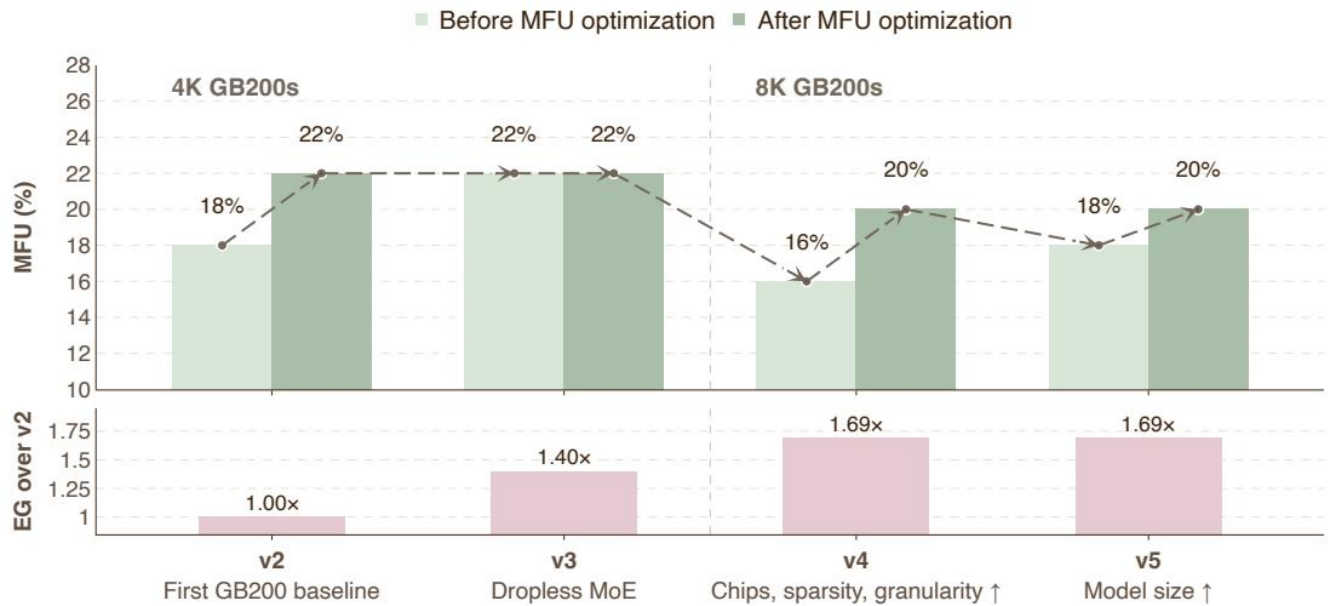
版本 3 保持了相同的整体模型架构，但从容量受限路由切换为无丢弃 (dropless) MoE 路由。从效率增益的角度看，这使我们能够采用更好的 MoE 负载均衡策略。这对 MFU 也有积极影响，因为消除了专家容量填充，从而同时减少了通信量和专家 GEMM 计算量。然而，无丢弃路由引入了新的同步开销，因为专家 token 数和

张量形状必须在执行期间动态通信。为缓解这些成本，我们将 token 数通信与其他运行时操作重叠，并将同步点移至专用流上。这些改动使我们在保持与版本 2 相当 MFU 的同时，实现了无丢弃路由的效率收益。

版本 4 引入了三项重要的架构改动：专家数量从 192 增加到 512，路由从 top-4 扩展到 top-8 专家选择，并使用了 LatentMoEs。增加专家数量有助于提升模型的学习容量，而 latentMoE 有助于在专家数量增加时管理 A2A 通信成本。同时，训练任务从 4,096 块 GPU 扩展到 8,192 块 GPU。LatentMoEs 减小了专家 GEMM 的规模，使得 CPU 启动开销和内核效率变得更加重要。结果是，初始 MFU 从 22% 下降到约 16%。使用 FA4 确定性内核（相对于块稀疏注意力后端提升 1.14 $\times$ ），再加上若干旨在降低 CPU 开销、提升运行时批处理效率的改进，将 MFU 从 16% 提升到 20%。

版本 5 将活跃参数量从 23B 增加到 35B，总参数量从 600B 增加到 1T。更大的模型显著增加了参数和激活内存需求。因此，初期部署依赖 ZeRO-3 优化器分片，但性能剖析显示，额外的参数 all-gather 操作导致反向传播受通信限制。为解决这一问题，我们通过激活卸载降低 GPU 内存压力，使训练系统得以回归 ZeRO-2 分片。消除 ZeRO-3 参数 all-gather 操作恢复了通信与计算之间的重叠，并提升了整体训练效率，使最终版本 5 配置能够维持约 20% 的 MFU。

图 11. 在 GB200 上训练的各类预训练配置中 MFU 和 EG 的演进。从 v2 起，每次模型改动都提升了 EG，但在部署优化前用上一代配置运行时，MFU 起初会下降。总体上，我们为每次预训练运行添加了 20+ 项优化，以超越 20% 的 MFU。注意，这里我们只列出具有显著基础设施影响的模型改动——它们并非模型各版本之间促进 EG 提升的全部改动。



3 强化学习攀登

预训练和中训练使 MAI-Base-1 具备了广泛的预测能力与知识，但它们并未指定模型应如何行事、如何解决长时程任务，或如何分配推理时计算。我们的强化学习（RL）攀登通过针对任务特定反馈优化模型来解决这一问题，使模型能够在作答前生成思维链（CoT），使用外部工具，与环境交互，并遵循偏好与安全信号。

由于 MAI-Thinking-1 是我们首款自研推理模型，我们的 RL 攀登从一个从未接触过推理轨迹的检查点开始。因此，模型必须从零开始发展推理能力，这使得长期训练稳定性成为核心挑战。我们通过三种机制实现这一点：(i) 对组相对策略优化（GRPO, Shao et al., 2024）进行两项简单但关键的调整，(ii) 在崩溃或基础策略更新后通过自蒸馏恢复 RL 攀登（第 3.1 节），以及 (iii) 消除训练与推理之间数值失配的基础设施改进（第 3.6 节）。

为支持并行开发，我们训练三个领域特定的专家模型：一个面向 STEM 与竞技编程（第 3.2 节），一个面向智能体编码与工具使用（第 3.3 节），一个面向有用性与安全性（第 3.4 节）。所有专家的 RL 攀登都遵循相同的配方，但在提示词分布以及模型因何获得奖励方面有所不同。如图 12 所示，这些专家模型随后通过监督微调（SFT）整合为单一模型。最后一轮轻量级 RL 攀登将这一整合模型转化为 MAI-Thinking-1——一个在所有领域都表现强劲模型（第 3.5 节）。

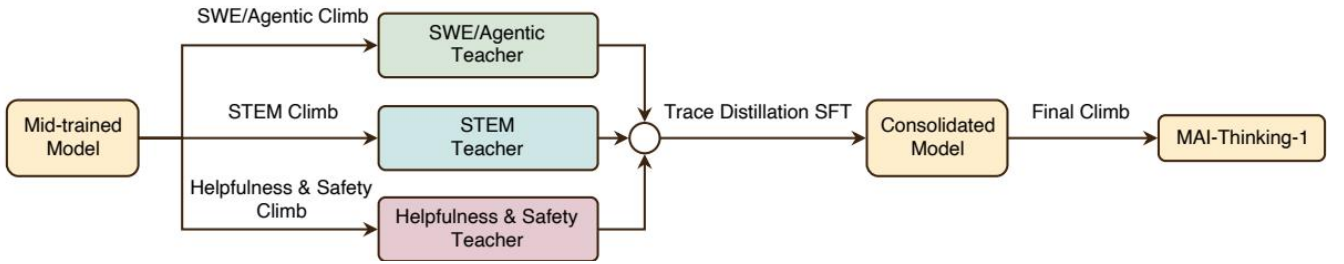


图 12. 我们各 RL 攀登的概览。我们从 MAI-Base-1 出发，使用强化学习训练三个专家模型。这些专家随后通过简单 SFT 被蒸馏为单个整合模型，该模型再经历一轮最终 RL 阶段，产生 MAI-Thinking-1。

3.1 强化学习配方

3.1.1 RL 目标

我们的强化学习攀登从一个策略 π_θ （即我们的中训练模型）出发。对于提示词 q ，rollout 策略采样一组共 G 个响应 $y_{1:G}$ ，每个响应 y_i 会收到一个标量奖励：

$$R_i = R(q, y_i). \quad (4)$$

奖励函数 R 与领域相关；通常它要么基于代码执行，要么基于提示式 AI 裁判或训练好的奖励模型的反馈。我们从 GRPO (Shao et al., 2024) 出发推导训练目标，并采用 token 级策略梯度 (Yu et al., 2025)：

$$J(\theta) = \mathbb{E}_{q \sim P(Q), y_{1:G} \sim \pi_{\text{old}}} \left[\frac{1}{\sum_{i=1}^G |y_i|} \sum_{i=1}^G \sum_{t=1}^{|y_i|} \min \left(r_{i,t}(\theta) A_i, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) A_i \right) \right], \quad (5)$$

其中 $P(Q)$ 是所有提示词上的分布， π_{old} 表示用于生成 rollouts 的策略。实践中，归一化在全局训练批次（跨所有数据并行 rank）的所有 token 上计算，使每个 token 无论响应长度如何都贡献相同。对于响应 y_i 和 token 位置 t ，重要性采样比率为

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(y_{i,t} \mid q, y_{i,<t})}{\pi_{\text{old}}(y_{i,t} \mid q, y_{i,<t})}. \quad (6)$$

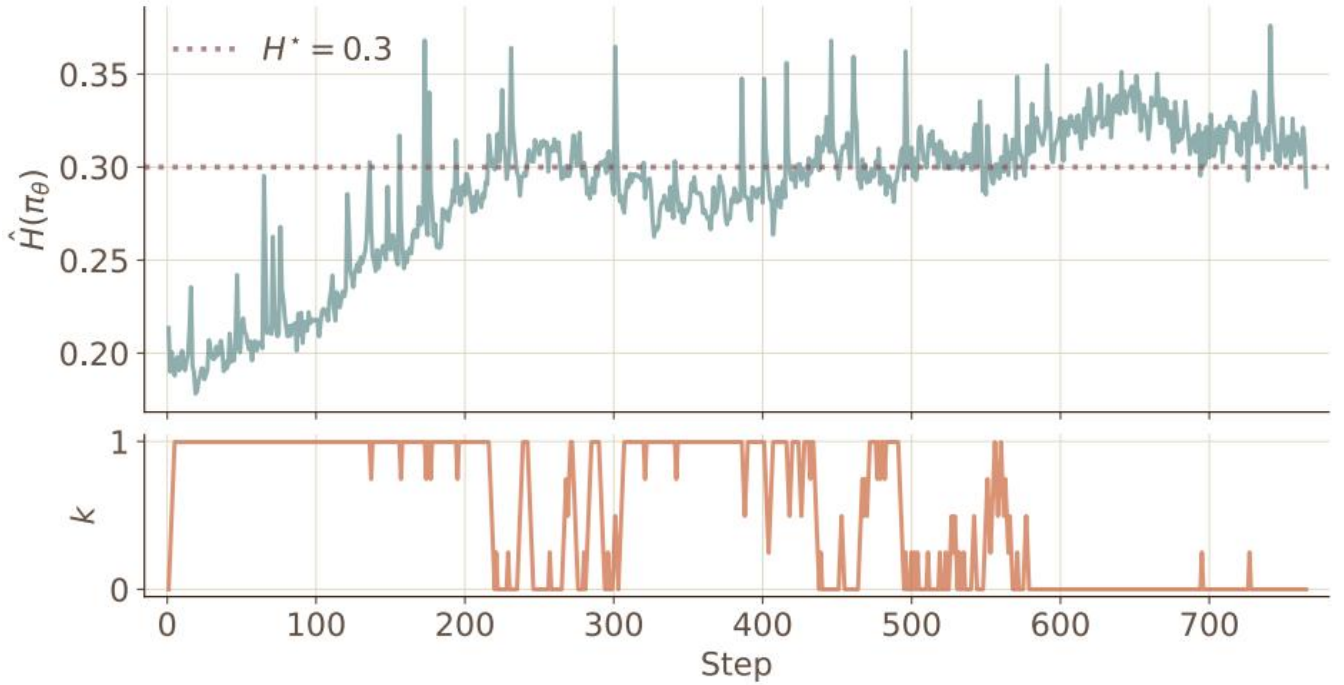
响应级优势 $A_i = (R_i - \text{mean}(R_{1:G})) / \text{std}(R_{1:G})$ 在响应 y_i 的所有 token 之间共享。

我们对该目标应用两项修改：自适应熵控制，动态调整上裁剪界以维持目标策略熵；以及外层比率裁剪，对我们的目标中未裁剪的分支设置上限，以防止梯度范数爆炸。

自适应熵控制。与 Yu et al. (2025) 和 Mistral-AI et al. (2025) 类似，我们使用分离的下、上裁剪界。我们用单个基础超参数 ϵ 对它们进行参数化，它控制基础信赖域宽度，再加上与熵相关的对裁剪上界的松弛量 k ：

$$r_{i,t}^{\text{tr}}(\theta) = \text{clip}(r_{i,t}(\theta), 1 - \epsilon, (1 - \epsilon)^{-1} + k).$$

图 13. 训练期间的自适应熵控制。上图：目标熵 $H^* = 0.3$ 的一次 RL 攀登中，800 步内观测到的熵 $\hat{H}(\pi_{\theta})$ 。下图： k 的对应取值，其中 $k_{\max} = 1.0$ ：当观测熵超过目标时减小，否则增大。调整 k 为调节策略熵提供了有效机制。



我们发现上界需要仔细调参：过大时会导致熵爆炸，过小时会导致熵坍缩（另见 Cui et al., 2025; Xi et al., 2025）。

为解决这一问题，我们基于当前策略的熵，使用一个简单的积分控制器动态调整 k 。在每个训练步，我们通过重要性加权估计器估计目标策略的每 token 熵：

$$\hat{H}(\pi_\theta) = \frac{1}{|\mathcal{T}|} \sum_{(i,t) \in \mathcal{T}} -\log \pi_\theta(y_{i,t} \mid q, y_{i,<t}) \cdot r_{i,t}(\theta), \quad (7)$$

其中 $\mathcal{T}$ 表示当前训练批次中所有（响应索引，token 索引）对的集合。给定目标熵 H^* ，控制器在每步之后以步长 $\delta \in \mathbb{R}^+$ 更新 k ：

$$k \leftarrow \text{clip} \left(k + \delta \cdot \text{sign} \left(H^* - \hat{H}(\pi_\theta) \right), 0, k_{\max} \right). \quad (8)$$

直观上，当熵过低时，增大 k 会放宽上裁剪界，使策略能够更激进地提高备选 token 的概率。当熵足够高时，减小 k 以收紧信赖域。如图 13 所示，该机制起到自动熵正则化器的作用，而无需在损失中添加显式的熵奖励项——我们发现后者不如自适应熵机制。

我们将 $k = 0$ 作为初值，使裁剪界 $1 - \epsilon$ 与 $(1 - \epsilon)^{-1}$ 互为乘法逆元，从而让初始裁剪区间在对数比率空间上对称。随后我们按照上述熵控制器在线更新 k 。

外层比率裁剪。GRPO 目标刻意保留两种情况不裁剪：(a) 优势为负且新策略赋予比旧策略更高的概率（即 $A_i < 0$ 且 $r_{i,t} > 1$ ），以及 (b) 优势为正且新策略赋予更低的概率（ $A_i > 0$ 且 $r_{i,t} < 1$ ）。其最初动机（Schulman et al., 2017）是：当策略朝正确方向自我修正时不加限制，仅限制利用优势估计的移动。

然而在实践中，我们发现这些未裁剪分支有时会导致灾难性的梯度范数尖峰。我们通过在所有分支上施加硬外层裁剪来解决这一问题：

$$r_{i,t}^{\text{out}}(\theta) = \text{clip}(r_{i,t}(\theta), r_{\min}, r_{\max}), \quad (9)$$

其中 $r_{\max}$ 设为较大的值，而 $r_{\min}$ 可以不加约束；这与 Ye et al. (2020) 提出的双重裁剪 PPO 目标在精神上类似。

这一两层策略在保留正常范围内比率的标准信赖域行为的同时，丢弃新旧概率存在极端差异的情况。实验表明，这减少了梯度范数尖峰，使攀登更加稳定。

3.1.2 奖励设计

虽然我们的任务特定组件在不同领域的 RL 攀登中有所不同，但我们在整个过程中使用相同的奖励分解：

$$R(q, y_i) = R_{\text{task}}(q, y_i) + w_{\text{lang}} \cdot R_{\text{lang}}(y_i) - w_{\text{len}} \cdot R_{\text{len}}(y_i). \quad (10)$$

这里， R_{task} 表示任务特定奖励， R_{lang} 是语言一致性奖励， R_{len} 是长度惩罚。系数 w_{lang} 和 w_{len} 是标量超参数。

语言一致性奖励。随着 RL 期间上下文长度的增加，我们观察到模型开始在其 CoT 中产生外语 token。这些混合语言 CoT 与训练和推理策略之间对数概率散度的尖峰相关，最终会破坏训练稳定性。为解决这一问题，我

们使用与 Guo et al. (2025) 类似的语言一致性奖励 $R_{\text{lang}}(y_i)$ 。由于英语在我们的训练分布中占主导地位，我们针对英语定义语言一致性：

$$R_{\text{lang}}(y_i) = \max\left(1 - \alpha \cdot n_{\text{non-english}}(y_i), 0\right), \quad (11)$$

其中 $n_{\text{non-english}}(y_i)$ 是响应 y_i 的 CoT 中非英语单词的数量， α 是逐词惩罚项。实践中我们发现，top-p 采样在阻止单个低概率外语 token 方面同样有效。

长度惩罚。沿用 Xiang et al. (2025)，我们将长度惩罚定义为：

$$R_{\text{len}}(y_i) = \rho_q \cdot \frac{|y_i|}{\ell_{\text{max}}} \quad (12)$$

其中 ρ_q 是问题 q 的通过率， ℓ_{max} 是最大 rollout 长度。该惩罚同时取决于响应长度和问题难度。通过率低的难题受到更弱的惩罚，允许模型探索更长的推理轨迹。简单题受到更强的惩罚，通过去除冗余循环和回避性行为，鼓励简洁且成本高效的推理。

3.1.3 采样策略

我们采用多种采样策略来提高 RL 攀登的效率和稳定性。它们作用于两个层面：选择训练哪些问题，以及控制如何为这些问题生成 rollouts。

用户与助手之间的对话。用户提出问题，助手负责解答。助手先在头脑中思考推理过程，再向用户给出答案。推理过程和答案分别放在 `<think>` `</think>` 和 `<answer>` `</answer>` 标签中，即 `<think>` 此处为推理过程 `</think>` `<answer>` 此处为答案 `</answer>`。用户：{QUERY} 助手：`<think>`

图 14. 用于第一轮自蒸馏之前的推理提示词模板，其中 {QUERY} 是实际用户查询的占位符（来自 Guo et al. (2025)）

问题采样。对于训练集中的每个问题 q ，我们从当前推理模型生成一组 rollouts。为降低推理成本，我们采用提前退出策略：先采样 $G_{\text{early}} < G$ 个响应并计算其经验通过率，即奖励为正的响应所占比例。如果早期通过率落在可接受区间 $[\rho_{\text{min}}^{\text{early}}, \rho_{\text{max}}^{\text{early}}]$ 内，则采样全部 G 个响应；否则丢弃该问题。

在生成全部 G 个响应之后，我们对完整组应用第二次通过率过滤 $[\rho_{\text{min}}, \rho_{\text{max}}]$ 。只有完整通过率落在该范围内的问题才会用于训练。这第二次过滤的主要动机是剔除方差极低的分组：如果几乎全部响应都正确或都不正确，该分组能提供的相对学习信号很有限。

Rollout 采样。我们使用 top-p 采样 (Holtzman et al., 2019)，用 π_{old} 采样 rollouts $y_{1:G}$ 。我们发现，继续对采样核之外 token 对应的 logits 进行反向传播可能导致灾难性的 off-policy 失配，使训练在几步之内就发散。为防止这一点，我们遵循 DeepSeek-AI et al. (2025b) 的做法，在训练期间排除这些 token：复用 rollout 采样时的 top-p 截断掩码，在 softmax 计算之前将所有被排除 token 的 logits 设为 $-\infty$ 。我们发现 top-p 掩码显著降低了 RL 训练期间的策略发散，代价是掩码存储与重放带来的额外开销。

为了提高攀登早期阶段的训练效率，我们最初将最大 rollout 长度限制在 8k token。随着训练推进，我们以 2 的幂逐级提高该上限，直到达到 128k token 的最终输出长度。我们发现这一长度扩展课程能显著降低低性能

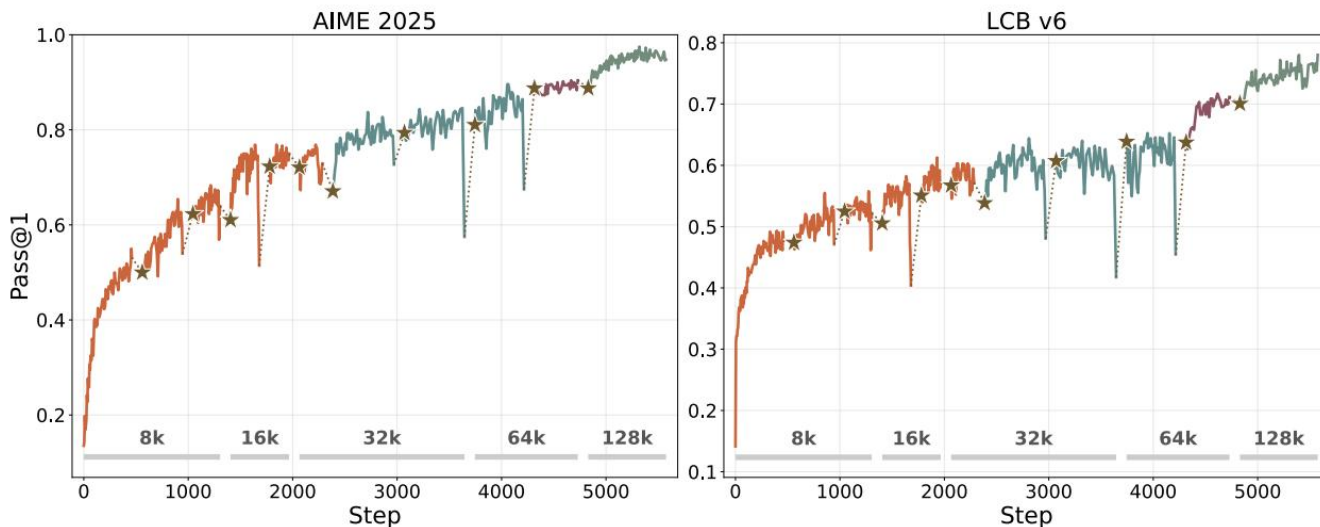
阶段的推理成本——该阶段很少需要长推理轨迹——同时仍能让模型随能力提升逐步适应更长的上下文。

3.1.4 自蒸馏

从中训练检查点实现强性能需要大量 RL 步的训练。我们利用自蒸馏 (Huang et al., 2022; Zelikman et al., 2022; Singh et al., 2023; Zhao et al., 2026) 让此类长时运行攀登更切实可行。自蒸馏的具体做法是：收集 RL 期间生成的 rollouts，并在一个中训练检查点上对这些 rollouts 做 SFT。得到的模型作为继续 RL 攀登的起点，同时保留先前 RL 阶段发现的能力。我们将自蒸馏用于多个目的：

- 对于最初的攀登，我们使用领域或任务特定的提示词来激发目标行为；例如，图 14 展示了我们 STEM 攀登使用的初始原始文本提示词。自蒸馏使我们只需更新 SFT 数据的格式，就能从提示词驱动迁移到使用原生对话格式。
- 另一个主要用途是从偶发的运行失败中恢复。在我们 RL 技术栈的早期——即第 3.6.4 节所述稳定性改进引入之前——推理与训练之间微小的数值差异有时会在训练过程中累积，导致攀登发散。在这种情况下，自蒸馏提供了一种简单有效的方式，将一次 RL 攀登的进度承接给下一次。从崩溃前的检查点恢复这一备选方案往往不可行，因为某些不稳定性在崩溃实际发生前许多步就已嵌入模型参数之中。

图 15. STEM 攀登期间在 AIME 2025 (左) 和 LiveCodeBench v6 硬子集 (右) 上的表现。自蒸馏通过 ★ 标记标出；不同的预训练/中训练模型版本以不同颜色显示。训练全程的最大输出长度在底部标出。自蒸馏是一种在崩溃后重置数值（可通过性能的突然下降观察到）以及在我们运行过程中更换基础策略的有效方式。随着我们在攀登中持续做出基础设施与算法层面的改进，因运行崩溃而触发的自蒸馏越来越少见。



- 当新的预训练和中训练检查点可用时，自蒸馏使我们能够将先前攀登的进展延续到下一代模型。
- 在自蒸馏过程中，可以应用过滤器剔除任何表现出不当奖励劫持形式的样本。

图 15 展示了自蒸馏在 STEM 攀登中的两种用途：既用于从数值不稳定中恢复，也用于更新基础策略。

关键发现与最佳实践。我们进行了大量消融实验，以弄清在实践中如何最好地进行自蒸馏。主要发现如下：

- $O(1M)$ 量级的推理轨迹足以匹配教师性能，同时保留 SFT 的稳定性优势。使用大得多的数据集收益递减，且有过度约束策略的风险——它会收窄策略的输出分布，使 RL 恢复后几乎没有探索空间 (Zheng et al., 2024; Gao et al., 2025)。

- 使用包含导致最终答案错误的轨迹进行训练，与仅使用成功轨迹训练效果相当 (Gandhi et al., 2025)。由于我们的 RL 运行通常产生的成功轨迹远超 $O(1M)$ ，我们最终将训练限定在成功轨迹上。

- 使用攀登后期阶段的轨迹很重要。纳入非常早期检查点的轨迹会导致明显的性能下降，且需要许多后续 RL 步才能恢复。仅从最终 RL 检查点生成轨迹，在恢复 RL 后性能较弱。一个可能的解释是：在一系列强检查点上收集的轨迹比从单一最终策略生成的轨迹具有更高的多样性，从而在 RL 中带来更好的探索。复用 RL 期间已生成的轨迹还能避免重采样所需的额外计算和时间。

- 在固定 token 预算下，提高提示词多样性的价值高于增加每个提示词的轨迹数量。我们还发现简单随机采样优于若干有偏选择策略，包括最短轨迹采样以及与 Yang et al. (2025) 探索的类似启发式过滤方法。

- 在自蒸馏期间，模型可能遗忘中训练学到的长上下文行为，尤其是当推理轨迹来自以较短最大长度训练的早期 RL 运行时。为缓解这一点，我们在长度扩展之前，将中训练数据与推理轨迹混合进行自蒸馏。

3.1.5 超参数

下面我们报告三个领域特定模型的主要 RL 攀登以及自蒸馏运行中通常使用的超参数。

RL 攀登。我们使用 AdamW 训练，其中 $\beta_1 = \beta_2 = 0.95$, $\epsilon = 10^{-15}$ ，且无权重衰减。我们使用恒定学习率 $\eta = 10^{-6}$ ，无 warmup 或衰减。打包后的全局批大小为 7040，未打包序列总数上限为 12000。由于我们的 RL 技术栈完全异步，随着生成长度增加，rollouts 的延迟会上升，而训练步耗时不会成比例增加（因为存在大量短生成）。这意味着较长的 rollouts——通常是难题——的 off-policy 程度会上升。为应对这一点，我们选择在较高长度下将学习率降至 9×10^{-7} ，以降低 off-policy 程度并提高稳定性。最大生成长度为 128k，但我们先以 8k、16k、32k 和 64k 训练。

对于带自适应熵控制的 GRPO，我们设置 $\epsilon = 0.6$, $k_{\max} = 2.5$ 、步长 $\delta := 0.25$ 以及目标熵 $H^* = 0.3$ 。对于外层裁剪，我们设置 $r_{\max} = 50$ ，并让 $r_{\min} = 0$ 不受约束。

对于语言一致性奖励，我们设置 $w_{\text{lang}} = 0.5$ ，逐词惩罚 $\alpha = 0.005$ 。对于长度惩罚，我们在 64k 长度扩展阶段之前固定 $w_{\text{len}} = 0.25$ 。由于该惩罚由 $\ell_{\max}$ 归一化，有效惩罚会随最大长度增加而自然减弱，使模型能够在更大的 token 预算下探索更长的推理轨迹。我们在 128k 扩展阶段移除长度惩罚 ($w_{\text{len}} = 0$)。

对于问题采样，我们设置共 $G = 128$ 次 rollouts，其中 $G_{\text{early}} = 16$ ， $[\rho_{\min}^{\text{early}}, \rho_{\max}^{\text{early}}] = [0.05, 0.8]$ 。通过率过滤使用 $[\rho_{\min}, \rho_{\max}] = [0.1, 0.8]$ 。对于 top-p 采样，我们使用 $p = 0.97$ 。更大的核取值能改善探索，但会增加传输采样掩码的基础设施开销；我们发现 $p = 0.97$ 在探索与训练效率之间取得了良好平衡。

我们在两次推理模型更新之间执行 5 个梯度步，并丢弃任何其生成策略超过 8 次推理更新（即落后 40 个梯度步）的 rollout。这在略微放宽 on-policy 新鲜度的同时显著提高了吞吐。我们使用 dropless MoE 训练，并将全局 MoE 负载均衡系数设为 $1 \cdot 10^{-5}$ 。

自蒸馏。对于自蒸馏 SFT，我们使用打包序列，全局批大小为 2,048，序列长度为 128k token。优化使用 AdamW，权重衰减 0.001，学习率采用余弦调度。最大学习率设为 $1.7 \cdot 10^{-5}$ ，最小学习率设为 $5.2 \cdot 10^{-6}$ ，学习率 warmup

比例为 2%。

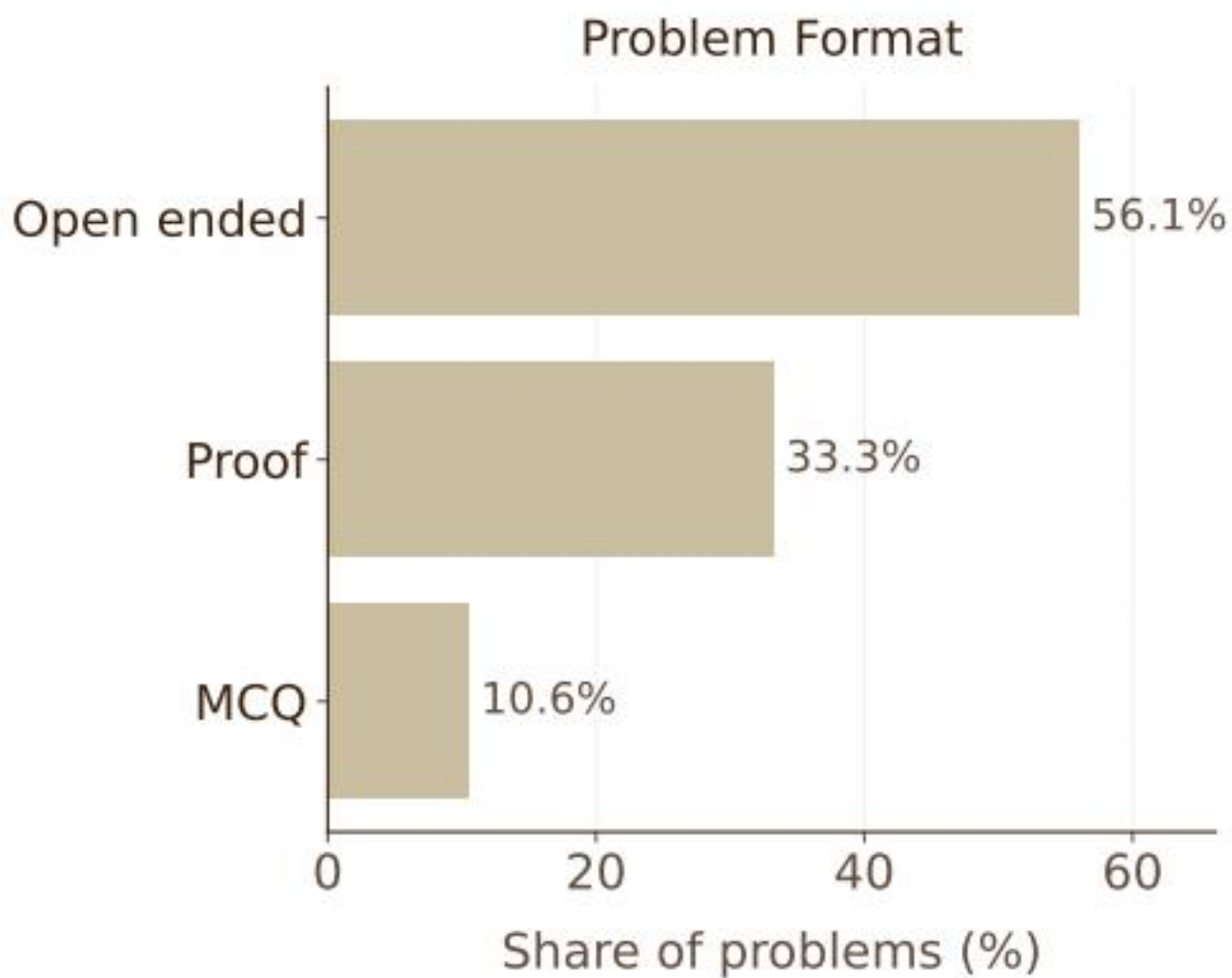
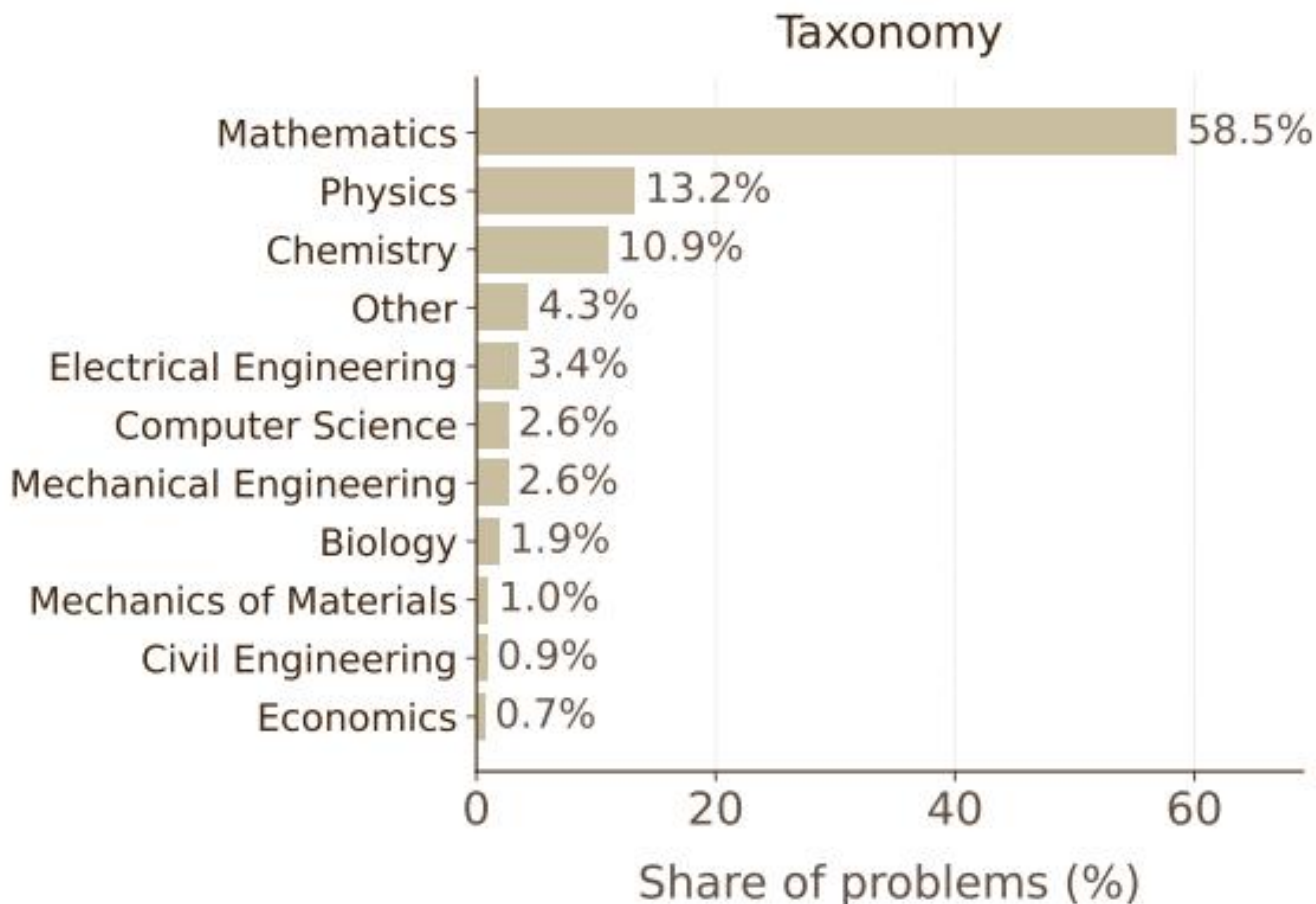


图 16. 我们的 STEM Mix 数据集按原始题目格式（左）和学科分类体系（右）的分布，其中 Other 汇总了各自占比低于数据集 0.5% 的主题。多项选择题和证明题在摄入期间被转换为开放式形式（转换不可行的予以剔除）；我们保留一小部分多项选择题，使模型对该格式保持熟悉。



在自蒸馏期间，我们仔细调优两个超参数：dropout (Srivastava et al., 2014) 和 MoE 负载均衡损失系数。我们使用相对较高的 0.15 的 dropout 率，这会提高熵并有助于防止模型坍缩，从而提升后续 RL 阶段的性能。由于领域特定 RL 中的数据分布远比预训练和中训练使用的混合数据窄，RL 期间的专家选择可能变得高度不均衡。然而，在 RL 中使用较大的负载均衡系数也会损害稳定的性能提升。为解决这一问题，我们在自蒸馏期间使用相对较大的负载均衡系数 $1 \cdot 10^{-2}$ ，而在 RL 期间使用小得多的系数 $1 \cdot 10^{-5}$ 。由于自蒸馏的上下文来自 RL 运行本身，因此分布相似，自蒸馏期间诱导的专家均衡效应在 RL 期间得以保留。

3.2 STEM 攀登

STEM 攀登是我们三项领域专属强化学习 (RL) 训练中时间最长的一次，旨在强化模型在单轮问题求解场景下的核心推理能力。它覆盖广泛的 STEM 领域，包括数学、物理、化学和竞赛编程。我们还观察到攀登期间 rollout 中反映出的模型行为演变，详见附录 D.1。由于数据质量是这次攀登成功的关键，我们重点介绍训练数据，以及我们为构建、验证和过滤这些数据而搭建的管线。

整个 STEM 攀登基于可验证数据对进行：为产生任务专属奖励 $R_{\text{task}}(q, y_i)$ ，我们从 y_i 中提取模型的最终答案，并将其与标准答案进行比较——既可使用 SymPy (Meurer 等, 2017) 之类的形式化验证器，也可使用 AI 评判器；对于竞赛编程，我们则将模型生成的代码片段在配套的题目专属测试用例集上运行。因此，每条 RL 数据实例要么是查询与标准答案组成的对 (q, a) ，要么是查询与 n 个测试用例组成的对 $(q, \{t_1, \dots, t_n\})$ 。在设

计获取这类数据对的管线时，我们主要考虑三个标准：高质量、难度适当和主题多样。

借助我们的 STEM 数据管线，我们处理了数百万份文档，产出用于 STEM 攀登的 STEM Mix 数据集，包含超过 5M 个样本；其中最具挑战性的部分包含超过 550k 对 (q, a) 数据。该数据集的一些特性见图 16。

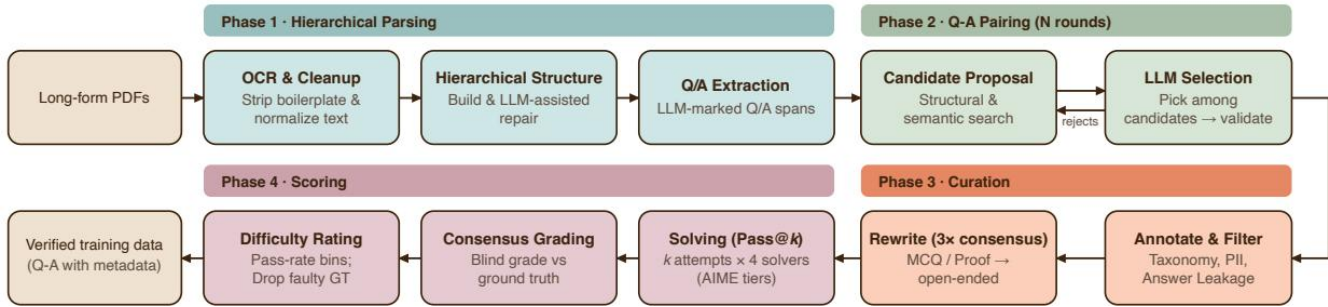


图 17. 从教科书和学术 PDF 中提取 (q, a) 数据对以构建 STEM Mix 的管线。

3.2.1 数据管线

我们的 STEM 数据管线将异构的原始来源——教科书、学术 PDF、论坛讨论、竞赛档案，以及从不同供应商获取的题目——转化为 (q, a) 数据对数据集。该管线基于可组合的、分阶段的架构构建：每个处理步骤都实现为独立的异步阶段。管线的来源专属实例按需选择和组合这些阶段中的子集。

下面我们介绍若干管线阶段，按四个阶段分组，以图 17 所示从长篇教科书和学术 PDF 中提取 (q, a) 数据对的示例为参照。对噪声或幻觉敏感的阶段会多次运行并采用共识投票。

分层解析。该阶段包含将原始文档转换为初始 (q, a) 数据对的各个步骤。根据文档类型，OCR 通过视觉-语言模型或 OCR 服务完成。文档被分块，无 STEM 内容的页面被丢弃，并移除样板文本。单独的阶段利用 OCR 输出中的结构线索构建分层表示，并修复断裂的交叉引用、错位的编号以及跨页拆分造成的伪影。最后，在问答 (QA) 抽取阶段，LLM 在清理后的文档结构中标记问题和答案片段，生成候选 (q, a) 数据对。

QA 配对。对于问题和答案出现在不同位置的数据源（例如章节末尾的习题、附录中的答案），多轮配对阶段将每个问题与其对应的答案进行匹配。问题首先利用结构信号和语义相似度与候选答案匹配；随后 LLM 从检索到的候选中选择最佳匹配的答案并验证配对。

整理。整理阶段的各步骤对 (q, a) 数据对进行标注、过滤和改写，以确保质量和格式一致性。其中若干步骤是基于 LLM 的分类：

- **可验证性：**条目被分为可验证与不可验证两类；不可验证的条目（例如没有可核查答案的开放式论坛讨论）会被丢弃。
- **题目类型：**我们区分不同的题目类型，包括开放式问题、选择题和证明题。
- **分类体系：**分层主题分类器为题目分配细粒度的 STEM 主题，涵盖数学、物理、化学、生物学、计算机科学和工程学。
- **PII 检测：**专门的分类器标记个人身份信息 (PII)；被标记的条目会被丢弃。

- 答案泄漏：检测并丢弃答案 a 明显包含在问题 q 中的 (q, a) 数据对。

另外两个整理阶段会对题目进行转换，使其对 RL 攀登更有用。转换（Conversion）将选择题和证明题改写为开放式形式；选择题往往可以通过猜测解决，会提供不可靠的奖励信号，而证明题不依赖更强的 AI 评判器则难以直接验证。该步骤运行三次，随后进行共识判定；未达成一致的条目被视为无法可靠转换而被丢弃。清理（Cleanup）移除多余的、非数学的文本以及与题目无关的外部引用。

评分。这一最终阶段量化题目难度，并过滤掉标准答案可能有误的条目。在求解阶段，四个模型档位各对每个问题求解 k 次，各档位的能力以其 AIME 2025 成绩作为代理指标。通过率被用于将条目归入不同的难度区间。

盲评阶段用于防范错误的标准答案。对于最强档位求解通过率较低的条目，我们将该模型的共识答案与标准答案以随机顺序呈现给评判器。如果评判器更倾向于共识答案，我们就判定该条目标标准答案可疑并将其丢弃；如果它更倾向于标准答案，则将其保留为真正困难的问题。

3.2.2 竞赛编程数据

竞赛编程数据使用一条独立的专用管线，与我们的主 STEM 管线有所不同；这是因为竞赛编程题目需要为每个提示提供一组测试用例 $t_1, \dots, t_n$ 。在 PDF 文档等非结构化来源中，很难找到带有全面测试用例的提示。因此我们依赖定向来源和供应商获取的数据，这省去了 STEM 数据管线中的许多提取和过滤步骤。

对于每个题目，我们获取参考解法并验证其能通过所有相关测试用例。我们共从多个来源收集了 160k 个题目，覆盖多种主题，包括分治、动态规划、图与树算法以及搜索算法。除测试用例之外，每个题目还包含运行时间和内存限制。我们的最终数据集支持 17 种编程语言，包括 Python、C++、C#、Java、JavaScript、Rust 和 TypeScript。

3.2.3 去重与去污染

我们的 STEM Mix 数据集和竞赛编程数据集都做了自身去重，并与我们在第 H 节报告的基准测试以及我们内部跟踪的奥林匹克竞赛和研究生级别 STEM 评测进行去重。我们使用三阶段管线进行去重：

1. 精确去重。作为第一步，我们使用 SHA-256 题目哈希识别完全重复的题目。
2. 词法模糊去重。我们使用字符级 n -gram shingling 结合 MinHash 局部敏感哈希来识别近似重复的题目。Jaccard 相似度高于给定阈值的题目对会被标记为重复。
3. 向量去重。最后，使用轻量级嵌入模型对题目进行嵌入。余弦相似度高于给定阈值的题目对会被标记为重复。

我们在每个阶段仔细调优超参数，以便在确保严格排除任何基准泄漏的同时，尽可能多地保留数据。

3.3 智能体攀登

智能体攀登训练 MAI-Thinking-1 解决需要与外部环境交互而非一次性文本应答的任务。在此设定下，模型必须拆解用户请求、选择工具或代码操作、观察结果，并在需要时跨多个步骤和轮次调整计划。训练信号将可验

证奖励（如软件环境中测试通过、或数据库达到目标状态）与针对难以精确刻画的行为方面（如任务理解、有用性和轨迹质量）的 AI 反馈奖励相结合。

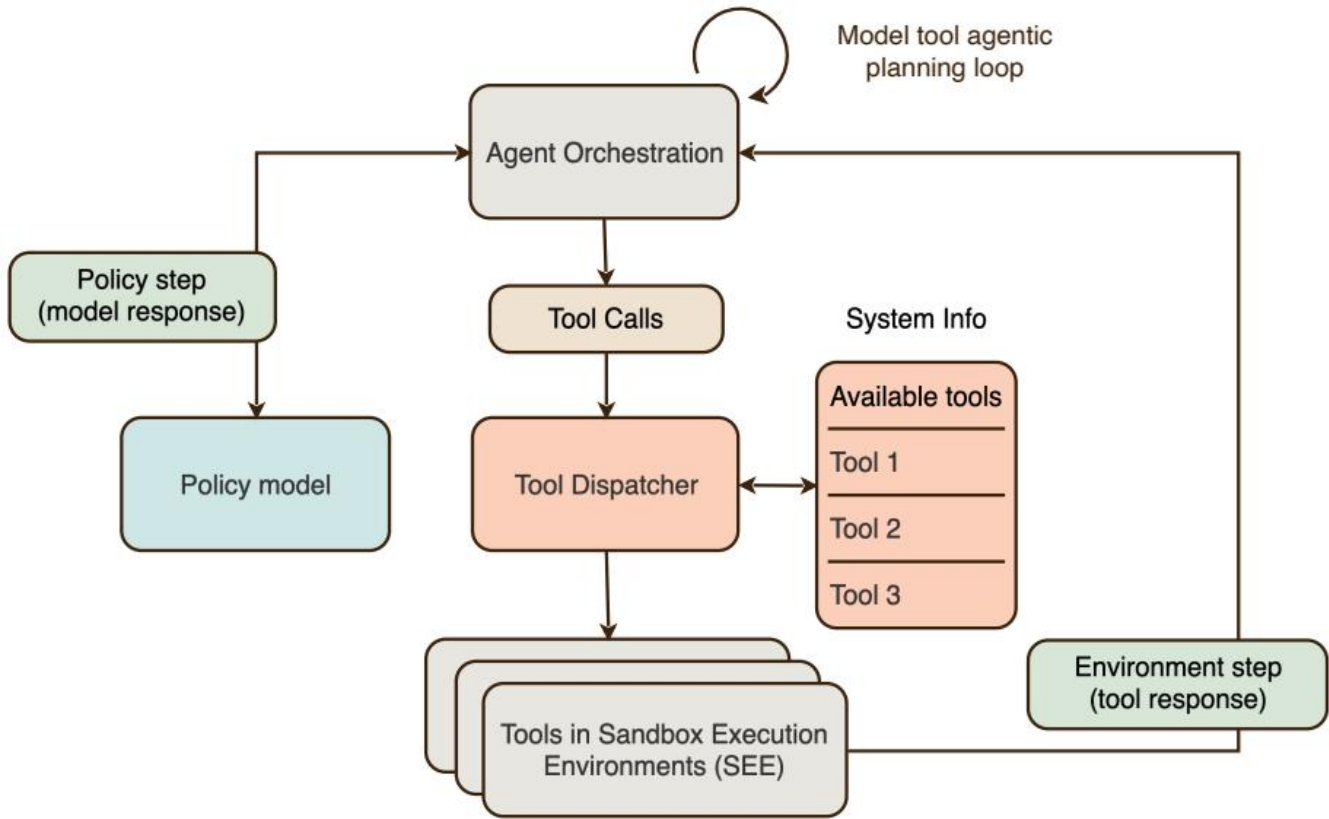


图 18. RL 训练中的智能体循环与多步编排。

我们聚焦两个智能体领域：(i) 软件工程（SWE）（第 3.3.1 节），涉及基于真实代码库构建的可执行软件工程环境；(ii) 通用工具使用（第 3.3.2 节），涉及在多步任务中调用结构化工具。

在实践中，我们联合攀登由智能体任务和推理型 STEM 任务构成的混合任务集，其中包含我们的竞赛编程混合集。我们发现，纳入 STEM 任务有助于稳定 RL 攀登过程，并对多步软件工程和工具调用性能产生正向迁移。反之，智能体任务对 STEM 相关的单次通过（single-pass）性能既无正向也无负向迁移。与 STEM 攀登类似，我们在整个攀登过程中观察到模型行为不断演化，示例见附录 D.2。智能体场景与 STEM 场景的不同之处在于其使用多步 rollout 和容器化环境，具体如后续小节所述。

多步 RL 框架。我们的智能体多步 RL 与第 3.1 节中单步推理 RL 配方采用相同的核心目标，但将 rollout 从单个采样响应扩展为由策略步骤与环境步骤（观测）构成的轨迹。每个 RL 环境由一个任务规范、一个用于执行工具的沙盒执行环境（Sandbox Execution Environment, SEE）会话，以及一组可验证或经评判的奖励组成，用于在跟踪环境状态的同时评估任务完成情况。在每个策略步骤中，模型可以发出工具调用或产生最终答案。工具调用在 SEE 会话内执行，其输出会在下一个策略步骤之前追加到上下文中。随后对完整轨迹的正确性进行评分，并可选择性地给出 AI 反馈，之后在所有策略步骤产生的所有 token 上统一进行信用分配。

编排框架是一个 ReAct 式循环（Yao et al., 2023）：它解析模型的推理和动作，将工具调用分派给 SEE，把

返回的观测追加到上下文，并将控制权交还给策略以进行下一步动作。之前所有步骤的 token 都保留在上下文中，并作为后续步骤的严格前缀。同一循环同时支持软件工程和通用工具使用两类工作负载。在 SWE 中，动作包括读写文件、运行 shell 命令，以及在针对问题定制的容器镜像内检查仓库状态。在通用工具使用中，动作调用由可变任务状态（如带种子数据的数据库）支撑的结构化工具，工具输出作为观测返回，供模型下一步动作使用。当模型不再发出工具调用，或 rollout 超过步骤、上下文长度或时间预算时，循环终止；随后 rollout 及其对应的 SEE 被送往评分器。评分器综合运用格式检查、基于规则的检查、可执行测试、可验证的状态比较和 AI 裁判。图 18 概括了我们的智能体 RL 框架。RL 环境来自两大领域：软件工程（第 3.3.1 节）和通用工具使用（第 3.3.2 节）。

沙盒执行环境。我们的沙盒执行环境（SEE）按需大规模提供隔离的容器环境。SEE 在 RL 训练期间用于为模型训练环境提供高度并行、低延迟的容器。它为每个智能体任务启动一个新容器，并在任务完成后销毁该容器。这保证了可复现性，支持安全探索，并防止任务之间的状态泄漏。

容器默认进行网络隔离，确保训练回合（episode）具有确定性，不受速率限制或瞬时故障等外部副作用的影响。当环境确实需要网络访问（例如安装软件包）时，流量通过缓存代理和域名白名单进行管控，以平衡环境保真度、训练可复现性与安全性的需求。

3.3.1 软件工程

训练前沿模型执行 SWE 任务，需要模型在真实的开发环境中与真实代码库交互。大量高质量的编码环境对于训练强大的编码模型至关重要。为实现这一目标，我们从真实代码库的贡献中收集自然产生的数据。为整理我们的 RL 环境库，我们依赖以下基础组件：大规模代码仓库数据集（如公开的 GitHub 代码）、从真实世界开源开发者的 pull request 构建可执行软件工程问题的流水线，以及用于执行的沙盒容器服务 SEE。

SWE RL 环境、任务与工具。每个 SWE RL 问题被打包为自包含的容器镜像，其中包含检出在特定 commit、预装全部依赖的仓库、问题描述以及用于评分的单元测试。由于镜像捕获了仓库的全部依赖，环境具有确定性，且 rollout 开始时无需任何配置即可立即使用。在 rollout 过程中，模型通过工具调用与容器交互：读取和编辑文件、运行 shell 命令以及浏览仓库。当模型发出完成信号或达到轮次上限时，评分器在同一个容器内执行测试，并将结果与预期输出进行比较，产生可验证的奖励信号。

模型可以使用两种常用工具：Bash——一种执行 bash 命令并返回标准输出和错误流的工具；以及 String replace editor——一种让模型通过精确字符串替换来编辑文件的工具，无需为此使用不符合人体工学的 bash 工具（Anthropic, 2025）。完整的工具描述和参数 schema 见附录 E。

自研可扩展 SWE 环境构建。已有工作（Pan et al., 2024; Chen et al., 2026b; Fu et al., 2026）探索了将来自互联网的原始代码仓库转化为 RL 环境。我们以公开 GitHub 仓库的 issue 和 pull request（PR）作为原始数据来源，并受 SWE-ReBench（Badertdinov et al., 2026a,b）启发，开发了一条可扩展的流水线，用于摄入真实世界可验证的 SWE 问题。为构建高质量的 RL 环境，我们确保问题描述为智能体正确实现解决方案提供足够的信息，同时不过度限定任务。我们还确保评分器提供足够的覆盖范围，同时避免过于具体的评估标准。流水线由以下阶段组成，更多基础设施细节见附录 G：

- 公开 GitHub PR 与 issue 收集。流水线从 1.02 亿个公开 GitHub PR 开始。首先，我们过滤 PR，以获得适合构建环境的子集。要入选，PR 必须已合并到其仓库的主分支，且修改的文件少于 15 个。每个 PR 还必须同时包含代码变更和测试变更，以便评分器将测试变更用作隐藏测试，评估模型修复问题的能力。测试变

更和代码变更根据补丁中的文件内容进行拆分。我们进一步基于 issue 关联关系过滤 PR，仅保留与 GitHub、Jira、Bugzilla、YouTrack、Phabricator、Launchpad 或 Linear 的 issue 相关联的 PR。经过此阶段，我们获得约 487 万个带有关联 issue 的 PR。

- 自动智能体环境构建。我们使用一个读取仓库状态并创建 Docker 文件以构建可执行容器镜像的 LLM 智能体，将选中的 PR 转换为可训练的环境。我们通过执行测试套件来验证每个镜像，并丢弃存在依赖或环境错误的构建。

- 参考评分信号提取。对每个候选问题，我们以 base commit 为基准分两个阶段执行仓库测试套件，得出参考测试结果：首先仅应用测试 diff（修复前，pre-fix），然后同时应用测试和代码 diff（修复后，post-fix）。由失败转通过（fail-to-pass, F2P）的测试构成问题解决信号，要求模型补丁使这些测试反转通过，才能获得解决问题的分数。在两个阶段均保持通过的测试，即持续通过（pass-to-pass, P2P），构成回归信号，确保生成的补丁不会破坏现有功能。没有存活 F2P 测试的问题被丢弃。

- 环境与评分器验证。在获得可执行环境、测试和代码变更、测试执行脚本以及预期的修复前后结果后，我们使用与 RL 训练期间相同的 SEE 训练基础设施对环境进行重新验证。这是对环境构建沙盒与训练沙盒之间潜在差异的最终合理性检查。我们验证空代码补丁无法通过评分器，而标准解法补丁能够通过，且各自经过多次试验。尽管上游补丁已知是正确的，但重新验证仍可能因集群差异（如网络出口策略、CPU/内存限制、执行超时）而失败。我们还过滤在重复执行中表现出非确定性测试行为的环境，以降低奖励噪声。

- 质量过滤与 SWE 环境改写。仅靠正确执行和严密的测试还不够，因为许多环境仍包含低质量或定义不充分的问题描述。例如，我们经常看到简短含糊的 bug 描述、不完整的规格说明，或类似“修复问题”这种只在隐藏测试中隐式编码了预期行为的描述。如果问题描述与评分器最终运行的测试所期望的内容之间存在较大差距，模型就只能猜测测试需要什么。

为提升环境质量，我们在同一环境中部署一个智能体，检查问题描述、仓库和测试，并从规格清晰度、测试质量、泄漏风险和可行性等方面对任务进行评分。对低质量环境，智能体会改写问题描述，使其更好地与测试要求对齐，同时避免不必要的信息泄漏或过度限定。

在最初的 487 万个候选问题中，208 万个（42.8%）成功通过自动智能体环境构建，745,452 个（15.3%）通过参考评分信号提取，265,617 个（5.5%）在 94,044 个独立仓库中通过了环境与评分器验证。

合成数据。虽然我们的环境构建流水线生成的许多环境未通过验证，但其中很大一部分仍然是有效的可执行环境。在大多数情况下，失败源于低质量的问题描述或不足的测试覆盖，而非无效的执行环境。为复用这些可执行环境，我们借鉴 BugPilot (Sonwane et al., 2025)、SWE-Smith (Yang et al., 2026) 和 SWE-Mirror (Wang et al., 2025a) 的方法，从与成功构建但未通过质量检查的环境相关联的 PR 中生成新的合成问题及相应测试。

防止 RL 训练中的奖励操纵。尽管 SWE 环境使用可验证、可执行的测试用例进行评分，它们仍然容易受到奖励操纵（reward hacking）的影响。我们使用 LLM 监控器审查 rollout，并对标记的示例进行人工审查。这识别出三种主要的奖励操纵类型：

- 互联网搜索。由于我们的 RL 环境来自 GitHub 上的公开仓库，PR（以及其中的标准解法）往往可以通过搜索获取。我们对沙盒执行环境实施网络访问控制来防止这种情况：当问题自包含时禁用互联网访问，否则仅允许问题所需的最低限度访问。

- 本地 git 历史搜索。Git 是软件仓库不可或缺的组成部分。有时智能体会搜索所有 git commit 和日志，试图找到隐藏在本地 git 数据库中的解答 commit。⁷删除所有 git commit 也不可行，因为 git 是一项有价值的技能，并能提供解决问题所需的正当信息。我们通过清除问题 base commit 之后的所有 commit、引用和分支来彻底净化环境，创建仓库“穿越”回原始状态的版本。

- 篡改测试。遵循 SWE-Bench (Jimenez et al., 2024) 使用的评估协议，我们在评分前重置所有被智能体修改的测试文件，防止智能体篡改测试用例使其通过评分。我们还在推理期间对智能体隐藏测试变更，仅在评分时应用，以进一步减少作弊机会。然而，仍然存在作弊途径，例如通过 monkey-patching 测试框架或修改等价性行为，这些难以通过简单地重置测试文件或引入隐藏测试来预防。我们使用 LLM 监控智能体行为，并持续加强测试文件检测、测试文件重置及其他反篡改启发式策略。

3.3.2 通用工具使用

与 SWE RL 环境相比，通用工具使用 RL 环境在可用工具和应用领域两方面都表现出显著更大的多样性。它们涵盖广泛的通用工具调用任务（例如库存管理、调度平台、报告生成、客户支持及其他面向企业的场景），强调与外部系统进行可靠、有状态的交互。

工具使用 RL 环境。每个工具使用 RL 问题被实例化为一个交互式、有状态的环境，由模拟真实 API 或 MCP (Model Context Protocol, 2026) 行为的模拟后端支撑。具体而言，每个问题由一条查询、一组带 schema⁸ 的可用工具、一个初始环境状态和一个评分器组成。与通常只涉及少量工具的 SWE 设置不同，工具使用环境以远为庞大的工具集模拟丰富的真实服务交互，单个环境中的工具数量往往超过 50 个。这一设定训练模型高效地选择合适的工具，并改善跨多样化工具使用场景的泛化能力。我们同时使用人工整理的环境和由我们的工具调用任务生成框架（下文介绍）生成的合成环境进行训练。

合成环境。为扩大训练数据的规模和多样性，我们合成生成自包含的封闭世界环境，包括带种子数据的数据库、工具定义和可验证的任务。我们的流水线只需对目标环境进行简单英文描述，即可端到端生成完整的工具使用环境。该方法延续了自动生成工具使用环境的已有工作 (Sullivan et al., 2025; Zhang et al., 2026; Wang et al., 2026; Mika, 2026)，特别是 FunReason-MT 流水线 (Xu et al., 2025b; Hao et al., 2025)。总体而言，我们的目标是复现常见的企业和消费者工具使用场景，例如旅行预订和库存管理。

我们的流水线包含三个主要阶段：(i) 环境引导——生成工具描述、实现函数，并用相关实体为数据库填充种子数据；(ii) 任务创建——采样可能的工具调用轨迹、创建与交互链相关的实体并构建用户请求；(iii) 验证与优化——执行上一阶段生成的动作，并移除过于相似的任务。每个阶段都涉及多次 LLM 调用，用于实体生成和函数实现等任务。我们还采用迭代的批判与改进循环来优化每个阶段的输出 (Dhuliawala et al., 2023; Madaan et al., 2023)。

我们生成环境特定的用户人设 (personas) (Ge et al., 2024)，以进一步丰富任务的多样性。为缓解过度积极的工具调用行为，我们还用包含工具描述但不需要使用工具的任务来扩充数据。总体而言，我们的流水线合成了超过 150 个环境和 130,000 个任务。

奖励设计。训练期间，我们混合使用环境专属评分器和跨环境评分器。环境专属评分器根据最终环境状态、工具使用模式和最终答案分配奖励。对合成环境，我们使用 LLM 裁判将任务拆分为多个子任务并分别评分。我

⁷例如，SWE-Bench 曾受此问题影响：<https://github.com/SWE-bench/SWE-bench/issues/465>。此后该问题已修复。

⁸我们使用 OpenAI 的 function-calling schema：<https://developers.openai.com/api/docs/guides/function-calling>。

们还采用鼓励高效工具使用的跨环境评分器，包括尽可能并行调用工具、避免重复调用，以及以有效的参数类型和参数正确调用工具。

3.4 有益性与安全攀登

有益性与安全（Helpfulness and Safety）RL 攀登以人类偏好、指令遵循、可引导性（steerability）、安全、诚实性与风格为判据，优化 MAI-Thinking-1 的通用有益性。（Ouyang et al., 2022; Radford et al., 2019; Li et al., 2024a）与其他攀登阶段不同，本阶段聚焦于性能无法被客观定义、也无法由机器验证的任务。

本节的组织方式是：先介绍 RL 攀登中使用的所有奖励信号（第 3.4.1 节），随后介绍各领域的具体数据配方与奖励设计。

3.4.1 奖励

与其他攀登阶段相比，有益性与安全攀登组合了更多样化的奖励类型，以引导模型行为中较为主观的方面。我们结合了基于人类偏好数据训练的奖励模型（Lambert et al., 2025）、AI 裁判反馈（通常由评分准则引导）（Gu et al., 2024），以及额外的可验证奖励（Peng et al., 2025; Pyatkin et al., 2026），形成聚合奖励信号。（Lambert et al., 2024; Wang et al., 2024c）

奖励模型。我们的奖励模型基于 MAI-Base-1 的后训练版本，我们对其微调，以预测以文本 token 表达的人类偏好。该模型仅使用由多家供应商的人类标注者收集的人类偏好数据进行训练。我们采用 Liu et al. (2025c) 所述的一套稳健的奖励劫持（reward hacking）缓解管线，以应对训练数据中的相关偏差。

训练。对于上下文 c 和 k 路并排比较（ k -way side-by-side）的响应 $y_1, \dots, y_k$ ，以及对应分数 $s_1, \dots, s_k \in [1; 5]$ ，奖励模型的输入为

$$c < \text{im_sep} > y_1 < \text{im_sep} > y_2 < \text{im_sep} > \dots < \text{im_sep} > y_k < \text{im_sep} >$$

其中训练目标是序列 $s_1 \dots s_k$ ，通过 SFT 训练。

推理。将多个候选响应放入同一个奖励模型上下文，可以在不同响应之间对逐点（pointwise）质量分数进行更好的校准。然而，由于奖励模型的自回归特性，这也增加了分数 $y_{i,i>1}$ 的噪声。为应对这一点，我们循环地应用奖励模型：对于给定上下文 c 和 k 个响应 $y_1, \dots, y_k$ ，我们按响应的循环置换 $(y_1, \dots, y_k), (y_2, \dots, y_k, y_1), \dots, (y_k, y_1, \dots, y_{k-1})$ 对奖励模型提示 k 次。对于这 k 次推理调用中的每一次，我们只解码第一个 token，并查看该 token 的完整概率分布；第 i 次调用对应的即第 i 个候选响应的奖励分数 s_i 。奖励信号 $R_{RM}(c, y_i)$ 随后被设置为 y_i 被评为最高质量 ($s_i = 5$) 的概率。我们发现，与我们考虑过的其他公式化方案相比，这能提供更稳定的攀登信号。

评估。我们结合两方面的数据评估奖励模型：由标注者评分的实际训练 rollout，以及奖励模型训练数据的验证集划分。

AI 裁判。除了微调奖励模型提供的人类偏好信号外，我们还采用 AI 裁判（Gu et al., 2024; Tan et al., 2025）提供反馈，其优势是能够快速适配并针对任何给定上下文定制。特别是，AI 反馈提供了一种快速杠杆，可以有针对性地塑造模型行为，而无需承担奖励模型数据收集和重新训练的时间成本。

可验证奖励。我们采用可验证奖励来提升某些领域的能力，例如指令遵循——这类领域中，对约束的遵循情况可以被直接检查。例如，对于要求“respond in a single paragraph”（用单段话回答）或“use fewer than 10 words”（用不超过 10 个词）的上下文，我们会纳入可验证奖励。与不可验证奖励相比，我们发现可验证奖励更不易被奖励劫持 (Zhao et al., 2025b; Wen et al., 2024)，对多轮训练（multi-epoching）的敏感性更低，并且通常有助于稳定训练。

可验证奖励还被用于缓解不可验证奖励中的偏差。例如，AI 反馈评分准则往往会对长度和风格要素产生向上压力，我们同时使用奖励模型和可验证奖励来缓解这一点。具体到长度，我们离线（offline）确定每个上下文可接受的响应长度分布，并对落在预定义分位数区间之外的响应施加惩罚。这使响应长度在训练过程中保持稳定，既不过度约束响应，也不会引入一种可被劫持的奖励——即通过把响应压缩得过分简短来钻空子的奖励。

奖励组合。在为有益性与安全攀登优化这些奖励时，会出现两个挑战。(Wang et al., 2024c) 首先，不同类型的奖励处于不同的尺度，无法直接比较。其次，奖励分布本身依赖上下文：对某些提示词而言窄，对另一些则宽。天真地将这些奖励相加会导致幅值最大的信号——无论其重要性如何——占据主导。此外，虽然我们希望联合优化各项奖励，但某些标准是不可让步的：例如，写得很好的响应如果不安全，仍然不可接受，无论其质量如何。我们通过两种互补的策略应对这些挑战，并根据上下文有选择地应用。

词典序奖励塑形 (Lexicographic reward shaping)。对于一组上下文，当一组 rollout 在更高重要性的奖励上得分完全相同时，较低重要性的奖励才被激活。这会带来严格的优先级排序：只有在 rollout 组内主要奖励打成平时，次级奖励才会影响梯度 (Skalse et al., 2022)。由于它基于组内相对比较，这一公式化方案也对每个奖励的绝对尺度不敏感。

门控奖励应用 (Gated reward application)。对于其他上下文，更高重要性的奖励必须先达到最低性能水平，之后才会应用较低重要性的奖励 (Achiam et al., 2017)。安全就是典型例子：不安全的响应获得最低奖励，并且永远不会被按响应质量评分。

这两种策略都以条件式或基于排序 (rank-based) 的逻辑取代加性组合，从而规避跨尺度校准问题，确保高优先级的目标绝不会为了低优先级目标上的收益而被牺牲。

3.4.2 指令遵循与可引导性

指令遵循 (instruction following, IF) 是 LLM 的核心能力：模型应遵循对话中用户直接指定的约束、API 用户通过开发者指令指定的约束，以及平台所有者通过特权系统指令指定的约束。模型应能按照预定义的优先级，在这些格式与行为约束之间被引导 (Wallace et al., 2024)。为构建稳健的指令遵循能力，我们构建了一个覆盖多种约束、场景与复杂度层级的数据集，数据来源包括合成数据与专家人工标注。

数据。我们使用两个不同的数据来源：专家编写的上下文和合成数据。我们发现，专家编写提示词中的复杂约束有助于启动能力，而合成数据则能实现最大覆盖。

合成数据生成。受先前工作启发 (Xu et al., 2025a; Dong et al., 2025)，我们构建了一条灵活的多阶段管线，生成带评估准则的真实指令与场景。它首先利用 LLM 生成提示指令和模型规格，该 LLM 由一个手工整理的约束分类体系（详见附录 F）引导，并配有一组多样化的种子。随后生成场景：这些场景是多语言的，覆盖简短与长篇对话，包含系统、开发者和用户消息等情形，并跨越超过 40 个领域，包括编程、写作、分析与旅行规划。

此外，我们还加入带有冲突的系统、开发者和用户指令的对抗性案例，以训练模型尊重指令层级（instruction hierarchy）。最后的批判与改写步骤会基于自然度、准则对齐度和接地性（groundedness）评估并更新每个场景。

数据过滤。我们通过多轮过滤来保持高质量数据，综合了各种质量启发式规则、复杂度过滤器以及拒绝采样。特别是，评分准则需要经过验证，以确保自包含性、无歧义性以及与所声明约束的对齐，同时提示词需要经过我们的安全政策筛查。参照 An et al. (2025) 的做法，我们还通过通过率（pass-rate）分析控制难度水平。我们只保留具有挑战性但能被我们的模型解决的样本。

奖励设计。为防止 RL 训练中出现退化响应和奖励劫持，我们使用混合奖励信号。对于具有确定性验证器的约束，我们使用基于规则的检查。这提供了快速、确定性、且校准良好的奖励信号。

此外，一个 LLM 裁判（第 3.4.1 节）会独立评估原子化评分准则，为每条准则产生一个二元判断。多次判断结果取平均以增强稳健性。最后，我们训练好的奖励模型评估一般响应质量，针对约束的 IF 裁判形成补充。随后，奖励按照第 3.4.1 节中的词典序评分聚合方案组合，其中 IF 专属奖励作为主要信号。

3.4.3 安全

与我们对支持人类自主性的承诺一致，我们将安全定义为模型在提供有益响应的同时保持符合我们政策的能力 (Microsoft, 2022)。

数据。为使训练与这一目标保持一致，我们为数据整理开发了一套分类体系。安全数据针对两种互补的失败模式：不安全合规（unsafe compliance），即模型完成了本应拒绝的请求；以及过度拒绝（over-refusal），即模型不必要地拒绝了合法请求。每个候选样本都依据政策分类体系进行标注，并被归入两类提示词之一：

表 8. 有害提示词与边界提示词的来源。

类别	管线	来源
有害	人类红队测试	供应商编写的提示词内部红队演练
	自动化攻击	基于模板的攻击，如 PyRIT (Munoz et al., 2024) 非交互式 LLM 生成，如 PAP (
边界	Do-Not-Refuse 分片能力数据	跨数据代际保留的精选数据经安全管线标注进入 do-not-refuse 分片

- 有害提示词：政策禁止提供部分或全部帮助的请求。响应策略为完全拒绝或部分拒绝（拒绝不安全的部分，同时提供安全的替代方案）。

- 边界提示词：涉及敏感领域但在政策范围内仍可回答的请求。响应策略为 do-not-refuse（不拒绝）：提供有边界、有帮助的回答，而不是含糊其辞或拒绝。

数据来源。表 8 总结了各类提示词的获取方式。有害提示词通过人类红队测试和自动化对抗生成两种途径收集。边界提示词来自跨数据代际保留的现有 do-not-refuse 分片，以及经由安全标注管线路由并被选入 do-not-refuse 分片的能力数据，从而让模型接触到应当保持可回答的政策邻近型（policy-adjacent）请求。

奖励设计。每个模型响应由安全裁判沿三个维度评分：

- 政策合规（Policy compliance）：衡量响应是否违反我们的安全政策。

- 响应参与度 (Response engagement): 将响应的参与程度与预期目标 (拒绝、部分拒绝或 do-not-refuse) 进行比较, 对过度拒绝以及严重请求上的合规行为都施加惩罚。
- 响应风格 (Response style): 衡量响应是否遵循预期的语气和指导原则, 例如对敏感的自残请求, 要承认其困难性, 而不是说教或向用户复述创伤事件。

裁判分数与来自奖励模型的奖励信号、(视数据来源而定) 额外的 AI 裁判信号相结合。简单的加权平均是不够的: 标量奖励对政策不合规的响应仍可能保持为正。⁹ 因此我们使用安全门控聚合器: 按第 3.4.1 节所述, 政策合规字段对奖励进行门控。如果安全裁判判定响应不合规, 则该 rollout 获得最低奖励, 无论其他分数如何; 否则使用正常的加权混合。

3.4.4 诚实性

我们将诚实性定义为: 模型在知道答案时能给出正确响应的能力, 以及在不知道答案时恰当表达不确定的能力。同时, 模型不应过度表达不确定, 因为单纯最小化事实错误只会促使模型做出更少的论断 (Min et al., 2023)。因此, 目标必须在事实精确性与信息量之间取得平衡。

表 9. 我们风格指南中目标行为描述的示例。

关注领域	示例指导
表情符号	尽量少用——仅在明确合适的休闲语境中使用。不要用表情符号作为列表标题或署名落款。
表格与 Markdown	项目符号列表和表格用于罗列多个要点、给出示例或呈现数据。当回答确需多个不同部分时, 用标
语气与正式程度	默认采用专业语气, 仅在用户直接要求时才采用轻松或富有感染力的语气。
版面布局	尽量少用开场白或收尾套话, 不写谄媚式的引言。信息层级清晰。回答便于快速浏览, 重要信息放
密度	避免大段文字堆砌。粗体字要有策略地使用。段落之间留出间距。句子保持足够简短易读。

数据。我们汇集了多样化的手工整理数据, 来源包括供应商、经 PII 过滤的消费者 Copilot 日志¹⁰以及合成生成的数据。受先前工作启发 (Zhang et al., 2024; Wei et al., 2024b,c), 我们的数据覆盖一个难度谱系: (a) 既有事实性查询, 包括短格式与长格式, 其响应可以与参考答案进行精确核验; (b) 有挑战性的事实性查询, 面向长尾或冷门主题——这些主题上模型的覆盖预期不稳定, 参考标签通过检索增强式核验生成; 以及 (c) 虚假前提查询, 即问题中包含错误的预设 (presupposition), 且不存在正确的肯定性回答。覆盖这些边界情形促使模型保持事实完整性, 并且只在缺乏相关知识时才表达不确定。

奖励设计。在事实性评分中, 每个 RL 样本的参考标签均通过检索增强生成与核验离线 (offline) 生成。随后每个模型响应由 LLM 裁判沿两个维度——事实性与置信度——评分, 得到五类之一: CONFIDENT_CORRECT (自信且正确)、UNCONFIDENT_CORRECT (不自信但正确)、NOT_ATTEMPTED (未作答)、UNCONFIDENT_INCORRECT (不自信且错误) 和 CONFIDENT_INCORRECT (自信但错误)。这些类别通过加权求和合并为标量奖励: 自信且正确的响应获得最高奖励, 自信的幻觉受到最严厉的惩罚, 放弃作答获得中性奖励, 而不自信但正确的响应获得降低的奖励, 以抑制过度表达不确定。

⁹在一项成对 rollout 审计中, 期望安全 Likert 分数与政策合规字段只有中等一致性 (Pearson 0.293, Spearman 0.344), 且 87.8% 的政策不合规响应获得的奖励模型分数不低于 3。

¹⁰消费者 Copilot 日志数据仅面向未选择退出将其数据用于模型训练的用户收集, 并排除微软 Copilot 隐私常见问题 (Privacy FAQ for Microsoft Copilot | Microsoft Support) 中所列的部分用户。

3.4.5 风格

我们制定了一份风格指南 (style guide)，规定优质输出应是什么样：亲切而不谄媚、结构易于扫读、语气依语境校准而非简单模仿用户。风格指南还涵盖何时以及如何使用表情符号、正式程度、数学与代码的记法，以及总体文本密度。表 9 给出了我们训练模型遵循的指导原则的高层示例。

数据。风格评分基于广泛的数据集：经 PII 过滤的微软消费者 Copilot 日志、供应商编写的上下文（静态与交互式），以及 Arena 对话。这些数据覆盖低至中等难度的提示词，排除复杂的指令遵循、编程、数学和 STEM。提示词按用户意图分类（如创意写作、实用指导、信息寻求和闲聊），并主动采集以针对模型较弱的领域。

表 10. 合并 SFT 数据混合按能力的组成。

能力	样本权重	Token 权重
STEM 与编程	56%	89%
智能体能力	11%	9%
通用有益性与安全	33%	2%

奖励设计。除了第 3.4.1 节介绍的奖励模型外，我们还使用带提示词的 LLM 裁判沿特定轴为输出评分，并对不良行为施加惩罚。这些带提示词的裁判在 0、1、2 的整数尺度上为响应评分，分别对应重大、轻微或无风格问题。我们发现，粗粒度裁判优于更细粒度的裁判或按提示词定制的评分准则，因为它们让 LLM 裁判有更大的灵活性，能针对给定的提示词与响应恰当地解释准则，从而让模型更难劫持裁判。风格裁判仅在可验证奖励与安全约束均已满足后才被应用，并根据领域与其他裁判一同加权。

3.5 将多种能力整合进单一模型

前面的章节描述了三个教师模型，它们分别针对 STEM 与竞赛编程（第 3.2 节）、智能体能力（第 3.3 节）以及帮助性与安全性（第 3.4 节）独立训练。我们分两个阶段将它们整合进单一模型，如图 12 所示。SFT 阶段复用第 3.1.4 节的自蒸馏管线，将其应用于每个专项教师模型，不过三个教师模型需要不同的过滤和拒绝采样策略。对于 STEM 与智能体教师模型，我们按照第 3.1.4 节的多样性发现，在每次攀升的多个检查点上采样推演，并优先使用较晚的检查点。我们为每个上下文保留多条正确推演，只施加轻度过滤以剔除退化的思维链 (CoT)。对于帮助性与安全性教师模型，我们使用 LLM 裁判和启发式过滤器，在正确性之外，还按风格、结构和已知缺陷对轨迹评分。

整合 SFT。SFT 阶段将三个教师模型蒸馏进单一模型。我们对数据混合配比和超参数进行了迭代，以平衡推理、智能体与通用帮助性能力，并在公开基准、内部基准和人工评估上进行验证。表 10 按样本权重和 token 权重报告了最终混合配比。我们发现按样本权重平衡混合配比很重要；相应地，token 分布由 STEM 和编程主导，这是因为它们的轨迹更长，而这在实践中并不会损害帮助性与安全性能力。与我们的标准自蒸馏方案不同，整合 SFT 训练 4 个 epoch，学习率从最大值 $1 \cdot 10^{-5}$ 衰减 $2\times$ 。

整合 RL。最后阶段的轻量 RL 进一步改进安全性、过度拒答和风格。该方案基于帮助性与安全性攀升（第 3.4 节），为保持推理性能做了一些调整。我们以 128k tokens 的最大序列长度训练，并在 RL 混合数据中保留少量 STEM 和编程数据；我们发现两者都很重要，否则复杂任务上的推理性能会在攀升过程中缓慢退化。

3.6 强化学习基础设施

我们的 RL 攀升依赖 Rocket，一个用于大规模异步分布式强化学习的内部框架，使用 YOLO 作为学习器（第 2.8 节），使用 SGLang（Zheng et al., 2024）进行模型推理。我们构建 Rocket，是因为我们的需求——即在 MAI-Thinking-1 所需的数千块 GPU 规模上支持异步 RL——是现有开源 RL 框架（von Werra et al., 2020; Hu et al., 2024; Sheng et al., 2024）无法满足的。Rocket 的核心 RL 数据流围绕单个控制器、一组问题工作节点和推演工作节点，以及生成模型输出的路由与推理服务器来组织。控制器、问题工作节点和推演工作节点各自实现为单个 Python 进程 / Ray actor，而路由与推理由基于 SGLang 的服务栈提供。Rocket 架构的总览如图 19 所示。

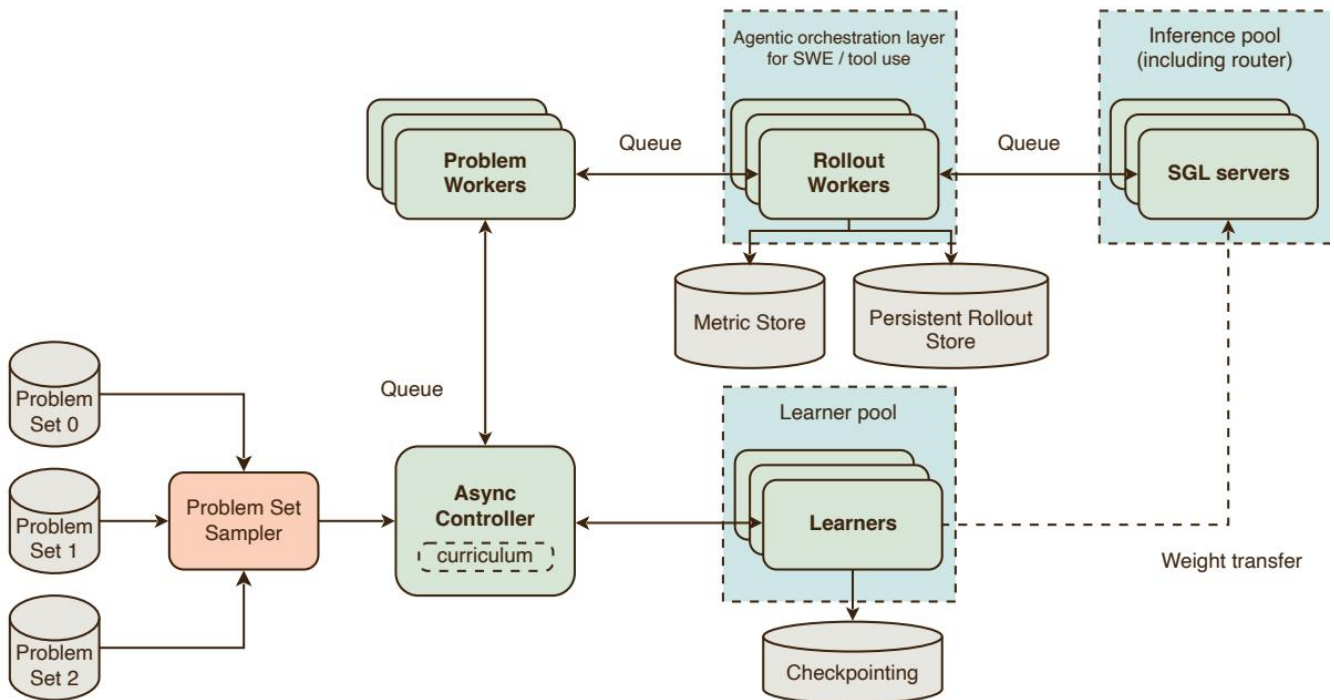


图 19. Rocket 总览：我们的大规模强化学习框架。

3.6.1 控制器

控制器加载 RL 任务并将其发送给问题工作节点处理。对于每个任务，控制器会收到一组已完成的推演及关联的评分元数据（通过/失败判定、奖励、归一化优势等）。控制器对完成的推演进行过滤，并分批发送给学习器。控制器的抽象使 Rocket 可以在 on-policy（同策略）RL 与 off-policy（离策略）RL 之间切换不同的控制器实现。实践中，我们在大规模运行中主要使用 off-policy（离策略）RL，而将 on-policy（同策略）RL 保留给较小的实验和调试。

3.6.2 问题工作节点

当问题工作节点从控制器收到 RL 任务时，它生成一组推演、为每条推演计算归一化优势，并将结果发回控制器。问题工作节点本身并不负责推演生成，而是向推演工作节点发送一组请求——每条推演一个请求。问题

工作节点为大规模异步 RL 实现容错，例如当某个推演工作节点失败时为任务重试推演。

单个请求到达问题工作节点的典型流程如下：

1. 早退出推演。按照第 3.1.3 节描述的早退出策略，问题工作节点从控制器接收 RL 任务，并向推演工作节点发出 16 个请求，必要时重试失败的请求。每条推演都会被评分，这些信息被聚合成该任务的整体通过率。如果整体通过率落在预设区间内，问题工作节点进入完整推演阶段；否则，该 RL 任务被中止。
2. 完整推演。问题工作节点再向推演工作节点发出 128 个请求，同样在必要时重试失败的请求。每条推演都会被评分，这些信息被聚合成该任务的整体通过率。如果整体通过率落在预设区间内，问题工作节点进入后处理阶段；否则，该 RL 任务被中止。
3. 后处理。问题工作节点为每条推演计算归一化优势，用于 RL 目标（第 3.1.1 节）。视具体问题而定，该步骤可能包括对长推演施加长度惩罚、应用 GRPO 奖励归一化或其他启发式规则。

3.6.3 推演工作节点

当推演工作节点收到来自问题工作节点的请求时，它生成一条推演，可选地进行评分，然后将结果发回调用方。在典型流程中，推演工作节点首先生成初始模型提示词并发送给模型推理服务器。响应返回后，推演工作节点解析响应、执行任何工具调用，并将结果合并进发给推理服务器的后续请求中。该过程持续进行，直到模型停止调用工具并输出最终答案，或超过预先设定的步数或推演耗时上限。

推演结束后，下一步是对模型的输出进行评分。评分器的输出包含数值奖励和可选的通过/失败判定。视问题而定，评分可以在推演工作节点上或问题工作节点上进行。如果各推演可以独立评估，评分由推演工作节点执行。例如，对于数学问题，如果模型的输出与问题的标准答案在数学上等价，评分器可能输出 (1, pass)，否则输出 (0, fail)。另一方面，有些评分器需要同时查看多条推演的答案，例如让 LLM 对不同模型的答案进行两两比较。在这种情况下，评分在问题工作节点上进行。单个问题可以有多个评分，此时由用户定义的策略将各评分聚合成一个整体奖励。

3.6.4 路由与推理

推理是系统最重要的组件之一：视任务的规模和特点而定，推理与学习器的 GPU 之比最高可达 5:1。在我们最大的 RL 任务中，共使用 4864 块 GB300 芯片，其中 4096 块专用于推理，其余 768 块专用于学习器。因此，推理性能和系统稳定性是一级关注点。为保障 RL 训练稳定性，控制学习器与推理引擎之间的数值差异至关重要。

推理配置。我们的 RL 推理栈构建在 SGLang 和 SGLang router (Zheng et al., 2024) 之上。我们选择 SGLang，是因为它满足了对丰富 RL 服务特性、面向开源模型的快速实验以及强劲生态发展速度的需求。在单个 SGLang worker 之上，我们使用 SGLang router 负责负载均衡、流量控制和前缀缓存。

推理性能。由于任务的大部分 GPU 用量来自推理，优化推理吞吐量对高效的 RL 至关重要。最小化请求延迟对控制陈旧性同样关键。我们根据推演发生在单轮还是多轮场景来定制优化。对于单轮工作负载，提示词通常较短，但生成（包括思考）最长可达 128k tokens，因此主要瓶颈是 KV 缓存内存。我们通过专家并行和注意力数据并行来降低 KV 缓存与权重的占用。我们还禁用前缀缓存，使滑动窗口注意力 token 在长生成过程中可以被完全逐出（我们的模型架构重度依赖滑动窗口注意力）。我们还通过 MLP 层的数据并行、DeepEP

(Deep-EveryParallel) (Zhao et al., 2025a) 和 EPLB (Expert Parallelism Load Balancer) (DeepSeek-AI, 2025) 进一步降低通信开销。对于多轮工作负载，提示词会变得很长，而生成往往较短，使这类负载以 prefill 为主。在这里我们重度依赖前缀缓存：在生产 RL 运行中，我们的前缀缓存命中率可达 97–98%。

推理系统稳定性。在用于训练 MAI-Thinking-1 的、由数千块推理芯片构成的 RL 规模下，单个副本会时常因节点故障而崩溃、挂起、变慢或失败；此时推理层必须优雅降级并重启，而不是让整个任务失败。我们依赖三个层面的纵深防御。在副本层面，每个 SGLang server 运行一个自看门狗，探测自身的生成端点并监控调度器内存；不健康的副本会触发自身的优雅重启，使编排器可以拉起一个干净的新副本。在路由层面，SGLang router 充当熔断器：当后端副本不健康时，它停止接收新请求，重新开放前执行多阶段探活，并定期重新发现那些悄然退出其活跃集合的副本。逐副本流量控制可防止慢副本积压请求，从而拖累整个推理池。在任务层面，存活监控器跟踪每类 actor 的存活数量（推理副本、router、推演工作节点、学习器 rank），如果任何一类低于允许阈值，就让任务失败以进行干净的重启。另一个正交的步进进度看门狗捕获更棘手的情况：一切看起来都活着，但训练已停止推进。最后，当推演工作节点对推理服务器的请求在半途失败时，会针对另一个副本重试，因此单个副本故障只会降低推理池吞吐量，而不会丢失推演。

推理与学习器之间的数值差异。RL 的一个关键方面是推理引擎与学习器之间的数值差异。YOLO（学习器）和 SGLang（推理引擎）使用不同的 kernel、调度和并行策略。即使是微小的逐 token logprob 差异，也会在长推演中累积，并可能破坏 off-policy（离策略）RL 中的重要性采样校正。为缓解这些问题，我们在 RL 运行中对学习器和推理引擎都使用 bf16；与我们评估过的更低精度方案相比，这产生了显著更小的数值差异和更稳定的训练。在 bf16 之上，我们还应用 MoE 路由重放 (Ma et al., 2025) 和 top-p 掩码重放（第 3.1.3 节）。

3.6.5 权重迁移

从同步 RL 转向异步 RL，使权重迁移变成一个反复出现的分布式同步问题：新的学习器权重必须每隔 k 步到达推理集群，而迁移时间会造成推理空闲和额外的陈旧性。核心难点在于，学习器与推理的分配以不同的方式对同一批张量进行分片：双方可能使用不同组合或程度的 FSDP、流水线并行、数据并行注意力与张量布局，参数还可能在精度、量化状态或矩阵布局上不同。因此，每次迁移都必须既重新分片字节，又执行逐张量变换。

迁移计划。我们将重新分片、数据搬移和逐张量变换编译成单一的迁移计划，在任务开始时计算一次，之后复用。对每个参数，规划器求学习器与推理服务器分片布局的交集，并为每个非空的子分片重叠输出一条记录，记录源 rank、目标 rank、字节范围以及 dtype 转换、布局置换等所需操作。重新分片隐含在这份交集中；每个学习器 rank 只发送与其配对的推理 rank 所需的切片，从而省去对完整张量的单独物化。变换在成本最低的一侧执行，运行时将打包、传输和解包流水线化，使连续的多个子分片在时间上重叠。为了将规划与集群规模解耦，该计划面向一台学习器加一台推理服务器的理想拓扑；在运行时，每个源-目标对会扩展为一个覆盖目标 rank 全部存活副本的迁移组，因此当副本加入、离开或被替换时，计划仍然有效。由于数据并行是复制而非分片参数，一个 DP 组就已持有全部模型参数——计划无需征用全部 D 个学习器 rank。我们将其限制在部分 DP 组上，每个 DP 组与一段互不相交的推理副本配对并独立运行。一个 36 台服务器的集群按四路切分后，并行执行四次 9 台服务器的迁移，在提升吞吐量的同时控制故障爆炸半径。

表 11. 训练后模型在 STEM 与智能体编码公开基准上的评测结果。其他模型的数据取自各自的官方模型卡。得分以“—”列出的模型未报告相应基准的数值。对于智能体编码评测，我们使用 256k 的总上下文长度。对于上述其他评测，我们使用 256k 的最大输出 token 数。

基准	MAI-Thinking-1	Sonnet 4.6	Opus 4.6	GPT 5.4	Kimi K2.6	DeepSeek V3.2	DeepSeek V4	GLM-5.1
AIME 2025	97.0	95.6	99.8	—	—	93.1	—	—
AIME 2026	94.5	—	—	—	96.4	—	—	95.3
HMMT Feb 2026	84.9	—	—	—	92.7	—	95.2	82.6
GPQA Diamond	84.2	89.9	91.3	92.8	90.5	82.4	90.1	86.2
LCB v6	87.7	—	—	—	89.6	83.3	93.5	—
Terminal-Bench 2.0	46.0	59.1	65.4	75.1	66.7	46.4	67.9	69.0
SWE-bench Verified	73.5	79.6	80.8	—	80.2	73.1	80.6	—
SWE-Bench Pro	52.8	—	53.4	57.7	58.6	—	55.4	58.4

4 评测

4.1 基准评测

我们将 MAI-Thinking-1 与多种开源和闭源权重的前沿模型在公开基准以及人工并排评测上进行了比较。这些评测涵盖广泛领域，凸显了模型在不同场景下的通用性：STEM、智能体编码、知识、指令遵循、长上下文、安全、健康、诚实性和工具调用。除非另有说明，MAI-Thinking-1 的所有基准评测结果均为 4 次运行的平均值，采用统一的推理设置，温度 $T = 1$ ，top-p 采样 $p = 0.97$ 。

表 11 报告了我们在 STEM 和智能体编码基准上的结果。其他模型的数值取自其官方模型卡和发布公告。综合来看，这些结果将 MAI-Thinking-1 置于其他流行 LLM 的有竞争力区间：它并不领先，但在广泛的基准类别上持续表现强劲。值得注意的是，MAI-Thinking-1 在 AIME 2025 上超过 Claude Sonnet 4.6，并在 SWE-Bench Pro 上接近 Claude Opus 4.6 的表现。

此外，我们所有的 SWE 智能体训练数据仅使用 bash 和字符串替换作为工具，不包含针对性的终端交互环境。因此，模型当前的 Terminal-Bench 表现反映的是从更广泛的智能体训练中获得的泛化能力，而非直接针对类似 Terminal-Bench 的环境进行训练。

数学、科学与竞赛编程。在数学方面，我们在 2025 和 2026 年版 AIME 以及 MathArena 的 2026 年 2 月 HMMT 基准 (Dekoninck et al., 2026) 上评测 MAI-Thinking-1。在科学方面，我们在研究生级 Google-Proof Q&A 基准 (GPQA) (Rein et al., 2023) 上评测，该基准包含知识密集型、研究生与研究级问题，主要覆盖 STEM 领域。在竞赛编程方面，我们在 LiveCodeBench v6 (LCB v6) (Jain et al., 2024) 上评测，该基准包含最新的竞赛编程问题。更多评测细节见附录 H。

表 12. 各公开基准上后训练模型的评测结果。Sonnet 4.6 的所有结果均通过我们自己的评测套件生成。对于 AdvancedIF，我们报告评分细则级分数。

	知识		指令遵循		长上下文	
模型						
MAI-Thinking-1	85	31	69	85	53	90
Sonnet 4.6	87	29	50	86	57	96

安全	诚实性					健康
模型						AIR-Bench CyberSec Instruct CyberSec Auto Long Fact Truthful QA Heal
MAI-Thinking-1	88	63	63	98	88	35
Sonnet 4.6	88	62	56	98	88	38

智能体编码与工具调用。在智能体编码方面，我们在 SWE-bench Verified (Chowdhury et al., 2024)、SWE-Bench Pro (Deng et al., 2025) 和 Terminal-Bench 2.0 (Merrill et al., 2026) 上评测 MAI-Thinking-1。在工具调用方面，我们在 BFCL v3 (Patil et al., 2025) 上评测 MAI-Thinking-1。与 STEM 评测不同，这些评测是多轮交互的，需要模型与环境交互。对于这三个基准，我们都使用一个非常简单的 ReAct 风格 (Yao et al., 2023) 循环来评测模型，该循环始终追加到图 18 所示的智能体循环。对于 SWE-bench Verified 和 SWE-Bench Pro，我们同时启用 bash 和字符串替换工具。对于 Terminal-Bench 2.0，我们仅启用 bash 工具，以模拟最精简的终端界面。为消除推理速度和基础设施的混杂因素 (Segato, 2026)，我们忽略 Terminal-Bench 2.0 的预定义超时。评测设置的更多细节见附录 I。

综合能力。在表 12 中，我们报告了涵盖知识、指令遵循、长上下文、安全、诚实性、健康和工具调用的基准结果。对于这些领域的基准，我们发现并非所有实验室都会在模型卡或模型公告中报告官方结果。为了提供与 MAI-Thinking-1 对比的基线，我们在这些基准上使用最大推理努力 (max reasoning effort) 和最大序列长度评测了 Sonnet 4.6，并将结果一并报告在表 12 中。在大多数基准上，我们发现我们的模型与 Sonnet 4.6 相当。

具体而言，我们报告 SimpleQA Verified (Haas et al., 2026) 和 MMLU-Pro (Wang et al., 2024b) 以衡量知识与推理能力，IFBench (Pyatkin et al., 2026)、AdvancedIF (He et al., 2025) 和 MultiChallenge (Deshpande et al., 2025) 以衡量指令遵循，GraphWalks 以衡量长上下文能力，AIR-Bench (Zeng et al., 2024b) 和 CyberSecEval 4 (Wan et al., 2024; Meta, 2024) 以衡量安全，TruthfulQA (Lin et al., 2022) 和 LongFact (Wei et al., 2024c) 以衡量诚实性，HealthBench (Arora et al., 2025) 和 MedXpertQA (Zuo et al., 2025) 以衡量健康知识任务。各评测设置的说明见附录 K。

4.2 人工并排评测

为补充上述以能力为导向的公开基准——它们聚焦于狭窄且客观定义的质量标准——我们还在广泛的真实世界任务上进行了人工评测。这些评测整体性地并排比较两个模型，重点关注整体有用性。并排评测有助于揭示单独审阅回复时不易察觉的质量差异。

表 13. 人工并排评测的任务分布。

任务类别	提示词占比
开放式问答、头脑风暴与建议、内容创作	各 13–14%
结构化问题求解、信息抽取、学业辅导、洞见生成、内容摘要	各 6–7%
任务规划、基于上下文的问答、其他文本分析	各 5%
个人支持、娱乐、闲聊、角色扮演	各 3–4%

评测任务选择。评测任务源自互补的多个来源，以确保全面覆盖真实用户需求，并具备较强的模型区分度。最

终集合包含 1276 个任务，全部为英文，其中 30% 为多轮任务。任务分布的详细信息见表 13。

第一个任务来源是遵循结构化分类法编写的专家提示词，涵盖复杂度各异真实世界用例，包括单轮和多轮对话。第二个来源是微软消费级 Copilot 产品的日志，经过仔细过滤，排除了包含个人可识别信息 (PII) 的提示词、不完整或缺乏必要对话上下文的提示词、对抗性提示词，以及需要自定义配置（如编码环境、图像生成能力或外部工具访问）的提示词。我们使用分层采样以确保用例覆盖，并在特异性和约束多样性等维度上平衡难度。

评分者池与评测流程。为开展这些模型评测，我们与由 Surge AI（一家信誉良好的数据标注供应商）管理的评测者合作。这些评测者是以英语为母语、来自各种通才和专业背景的人士。评测者须经过多阶段资格筛选流程，该流程评估他们评测核心 LLM 能力与失败模式的能力，包括事实核查、阅读理解和指令遵循。培训材料包括评分说明和常见失败模式的示例。

对于每个提示词，评分者首先需要从多个维度分别仔细评估 MAI 和其他模型的回复：指令遵循（包括显式和隐式）、事实性（使用搜索引擎辅助事实核查）、简洁性与相关性、完整性，以及风格与语气。在每个维度上，评分者判断回复是否存在无问题、轻微问题或重大问题。最后一步，评分者在两个回复之间给出整体偏好评分，采用 7 点 Likert 量表，范围从“远差于”(-1.5) 到“远优于”(1.5)。我们观察到评分者池具有较强的一致性，验证了评分在可接受的噪声阈值内是稳定且可复现的。

结果。表 14 展示了整体成对偏好在 $[-1.5, 1.5]$ 区间上的结果，以及各质量维度在 $[-1, 1]$ 区间上的差值。总体而言，评分者更偏好 MAI-Thinking-1 而非 Sonnet 4.6，但更偏好 Opus 4.6 而非 MAI-Thinking-1。对比 Sonnet 4.6，MAI-Thinking-1 在 49% 的比较中获胜，6% 持平，45% 落败。对比 Opus 4.6，MAI-Thinking-1 在 43% 中获胜，5% 持平，52% 落败。在定向维度上，评分者认为 MAI-Thinking-1 在简洁性与相关性以及风格与语气上优于 Sonnet 4.6，而在指令遵循、事实性和完整性上大致相当（在噪声范围内）。

4.3 内部安全评测

安全与过度拒答。我们构建了一个内部基准，用于衡量模型在被判定为低风险请求（模型应当回答）的提示词上的过度拒答。拒答判定器 (refusal judge) 随后根据既定策略对每个回复打分，标记拒答、回避或不当的部分拒答。过度拒答率是回复未能依从的提示词占比，有用性定义为 1 减去该比率。结合高敏感条目上的安全通过率（1-5 Likert 量表上判定器安全得分 > 3），这揭示了理想的模型行为：对更高风险、有害的请求更安全，对低风险、良性的请求更有帮助。评测方法与数据集构建的详细说明见附录 J。

表 14. MAI-Thinking-1 对比 Sonnet 4.6 和 Opus 4.6 的人工评测结果。正值表示偏好 MAI-Thinking-1，负值表示偏好 Sonnet 或 Opus。

模型表现	对比 Sonnet 4.6	对比 Opus 4.6
整体并排偏好	0.07 ± 0.06	-0.07 ± 0.06
指令遵循 Δ	-0.01 ± 0.02	-0.04 ± 0.02
事实性 Δ	-0.02 ± 0.02	-0.03 ± 0.02
简洁性与相关性 Δ	0.11 ± 0.02	0.07 ± 0.02
完整性 Δ	-0.01 ± 0.02	-0.02 ± 0.02
风格与语气 Δ	0.08 ± 0.02	0.05 ± 0.02

图 20 展示了 MAI-Thinking-1 对比 Sonnet 4.6 的安全-有用性平衡。在八个类别中的五个上，MAI-Thinking-1 位于 Sonnet 4.6 的上方和/或右方，表明表现更优，其中在化学、生物、放射与核（CBRN）、自伤以及选举与政治上的提升最大。

越狱攻击。我们从供应商、内部红队以及包括 HarmBench (Mazeika et al., 2024) 和 StrongREJECT (Souly et al., 2024) 在内的开源基准中收集了 2.5K 个独特的种子场景，构建了内部越狱评测套件。我们对收集到的提示词进行扩充，最终形成约 9.5K 条越狱提示词的评测集。我们按变换程度和攻击者自适应性将这些提示词分为三类：基础技术、组合技术与自适应技术（定义见图 21）。基础技术（Foundational Techniques）是单步变换，通过越狱包装或提示词模板等简单修改保留有害意图。组合技术（Compositional Techniques）结合多种变换或结构化改写，包括基于模板的攻击，如 PyRIT (Munoz et al., 2024)、PAP 风格变换 (Zeng et al., 2024a)，以及非英语或混合语言变体。自适应技术（Adaptive Techniques）引入交互、搜索或多轮结构，包括 TAP (Mehrotra et al., 2024) 和多轮攻击 (Russovich et al., 2024)。

图 21 报告了这三类攻击的成功率（ASR）；数值越低表示安全性越强。在这些提示词变换类型上，MAI-Thinking-1 取得了与 Sonnet 4.6 和 Opus 4.6 相当的低 ASR。

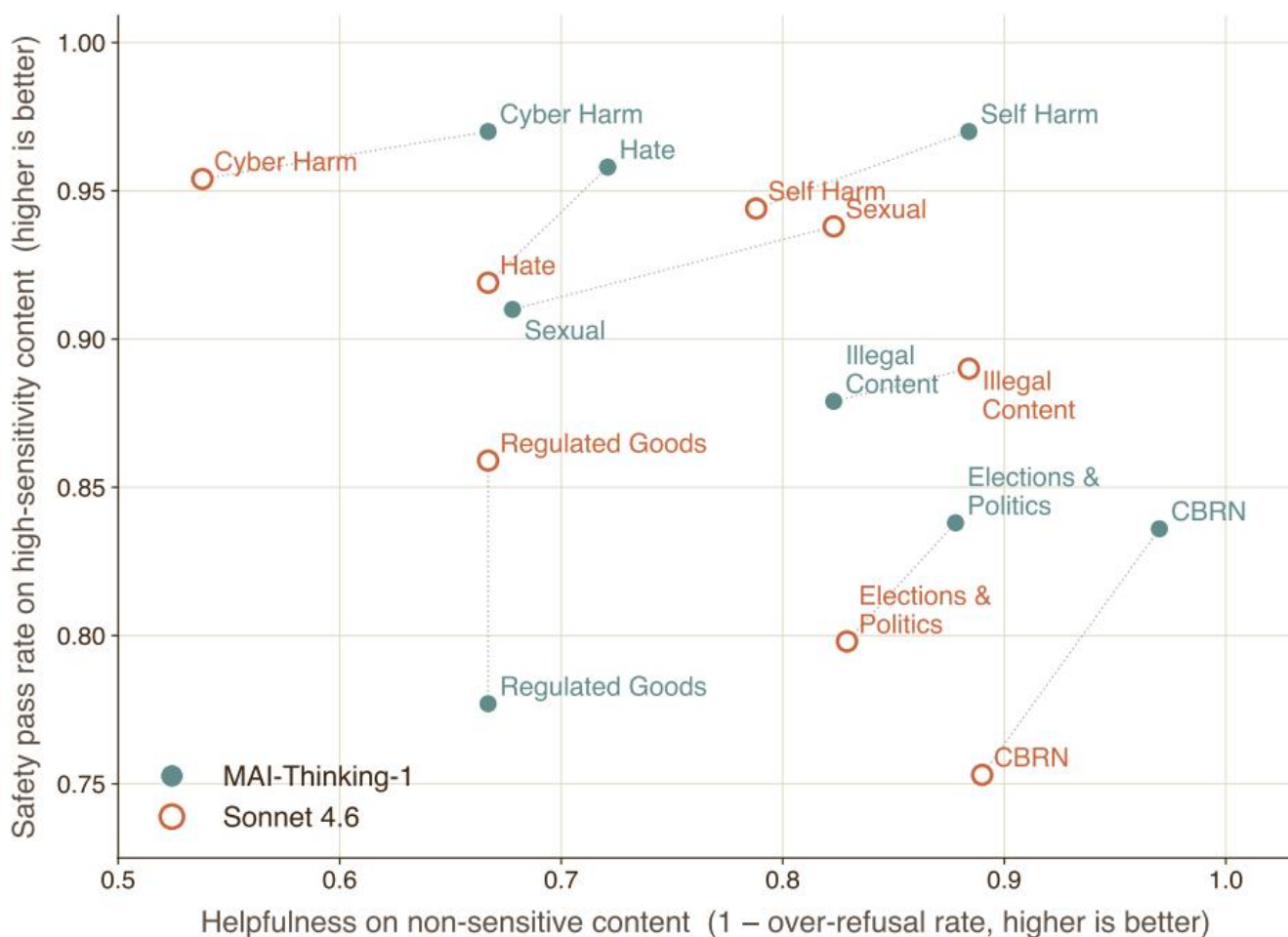
5 安全红队测试

为了评估自动化基准之外的安全特性，我们与模型开发周期并行开展了红队测试。目标是揭示自动化评估无法可靠检测的对抗性漏洞、新型攻击向量和危害类别缺口，并将发现结果按滚动节奏反馈到训练数据收集与策略细化中。

5.1 内部红队测试

方法论。内部红队测试由 MAI 红队成员（安全研究人员和招募的外部标注人员）执行，并在整个开发周期中对不同模型版本开展。

图 20. 按危害类别的安全-有用性权衡。指标为每个提示两次生成结果的平均值。对于给定的危害类别，安全-有用性权衡更好的模型位于另一模型的右上方。两个模型之间用虚线连接。



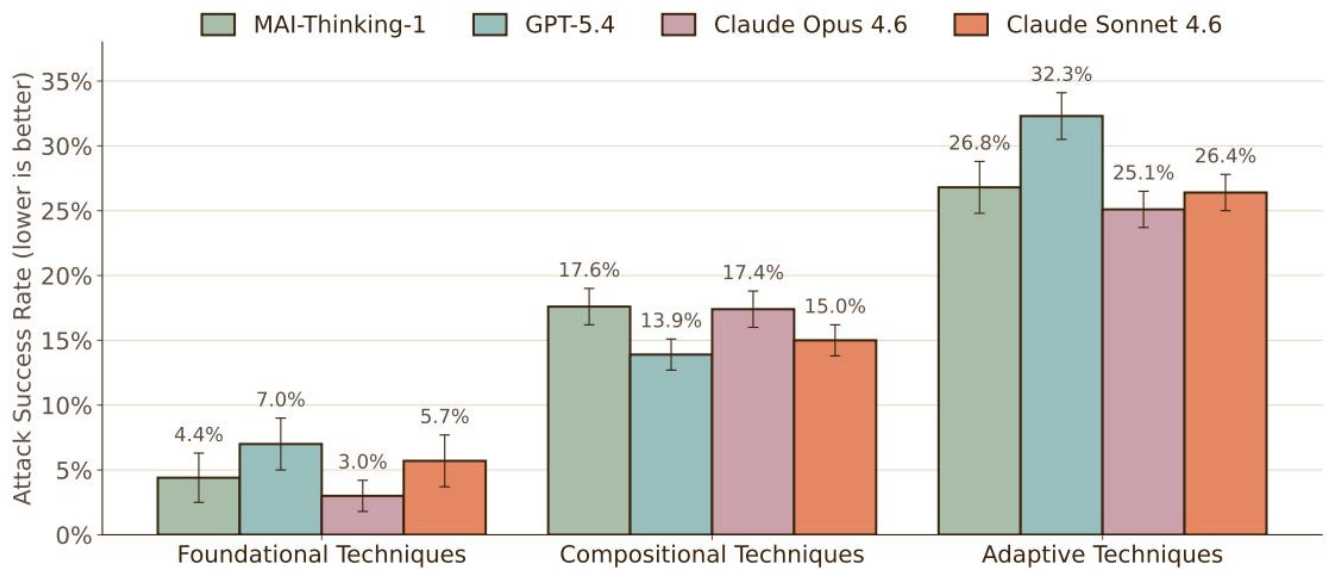
在覆盖 MAI-Thinking-1 开发早期、中期和后期阶段的 15 次合作中，红队成员在 25 个政策类别上执行了超过 2,170 个基于目标的对抗性场景。每个场景运行 5–10 轮对话，以允许升级到首轮拒绝之外。随着后续模型版本能力增强，我们提高了评估的范围和难度。

标注人员经过对抗性提示能力筛选，并获准访问内部模型部署。他们收到一份预填充的基于目标的场景追踪表，每个场景都附有支持性上下文和目标政策类别，并被指示在多轮对话中诱导政策违规。违规情况标注了结果、严重程度、首次违规的轮次、最严重违规的文本以及其他元数据。该计划有几个范围约束：测试主要使用英语，非英语输入仅在有限情况下作为越狱向量使用，而非系统性评估；智能体工具使用和多模态输入不在范围内。

发现与缓解措施。该计划的一个核心产出是攻击模式分类法，这些模式在红队成员和模型检查点之间独立反复出现，我们认为这证明这些模式（而非单个提示）才是持久的对抗面。其中六种模式最为突出：在良性借口下进行多轮升级；虚构或小说式框架；凭证化身份借口；逐步递归或格式漂移（反复要求扩展、重排格式或落实先前含糊其辞的回答）；上下文内年龄指示绕过；以及权威文档伪造。在各类别中，同样的模式驱动了大多数红队成功案例，该分类法现在作为后续周期中提示收集和评判准则开发的覆盖清单。

发现结果按滚动节奏反馈到训练中。一个突出的反复出现的模式是虚构框架绕过，会暴露自残内容，包括多语

言变体。该模式催生了一组精选数据，被扩展进安全 evaluation，并用于对抗性提示收集的种子扩展，以纳入图 21。按变换类型分组的越狱评估攻击成功率。基础技术（Foundational Techniques）是保留有害意图的单步变换，通过越狱包装器或提示模板等简单修改实现。组合技术（Compositional Techniques）组合多种变换或结构化改写，包括基于模板的攻击，如 PyRIT (Munoz et al., 2024)、PAP 风格变换 (Zeng et al., 2024a) 以及非英语或混合语言变体。自适应技术（Adaptive Techniques）引入交互、搜索或多轮结构，包括 TAP (Mehrotra et al., 2024) 和多轮攻击 (Rusinovich et al., 2024)。攻击成功率越低表示安全性越强。误差棒表示 95% 置信区间。第三方模型结果包含服务方侧安全过滤。



SFT 和 RL 安全配比。

在红队测试中识别出的最高优先级修复类别中，从缓解前到最终候选版本，攻击成功率总体下降了约 22%。我们将越狱成功率降低了约 44%，仇恨与公平问题降低了约 43%，儿童安全问题降低了约 30%，心理健康攻击降低了约 20%。

5.2 独立红队测试

额外的红队测试由微软 AI 红队 (AIRT) 和第三方供应商独立进行。这些合作聚焦于静态评估最薄弱的风险领域：自动化对抗攻击方法、代码与网络滥用安全、心理社会与心理健康危害，以及多语言覆盖。结构化危险能力/能力提升评估 (Phuong et al., 2024; Li et al., 2024b) 不在本次发布范围内。

自适应剪枝攻击树 (TAP) 攻击 (Mehrotra et al., 2024) 被揭示为一个鲁棒性缺口。作为回应，我们构建了一条针对性的对抗性数据管线：首先广泛生成现实有害场景，再通过多样化的攻击变换模板集进行扩展，然后使用 TAP 风格的自适应细化，针对当前模型优化生成的提示，直到它们产生可靠的模型特定失败。这一闭环过程将外部红队的发现转化为针对性的修复数据，大幅降低了 TAP 越狱敏感性，并在相同攻击向量上使模型达到与最先进模型相当的水平。

低资源语言框架 (Deng et al., 2024) 被标记为另一个漏洞。用英语可靠拒绝的内容在约鲁巴语、泰卢固语、阿姆哈拉语、缅甸语、高棉语和马来语中被诱导出来。我们的回应是扩展安全训练数据配比，加入多语言对抗性

种子，将高回报的英语攻击模式翻译并重新定向到受影响的语言。这显著缩小了目标语言上英语/非英语之间的差距，尽管长尾低资源语言的鲁棒性仍是持续投入的领域。

6 集群环境

MAI-Thinking-1 需要一个集群环境，既要足够可组合，以支持跨数据、模型、训练、评估和服务组件的快速迭代；又要足够可扩展，以便在前沿规模上测试改进；还要足够可靠，使测得的模型质量提升而非基础设施噪声成为关注焦点。

因此，我们将集群环境视为模型开发的活跃组成部分。在训练中，目标是在保持数值正确性、确定性恢复、高 MFU 和高 goodput 的同时，最大化每个墙钟日的有效 FLOPs。在推理中，目标是在保持质量、长上下文正确性、可预测延迟和部署效率的同时，最大化每秒和每瓦特的有效 token 数。两种场景下都适用相同的系统原则：拓扑很重要，内存移动很重要，静默的正确性失败不可接受，聚合吞吐只有在转化为可靠的模型进展或可靠的推理服务能力时才有用。

本节介绍该环境的三个组成部分。第 6.1 节介绍训练集群如何将原始加速器容量转化为稳定的实验基质。第 6.2 节介绍用于衡量所分配 GPU 是否产生有用学习进展的稳定性、确定性和 goodput 指标。第 6.3 节简要介绍在微软 MAIA-200 硬件 (Microsoft, 2026) 上运行 MAI-Thinking-1 得到的推理结果。

6.1 训练集群

该集群被设计为一个系统级优化问题。所需算力包络由规模定律估算、数据质量目标、token 预算和实际观测到的训练效率推导得出。随后，基础设施规划将该包络转化为数据中心电力与散热、GPU、CPU、NVLink 纵向扩展域、InfiniBand 横向扩展网络、存储、系统软件、诊断、调度、检查点与可观测性等环节中可用的 FLOPs。

规划目标并非仅理论峰值 FLOPs。长时间运行的训练任务会因 MFU 损失、检查点开销、重计算、硬件损耗、维护、验证负载和恢复缓冲区而损失可用容量。因此，我们针对可用训练容量进行优化：即所交付硬件中能够被调度、保持健康、以高 MFU 运行并在故障发生时快速恢复的部分。

MAI-Base-1 在包含 8K 张 GB200 GPU 的单一逻辑集群上完成训练，MAI-Thinking-1 的 RL 爬升则使用 4.6K 张 GB300 运行。通过将训练保持在同一代同构加速器上、处于已知良好的机架健康边界内、在稳定的调度器行为下运行，并贴近可预测的存储路径，这一选择降低了实验方差。更广泛的实验室环境还包括用于开发、验证、对比剖析和下一代系统上线的 H100、GB200 与 GB300 系统，但主训练运行优先保证局部性、拓扑稳定性和运维同构性。

有关我们集群环境的更多细节见附录 L。

6.2 训练稳定性、确定性与有效产出指标

训练稳定性通过集群使任务持续高效推进、从故障中快速恢复并在各重启路径上保持数值正确性的能力来衡量。我们同时追踪显式故障与静默效率损失。显式故障包括崩溃循环、节点故障、InfiniBand 与 NVLink 链路抖动和链路中断、内存溢出错误、Pod 终止、检查点停滞以及人工重新入队。静默效率损失包括 MFU 下降、

重计算、冗长的启动路径、缓慢的 actor 调度、检查点引发的停滞、内存性能退化，以及不会立即导致任务崩溃但会降低有效吞吐量的网络状况。

确定性被视为一等基础设施属性。在前沿规模下，确定性训练不仅是模型代码层面的事，它还取决于完整执行基座（execution substrate）的正确性与稳定性。集群必须消除静默数据损坏（Hochschild et al., 2021; Dixit et al., 2021），保持通信拓扑稳定，防止不健康的设备与链路进入任务，并在检查点与重启边界之间保持浮点归约和累加的顺序。

有效产出（goodput）是首要的生产 KPI。我们将有效产出定义为理想训练时长与实际墙钟时长之比（Jiang et al., 2024; Grattafiori et al., 2024），并将两者差距分解为各类开销。这一框架使失败成本变得显式化。一次失败不仅耗费到重启为止的时间，还可能强制重算自上一个持久检查点以来的内容，引发启动与 actor 调度延迟，扰乱任务放置，阻塞检查点推进，或在恢复后导致 MFU 降低。同理，即使任务仍在运行，MFU 下降也会被当作生产事故处理，因为集群仍在消耗已分配的 GPU 小时数，而产出的训练进展低于预期。

6.2.1 MFU 与有效产出

总体趋势是从可靠性向效率的转变。第一层改进来自减少中断，如崩溃循环、节点故障、链路抖动、重新入队、内存溢出（OOM）错误和检查点停滞。第二层、也更难的一层改进来自消除静默的有效产出损失，如 MFU 下降、重计算、冗长的启动路径、缓慢的进程调度、检查点引发的停滞，以及网络或内存性能退化。在前沿规模下，两层改进都很重要，因为每多一小时开销都会在上千张 GPU 上成倍放大。运营目标是将有效产出视为首要生产 KPI：每种故障模式、运行时变慢、检查点事件和 MFU 回退都有明确的负责人、检测信号、预防路径，以及对可用 FLOPs 的量化影响。

MAI-Base-1 预训练在 8K 张 GPU 上达到了 90.0% 的有效产出，尽管其规模大于此前的预训练任务。总开销降至 51 小时。重计算——即回退到检查点后重新生成先前已计算步骤所花的时间——降至 6.5 小时，仅占总开销的 15%。非推进时间降至 14 小时，占 27%，表明系统在保持存活、避免重复返工以及在无需长时间人工干预的情况下恢复方面有了显著改善。不过，最终运行也清楚暴露了下一个瓶颈：MFU 下降开销成为剩余类别中最大的一项，达 18 小时，占总开销的 35%。这主要来自检查点写入、网络退化、内存压力和硬件健康状况迁移。故障趋势虽有改善，但并未消失。

6.3 推理效率与模型部署

随着推理模型日益重要，推理成本、延迟和服务可扩展性成为部署的关键约束。因此，我们在模型开发的整个过程中将推理效率视为一等目标，在技术栈各层寻找优化机会：模型架构、服务引擎，乃至部署硬件的选择。

为提升每瓦性能，我们在微软的 MAIA-200 硬件上实现 MAI-Thinking-1。与基于 GB200 的部署相比，在相同机架功率预算下，MAIA-200 的 token 生成吞吐量高出 40% 以上。每瓦性能的提升使数据中心电力得到更高效的利用，并支持在更大规模上承载推理负载。

6.4 可持续性与社区优先的 AI 基础设施倡议

对我们而言，AI 的发展必须兼顾社会与环境可持续性，并惠及我们建设与运营基础设施所在的社区。微软承诺实现碳负排放、水正效益、零废弃，并保护生态系统。2025 年，公司在实现全球年用电量 100% 由可再生能源匹配的目标上达成了一项里程碑。

MAI-Thinking-1 主要在微软自有的亚利桑那州凤凰城（Phoenix, AZ）基础设施上完成训练，并在得克萨斯州达拉斯（Dallas, TX）完成后续训练。凤凰城的设施按照 LEED 金级认证标准建设，以表彰其在环境可持续性和能源效率方面的卓越表现。例如，我们这里的备用发电机使用可再生柴油，与传统石油基柴油相比可降低净碳排放。

微软的 Community-First AI Infrastructure 倡议承诺最小化环境影响，并投资于其建设与运营所在社区。该方式以与凤凰城地区公用事业公司、水务部门和环保组织的紧密合作为基础，在降低整体系统需求的同时强化共享基础设施。

微软已承诺投入超过 \$50M 用于扩大公共水资源资产，包括与古德伊尔市（City of Goodyear）合作建设市政蓄水设施，并为惠及社区的供水与污水管道延伸及更广泛的系统升级提供资金。这些投资与大自然保护协会（The Nature Conservancy）以及 Gila River 蓄水系统等合作伙伴共同开展的保护行动相配合，帮助补给地下水并改善流域层面的用水效率。

与此同时，微软通过设在埃斯特雷拉山社区学院（Estrella Mountain Community College）和格兰代尔社区学院（Glendale Community College）的数据中心学院（Datacenter Academy）投资长期人才培养，确保凤凰城基础设施的增长转化为社区持久的经济机遇。

7 结论与未来方向

我们介绍了我们的爬山式优化机器——一种优化流水线每一个组成部分（从数据和基础设施到 RL 配方与评测）的模型开发方法。MAI-Thinking-1 是该机器产出的首个模型：一个 35B active / 1T total 参数的 MoE，其训练不依赖对第三方模型的蒸馏。在 STEM 推理与软件工程任务上，MAI-Thinking-1 位列其参数量级中最强模型之列。

MAI-Thinking-1 是起点而非终点。展望未来，我们计划将爬山式优化扩展到更多模态、更大规模和更精细的能力。AI 的进步不是任何单一模型的产物，而是可以被可靠改进的流水线的产物。我们将继续打磨我们的流水线，并期待分享未来的爬升进展。

参考文献

Amro Abbas, Kushal Tirumala, Dániel Simig, Surya Ganguli, and Ari S. Morcos. Semdedup: Data-efficient learning at web-scale through semantic deduplication, 2023. URL <https://arxiv.org/abs/2303.09540>.

Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In International Conference on Machine Learning, volume 70, pages 22–31. PMLR, 2017.

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alex Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In ICLR, 2026. URL <https://openreview.net/forum?id=RQm2KQTM5r>.

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints, 2023. URL

<https://arxiv.org/abs/2305.13245>.

Chenxin An, Zhihui Xie, Xiaonan Li, Lei Li, Jun Zhang, Shansan Gong, Ming Zhong, Jingjing Xu, Xipeng Qiu, Mingxuan Wang, and Lingpeng Kong. Polaris: A post-training recipe for scaling reinforcement learning on advanced reasoning models, 2025. URL <https://hkunlp.github.io/blog/2025/Polaris>.

L.W. Anderson and D.R. Krathwohl. A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom’s Taxonomy of Educational Objectives. Longman, 2001.

Anthropic. Raising the bar on SWE-bench Verified with Claude 3.5 Sonnet. <https://www.anthropic.com/engineering/swe-bench-sonnet>, January 2025.

Rahul K Arora, Jason Wei, Rebecca Soskin Hicks, Preston Bowman, Joaquin Quiñonero-Candela, Foivos Tsimpourlas, Michael Sharman, Meghan Shah, Andrea Vallone, Alex Beutel, et al. Healthbench: Evaluating large language models towards improved human health. arXiv preprint arXiv:2505.08775, 2025.

Anthony C. Atkinson, Alexander N. Donev, and Randall D. Tobias. Optimum Experimental Designs, with SAS. Oxford University Press, 2007. ISBN 978-0-19-929659-0.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. arXiv preprint arXiv:2108.07732, 2021.

Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. Swe-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. Advances in Neural Information Processing Systems, 38, 2026a.

Ibragim Badertdinov, Maksim Nekrashevich, Anton Shevtsov, and Alexander Golubev. Swe-rebench v2: Language-agnostic swe task collection at scale. arXiv preprint arXiv:2602.23866, 2026b.

Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. arXiv preprint arXiv:2412.15204, 2024.

Bowen Baker, Joost Huizinga, Leo Gao, Zehao Dou, Melody Y Guan, Aleksander Madry, Wojciech Zaremba, Jakub Pachocki, and David Farhi. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation. arXiv preprint arXiv:2503.11926, 2025.

Adrien Barbaresi. Trafilatura: A Web Scraping Library and Command-Line Tool for Text Discovery and Extraction. In Proceedings of the ACL-IJCNLP 2021 System Demonstrations, pages 122–131, 2021. doi: 10.18653/v1/2021.acl-demo.15.

Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer, 2020.

James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Yash Katariya, Chris Leary, Dougal

Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/jax-ml/jax>.

Andrei Z. Broder. On the resemblance and containment of documents. In *Proceedings of compression and complexity of sequences 1997*, pages 21–29. IEEE, 1997.

Brad Calder, Ju Wang, Aaron Ogus, Niranjan Nilakantan, Arild Skjolsvold, Sam McKelvie, Yikang Xu, Shashwat Srivastav, Jiesheng Wu, Huseyin Simitci, et al. Windows azure storage: a highly available cloud storage service with strong consistency. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*, pages 143–157, 2011.

Mayee F. Chen, Tyler Murray, David Heineman, Matt Jordan, Hannaneh Hajishirzi, Christopher Ré, Luca Soldaini, and Kyle Lo. Olmix: A framework for data mixing throughout LM development. *arXiv preprint arXiv:2602.12237*, 2026a. URL <https://arxiv.org/abs/2602.12237>.

Mouxian Chen, Lei Zhang, Yunlong Feng, Xuwu Wang, Wenting Zhao, Ruisheng Cao, Jiayi Yang, Jiawei Chen, Mingze Li, Zeyao Ma, et al. Swe-universe: Scale real-world verifiable environments to millions. *arXiv preprint arXiv:2602.02361*, 2026b.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ip-polito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shrivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayanan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. PaLM: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023. URL <http://jmlr.org/papers/v24/22-1144.html>. *arXiv:2204.02311*.

Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljubei, Mia Glaese, Carlos E. Jimenez, John Yang, et al. Introducing SWE-bench Verified, 2024. URL <https://openai.com/index/introducing-swe-bench-verified/>.

Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, Zhiyuan Liu, Hao Peng, Lei Bai, Wanli Ouyang, Yu Cheng, Bowen Zhou, and Ning Ding. The entropy mechanism of reinforcement learning for reasoning language models, 2025. URL <https://arxiv.org/abs/2505.22617>.

Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Zhao, Xiaodong Sun, Aixin Liu, and Wenfeng Liang. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. In

Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2024.

DeepSeek-AI. Expert Parallelism Load Balancer (EPLB). <https://github.com/deepseek-ai/EPLB>, 2025. GitHub repository.

DeepSeek-AI. DeepSeek-V4: Towards highly efficient million-token context intelligence, 2026. Technical report, <https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro>.

DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du,

R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanbiao Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. Deepseek-v3 technical report, 2025a. URL <https://arxiv.org/abs/2412.19437>.

DeepSeek-AI, Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenhao Xu, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Erhang Li, Fangqi Zhou, Fangyun Lin, Fucong Dai, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Hao Li, Haofen Liang, Haoran Wei, Haowei Zhang, Haowen Luo, Haozhe Ji, Honghui Ding, Hongxuan Tang, Huanqi Cao, Huazuo Gao, Hui Qu, Hui Zeng, Jialiang Huang, Jiashi Li, Jiaxin Xu, Jiewen Hu, Jingchang Chen, Jingting Xiang, Jingyang Yuan, Jingyuan Cheng, Jinhua Zhu, Jun Ran, Junguang Jiang, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong,

Kaige Gao, Kang Guan, Kexin Huang, Kexing Zhou, Kezhao Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Wang, Liang Zhao, Liangsheng Yin, Lihua Guo, Lingxiao Luo, Linwang Ma, Litong Wang, Liyue Zhang, M. S. Di, M. Y. Xu, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Panpan Huang, Peixin Cong, Peiyi Wang, Qiancheng Wang, Qihao Zhu, Qingyang Li, Qinyu Chen, Qiushi Du, Ruiling Xu, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runqiu Yin, Runxin Xu, Ruomeng Shen, Ruoyu Zhang, S. H. Liu, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaofei Cai, Shaoyuan Chen, Shengding Hu, Shengyu Liu, Shiqiang Hu, Shirong Ma, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, Songyang Zhou, Tao Ni, Tao Yun, Tian Pei, Tian Ye, Tianyuan Yue, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjie Pang, Wenjing Luo, Wenjun Gao, Wentao Zhang, Xi Gao, Xiangwen Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaokang Zhang, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xingyou Li, Xinyu Yang, Xinyuan Li, Xu Chen, Xuecheng Su, Xuehai Pan, Xuheng Lin, Xuwei Fu, Y. Q. Wang, Yang Zhang, Yanhong Xu, Yanru Ma, Yao Li, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Qian, Yi Yu, Yichao Zhang, Yifan Ding, Yifan Shi, Yiliang Xiong, Ying He, Ying Zhou, Yinmin Zhong, Yishi Piao, Yisong Wang, Yixiao Chen, Yixuan Tan, Yixuan Wei, Yiyang Ma, Yiyuan Liu, Yonglun Yang, Yongqiang Guo, Yongtong Wu, Yu Wu, Yuan Cheng, Yuan Ou, Yuanfan Xu, Yudian Wang, Yue Gong, Yuhan Wu, Yuheng Zou, Yukun Li, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehua Zhao, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhixian Huang, Zhiyu Wu, Zhuoshu Li, Zhuping Zhang, Zian Xu, Zihao Wang, Zihui Gu, Zijia Zhu, Zilin Li, Zipeng Zhang, Ziwei Xie, Ziyi Gao, Zizheng Pan, Zongqing Yao, Bei Feng, Hui Li, J. L. Cai, Jiaqi Ni, Lei Xu, Meng Li, Ning Tian, R. J. Chen, R. L. Jin, S. S. Li, Shuang Zhou, Tianyu Sun, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xinnan Song, Xinyi Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, Dongjie Ji, Jian Liang, Jianzhong Guo, Jin Chen, Leyi Xia, Miaojun Wang, Mingming Li, Peng Zhang, Ruyi Chen, Shangmian Sun, Shaoqing Wu, Shengfeng Ye, T. Wang, W. L. Xiao, Wei An, Xianzu Wang, Xiaowen Sun, Xiaoxiang

Wang, Ying Tang, Yukun Zha, Zekai Zhang, Zhe Ju, Zhen Zhang, and Zihua Qu. Deepseek-v3.2: Pushing the frontier of open large language models, 2025b. URL <https://arxiv.org/abs/2512.02556>.

Mostafa Dehghani, Josip Djolonga, Basil Mustafa, Piotr Padlewski, Jonathan Heek, Justin Gilmer, Andreas Steiner, Mathilde Caron, Robert Geirhos, Ibrahim Alabdulmohsin, Rodolphe Jenatton, Lucas Beyer, Michael Tschannen, Anurag Arnab, Xiao Wang, Carlos Riquelme, Matthias Minderer, Joan Puigcerver, Utku Evci, Manoj Kumar, Sjoerd van Steenkiste, Gamaleldin F. Elsayed, Aravindh Mahendran, Fisher Yu, Avital Oliver, Fantine Huot, Jasmijn Bastings, Mark Patrick Collier, Alexey Gritsenko, Vighnesh Birodkar, Cristina Vasconcelos, Yi Tay, Thomas Mensink, Alexander Kolesnikov, Filip Pavetić, Dustin Tran, Thomas Kipf, Mario Lučić, Xiaohua Zhai, Daniel Keysers, Jeremiah Harmsen, and Neil Houlsby. Scaling vision transformers to 22 billion parameters, 2023. URL <https://arxiv.org/abs/2302.05442>.

Jasper Dekoninck, Nikola Jovanović, Tim Gehringer, Kári Rognvaldsson, Ivo Petrov, Chenhao Sun, and Martin Vechev. Beyond benchmarks: Matharena as an evaluation platform for mathematics with llms. 2026. URL <https://arxiv.org/abs/2605.00674>.

Xiang Deng, Jef Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Laufer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? CoRR, abs/2509.16941, 2025.

Yue Deng, Wenxuan Zhang, Sinno Jialin Pan, and Lidong Bing. Multilingual jailbreak challenges in large language models, 2024. URL <https://arxiv.org/abs/2310.06474>.

Kaustubh Deshpande, Ved Sirdeshmukh, Johannes Baptist Mols, Lifeng Jin, Ed-Yeremai Hernandez-Cardona, Dean Lee, Jeremy Kritz, Willow E Primack, Summer Yue, and Chen Xing. Multichallenge: A realistic multi-turn conversation evaluation benchmark challenging to frontier llms. In Findings of the Association for Computational Linguistics: ACL 2025, pages 18632–18702, 2025.

Nolan Dey, Daria Soboleva, Faisal Al-Khateeb, Bowen Yang, Ribhu Pathria, Hemant Khachane, Shaheer Muhammad, Zhiming, Chen, Robert Myers, Jacob Robert Steeves, Natalia Vassilieva, Marvin Tom, and Joel Hestness. BTLM-3B-8K: 7b parameter performance in a 3b parameter model, 2023. URL <https://arxiv.org/abs/2309.11568>.

Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. Chain-of-verification reduces hallucination in large language models. arXiv [cs.CL], September 2023. doi: 10.48550/arXiv.2309.11495.

Harish Dattatraya Dixit, Sneha Pendharkar, Matt Beadon, Chris Mason, Tejasvi Chakravarthy, Bharath Muthiah, and Sriram Sankar. Silent data corruptions at scale. arXiv preprint arXiv:2102.11245, 2021. URL <https://arxiv.org/abs/2102.11245>.

Guanting Dong, Keming Lu, Chengpeng Li, Tingyu Xia, Bowen Yu, Chang Zhou, and Jingren Zhou. Selfplay with execution feedback: Improving instruction-following capabilities of large language models. In International Conference on Learning Representations, volume 2025, pages 39286–39313, 2025.

Zhengxiao Du, Aohan Zeng, Yuxiao Dong, and Jie Tang. Understanding emergent abilities of language models from the loss perspective. In Advances in Neural Information Processing Systems, 2024.

Essential AI, :, Andrew Hojel, Michael Pust, Tim Romanski, Yash Vanjani, Ritvik Kapila, Mohit Parmar, Adarsh Chaluvaraju, Alok Tripathy, Anil Thomas, Ashish Tanwer, Darsh J Shah, Ishaan Shah, Karl Stratos, Khoi Nguyen, Kurt Smith, Michael Callahan, Peter Rushton, Philip Monk, Platon Mazarakis, Saad Jamal, Saurabh Srivastava, Somanshu Singla, and Ashish Vaswani. Essential-web v1.0: 24t tokens of organized web data, 2025. URL <https://arxiv.org/abs/2506.14111>.

Dayuan Fu, Shenyu Wu, Yunze Wu, Zerui Peng, Yaxing Huang, Jie Sun, Ji Zeng, Mohan Jiang, Lin Zhang, Yukun Li, et al. davinci-env: Open swe environment synthesis at scale. arXiv preprint arXiv:2603.13023, 2026.

Kanishk Gandhi, Ayush Chakravarthy, Anikait Singh, Nathan Lile, and Noah D Goodman. Cognitive behaviors that enable self-improving reasoners, or, four habits of highly effective stars. arXiv preprint

arXiv:2503.01307, 2025.

Bofei Gao, Yejie Wang, Yibo Miao, Ruoyu Wu, Feifan Song, Longhui Yu, Tianyu Liu, and Baobao Chang. Towards a better initial policy model for scalable long-CoT reinforcement learning. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, Findings of the Association for Computational Linguistics: ACL 2025, pages 7652–7665, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi: 10.18653/v1/2025.findings-acl.397. URL <https://aclanthology.org/2025.findings-acl.397/>.

Tao Ge, Xin Chan, Xiaoyang Wang, Dian Yu, Haitao Mi, and Dong Yu. Scaling synthetic data creation with 1,000,000,000 personas. arXiv [cs.CL], June 2024. doi: 10.48550/arXiv.2406.20094.

Gemma Team. Gemma 2: Improving open language models at a practical size. arXiv preprint arXiv:2408.00118, 2024.

Gemma Team. Gemma 4 model card. https://ai.google.dev/gemma/docs/core/model_card_4, 2026.

Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivi re, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, Ga l Liu, Francesco Visin, Kathleen Kenealy, Lucas Beyer, Xiaohai Zhai, Anton Tsitsulin, Robert Busa-Fekete, Alex Feng, Noveen Sachdeva, Benjamin Coleman, Yi Gao, Basil Mustafa, Iain Barr, Emilio Parisotto, David Tian, Matan Eyal, Colin Cherry, Jan-Thorsten Peter, Danila Sinopalnikov, Surya Bhupatiraju, Rishabh Agarwal, Mehran Kazemi, Dan Malkin, Ravin Kumar, David Vilar, Idan Brusilovsky, Jiaming Luo, Andreas Steiner, Abe Friesen, Abhanshu Sharma, Abheesht Sharma, Adi Mayrav Gilady, Adrian Goedeckemeyer, Alaa Saade, Alex Feng, Alexander Kolesnikov, Alexei Bendebury, Alvin Abdagic, Amit Vadi, Andr s Gy r gy, Andr  Susano Pinto, Anil Das, Ankur Bapna, Antoine Miech, Antoine Yang, Antonia Paterson, Ashish Shenoy, Ayan Chakrabarti, Bilal Piot, Bo Wu, Bobak Shahriari, Bryce Petrini, Charlie Chen, Charline Le Lan, Christopher A. Choquette-Choo, CJ Carey, Cormac Brick, Daniel Deutsch, Danielle Eisenbud, Dee Cattle, Derek Cheng, Dimitris Paparas, Divyashree Shivakumar Sreepathihalli, Doug Reid, Dustin Tran, Dustin Zelle, Eric Noland, Erwin Huizenga, Eugene Kharitonov, Frederick Liu, Gagik Amirkhanyan, Glenn Cameron, Hadi Hashemi, Hanna Klimczak-Pluci nska, Harman Singh, Harsh Mehta, Harshal Tushar Lehri, Hussein Hazimeh, Ian Ballantyne, Idan Szpektor, Ivan Nardini, Jean Pouget-Abadie, Jetha Chan, Joe Stanton, John Wieting, Jonathan Lai, Jordi Orbay, Joseph Fernandez, Josh Newlan, Ju yeong Ji, Jyotinder Singh, Kat Black, Kathy Yu, Kevin Hui, Kiran Vodrahalli, Klaus Greff, Linhai Qiu, Marcella Valentine, Marina Coelho, Marvin Ritter, Matt Hofman, Matthew Watson, Mayank Chaturvedi, Michael Moynihan, Min Ma, Nabila Babar, Natasha Noy, Nathan Byrd, Nick Roy, Nikola Momchev, Nilay Chauhan, Noveen Sachdeva, Oskar Bunyan, Pankil Botarda, Paul Caron, Paul Kishan Rubenstein, Phil Culliton, Philipp Schmid, Pier Giuseppe Sessa, Pingmei Xu, Piotr Stanczyk, Pouya Tafti, Rakesh Shivanna, Renjie Wu, Renke Pan, Reza Rokni, Rob Willoughby, Rohith Vallu, Ryan Mullins, Sammy Jerome, Sara Smoot, Sertan Girgin, Shariq Iqbal, Shashir Reddy, Shruti Sheth, Siim P der, Sijal Bhatnagar, Sindhu Raghuram Panyam, Sivan Eiger, Susan Zhang, Tianqi Liu, Trevor Yacovone, Tyler Liechty, Uday Kalra, Utku Evci, Vedant Misra, Vincent Roseberry, Vlad Feinberg, Vlad Kolesnikov, Woohyun Han, Woosuk Kwon, Xi Chen, Yinlam Chow, Yuvein Zhu, Zichuan Wei,

Zoltan

Egyed, Victor Cotruta, Minh Giang, Phoebe Kirk, Anand Rao, Kat Black, Nabila Babar, Jessica Lo, Erica Moreira, Luiz Gustavo Martins, Omar Sanseviero, Lucas Gonzalez, Zach Gleicher, Tris Warkentin, Vahab Mirrokni, Evan Senter, Eli Collins, Joelle Barral, Zoubin Ghahramani, Raia Hadsell, Yossi Matias, D. Sculley, Slav Petrov, Noah Fiedel, Noam Shazeer, Oriol Vinyals, Jef Dean, Demis Hassabis, Koray Kavukcuoglu, Clement Farabet, Elena Buchatskaya, Jean-Baptiste Alayrac, Rohan Anil, Dmitry, Lepikhin, Sebastian Borgeaud, Olivier Bachem, Armand Joulin, Alek Andreev, Cassidy Hardin, Robert Dadashi, and Léonard Hussenot. Gemma 3 technical report, 2025. URL <https://arxiv.org/abs/2503.19786>.

Nathan Godey, Wissam Antoun, Rian Touchent, Rachel Bawden, Éric de la Clergerie, Benoît Sagot, and Djamé Seddah. Gaperon: A peppered English-French generative language model suite. arXiv preprint arXiv:2510.25771, 2025. doi: 10.48550/arXiv.2510.25771. URL <https://arxiv.org/abs/2510.25771>.

Aaron Gokaslan and Vanya Cohen. Openwebtext corpus. <https://skylion007.github.io/OpenWebTextCorpus>, 2019.

Richard L Graham, Lion Levi, Devendar Burreddy, Gil Bloch, Gilad Shainer, David Cho, George Elias, Daniel Klein, Joshua Ladd, Ophir Maor, et al. Scalable hierarchical aggregation and reduction protocol (sharp) tm streaming-aggregation hardware design and evaluation. In International Conference on High Performance Computing, pages 41–59. Springer, 2020.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The Llama 3 herd of models. arXiv preprint arXiv:2407.21783, 2024. URL <https://arxiv.org/abs/2407.21783>.

Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, et al. A survey on llm-as-a-judge. The Innovation, 2024.

Melody Y Guan, Miles Wang, Micah Carroll, Zehao Dou, Annie Y Wei, Marcus Williams, Benjamin Arnav, Joost Huizinga, Ian Kivlichan, Mia Glaese, et al. Monitoring monitorability. arXiv preprint arXiv:2512.18311, 2025.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen,

Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu,

Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, September 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-09422-z. URL <http://dx.doi.org/10.1038/s41586-025-09422-z>.

Lukas Haas, Gal Yona, Giovanni D’Antonio, Sasha Goldshtein, and Dipanjan Das. SimpleQA Verified: A reliable factuality benchmark to measure parametric knowledge, 2026. URL <https://arxiv.org/abs/2509.07968>.

Bingguang Hao, Zengzhuang Xu, Maolin Wang, Yuntao Wen, Yicheng Chen, Cunyin Peng, Long Chen, Dong Wang, Xiangyu Zhao, Jinjie Gu, Chenyi Zhuang, and Ji Zhang. FunReason: Enhancing large language models’ function calling via self-refinement multiscale loss and automated data refinement. *arXiv [cs.LG]*, May 2025. doi: 10.48550/arXiv.2505.20192.

Yun He, Wenzhe Li, Hejia Zhang, Songlin Li, Karishma Mandyam, Sopan Khosla, Yuanhao Xiong, Nanshu Wang, Xiaoliang Peng, Beibin Li, et al. Advancedif: Rubric-based benchmarking and reinforcement learning for advancing llm instruction following. *arXiv preprint arXiv:2511.10507*, 2025.

David Heineman, Valentin Hofmann, Ian Magnusson, Yuling Gu, Noah Smith, Hanna Hajishirzi, Kyle Lo, and Jesse Dodge. Signal and noise: A framework for reducing uncertainty in language model evaluation. *arXiv preprint arXiv:2508.13144*, 2025.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

Alex Henry, Prudhvi Raj Dachapally, Shubham Shantaram Pawar, and Yuxuan

Chen. Query-key normalization for transformers. *CoRR*, abs/2010.04245, 2020. URL <https://arxiv.org/abs/2010.04245>.

Rebecca Soskin Hicks, Mikhail Trofimov, Dominick Lim, Rahul K Arora, Foivos Tsimpourlas, Preston Bow-

man, Michael Sharman, Chi Tong, Kavin Karthik, Arnav Dugar, et al. Healthbench professional: Evaluating large language models on real clinician chats. arXiv preprint arXiv:2604.27470, 2026.

Peter H. Hochschild, Paul Turner, Jeffrey C. Mogul, Rama Govindaraju, Parthasarathy Ranganathan, David E. Culler, and Amin Vahdat. Cores that don’t count. In Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS ’21), pages 9–16, Ann Arbor, MI, USA, 2021. ACM. doi: 10.1145/3458336.3465297.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, DDL Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. arXiv preprint arXiv:2203.15556, 10, 2022.

Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. arXiv preprint arXiv:1904.09751, 2019.

Jian Hu, Xibin Wu, Wei Shen, Jason Klein Liu, Zilin Zhu, Weixun Wang, Songlin Jiang, Haoran Wang, Hao Chen, Bin Chen, Weikai Fang, Xianyu, Yu Cao, Haotian Xu, and Yiming Liu. OpenRLHF: An easy-to-use, scalable and high-performance RLHF framework. arXiv preprint arXiv:2405.11143, 2024. URL <https://arxiv.org/abs/2405.11143>.

Jiaxin Huang, Shixiang Shane Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu, and Jiawei Han. Large language models can self-improve, 2022. URL <https://arxiv.org/abs/2210.11610>.

Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. GPipe: Efficient training of giant neural networks using pipeline parallelism, 2019.

Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. Gpt-4o system card. arXiv preprint arXiv:2410.21276, 2024.

Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. DeepSpeed Ulysses: System optimizations for enabling training of extreme long sequence transformer models, 2023. URL <https://arxiv.org/abs/2309.14509>.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. arXiv preprint arXiv:2403.07974, 2024.

Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, Yulu Jia, Sun He, Hongmin Chen, Zhihao Bai, Qi Hou, Shipeng Yan, Ding Zhou, Yiyao Sheng, Zhuo Jiang, Haohan Xu, Haoran Wei, Zhang Zhang, Pengfei Nie, Leqi Zou, Sida Zhao, Liang Xiang, Zherui Liu, Zhe Li, Xiaoying Jia, Jianxi Ye, Xin Jin, and Xin Liu. MegaScale: Scaling large language model training to more than 10,000 GPUs. In 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24). USENIX Association, 2024. URL <https://www.usenix.org/conference/nsdi24/presentation/jiang-ziheng>. arXiv:2402.15627.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In International Conference on Learning Representations, volume 2024, pages 54107–54157, 2024.

Armand Joulin, Edouard Grave, Piotr Bojanowski, Matthijs Douze, H erve J egou, and Tomas Mikolov. Fast-Text.zip: Compressing text classification models. arXiv preprint arXiv:1612.03651, 2016.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. arXiv preprint arXiv:2001.08361, 2020.

Amirhossein Kazemnejad, Inkit Padhi, Karthikeyan Natesan Ramamurthy, Payel Das, and Siva Reddy. The impact of positional encoding on length generalization in transformers, 2023. URL <https://arxiv.org/abs/2305.19466>.

Kimi Team, Yifan Bai, Yiping Bao, Y. Charles, Cheng Chen, Guanduo Chen, Haiting Chen, Huarong Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, Zhuofu Chen, Jialei Cui, Hao Ding, Mengnan Dong, Angang Du, Chenzhuang Du, Dikang Du, Yulun Du, Yu Fan, Yichen Feng, Kelin Fu, Bofei Gao, Chenxiao Gao, Hongcheng Gao, Peizhong Gao, Tong Gao, Yuyao Ge, Shangyi Geng, Qizheng Gu, Xinran Gu, Longyu Guan, Haiqing Guo, Jianhang Guo, Xiaoru Hao, Tianhong He, Weiran He, Wenyang He, Yunjia He, Chao Hong, Hao Hu, Yangyang Hu, Zhenxing Hu, Weixiao Huang, Zhiqi Huang, Zihao Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yongsheng Kang, Guokun Lai, Cheng Li, Fang Li, Haoyang Li, Ming Li, Wentao Li, Yang Li, Yanhao Li, Yiwei Li, Zhaowei Li, Zheming Li, Hongzhan Lin, Xiaohan Lin, Zongyu Lin, Chengyin Liu, Chenyu Liu, Hongzhang Liu, Jingyuan Liu, Junqi Liu, Liang Liu, Shaowei Liu, T. Y. Liu, Tianwei Liu, Weizhou Liu, Yangyang Liu, Yibo Liu, Yiping Liu, Yue Liu, Zhengying Liu, Enzhe Lu, Haoyu Lu, Lijun Lu, Yashuo Luo, Shengling Ma, Xinyu Ma, Yingwei Ma, Shaoguang Mao, Jie Mei, Xin Men, Yibo Miao, Siyuan Pan, Yebo Peng, Ruoyu Qin, Zeyu Qin, Bowen Qu, Zeyu Shang,

Lidong Shi, Shengyuan Shi, Feifan Song, Jianlin Su, Zhengyuan Su, Lin Sui, Xinjie Sun, Flood Sung, Yunpeng Tai, Heyi Tang, Jiawen Tao, Qifeng Teng, Chaoran Tian, Chensi Wang, Dinglu Wang, Feng Wang, Hailong Wang, Haiming Wang, Jianzhou Wang, Jiaying Wang, Jinhong Wang, Shengjie Wang, Shuyi Wang, Si Wang, Xinyuan Wang, Yao Wang, Yejie Wang, Yiqin Wang, Yuxin Wang, Yuzhi Wang, Zhaoji Wang, Zhengtao Wang, Zhengtao Wang, Zhexu Wang, Chu Wei, Qianqian Wei, Haoning Wu, Wenhao Wu, Xingzhe Wu, Yuxin Wu, Chenjun Xiao, Jin Xie, Xiaotong Xie, Weimin Xiong, Boyu Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinran Xu, Yangchuan Xu, Ziyao Xu, Jing Xu, Jing Xu, Junjie Yan, Yuzi Yan, Hao Yang, Xiaofei Yang, Yi Yang, Ying Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Xingcheng Yao, Wenjie Ye, Zhuorui Ye, Bohong Yin, Longhui Yu, Enming Yuan, Hongbang Yuan, Mengjie Yuan, Siyu Yuan, Haobing Zhan, Dehao Zhang, Hao Zhang, Wanlu Zhang, Xiaobin Zhang, Yadong Zhang, Yangkun Zhang, Yichi Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Haotian Zhao, Yikai Zhao, Zijia Zhao, Huabin Zheng, Shaojie Zheng, Longguang Zhong, Jianren Zhou, Xinyu Zhou, Zaida Zhou, Jinguo Zhu, Zhen Zhu, Weiyu Zhuang, and Xinxing Zu. Kimi K2: Open agentic intelligence, 2026. URL <https://arxiv.org/abs/2507.20534>.

- Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems*, 5:341–353, 2023.
- Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, et al. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.
- Nathan Lambert, Valentina Pyatkin, Jacob Morrison, LJ Miranda, Bill Yuchen Lin, Khyathi Chandu, Nouha Dziri, Sachin Kumar, Tom Zick, Yejin Choi, et al. Rewardbench: Evaluating reward models for language modeling. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 1755–1797, 2025.
- Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. Deduplicating training data makes language models better. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8424–8445, 2022.
- Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding, 2020. URL <https://arxiv.org/abs/2006.16668>.
- Junyi Li, Charith Peris, Ninareh Mehrabi, Palash Goyal, Kai-Wei Chang, Aram Galstyan, Richard Zemel, and Rahul Gupta. The steerability of large language models toward data-driven personas. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7290–7305, 2024a.
- Nathaniel Li, Alexander Pan, Anjali Gopal, Summer Yue, Daniel Berrios, Alice Gatti, Justin D. Li, Ann-Kathrin Dombrowski, Shashwat Goel, et al. The WMDP benchmark: Measuring and reducing malicious use with unlearning. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024b. URL <https://arxiv.org/abs/2403.03218>.
- Wanchao Liang, Tianyu Liu, Less Wright, Will Constable, Andrew Gu, Chien-Chin Huang, Iris Zhang, Wei Feng, Howard Huang, Junjie Wang, et al. TorchTitan: One-stop pytorch native solution for production ready llm pre-training. *arXiv preprint arXiv:2410.06511*, 2024.
- Stephanie Lin, Jacob Hilton, and Owain Evans. Truthfulqa: Measuring how models mimic human falsehoods, 2022. URL <https://arxiv.org/pdf/2109.07958>.
- Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.
- Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, Yanru Chen, Huabin Zheng, Yibo Liu, Shaowei Liu, Bohong Yin, Weiran He, Han Zhu, Yuzhi Wang, Jianzhou Wang, Mengnan Dong, Zheng Zhang, Yongsheng Kang, Hao Zhang, Xinran Xu,

Yutao Zhang, Yuxin Wu, Xinyu Zhou, and Zhilin Yang. Muon is scalable for LLM training, 2025a. URL <https://arxiv.org/abs/2502.16982>.

Qian Liu, Xiaosen Zheng, Niklas Muennighoff, Guangtao Zeng, Longxu Dou, Tianyu Pang, Jing Jiang, and Min Lin. Regmix: Data mixture as regression for language model pre-training. In International Conference on Learning Representations, volume 2025, pages 38305–38339, 2025b.

Tianqi Liu, Wei Xiong, Jie Ren, Lichang Chen, Junru Wu, Rishabh Joshi, Yang Gao, Jiaming Shen, Zhen Qin, Tianhe Yu, Daniel Sohn, Anastasiia Makarova, Jeremiah Liu, Yuan Liu, Bilal Piot, Abe Ittycheriah, Aviral Kumar, and Mohammad Saleh. Rrm: Robust reward model training mitigates reward hacking, 2025c. URL <https://arxiv.org/abs/2409.13156>.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101, 2017.

Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abulkhanov, Indraneil Paul, Zhuang Li, Wen-Ding Li, Megan Risdal, Jia Li, Jian Zhu, Terry Yue Zhuo, Evgenii Zheltonozhskii, Nii Osae Osae Dade, Wenhao Yu, Lucas Krauß, Naman Jain, Yixuan Su, Xuanli He, Manan Dey, Edoardo Abati, Yekun Chai, Niklas Muennighoff, Xiangru Tang, Muhtasham Oblokulov, Christopher Akiki, Marc Marone, Chenghao Mou, Mayank Mishra, Alex Gu, Binyuan Hui, Tri Dao, Armel Zebaze, Olivier Dehaene, Nicolas Patry, Canwen Xu, Julian McAuley, Han Hu, Torsten Scholak, Sebastien Paquet, Jennifer Robinson, Carolyn Jane Anderson, Nicolas Chapados, Mostofa Patwary, Nima Tajbakhsh, Yacine Jernite, Carlos Muñoz Ferrandis, Lingming Zhang, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. Star-Coder 2 and The Stack v2: The next generation, 2024. URL <https://arxiv.org/abs/2402.19173>.

Zhiyuan Lu, Chenliang Li, Yingcheng Shi, Weizhou Shen, Ming Yan, and Fei Huang. CorpusQA: A 10 million token benchmark for corpus-level analysis and reasoning. 2025.

Wenhan Ma, Hailin Zhang, Liang Zhao, Yifan Song, Yudong Wang, Zhifang Sui, and Fuli Luo. Stabilizing MoE reinforcement learning by aligning training and inference routers, 2025. URL <https://arxiv.org/abs/2510.11370>.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. arXiv [cs.CL], March 2023. doi: 10.48550/arXiv.2303.17651.

Rabeeh Karimi Mahabadi, Sanjeev Satheesh, Shrimai Prabhumoye, Mostofa Patwary, Mohammad Shoeybi, and Bryan Catanzaro. Nemotron-CC-Math: A 133 billion-token-scale high quality math pretraining dataset, 2025. URL <https://arxiv.org/abs/2508.15096>.

Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, et al. Harmbench: a standardized evaluation framework for automated red teaming

and robust refusal. In Proceedings of the 41st International Conference on Machine Learning, pages 35181–35224, 2024.

MediaWiki. Wikitext 1.0.0 Specification. <https://www.mediawiki.org/wiki/Specs/wikitext/1.0.0>, 2026. Accessed: 2026-05-31.

Anay Mehrotra, Manolis Zampetakis, Paul Kassianik, Blaine Nelson, Hyrum Anderson, Yaron Singer, and Amin Karbasi. Tree of attacks: Jailbreaking black-box llms automatically, 2024. URL <https://arxiv.org/abs/2312.02119>.

Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL <https://arxiv.org/abs/2601.11868>.

Meta. PurpleLlama CyberSecEval 4. <https://github.com/meta-llama/PurpleLlama>, 2024.

Aaron Meurer, Christopher P. Smith, Mateusz Paprocki, Ondřej Čertík, Sergey B. Kirpichev, Matthew Rocklin, Amit Kumar, Sergiu Ivanov, Jason K. Moore, Sartaj Singh, Thilina Rathnayake, Sean Vig, Brian E. Granger, Richard P. Muller, Francesco Bonazzi, Harsh Gupta, Shivam Vats, Fredrik Johansson, Fabian Pedregosa, Matthew J. Curry, Andy R. Terrel, Štěpán Roučka, Ashutosh Saboo, Isuru Fernando, Sumith Kulal, Robert Cimrman, and Anthony Scopatz. SymPy: symbolic computing in python. *PeerJ Computer Science*, 3:e103, January 2017. ISSN 2376-5992. doi: 10.7717/peerj-cs.103. URL <https://doi.org/10.7717/peerj-cs.103>.

Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, et al. Mixed precision training. *arXiv preprint arXiv:1710.03740*, 2017.

Paulius Micikevicius, Dusan Stolic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisen-thwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, Naveen Mellempudi, Stuart Oberman, Mohammad Shoeybi, Michael Siu, and Hao Wu. Fp8 formats for deep learning, 2022. URL <https://arxiv.org/abs/2209.05433>.

Microsoft. Microsoft responsible AI standard, v2. Technical report, Microsoft Corporation, 2022. URL <https://go.microsoft.com/fwlink/?linkid=2311742>. Accessed: 2026-06-01.

Microsoft. Maia 200: The AI accelerator built for inference. <https://blogs.microsoft.com/blog/2026/01/26/maia-200-the-ai-accelerator-built-for-inference/>, 2026.

Mika. General agent: A self-evolving, synthetic agent environment. <https://www.primeintellect.ai/blog/general-agent>, May 2026. Prime Intellect Blog.

Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. Factscore: Fine-grained atomic evaluation of factual precision in long form text generation, 2023. URL <https://arxiv.org/pdf/2305.14251>.

Mistral-AI, :, Abhinav Rastogi, Albert Q. Jiang, Andy Lo, Gabrielle Berrada, Guillaume Lample, Jason Rute, Joep Barmentlo, Karmesh Yadav, Kartik Khandelwal, Khyathi Raghavi Chandu, Léonard Blier, Lucile Saulnier, Matthieu Dinot, Maxime Darrin, Neha Gupta, Roman Soletskyi, Sagar Vaze, Teven Le Scao, Yihan Wang, Adam Yang, Alexander H. Liu, Alexandre Sablayrolles, Amélie Héliou, Amélie Martin, Andy Ehrenberg, Anmol Agarwal, Antoine Roux, Arthur Darcet, Arthur Mensch, Baptiste Bout, Baptiste Rozière, Baudouin De Monicault, Chris Bamford, Christian Wallenwein, Christophe Renaudin, Clémence Lanfranchi, Darius Dabert, Devon Mizelle, Diego de las Casas, Elliot Chane-Sane, Emilien Fugier, Emma Bou Hanna, Gauthier Delerce, Gauthier Guinet, Georgii Novikov, Guillaume Martin, Himanshu Jaju, Jan Ludziejewski, Jean-Hadrien Chabran, Jean-Malo Delignon, Joachim Studnia, Jonas Amar, Joselin Somerville Roberts, Julien Denize, Karan Saxena, Kush Jain, Lingxiao Zhao, Louis Martin, Luyu Gao, Léo Renard Lavaud, Marie Pellat, Mathilde Guillaumin, Mathis Felardos, Maximilian Augustin, Mickaël Seznec, Nikhil Raghuraman, Olivier Duchenne, Patricia Wang, Patrick von Platen, Patryk Saffer, Paul Jacob, Paul Wambergue, Paula Kurylowicz, Pavankumar Reddy Muddireddy, Philomène Chagniot, Pierre Stock, Praveesh Agrawal, Romain Sauvestre, Rémi Delacourt, Sanchit Gandhi, Sandeep Subramanian, Shashwat Dalal, Siddharth Gandhi, Soham Ghosh, Srijan Mishra, Sumukh Aithal, Szymon Antoniak, Thibault Schueller, Thibaut Lavril, Thomas Robert, Thomas Wang, Timothée Lacroix, Valeriia Nemychnikova, Victor Paltz, Virgile Richard, Wen-Ding Li, William Marshall, Xuanyu Zhang, and Yunhao Tang. Magistral, 2025. URL <https://arxiv.org/abs/2506.10910>.

Model Context Protocol. What is the Model Context Protocol (MCP)?, 2026. URL <https://modelcontextprotocol.io/docs/started/intro>.

Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI), pages 561–577, Carlsbad, CA, USA, 2018.

Gary D. Lopez Munoz, Amanda J. Minnich, Roman Lutz, Richard Lundeen, Raja Sekhar Rao Dheekonda, Nina Chikanov, Bolor-Erdene Jagdagdorj, Martin Pouliot, Shiven Chawla, Whitney Maxwell, Blake Bullwinkel, Katherine Pratt, Joris de Gruyter, Charlotte Siska, Pete Bryan, Tori Westerhof, Chang Kawaguchi, Christian Seifert, Ram Shankar Siva Kumar, and Yonatan Zunger. Pyrit: A framework for security risk identification and red teaming in generative ai system, 2024. URL <https://arxiv.org/abs/2410.02828>.

Muse Spark Team. Muse Spark Eval Methodology. <https://ai.meta.com/static-resource/muse-spark-eval>

methodology, 2026.

NVIDIA, :, Aaron Blakeman, Aaron Grattafiori, Aarti Basant, Abhibha Gupta, Abhinav Khattar, Adi Renduchintala, Aditya Vavre, Akanksha Shukla, Akhiad Bercovich, Aleksander Ficek, Aleksandr Shaposhnikov, Alex Kondratenko, Alexander Bukharin, Alexandre Milesi, Ali Taghibakhshi, Alisa Liu, Amelia Barton, Ameya Sunil Mahabaleshwarkar, Amir Klein, Amit Zuker, Amnon Geifman, Amy Shen, Anahita Bhiwandiwalla, Andrew Tao, Anjolie Agrusa, Ankur Verma, Ann Guan, Anubhav Mandarwal, Arham Mehta, Ashwath Aithal, Ashwin Poojary, Asif Ahamed, Asit Mishra, Asma Kuriparambil Thekkumpate, Ayush Dattagupta, Banghua Zhu, Bardiya Sadeghi, Barnaby Simkin, Ben Lanir, Benedikt Schiferer, Besmira Nushi, Bilal Kartal, Bitu Darvish Rouhani, Boris Ginsburg, Brandon Norick, Brandon Soubasis, Branislav Kisacanin, Brian Yu, Bryan Catanzaro, Carlo del Mundo, Chantal Hwang, Charles Wang, Cheng-Ping Hsieh, Chenghao Zhang, Chenhan Yu, Chetan Mungekar, Chintan Patel, Chris Alexiuk, Christopher Parisien, Collin Neale, Cyril Meurillon, Damon Mosk-Aoyama, Dan Su, Dane Corneil, Daniel Afrimi, Daniel Lo, Daniel Rohrer, Daniel Serebrenik, Daria Gitman, Daria Levy, Darko Stosic, David Mosallanezhad, Deepak Narayanan, Dhruv Nathawani, Dima Rekesh, Dina Yared, Divyanshu Kakwani, Dong Ahn, Duncan Riach, Dusan Stosic, Edgar Minasyan, Edward Lin, Eileen Long, Eileen Peters Long, Elad

Segal, Elena Lantz, Ellie Evans, Elliott Ning, Eric Chung, Eric Harper, Eric Tramel, Erick Galinkin, Erik Pounds, Evan Briones, Evelina Bakhturina, Evgeny Tsykunov, Faisal Ladhak, Fay Wang, Fei Jia, Felipe Soares, Feng Chen, Ferenc Galko, Frank Sun, Frankie Siino, Gal Hubara Agam, Ganesh Ajjanagadde, Gantavya Bhatt, Gargi Prasad, George Armstrong, Gerald Shen, Gorkem Batmaz, Grigor Nalbandyan, Haifeng Qian, Harsh Sharma, Hayley Ross, Helen Ngo, Herbert Hum, Herman Sahota, Hexin Wang, Himanshu Soni, Hiren Upadhyay, Huizi Mao, Huy C Nguyen, Huy Q Nguyen, Iain Cunningham, Ido Galil, Ido Shahaf, Igor Gitman, Ilya Loshchilov, Itamar Schen, Itay Levy, Ivan Moshkov, Izik Golan, Izzy Putterman, Jan Kautz, Jane Polak Scowcroft, Jared Casper, Jatin Mitra, Jeffrey Glick, Jenny Chen, Jesse Oliver, Jian Zhang, Jiaqi Zeng, Jie Lou, Jimmy Zhang, Jinhang Choi, Jining Huang, Joey Conway, Joey Guman, John Kamalu, Johnny Greco, Jonathan Cohen, Joseph Jennings, Joyjit Daw, Julien Veron Vialard, Junkeun Yi, Jupinder Parmar, Kai Xu, Kan Zhu, Kari Briski, Katherine Cheung, Katherine Luna, Keith Wyss, Keshav Santhanam, Kevin Shih, Kezhi Kong, Khushi Bhardwaj, Kirthi Shankar, Krishna C. Puvvada, Krzysztof Pawelec, Kumar Anik, Lawrence McAfee, Laya Sleiman, Leon Derczynski, Li Ding, Lizzie Wei, Lucas Liebenwein, Luis Vega, Maanu Grover, Maarten Van Segbroeck, Maer Rodrigues de Melo, Mahdi Nazemi, Makesh Narsimhan Sreedhar, Manoj Kilaru, Maor Ashkenazi, Marc Romeijn, Marcin Chochowski, Mark Cai, Markus Kliegl, Maryam Moosaei, Matt Kulka, Matvei Novikov, Mehrzad Samadi, Melissa Corpuz, Mengru Wang, Meredith Price, Michael Andersch, Michael Boone, Michael Evans, Miguel Martinez, Mikail Khona, Mike Chrzanowski, Minseok Lee, Mohammad Dabbah, Mohammad Shoeybi, Mostofa Patwary, Nabin Mulepati, Najeeb Nabwani, Natalie Hereth, Nave Assaf, Negar Habibi, Neta Zmora, Netanel Haber, Nicola Sessions, Nidhi Bhatia, Nikhil Jukar, Nikki Pope, Nikolai Ludwig, Nima Tajbakhsh, Nir Ailon, Nirmal Juluru, Nishant Sharma, Oleksii Hrinchuk, Oleksii Kuchaiev, Olivier Delalleau, Oluwatobi Olabiyi, Omer Ullman Argov, Omri Puny, Oren Tropp, Ouyue Xie, Parth Chadha, Pasha Shamis, Paul Gibbons, Pavlo Molchanov, Pawel Morkisz, Peter Dykas, Peter Jin, Pinky Xu, Piotr Januszewski, Pranav Prashant Thombre, Prasoon Varshney, Pritam Gundecha, Przemek Tredak, Qing Miao, Qiyu Wan, Rabeeh Karimi Mahabadi, Rachit Garg, Ran El-Yaniv, Ran Zilber-

stein, Rasoul Shafipour, Rich Harang, Rick Izzo, Rima Shahbazyan, Rishabh Garg, Ritika Borkar, Ritu Gala, Riyad Islam, Robert Hesse, Roger Waleffe, Rohit Watve, Roi Koren, Ruoxi Zhang, Russell Hewett, Russell J. Hewett, Ryan Prenger, Ryan Timbrook, Sadegh Mahdavi, Sahil Modi, Samuel Krizan, Sangkug Lim, Sanjay Kariyappa, Sanjeev Satheesh, Saori Kaji, Satish Pasumarthi, Saurav Muralidharan, Sean Narentharen, Sean Narenthiran, Seonmyeong Bak, Sergey Kashirsky, Seth Poulos, Shahar Mor, Shanmugam Ramasamy, Shantanu Acharya, Shaona Ghosh, Sharath Turuvekere Sreenivas, Shelby Thomas, Shiqing Fan, Shreya Gopal, Shrimai Prabhumoye, Shubham Pachori, Shubham Toshniwal, Shuoyang Ding, Siddharth Singh, Simeng Sun, Smita Ithape, Somsubhra Majumdar, Soumye Singhal, Stas Sergienko, Stefania Alborghetti, Stephen Ge, Sugam Dipak Devare, Sumeet Kumar Barua, Suseella Panguluri, Suyog Gupta, Sweta Priyadarshi, Syeda Nahida Akter, Tan Bui, Teodor-Dumitru Ene, Terry Kong, Thanh Do, Tijmen Blankevoort, Tim Moon, Tom Balough, Tomer Asida, Tomer Bar Natan, Tomer Ronen, Tugrul Konuk, Twinkle Vashishth, Udi Karpas, Ushnish De, Vahid Noorozi, Vahid Noroozi, Venkat Srinivasan, Venmugil Elango, Victor Cui, Vijay Korthikanti, Vinay Rao, Vitaly Kurin, Vitaly Lavrukhin, Vladimir Anisimov, Wanli Jiang, Wasi Uddin Ahmad, Wei Du, Wei Ping, Wenfei Zhou, Will Jennings, William Zhang, Wojciech Prazuch, Xiaowei Ren, Yashaswi Karnati, Yejin Choi, Yev Meyer, Yi-Fu Wu, Yian Zhang, Yigong Qin, Ying Lin, Yonatan Geifman, Yonggan Fu, Yoshi Subara, Yoshi Suhara, Yubo Gao, Zach Moshe, Zhen Dong, Zhongbo Zhu, Zihan Liu, Zijia Chen, and Zijie Yan. NVIDIA Nemotron 3: Efficient and open intelligence, 2025. URL <https://arxiv.org/abs/2512.20856>.

OpenAI. tiktoken: A fast bpe tokeniser for use with openai’s models, 2022. URL <https://github.com/openai/tiktoken>.

OpenAI. gpt-oss-120b & gpt-oss-20b model card. CoRR, abs/2508.10925, 2025. doi: 10.48550/ARXIV.2508.10925. URL <https://doi.org/10.48550/arXiv.2508.10925>.

OpenAI. simple-evals: Lightweight library for evaluating language models. <https://github.com/openai/simple-evals>, 2025. GitHub repository. Accessed: 2026-05-31.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Gray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022. URL <https://openreview.net/forum?id=TG8KACxEON>.

Kaan Ozkara, Tao Yu, and Youngsuk Park. Stochastic rounding for llm training: Theory and practice, 2025. URL <https://arxiv.org/abs/2502.20566>.

Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with swe-gym. *arXiv preprint arXiv:2412.21139*, 2024.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large

language models. In Proceedings of Forty-second International Conference on Machine Learning, 2025.

Hao Peng, Yunjia Qi, Xiaozhi Wang, Bin Xu, Lei Hou, and Juanzi Li. Verif: Verification engineering for reinforcement learning in instruction following. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pages 30312–30327, 2025.

Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, Hugh

Zhang, Chen Bo Calvin Zhang, Mohamed Shaaban, John Ling, Sean Shi, et al. Humanity’s last exam. arXiv preprint arXiv:2501.14249, 2025.

Mary Phuong, Matthew Aitchison, Elliot Catt, Sarah Cogan, Alexandre Kaskasoli, Victoria Krakovna, David Lindner, Matthew Rahtz, Yannis Assael, et al. Evaluating frontier models for dangerous capabilities, 2024. URL <https://arxiv.org/abs/2403.13793>.

Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tiny-Benchmarks: evaluating LLMs with fewer examples, 2024. URL <https://arxiv.org/abs/2402.14992>.

Fabrice Prigent. UT1 URL Blacklist, 2026. URL <http://dsi.ut-capitole.fr/blacklists/>. Accessed 2026.

Valentina Pyatkin, Saumya Malik, Victoria Graf, Hamish Ivison, Shengyi Huang, Pradeep Dasigi, Nathan Lambert, and Hanna Hajishirzi. Generalizing verifiable instruction following. Advances in Neural Information Processing Systems, 38, 2026.

Penghui Qi, Xinyi Wan, Guangxing Huang, and Min Lin. Zero bubble pipeline parallelism, 2023.

Zihan Qiu, Zeyu Huang, Bo Zheng, Kaiyue Wen, Zekun Wang, Rui Men, Ivan Titov, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Demons in the detail: On implementing load balancing loss for training specialized mixture-of-expert models, 2025. URL <https://arxiv.org/abs/2501.11873>.

Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. OpenAI, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf. Accessed: 2024-11-15.

Jack W. Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, Eliza Rutherford, Tom Hennigan, Jacob Menick, Albin Cassirer, Richard Powell, George van den Driessche, Lisa Anne Hendricks, Maribeth Rauh, Po-Sen Huang, Amelia Glaese, Johannes Welbl, Sumanth Dathathri, Saffron Huang, Jonathan Uesato, John Mellor, Irina Higgins, Antonia Creswell, Nat McAleese, Amy Wu, Erich Elsen, Siddhant Jayakumar,

Elena Buchatskaya, David Budden, Esme Sutherland, Karen Simonyan, Michela Paganini, Laurent Sifre, Lena Martens, Xiang Lorraine Li, Adhiguna Kuncoro, Aida Nematzadeh, Elena Gribovskaya, Domenic Donato, Angeliki Lazaridou, Arthur Mensch, Jean-Baptiste Lespiau, Maria Tsimpoukelli, Nikolai Grigorev, Doug Fritz, Thibault Sottiaux, Mantas Pajarskas, Toby Pohlen, Zhitao Gong, Daniel Toyama, Cyprien de Masson d’Autume, Yujia Li, Tayfun Terzi, Vladimir Mikulik, Igor Babuschkin, Aidan Clark, Diego de Las Casas, Aurelia Guy, Chris Jones, James Bradbury, Matthew Johnson, Blake Hechtman, Laura Weidinger, Iason Gabriel, William Isaac, Ed Lockhart, Simon Osindero, Laura Rimell, Chris Dyer, Oriol Vinyals, Kareem Ayoub, Jeff Stanway, Lorrayne Bennett, Demis Hassabis, Koray Kavukcuoglu, and Geoffrey Irving. Scaling language models: Methods, analysis & insights from training gopher, 2022. URL <https://arxiv.org/abs/2112.11446>.

Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In SC20: international conference for high performance computing, networking, storage and analysis, pages 1–16. IEEE, 2020.

Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining, pages 3505–3506, 2020.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. GPQA: A graduate-level google-proof q&a benchmark. arXiv preprint arXiv:2311.12022, 2023.

Leonard Richardson. Beautiful soup documentation. <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>, 2026. Beautiful Soup version 4.14.3; accessed 31 May 2026.

Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers, 2020. URL <https://arxiv.org/abs/2003.05997>.

Mark Russinovich, Ahmed Salem, and Ronen Eldan. Great, now write an article about that: The crescendo multi-turn LLM jailbreak attack, 2024. URL <https://arxiv.org/abs/2404.01833>.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL <https://arxiv.org/abs/1707.06347>.

Gian Segato. Quantifying infrastructure noise in agentic coding evals. Engineering at Anthropic. <https://www.anthropic.com/engineering/infrastructure-noise>, February 2026.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.

Noam Shazeer. Glu variants improve transformer. arXiv preprint arXiv:2002.05202, 2020.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. arXiv preprint arXiv:2409.19256, 2024.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. Openai gpt-5 system card. arXiv preprint arXiv:2601.03267, 2025a.

Avi Singh, John D Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J Liu, James Harrison, Jaehoon Lee, Kelvin Xu, et al. Beyond human data: Scaling self-training for problem-solving with language models. arXiv preprint arXiv:2312.06585, 2023.

Shivalika Singh, Angelika Romanou, Clémentine Fourrier, David Ifeoluwa Adelani, Jian Gang Ngui, Daniel Vila-Suero, Peerat Limkonchotiwat, Kelly Marchisio, Wei Qi Leong, Yosephine Susanto, et al. Global mmlu: Understanding and addressing cultural and linguistic biases in multilingual evaluation. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 18761–18799, 2025b.

Joar Skalse, Lewis Hammond, Charlie Griffin, and Alessandro Abate. Lexicographic multi-objective reinforcement learning. In International Joint Conference on Artificial Intelligence, pages 3430–3436, 2022.

Shaden Smith, Mostofa Patwary, Brandon Norick, Patrick LeGresley, Samyam Rajbhandari, Jared Casper, Zhun Liu, Shrimai Prabhumoye, George Zerveas, Vijay Korthikanti, Elton Zhang, Rewon Child, Reza Yazdani Aminabadi, Julie Bernauer, Xia Song, Mohammad Shoeybi, Yuxiong He, Michael Houston, Saurabh Tiwary, and Bryan Catanzaro. Using DeepSpeed and Megatron to train Megatron-Turing NLG 530B, a large-scale generative language model, 2022. URL <https://arxiv.org/abs/2201.11990>.

Atharv Sonwane, Isadora White, Hyunji Lee, Matheus Pereira, Lucas Caccia, Minseon Kim, Zhengyan Shi, Chinmay Singh, Alessandro Sordoni, Marc-Alexandre Côté, et al. Bugpilot: Complex bug generation for efficient learning of swe skills. arXiv preprint arXiv:2510.19898, 2025.

Ben Sorscher, Robert Geirhos, Shashank Shekhar, Surya Ganguli, and Ari S. Morcos. Beyond neural scaling laws: beating power law scaling via data pruning, 2023. URL <https://arxiv.org/abs/2206.14486>.

Alexandra Souly, Qingyuan Lu, Dillon Bowen, Tu Trinh, Elvis Hsieh, Sana Pandey, Pieter Abbeel, Justin Svegliato, Scott Emmons, Olivia Watkins, et al. A strongreject for empty jailbreaks. *Advances in Neural Information Processing Systems*, 37:125416–125440, 2024.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.*, 15(1):1929–1958, January 2014. ISSN 1532-4435.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023. URL <https://arxiv.org/abs/2104.09864>.

Michael Sullivan, Mareike Hartmann, and Alexander Koller. Procedural environment generation for tooluse

agents. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pages 18555–18573, 2025.

Sijun Tan, Siyuan Zhuang, Kyle Montgomery, William Tang, Alejandro Cuadron, Chenguang Wang, Raluca Popa, and Ion Stoica. Judgebench: A benchmark for evaluating llm-based judges. In International Conference on Learning Representations, volume 2025, pages 63277–63303, 2025.

Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, Nathan Lambert, Dustin Schwenk, Oyvind Tafjord, Taira Anderson, David Atkinson, Faeze Brahman, Christopher Clark, Pradeep Dasigi, Nouha Dziri, Michal Guerquin, Hamish Ivison, Pang Wei Koh, Jiacheng Liu, Saumya Malik, William Merrill, Lester James V. Miranda, Jacob Morrison, Tyler Murray, Crystal Nam, Valentina Pyatkin, Aman Rangapur, Michael Schmitz, Sam Skjonsberg, David Wadden, Christopher Wilhelm, Michael Wilson, Luke Zettlemoyer, Ali Farhadi, Noah A. Smith, and Hannaneh Hajishirzi. 2 OLMo 2 Furious. 2024. URL <https://arxiv.org/abs/2501.00656>.

Kushal Tirumala, Daniel Simig, Armen Aghajanyan, and Ari S. Morcos. D4: Improving LLM pretraining via document de-duplication and diversification, 2023. URL <https://arxiv.org/abs/2308.12284>.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, Advances in Neural Information Processing Systems, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c0530000000-Paper.pdf.

Kiran Vodrahalli, Santiago Ontanon, Nilesh Tripuraneni, Kelvin Xu, Sanil Jain, Rakesh Shivanna, Jeffrey Hui, Nishanth Dikkala, Mehran Kazemi, Bahare Fatemi, Rohan Anil, Ethan Dyer, Siamak Shakeri, Roopali Vij, Harsh Mehta, Vinay Ramasesh, Quoc Le, Ed Chi, Yifeng Lu, Orhan Firat, Angeliki Lazaridou, Jean-Baptiste Lespiau, Nithya Attaluri, and Kate Olszewska. Michelangelo: Long context evaluations beyond haystacks via latent structure queries, 2024. URL <https://arxiv.org/abs/2409.12640>.

Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformers Reinforcement Learning, 2020. URL <https://github.com/huggingface/trl>.

Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training llms to prioritize privileged instructions. arXiv preprint arXiv:2404.13208, 2024.

Shengye Wan, Cyrus Nikolaidis, Daniel Song, David Molnar, James Crnkovich, Jayson Grace, Manish Bhatt, Sahana Chennabasappa, Spencer Whitman, Stephanie Ding, Vlad Ionescu, Yue Li, and Joshua Saxe. Cyberseceval 3: Advancing the evaluation of cybersecurity risks and capabilities in large language models, 2024. URL <https://arxiv.org/abs/2408.01605>.

Junhao Wang, Daoguang Zan, Shulin Xin, Siyao Liu, Yurong Wu, and Kai Shen. Swe-mirror: Scaling issue-resolving datasets by mirroring issues across repositories. arXiv preprint arXiv:2509.08724, 2025a.

Lean Wang, Huazuo Gao, Chenggang Zhao, Xu Sun, and Damai Dai. Auxiliary-loss-free load balancing strategy for mixture-of-experts, 2024a. URL <https://arxiv.org/abs/2408.15664>.

Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhramil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, et al. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. *Advances in Neural Information Processing Systems*, 37:95266–95290, 2024b.

Zengzhi Wang, Fan Zhou, Xuefeng Li, and Pengfei Liu. Octothinker: Mid-training incentivizes reinforcement learning scaling. *arXiv preprint arXiv:2506.20512*, 2025b. URL <https://arxiv.org/abs/2506.20512>.

Zhaoyang Wang, Canwen Xu, Boyi Liu, Yite Wang, Siwei Han, Zhewei Yao, Huaxiu Yao, and Yuxiong He. Agent world model: Infinity synthetic environments for agentic reinforcement learning. *arXiv preprint arXiv:2602.10090*, 2026.

Zihao Wang, Chirag Nagpal, Jonathan Berant, Jacob Eisenstein, Alex D’Amour, Sanmi Koyejo, and Victor Veitch. Transforming and combining rewards for aligning large language models. *arXiv preprint arXiv:2402.00742*, 2024c.

Jason Wei, Nguyen Karina, Hyung Won Chung, Yunxin Joy Jiao, Spencer Papay, Amelia Glaese, John Schulman, and William Fedus. Measuring short-form factuality in large language models. *arXiv preprint arXiv:2411.04368*, 2024a.

Jason Wei, Nguyen Karina, Hyung Won Chung, Yunxin Joy Jiao, Spencer Papay, Amelia Glaese, John Schulman, and William Fedus. Measuring short-form factuality in large language models, 2024b. URL <https://arxiv.org/pdf/2411.04368>.

Jerry Wei, Chengrun Yang, Xinying Song, Yifeng Lu, Nathan Hu, Jie Huang, Dustin Tran, Daiyi Peng, Ruibo Liu, Da Huang, Cosmo Du, and Quoc V. Le. Long-form factuality in large language models, 2024c. URL <https://arxiv.org/pdf/2403.18802>.

Bosi Wen, Pei Ke, Xiaotao Gu, Lindong Wu, Hao Huang, Jinfeng Zhou, Wenchuang Li, Binxin Hu, Wendy Gao, Jiaxin Xu, et al. Benchmarking complex instruction-following with multiple constraints composition. *Advances in Neural Information Processing Systems*, 37:137610–137645, 2024.

Mitchell Wortsman, Peter J. Liu, Lechao Xiao, Katie Everett, Alex Alemi, Ben Adlam, John D. Co-Reyes, Izzeddin Gur, Abhishek Kumar, Roman Novak, Jeffrey Pennington, Jascha Sohl-dickstein, Kelvin Xu, Jaehoon Lee, Justin Gilmer, and Simon Kornblith. Small-scale proxies for large-scale transformer training instabilities, 2023. URL <https://arxiv.org/abs/2309.14322>.

Zhiheng Xi, Xin Guo, Yang Nan, Enyu Zhou, Junrui Shen, Wenxiang Chen, Jiaqi Liu, Jixuan Huang, Zhihao Zhang, Honglin Guo, Xun Deng, Zhikai Lei, Miao Zheng, Guoteng Wang, Shuo Zhang, Peng Sun, Rui Zheng, Hang Yan, Tao Gui, Qi Zhang, and Xuanjing Huang. BAPO: Stabilizing off-policy reinforcement learning for llms via balanced policy optimization with adaptive clipping, 2025. URL <https://arxiv.org/abs/2510.18927>.

Violet Xiang, Chase Blagden, Rafael Rafailov, Nathan Lile, Sang Truong, Chelsea Finn, and Nick Haber. Just enough thinking: Efficient reasoning with adaptive length penalties reinforcement learning, 2025. URL

<https://arxiv.org/abs/2506.05256>.

Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, Qingwei Lin, and Daxin Jiang. WizardLM: Empowering large pre-trained language models to follow complex instructions, 2025a. URL <https://arxiv.org/abs/2304.12244>.

Zengzhuang Xu, Bingguang Hao, Zechuan Wang, Yuntao Wen, Xinyi Xu, Yang Liu, Long Chen, Dong Wang, Maolin Wang, Tong Zhao, Yicheng Chen, Cunyin Peng, Jinjie Gu, Leilei Gan, Xiangyu Zhao, Chenyi Zhuang, and Shi Gu. Funreason-mt technical report: Advanced data synthesis solution for real-world multi-turn tool-use, 2025b. URL <https://arxiv.org/abs/2510.24645>.

Zijie Yan, Hongxiao Bai, Xin Yao, Dennis Liu, Tong Liu, Hongbin Liu, Pingtian Li, Evan Wu, Shiqing Fan, Li Tao, et al. Scalable training of mixture-of-experts models with megatron core. arXiv preprint arXiv:2603.07685, 2026.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.

John Yang, Kilian Lieret, Carlos Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. Swe-smith: Scaling data for software engineering agents. *Advances in Neural Information Processing Systems*, 38, 2026.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

Deheng Ye, Zhao Liu, Mingfei Sun, Bei Shi, Peilin Zhao, Hao Wu, Hongsheng Yu, Shaojie Yang, Xipeng Wu, Qingwei Guo, Qiaobo Chen, Yinyuting Yin, Hao Zhang, Tengfei Shi, Liang Wang, Qiang Fu, Wei Yang, and Lanxiao Huang. Mastering complex control in moba games with deep reinforcement learning, 2020. URL <https://arxiv.org/abs/1912.09729>.

Jiasheng Ye, Peiju Liu, Tianxiang Sun, Yunhua Zhou, Jun Zhan, and Xipeng Qiu. Data Mixing Laws: Optimizing data mixtures by predicting language modeling performance. In *International Conference on Learning Representations*, 2025. doi: 10.48550/arXiv.2403.16952. URL <https://arxiv.org/abs/2403.16952>.

Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao,

- Yonghui Wu, and Mingxuan Wang. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL <https://arxiv.org/abs/2503.14476>.
- Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, et al. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pages 9556–9567, 2024.
- Xiang Yue, Tianyu Zheng, Yuansheng Ni, Yubo Wang, Kai Zhang, Shengbang Tong, Yuxuan Sun, Botao Yu, Ge Zhang, Huan Sun, et al. Mmmu-pro: A more robust multi-discipline multimodal understanding benchmark. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 15134–15186, 2025.
- Ted Zadouri, Markus Hoehnerbach, Jay Shah, Timmy Liu, Vijay Thakkar, and Tri Dao. Flashattention-4: Algorithm and kernel pipelining co-design for asymmetric hardware scaling, 2026. URL <https://arxiv.org/abs/2603.05451>.
- Pedram Zamirai, Jian Zhang, Christopher R. Aberger, and Christopher De Sa. Revisiting bfloat16 training. CoRR, abs/2010.06192, 2020. URL <https://arxiv.org/abs/2010.06192>.
- Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. Star: Bootstrapping reasoning with reasoning, 2022. URL <https://arxiv.org/abs/2203.14465>.
- Yi Zeng, Hongpeng Lin, Jingwen Zhang, Diyi Yang, Ruoxi Jia, and Weiyan Shi. How Johnny can persuade LLMs to jailbreak them: Rethinking persuasion to challenge AI safety by humanizing llms, 2024a. URL <https://arxiv.org/abs/2401.06373>.
- Yi Zeng, Yu Yang, Andy Zhou, Jeffrey Ziwei Tan, Yuheng Tu, Yifan Mai, Kevin Klyman, Minzhou Pan, Ruoxi Jia, Dawn Song, Percy Liang, and Bo Li. AIR-Bench 2024: A safety benchmark based on risk categories from regulations and policies, 2024b. URL <https://arxiv.org/abs/2407.17436>.
- Biao Zhang and Rico Sennrich. Root mean square layer normalization, 2019. URL <https://arxiv.org/abs/1910.07467>.
- Hanning Zhang, Shizhe Diao, Yong Lin, Yi Fung, Qing Lian, Xingyao Wang, Yangyi Chen, Heng Ji, and Tong Zhang. R-tuning: Instructing large language models to say ‘i don’t know’. In Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), pages 7113–7139, 2024.
- Puzhen Zhang, Weijie Bai, Wendong Fan, and Guohao Li. Toolathlon-GYM: Large-Scale Long-Horizon Environments for Tool-Use Agents, 2026. URL https://github.com/eigent-ai/toolathlon_gym.
- Chenggang Zhao, Shangyan Zhou, Liyue Zhang, Chengqi Deng, Zhean Xu, Yuxuan Liu, Kuai Yu, Jiashi Li, and Liang Zhao. DeepEP: an efficient expert-parallel communication library. <https://github.com/deepseek-ai/DeepEP>, 2025a.
- Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Selfdistilled reasoner: On-policy self-distillation for large language models, 2026. URL <https://arxiv.org/abs/2601.18734>.

Yulai Zhao, Haolin Liu, Dian Yu, Sunyuan Kung, Meijia Chen, Haitao Mi, and Dong Yu. One token to fool llm-as-a-judge. arXiv preprint arXiv:2507.08794, 2025b.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: efficient execution of structured language model programs. In Proceedings of the 38th International Conference on Neural Information Processing Systems, NIPS '24, Red Hook, NY, USA, 2024. Curran Associates Inc. ISBN 9798331314385.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. arXiv preprint arXiv:2311.07911, 2023.

Yuxin Zuo, Shang Qu, Yifei Li, Zhangren Chen, Xuekai Zhu, Ermo Hua, Kaiyan Zhang, Ning Ding, and Bowen Zhou. Medxpertqa: Benchmarking expert-level medical reasoning and understanding. arXiv preprint arXiv:2501.18362, 2025.

8 引用格式

引用本工作时，请使用以下 *BIBTEX* 条目：

```
@techreport{mai_thinking_1,  
title={MAI-Thinking-1: Building a Hill-Climbing Machine},  
author={The Microsoft AI Team},  
institution={Microsoft AI},  
year={2026},  
urldate={2026-06-02},  
url={https://microsoft.ai/pdf/mai-thinking-1.pdf}  
}
```

9 预训练数据管线细节

本附录进一步介绍第 2.4 节所述主要预训练数据源的处理细节。

9.1 网页 HTML

我们的网页 HTML 语料大部分来自内部抓取。在完成初始的页面发现与筛选后，我们抓取并解析了约 1.2 万亿个页面。HTML 使用 Trafilatura (Barbaresi, 2021) 转换为文本，输出抽取出的文本以及语言标识和相应的置信度分数。在抽取过程中，我们推断文档编码，而不是在处理前假定其为 UTF-8。这种方式在处理带有旧式、缺失或错误声明编码的多语言网页数据时更具鲁棒性。

解析完成后，所有页面都要经过政策合规、成人内容、与网页 PDF 语料重叠以及域名黑名单等过滤。除第 2.4 节所述的微软标准政策外，我们还应用 UT1 黑名单 (Prigent, 2026) 来移除成人内容与盗版相关域名。总体而言，这些过滤将语料从 1.2 万亿页缩减至 7940 亿页。鉴于网络上 AI 生成内容日益普遍，我们还使用内部 AI 内容检测模型对页面评分，并通过人工检查识别包含大量 AI 生成内容的域名；这些域名将从训练语料中过滤掉。

过滤后的网页语料随后进行精确去重与模糊去重。我们先通过转小写和去除首尾空白来规范化文档，然后使用 MD5 哈希移除完全重复项。这一步将语料从 7940 亿缩减至 4230 亿篇文档。我们的模糊去重采用第 2.4.3 节所述的 LSH 方法与参数。模糊去重后，内部抓取语料包含 734 亿篇英文文档和 1165 亿篇非英文文档。

我们对 Common Crawl 使用相同的管线。从约 3000 亿个页面出发，我们为每个 URL 保留最新抓取的内容，将语料缩减至约 1000 亿个页面。经过过滤、去重、与内部网页语料合并，以及最后一轮基于精确 URL 和内容级的模糊去重后，Common Crawl 部分包含 242 亿个页面。

在下游处理中，我们使用 Qwen3-Embedding-0.6B (Yang et al., 2025) 为所有网页计算文本嵌入。得到的语料随后通过面向通用网页、STEM 页面、代码页面和关键领域的专属管线进行处理。

通用网页。对于通用网页，我们结合过滤与质量分箱，以便对这一大规模语料进行更细粒度的数据配比。首先，我们将属性模型应用于每篇文档的文本嵌入。这些属性模型对与预训练相关的属性进行评分，如教育价值、事实准确性与可靠性、写作质量、信息密度、推理内容以及通识知识价值。在这些属性之上，我们训练一个质量模型，并过滤掉底部 70% 的英文文档。随后应用 Gopher 风格 (Rae et al., 2022) 的启发式过滤器，其基于词统计、标点模式、常见 n-gram 及相关文档质量特征。这一步从内部抓取语料中保留约 46 亿篇英文文档，从英文 Common Crawl 中保留 28 亿篇。为便于数据配比，这些文档结合元数据与基于嵌入的质量评分进行分箱。多语言网页数据的处理方式与英文通用网页数据类似，但采用语言感知的过滤与质量评分。

除广泛的通用网页管线外，我们还构建了一个高质量的人工策展网页内容子集，以更好地捕捉引人入胜、文笔上乘的长文内容。我们从人工策展渠道（如杂志和长文阅读聚合站）以及社交平台上的外部引用链接获取候选域名，并从网页语料中取回相应页面。

STEM 页面。预训练阶段对高质量 STEM 内容的广泛覆盖，对于下游的数学与科学推理能力至关重要。然而，这类内容在网上相对稀少，且通常包含公式、代码片段和技术标记等结构化元素，通用网页处理管线难以很好地处理这些内容。因此，我们构建了一条专用管线来识别并抽取网页中的 STEM 内容。

我们使用基于嵌入的分类器从三个维度识别候选 STEM 页面：主题、教育价值和教育水平。主题由七个二分类器建模，分别对应数学、物理、统计、化学、生物、工程和计算机科学。一个共享的教育价值分类器用于将教育类内容（如研究论文、Q&A 论坛、教程、讲义和博客）与低价值页面（如产品列表、引文索引、商业内容和模板化页面）区分开。最后，一个教育水平分类器将每篇文档划分为高中、本科或研究生水平，用于数据配比。为提高语料质量，我们还应用了一套针对 STEM 内容定制的启发式规则，因为通用网页过滤器往往会过度惩罚包含公式和数值表达式的文档。这些启发式规则会移除 PDF URL、计算器域名、专利域名、已知 AI 生成内容域名以及被判定为低质量的域名，并根据词数、数字占比、字母占比、重复行比例、停用词密度以及以省略号结尾的行占比来过滤文档。

由于 Trafilatura 并不总能很好地保留数学公式和代码，我们为 STEM 文档构建了定制的抽取管线。从原始 HTML 出发，我们对 MathML 和 LATEX 内容进行规范化，并将文档转换为 Markdown。随后将文档切分为多个小节，每个公式和代码块作为独立单元保留，并使用语言模型将每个小节分类为保留或移除。该模型仅限于二元取舍决策，不能生成合成内容。最后，我们使用 OpenWebMath 风格的后处理 (Gokaslan and Cohen, 2019) 对抽取出的内容进行清洗和规范化，使公式的定界保持一致。

该管线产出一个 680B tokens 的英文 STEM 语料，其中 76B 与数学相关。对于多语言数据，我们额外获得 760B 的 STEM tokens，其中包含 58B 高质量且与数学相关的 tokens。

代码页面。对于代码类网页，我们复用 STEM 页面管线，包括主题分类、教育价值分类和基于 LLM 的抽取，并聚焦于计算机科学主题进行过滤。为进一步提升质量，我们使用 Qwen3-30B (Yang et al., 2025) 对每篇候选文档进行评分。评判提示 (judge prompt) 使用 GEPA / DSPy (Agrawal et al., 2026) 配合约 2,000 个人工标注进行优化。在通过额外启发式规则过滤掉低质量文档后，我们获得约 233B tokens 的数据集。

关键领域。对于选定的高价值领域，我们使用改进的抽取管线，以更好地保留领域特有的结构与内容。例如，对于 Wikipedia，我们在收录上宁偏宽松，避免在标准抽取可能丢弃重要页面元素时遗漏有用信息。

9.2 网页 PDF

我们收集了一个网络抓取的 PDF 语料，约含 10B 篇文档，涵盖教育、生活、技术和专业文档。基于从 PDF 字节中抽取出的文本，我们使用启发式规则和分类器将其过滤至 620M 篇文档，供进一步处理。为将文档转换为可训练文本，我们使用 Azure Document Intelligence (Azure DI)。

我们对 OCR 输出应用若干后处理步骤：

- 公式与表格规范化
- 移除模板化内容，如图眉、页脚和图中检测到的文本
- 移除 OCR 伪影，如字形错误
- 行展开：去除换行符和连字符
- 移除学术文档中的参考文献部分

我们使用文本级分类器和 PDF 元数据启发式规则来移除 SEO 和垃圾内容。特别是，PDF 元数据字段 “creator” 和 “producer” 是过滤时高精度的有效信号。我们进一步使用文本长度、gzip 可压缩性、字母数字占比和表格占比等启发式规则以及质量分类器对文档进行过滤。

经过模糊去重后，PDF 数据集包含 1.8T 英文 tokens 和 1.85T 多语言 tokens。为支持数据配比，我们将文档分类为教育类或非教育类。教育类文档进一步划分为数学、计算机科学、STEM 和非 STEM，并指定为高中、本科和研究生中的某一教育水平。

9.3 图书与期刊

我们从多家供应商处获取图书和期刊，包括通过与出版商的直接协议。针对每家供应商，我们构建一条专用的摄取管线，以适配其底层数据格式，并与内容附带的用途限制保持一致。每条管线将原始内容转换为规范的文本格式。标准化后的文本随后经过 OCR 伪影清洗、启发式过滤和内容筛选。清洗后的文本经过精确去重、标题去重和模糊去重，以减少不必要的重复。最后，使用 LLM 进行标注，为语料中的每篇文档赋予主题和质量标签，以评估每部作品对 LLM 预训练的用处，并更好地控制数据配比。

9.4 公开 GitHub

我们 7.4T-token 的代码预训练语料大部分来自公开 GitHub 仓库，并组织为三个数据集：files、commits 和 pull requests (PRs)。三个数据集均经过共享的过滤、去重、去污染和质量评分管线。

共享处理。我们首先应用启发式文件级过滤器，以移除非代码和低质量数据。这些过滤器包括：

- 移除 `node_modules`、`build`、`__pycache__` 和 `.vscode` 等垃圾文件夹中的文件。
- 通过路径模式检测生成的代码，如 `_pb2.py` 之类的 `protobuf` 输出以及 `.d.ts` 之类的 `TypeScript` 声明文件。
- 排除非代码文件类型，如二进制文件、图像、压缩包、文档和字体。
- 拒绝超过 30K 字符或包含过长行的文件。
- 字符构成检查，用于过滤掉数据转储。

随后我们通过 `SHA-512` 哈希进行精确去重，再进行基于 `MinHashLSH` 的模糊去重，以及使用 `Qwen3-Embedding-0.6B` (Yang et al., 2025) 的 1024 维嵌入上的余弦相似度进行语义去重。我们还会针对推理训练和评测中使用的编程题进行去污染。最后，每个样本被赋予一个质量分数，并放入离散的质量分箱，以支持质量感知采样。

在完成各数据集特有的处理后，每个数据集中的样本被贪婪地打包为定长序列，`files`、`commits` 和 `PRs` 分别产生约 1.26T、4.5T 和 1.19T tokens。

Files。原始 `files` 数据集包含所有仓库中每个文件的最新版本。我们使用与 `StarCoder2` (Lozhkov et al., 2024) 中所述类似的过滤器进行额外的启发式过滤，并特别丢弃 `token` 数量较大的仓库中的非编程文件。文件按仓库分组，按仓库结构的深度优先遍历顺序排序，并拼接为仓库级序列。

Commits。原始 `commits` 数据集包含从每个仓库抽取的最新的 10K 条提交。每一行包含所有被修改文件的提交前状态以及该提交所应用的补丁。作为某条 `PR` 组成部分的提交会被移除，以避免跨数据集冗余。被修改文件的提交前状态用作 `loss-masked` 前缀，而可训练文本仅由提交补丁组成，以 `git-diff` 或 `search-and-replace` 格式呈现。

Pull requests。原始 `PR` 数据集通过从克隆的仓库中抽取所有组成提交并为每条 `PR` 关联相关元数据而构建。每一行包含 `PR` 标题、正文、关联的 `issue`、`issue` 评论、`reviews`、`threads`、`review` 评论、提交前文件状态以及按顺序排列的组成提交列表。`PR` 数据集还额外针对 `SWE-bench Verified` 进行去污染，移除基准测试中使用的所有 `PR`。被修改文件的提交前状态用作 `loss-masked` 前缀；可训练文本包含其余字段，包括 `PR` 标题、正文、关联的 `issue`、`reviews`、评论以及各组成提交的补丁。当元数据可映射到特定提交（如 `review` 评论和 `threads`）时，它会被紧跟在相应提交补丁之后放置，而非放在 `PR` 级别。我们还对过长的前缀应用前缀压缩，随机选择每条 `git` 补丁位置周围的行，以适应固定的 `token` 预算。压缩后仍超过 256K tokens 的序列将被丢弃。

10 长上下文扩展

我们的预训练目标是 256K token 的上下文窗口。我们没有在整个预训练或中期训练阶段都采用完整的 256K 上下文长度——由于长序列下 MFU 较低，这种做法并不实际——而是采用了分阶段的方法：模型在 16K 上下文下完成预训练，在 64K 上下文下完成中期训练（阶段 1），随后在一个短暂的专用阶段（阶段 2）扩展到 256K。本节介绍支撑这一配方的消融实验与评测。除非另有说明，所有消融实验均在小规模模型上进行。

10.1 数据

我们针对上下文扩展阶段尝试了多种数据混合变体，包括提升长上下文文档的权重以及调整各领域间的混合比例。最终，这些修改都没有带来有意义的改进。最简单的做法——沿用前一中期训练阶段的数据混合，并按目标 256K 序列长度重新打包——效果很好，这也是我们在最终模型中所采用的方案。

10.2 评测设置

我们在多个长上下文评测基准上评估质量，其中部分介绍如下。

Code NLL。我们将一个内部代码仓库序列化为线性 token 流，并提取 256K-token 的块。对每个块，末尾的 16K token 作为我们计算 NLL 的 suffix，同时我们从紧邻的前序 token 开始逐步扩展 prefix，使总上下文 (prefix + suffix) 为 16K、32K、64K、128K 或 256K。理想的长上下文模型应能利用额外的 prefix——其中包含来自同一仓库的相关代码——来进一步降低 suffix 的损失。如图 22(b) 所示，上下文扩展后的 checkpoint 恰好表现出这一行为：其 NLL 从 16K 到 256K 单调下降，证明模型有效利用了完整的上下文窗口。相比之下，在较短上下文长度下中期训练且未做扩展的模型，一旦上下文长度超过训练分布范围的 $> 2x$ ，其 NLL 就会上升。

Retrieval NLL。为了检验模型无论相关信息位于上下文中的哪个位置都能检索并利用，我们设计了一个受控实验：选取一份 32K-token 的文档（来自后训练的每日 standup 笔记），将其切分为两个 16K-token 的块，即 A (suffix) 和 B (prefix)，其中 prefix B 为预测 suffix A 提供了有用的上下文（即 $NLL(A | B) \ll NLL(A)$ ）。然后我们在 A 和 B 之间逐步插入无关块（与预训练相关的设计文档），将总上下文长度增加到 256K token。如果没有专门的长上下文训练，随着相关 prefix 被推得越来越远，模型的 NLL 会大幅恶化，表明模型越来越容易被中间插入的无关内容干扰（图 22(a)）。相比之下，长上下文 checkpoint 无论相关块之间距离多远，都能保持近乎平坦的 NLL，证明了在完整 256K 上下文窗口内的稳健信息检索能力。

Generative QA。我们在内部仓库文档上构建了一个简单的问答基准，将问题放置在上下文中的不同位置，并测量答案准确率随总上下文长度和证据位置的变化。在该基准上，模型表现出高达 $4\times$ 的长度外推能力：在 32K 上下文中训练的模型可以在最长 128K token 的上下文中正确回答问题（图 22(d)）。我们观察到，随着总上下文长度超出训练分布，模型在从近期（上下文末尾）位置而非较远位置检索信息时会遇到不成比例的困难（图 22(e)）。这可以用分布偏移来解释——当用 64K 输入评测一个在 32K 下训练的模型时，前 32K token 落在训练分布之内，而末尾的 token 则占据了模型从未训练过的位置。长上下文训练解决了这种不对称性。

10.3 渐进式上下文长度扩展

一个自然的问题是：不在我们期望的最大上下文长度上进行中期训练，会牺牲多少质量？图 22(b) 通过在 code NLL 上比较五种配置回答了这个问题：四次中期训练分别采用 16K、32K、64K 和 128K 上下文长度，各训练 1T token（均基于 16K 预训练），外加一种渐进式方案——先在 32K 下中期训练 1T token，再用约 100B 额外 token 扩展到 256K。

在 128K 上下文下，渐进式 checkpoint 的 NLL 与完整 1T-token 128K 中期训练的结果持平，尽管它在长上下文长度上使用的 token 数要少得多。此外，所有五种配置在短上下文长度（16K 和 32K）下都收敛到大致相同的 NLL，这证实了在中期训练阶段使用更长的上下文长度训练并不会显著影响短上下文质量。这意味着，

如果末尾一个简短的扩展阶段就能达到同样的效果，就无需为昂贵的长上下文中期训练买单。该图支持一条通用的配方：在较短、对 MFU 友好的上下文长度上进行中期训练，并在最后进行扩展。

10.4 适应速度

图 22(c) 展示了某次消融实验在上下文扩展阶段 256K code NLL 的变化轨迹。适应的速度非常快：绝大部分 NLL 的改善发生在训练迭代的前 1–10% 内，此后曲线趋于平稳。这种快速收敛表明，模型在上下文扩展期间并非在获取全新的能力，而是在为早期训练中未出现过的序列长度校准其位置编码与注意力机制。利用长上下文所需的表示在预训练和中期训练中已经具备；扩展阶段只是针对分布外的位置对它们进行调整。

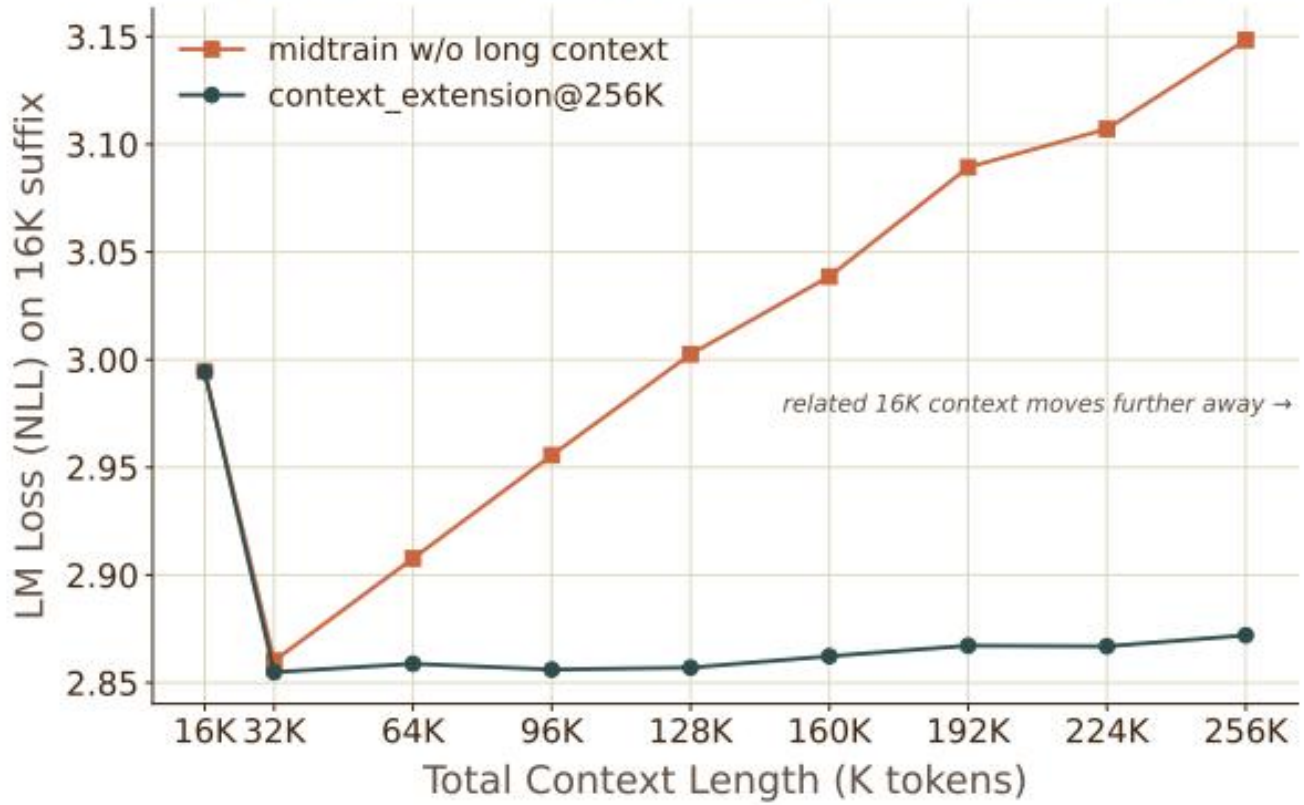
10.5 最终配方

对于最终模型，我们选择了 64K 中期训练，随后用 140B token 进行 256K 扩展。虽然消融分析（主要基于 NLL 指标）表明，更短的中期训练上下文长度和更短的扩展 token 预算就足够了，但上下文长度与后训练之间的相互作用尚未被充分刻画，因此我们选择了更保守的方案。总体而言，上下文扩展阶段相对于训练的其他部分仍然快速且廉价。展望未来，同样的分阶段方法可以自然地扩展到更长的上下文窗口——这个简短的最终扩展阶段可以用适度的计算成本，将目标从 256K 提升到 1M token 甚至更长。

(b)

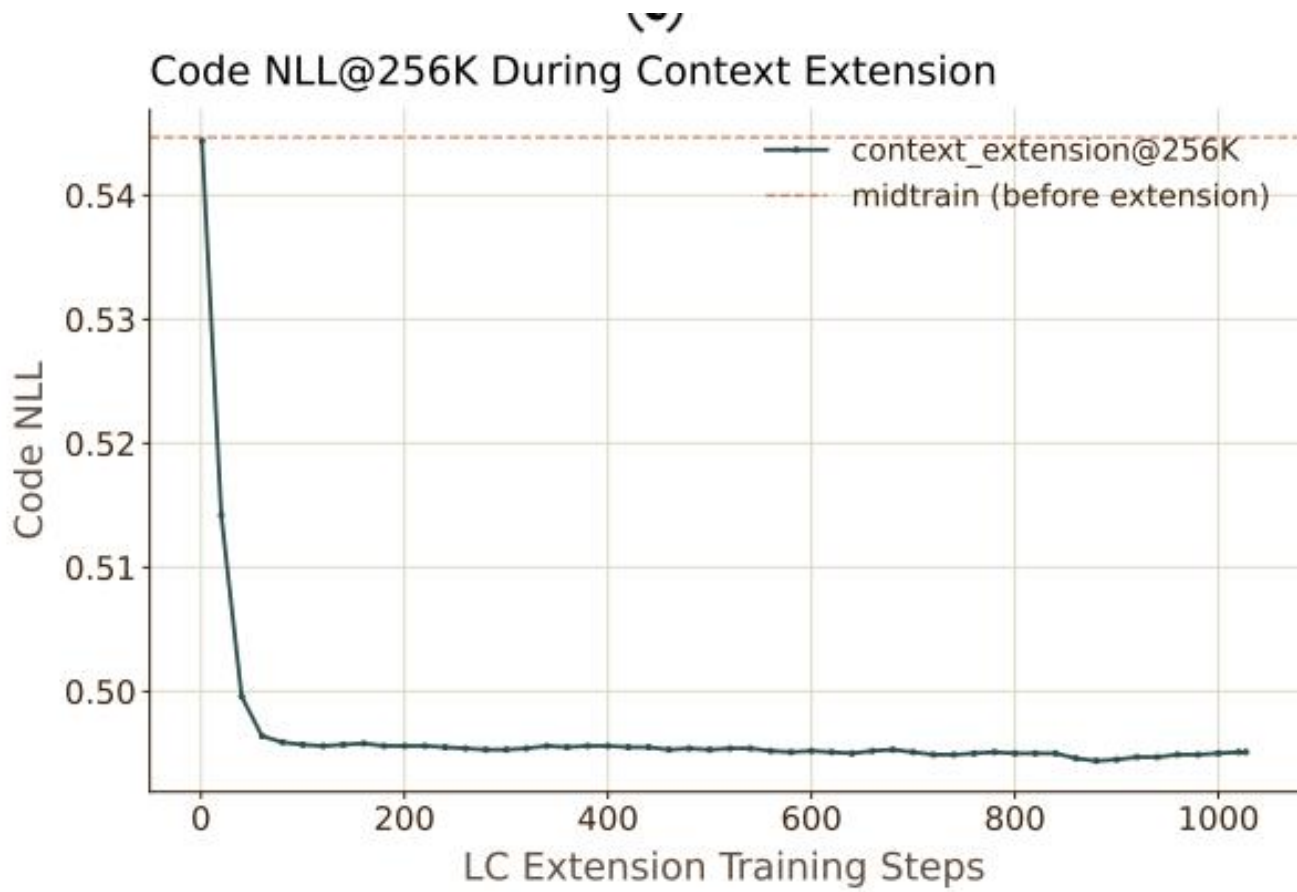
(a)

Can the model use related context at a distance?



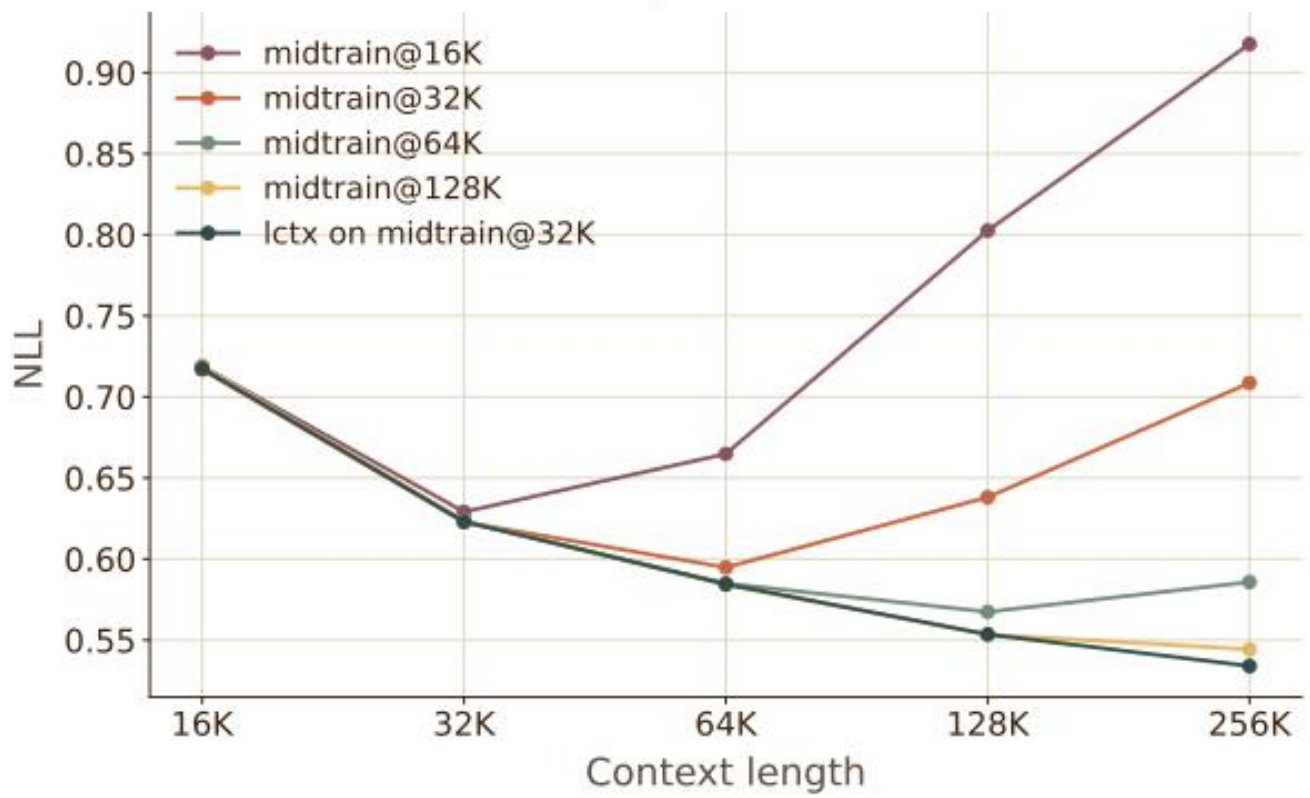
(c)

(d)



(d)

Code NLL vs Context Length



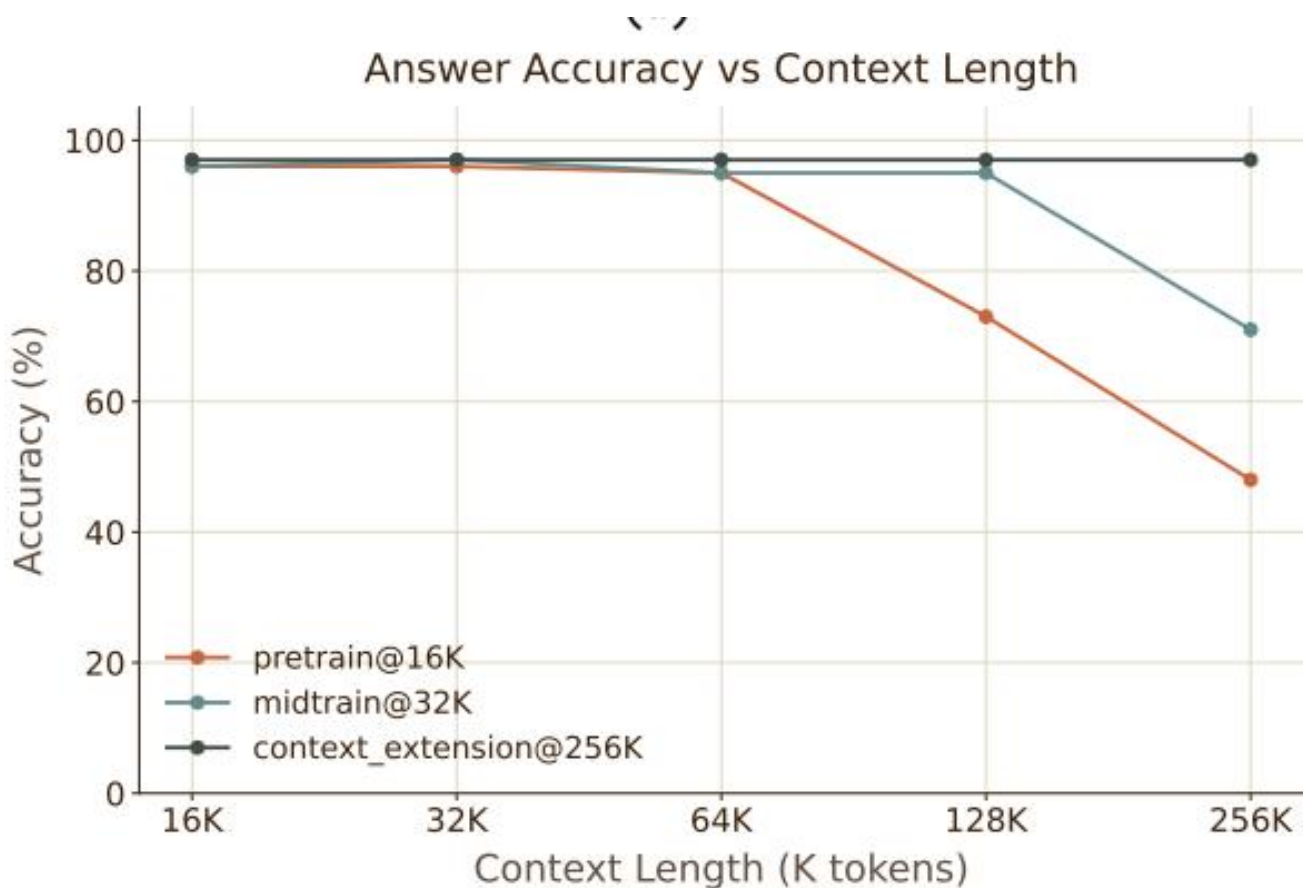
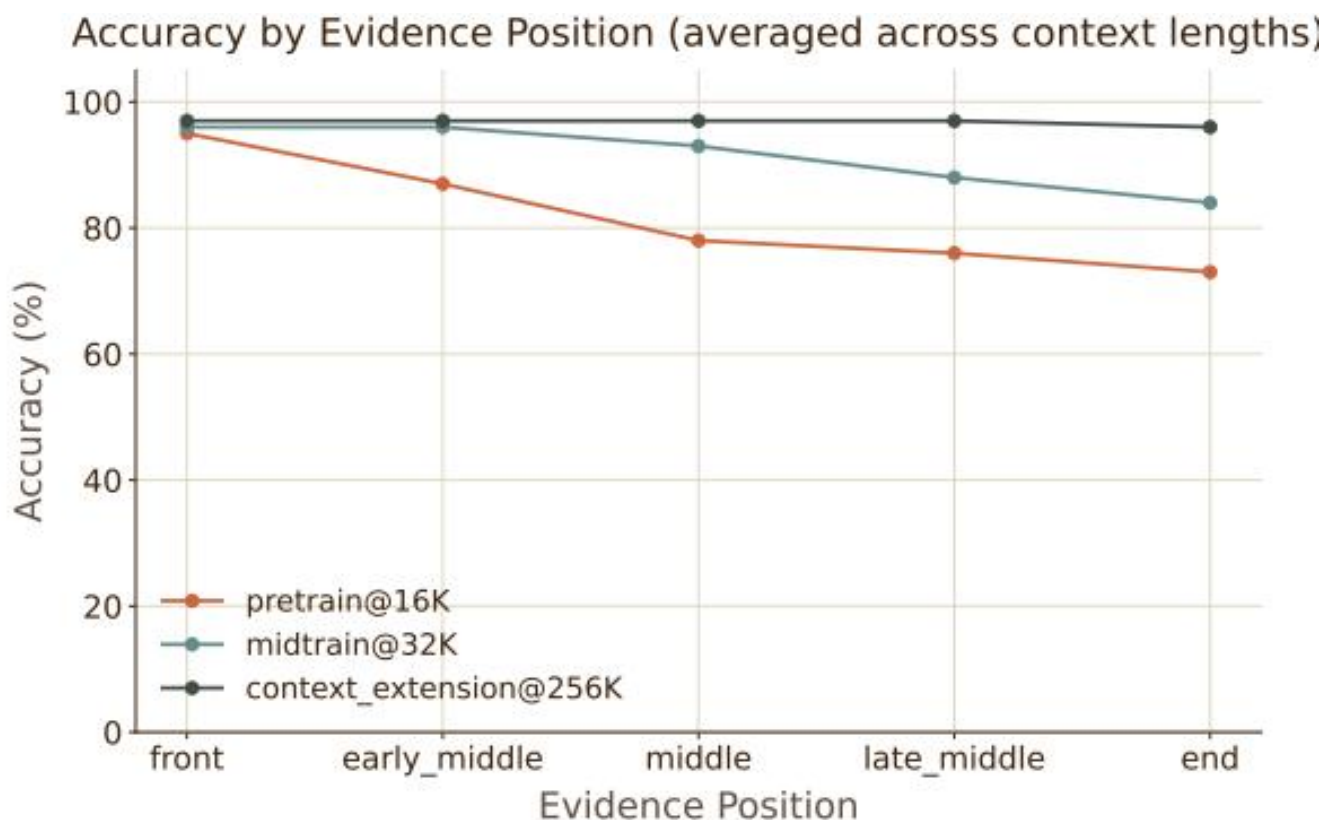


图 22. 长上下文扩展评估。(a) Retrieval NLL：随着相关的 16K prefix 被无关上下文推得越来越远，suffix 的 NLL 变化情况。长上下文 checkpoint 无论距离多远都保持 NLL 平坦；没有长上下文训练时，NLL 会大幅恶化。(b) 五种配置下 Code NLL 随上下文长度的变化：分别在 16K、32K、64K 和 128K 下进行中期训练（各 1T token），外加一种渐进式方案（32K 中期训练 + 简短的 256K 扩展）。渐进式 checkpoint 的表现与 128K 中期训练持平甚至更优。(c) 上下文扩展阶段 256K code NLL 的变化。大部分改进发生在训练迭代的前 1–10% 内。(d) 在 generative QA 基准上，答案准确率随上下文长度的变化。预训练 base 模型和中期训练（32K）模型一旦超出其训练上下文长度，准确率就会急剧下降，而长上下文 checkpoint 在最长 256K 的范围内都能保持接近完美的准确率。(e) 按证据位置统计的答案准确率（跨上下文长度取平均）。没有长上下文训练时，随着证据向上下文末尾——即分布外区域——移动，准确率会下降。长上下文训练解决了这种不对称性。



11 推理轨迹的演进

我们的模型从零开始攀登，不蒸馏任何第三方思维链（CoTs）。这为我们研究思维链随模型增强而如何演化提供了一个干净的视角。在本节中，我们研究模型在 RL 训练期间思维链中浮现的一些行为。我们分析了 STEM 攀登（第 3.2 节）和智能体攀登（第 3.3 节）的 rollout，考察行为原型的变迁，这些变迁反映在思维链中。（Guan et al., 2025; Baker et al., 2025）

11.1 STEM 思维链

弱模型靠猜，强模型下苦功。考虑来自 AIME 2024 的如下题目（答案：240）。

存在恰好三个正实数 k ，使得定义在正实数上的函数 $f(x) = \frac{(x-18)(x-72)(x-98)(x-k)}{x}$ 恰好有两个正实数 x 取得最小值。求这三个 k 值之和。

关键区别不在初始的代数设定，而在最后的验证步骤。强模型推导出四个代数候选值 k in

8, 32, 200, 512，并通过定义域条件 $x > 0$ 进行筛选，排除 512。弱模型则从可见的根 18、72、98 猜测候选极小值点，产生 40 和 152 这样的虚假值。

11.2 弱模型摘录：猜测极小值点，捏造根

或者，另外两个 k 值可能对应 x_1, x_2 以另一种方式与 18、72、98 相关，比如当 k 使得 $x_1 = 18, x_2 = 72$ 或类似情况时。若 x_1, x_2 为 18、72，则由 (1) 得 $x_1 + x_2 = 90 = 94 + k/2 \rightarrow k/2 = 90 - 94 = -4 \rightarrow k = -8$ ，不是正数，所以排除。若 x_1, x_2 为 18、98： $116 = 94 + k/2 \rightarrow k/2 = 20 \rightarrow k = 40$ 。若 x_1, x_2 为 72、98： $170 = 94 + k/2 \rightarrow k/2 = 76 \rightarrow k = 152$ 。所以可能的 k 是 40、152、512。验证 $k = 40$ ：若 $k = 40$ ，则 $x_1 + x_2 = 94 + 40/2 = 114$ ， $x_1 x_2 = \sqrt{18 \cdot 72 \cdot 98 \cdot 40}$ 。[...] 18、98、72、40 之间有关联吗？[...] 不确定，但也许可行。所以这三个 k 值可能就是 40、152、512。于是和为 $40 + 152 + 512 = 704$ ？等等， $40 + 152 = 192$ ， $192 + 512 = 704$ 。所以和是 704。

11.3 强模型摘录：逐一推导每个候选值并检查定义域

[...] 现在逐一检查： $k = 8$ ：二次式 $x^2 - 98x + 1008$ 的根为 $x_1 \approx 11.68 > 0, x_2 \approx 86.33 > 0$ 。成立。因此，对 $k = 8$ ， $f(x; 8) = m + \frac{(x-x_1)^2(x-x_2)^2}{x}$ ，其中 $m = -10368$ ，在 $x > 0$ 上的全局最小值 m 恰好由 x_1, x_2 取得。成立，相符（这两点位于两个负区间 (8, 18) 和 (72, 98) 内）。 $k = 32$ ：二次式 $x^2 - 110x + 2016$ 的根为 $x_1 \approx 23.24 > 0, x_2 \approx 86.76 > 0$ 。成立。全局最小值 $m = -7200$ 恰好在这两点取得（区间 (18, 32) 和 (72, 98)）。 $k = 200$ ：二次式 $x^2 - 194x + 5040$ 的根为 $x_1 \approx 30.90 > 0, x_2 \approx 163.10 > 0$ 。成立。全局最小值 $m = -194688$ 恰好在这两点取得（区间 (18, 72) 和 (98, 200)）。 $k = 512$ ：二次式 $x^2 - 350x - 8064$ 的根为 $x_1 \approx 371.70 > 0, x_2 \approx -21.70 < 0$ 。因此，对 $k = 512$ ， $f(x; 512) = m + \frac{(x-x_1)^2(x-x_2)^2}{x}$ ，其中 $x_2 < 0 \Rightarrow f(x) \geq m$ 对所有 $x > 0$ 成立，但等号仅在 $x_1 > 0$ 处取得。所以全局最小值恰好在一个正实数 x 处取得（而不是两个）。此外我们还看到有另一个局部极小值，但其值高于 m 。因此 $k = 512$ 被排除。现在我们得到三个值： $k = 8, 32, 200$ 。再快速确认一下：对这些 k ，没有其他点使得 $f(x) = m$ （题目表述为“恰好两个正实数 x 取得最小值”）。由于 $f(x) - m = \frac{(x-x_1)^2(x-x_2)^2}{x}$ 且 $x_1 \neq x_2$ ，等号仅在 $x = x_1$ 或 $x = x_2$ 时成立。成立，且两者都 > 0 。再检查定义域为 $x > 0$ 。现在我们得到三个值： $k = 8, 32, 200$ ，所以和为 240。[...]

弱模型暴力搜索，强模型寻找不变量。考虑以下 AIME25 题目。

设 N 为满足如下条件的有序正整数三元组 (a, b, c) 的个数： $a, b, c \leq 3^6$ 且 $a^3 + b^3 + c^3$ 是 3^7 的倍数。求 N 除以 1000 的余数。

弱模型试图通过假设立方运算是模 3^7 的单位群上的双射来简化同余式。这把三次条件变成线性条件，但这一假设是错误的： $\gcd(3, \varphi(3^7)) = 3$ ，而不是 1。相比之下，强模型识别出相关的不变量：对于 3 的幂，单位立方数并非任意的单位。它们构成一个指数为 3 的子群，其特征是同余于 $\pm 1 \pmod{9}$ 。

11.4 弱模型摘录：错误的线性化

或者，观察模 3 的条件：如果 3 不整除其中某些数。### 情况 1：3 不整除 a ，3 不整除 b ，3 不整除 c （都是模 3 的单位）那么 a, b, c 属于 $(\mathbb{Z}/3^7\mathbb{Z})^*$ 。由于 $x \mapsto x^3$ 是双射，设 $x \equiv u^3 \pmod{3^7}$ ， $y \equiv v^3 \pmod{3^7}$ ， $z \equiv w^3 \pmod{3^7}$ ，其中 u, v, w 属于 $(\mathbb{Z}/3^7\mathbb{Z})^*$ 。[...] 实际上，更仔细地看，由于立方运算是单位群上的双射，映射 $(a, b, c) \pmod{3^7} \mapsto (a^3, b^3, c^3)$

$b \pmod{3^7}$ 是 $(\mathbb{Z}/3^7\mathbb{Z})^* \times (\mathbb{Z}/3^7\mathbb{Z})^* \times (\mathbb{Z}/3^7\mathbb{Z})^*$ 上的双射。因此，满足 $3 \nmid a, b, c$ 且 $a^3 + b^3 + c^3 \equiv 0 \pmod{3^7}$ 的解的个数，等于满足 $3 \nmid a, b, c$ 且 $a + b + c \equiv 0 \pmod{3^7}$ 的解的个数。

11.5 强模型摘录：识别不变量

再回顾指数提升引理 / 模 3 的幂的单位立方结构：对 $x \not\equiv 0 \pmod{3}$ ，看 $x \pmod{3^e}$ 。[...] 注意对 $x \not\equiv 0 \pmod{3}$ ，由欧拉定理， $x^{\phi(3^e)} = x^{2 \cdot 3^{e-1}} \equiv 1 \pmod{3^e}$ ，且模 3^e ($e \geq 2$) 的乘法群是阶为 $2 \cdot 3^{e-1}$ 的循环群。映射 $x \mapsto x^3$ 的像是唯一的指数为 3 的子群。 [...]

另外，对 $3 \nmid y$ ，看 $y^3 \pmod{3^m}$ 。[...] 注意对 $e \geq 2$ ， $(\mathbb{Z}/3^e\mathbb{Z})^* = \langle -1 \rangle \times \langle 4 \rangle \cong C_2 \times C_{3^{e-1}}$ 。元素 4 模 3^e 的阶为 3^{e-1} (1-单位的一个生成元)。[...] 那么 $y \equiv (-1)^s 4^t \pmod{3^e}$ ， $s \in \{0, 1\}, t \in \{0, \dots, 3^{e-1} - 1\}$ ，且 $y^3 \equiv (-1)^{\{3s\}} 4^{\{3t\}} \equiv (-1)^s (4^3)^t \pmod{3^e}$ 。[...] 另外，像 $\{y^3 \pmod{3^e} : 3 \nmid y\}$ 是 $(\mathbb{Z}/3^e\mathbb{Z})^*$ 中指数为 3 的子群：它由满足 $z \equiv \pm 1 \pmod{9}$ 的元素 z 构成。

强模型是多疑的。另一个来自 AIME25 的题目。这里，强模型在存疑时会停下来重新审视自己的解答（“等等，重新检查一下”），抓住了边界情形，从而答对。

11.6 弱模型摘录：把空格子标为“极大”且从不复核

零棋子的放置可能是极大的吗？[...] 那么对空格子，能添加棋子吗？能，可以在任意格子添加白棋。但在我们的情形中，若所有行都是 W、所有列都是 B，你能添加白棋吗？我们看看：要在格子 (i, j) 添加白棋，该格子会变成白色。所以第 i 行必须是白色，没问题（第 i 行是 W）。第 j 列必须是白色，但第 j 列是 B，所以不能添加白棋。能添加黑棋吗？第 i 行必须是黑色，但第 i 行是 W，所以不行。所以实际上，若所有行都是 W、所有列都是 B，空格子就是极大的（无法添加任何棋子）。因此放置零个棋子是允许的。[...] 所以总放置数为 $\text{Case 1} + \text{Case 2} + \text{Case 3} = 960 + 32 + 32 = 1024$ 。

11.7 强模型摘录：质疑自己的逆命题并用小例子检验

以这个例子为例： $s_i = W$ 对所有 i ， $t_j = B$ 对所有 $j \Rightarrow S = \emptyset$ ， c 是空函数。现在取这个 $(S, c) = (\emptyset, \emptyset)$ （空放置）。我们能在格子 $(1, 1)$ 再放一个棋子吗？[...] 选 $x = W$ 。那么第 1 行有白棋：没问题。第 1 列有白棋：没问题。条件 1、2 满足。因此，放置 $(\emptyset, \emptyset)$ 不满足条件 3。但根据我们上面的一般论证，我们写过：“反之，若任取 $s_i, t_j \in \{W, B\}$ ，定义 $S = \{s_i == t_j\}$ ， $c(i, j) = s_i$ ，则 (S, c) 满足条件 3。”这在逻辑上有问题吗？等等，我们针对这个例子重新检查这个论证。[...] ### 小测试用例我们试一个更小的例子来检验逻辑，比如 1×1 网格，1 个白棋、1 个黑棋。求个数。[...] 现在用我们的公式逻辑：网格 $1 \times 1 \Rightarrow 1$ 行、1 列。求 $|Y| =$ 满足 $\{s_1\} = \{t_1\}$ 的 $(s_1, t_1) \in \{W, B\}^2$ 的个数。[...] 所以 (W, W) 和 (B, B) 两种：2 对。成立，完全吻合。

11.8 智能体思维链

强模型会编写并运行单元测试，而弱模型只做基本的健全性检查。在 SWE 任务上进行智能体强化学习 (agentic RL) 训练的过程中，涌现出的一种行为是定期运行单元测试来验证工作成果。

弱模型摘录：做基本的健全性检查，以确认所提方案满足问题陈述中的需求，但不会真正运行单元测试。

看看我所做的修改，我已经成功实现了问题陈述中的所有需求。让我回顾一下完成了什么：对于 `model.py`：添加了带 “key” 字段（类型为 `str`）的 `SeedDict TypedDict` 类；添加了正确的类型导入和类型定义（`SeedSubjectString`、`SeedSubjectType`、`Seed`）；创建了 `_normalize_seed()` 辅助方法，以一致地处理所有 `seed` 格式；为

List 和 Seed 类的所有公开方法添加了显式返回类型和输入参数类型；重构了 `add_seed()`、`remove_seed()` 和 `_index_of_seed()`，以支持所有 seed 格式，并使用规范化字符串键进行重复检测；修复了 `get_export_list()`，使其返回包含三个键（“authors”、“works”、“editions”）的字典，每个键都包含完全加载的 Thing 实例；更新了 `get_seeds()` ...；为 Seed 类的所有方法添加了类型注解 ...；增强了 Seed 类以处理 “author” 和 “redirect” 类型字符串。对于 `lists.py`：添加了 `subject_key_to_seed()` 函数 ...；添加了 `is_seed_subject_string()` 函数 ...。该实现提升了类型安全性和代码可读性，并保持了向后兼容性。seed 处理逻辑通过一致的规范化变得更加健壮，类型注解也提供了更好的静态分析支持。问题陈述中的所有需求均已满足。

11.9 强模型摘录：审查问题需求并运行单元测试以验证工作

测试仍然通过。所以所有核心功能都已就绪。让我看看整体情况。我需要考虑是否已经满足了问题的所有需求。对于这些需求：- List 和 Seed 的所有公开方法都有类型注解——我已为关键方法添加了注解 - `model.py` 中定义了 `SeedDict TypedDict` - `is_seed_subject_string` 函数——我创建了它 - `subject_key_to_seed` 函数能正确处理规范化 ...——让我验证一下 - `get_export_list` 返回结构正确的字典，其中包含字典列表 - `add_seed` 和 `remove_seed` 处理所有格式，并有更好的重复检测 ... 总的来说，我认为核心需求都已满足。所有现有测试持续通过，证实了向后兼容性。

强模型会做“证据考古”，而弱模型则执着于编辑机制。爬升过程中的强 checkpoint 通常带有这样的 CoT：在打补丁之前先在仓库中寻找证据（被回滚的提交、payload、测试）。弱 CoT 能识别出正确的目标，但会在后续轮次中耗费精力于精确的编辑机制和损坏恢复，范围小，且有一定程度的执着。

弱模型摘录：专注于大量有针对性的编辑和编辑上的省力操作，每次只迈小步。

- 我需要先探索代码库结构，了解 Leaflet 是如何组织的。由于任务涉及为各种图层类型添加 `‘toGeoJSON()’` 方法，我应该首先

: 1. 了解整体项目结构 2. 定位这些图层类型（Marker、Polyline、Polygon、LayerGroup 等）的源文件 3. 了解 Leaflet 中使用的现有代码模式和约定。让我先总览一下 ... - 现在我对代码库有了很好的了解。让我规划实现：** 理解坐标转换：** - Leaflet 内部使用 `‘[lat, lng]’` - GeoJSON

要求 `‘[lng, lat]’` 的顺序 - 对于 Marker: `‘{type: ‘Point’, coordinates: [lng, lat]}’` - 对于 Polyline: `‘{type: ‘LineString’, coordinates: [[lng, lat], ...]}’` - 对于 Polygon: `‘{type: ‘Polygon’, coordinates: [[[lng, lat], ...repeat first...], [ho...`

- 我需要确保我的 `old_str` 与文件中的内容完全一致。让我查看 `Marker.js` 文件末尾的更多内容，以了解确切的结构和行尾格式。

- 现在我看到确切的结构了。文件使用了混合缩进（有些地方用制表符，有些地方用空格——我需要保持这种风格）。让我按照相同的格式风格为 Marker 添加 `‘toGeoJSON()’` 方法。

- 再看一下文件，我注意到它使用制表符缩进而非空格。我需要匹配文件中使用的确切制表符缩进。

- 让我查看从第 265 行到末尾的确切范围，以了解精确的格式。

- 现在我能看到带制表符的确切格式（显示为空格，但它们很可能是制表符）。让我添加 `toGeoJSON` 方法。

强模型摘录：充分探索仓库并收集大量证据，而不急于采取行动。

- 让我先探索代码库。我需要了解 check enforcer 当前的工作方式——它如何处理 CI 检查、webhook，以及如何处理 GitHub 事件。这将帮助我理解需要修改什么，才能让 GitHub Actions 与 Azure Pipelines 一起成为一等公民。

让我阅读主要的代码文件——main.go、types.go 以及其他关键文件。我需要了解整体架构。问题提到了 Azure Pipelines 的 check-suite 处理、webhook 和 CI 评估逻辑。让我看看当前是如何实现的。

- 我看到结构了。代码当前处理：1. 包含 ‘/check-enforcer‘ 命令的 issue 评论 2. 来自 Azure Pipelines 的 check suite 事件（应用名为 “Azure Pipelines”）。但问题说我们需要处理：1. 将 GitHub Actions 作为一等信号（而不只是 Azure Pipelines）2. 可能存在多个相关套件——目前 github_client.go 中的 GetCheckSuiteStatus 我需要看一下 3. workflow-run 事件 ...

- 我看到 GetCheckSuiteStatus (github_client.go 第 105-138 行) 会获取所有 check suite，然后找到第一个与 AppTarget（当前代码中是 “Azure Pipelines”）匹配的套件。但它只返回第一个匹配套件的 status/conclusion。但问题说 “从相关套件评估整体提交状态，而不是假设” 只有一个应用或一种事件形态参与” 和 “mul...

- 看看代码库，我看到了当前的实现。让我读更多测试文件，了解预期是什么。我看到 main_test.go 中有测试。我还需要更精确地理解需求。让我看所有测试，以了解预期行为。我看到了 main_test.go、github_client_test.go、types_test.go。让我也读一下 README 和文档来了解上下文。

- 让我继续阅读测试，以了解完整的预期行为。
- 我看到测试了。现在让我看看其他测试文件，以获得更全面的认识。另外让我搜索 “workflow_run”、“GitHub Actions” 等术语，以理解完整问题语境中的需求。更仔细地看这个问题——我需要理解什么是 “workflow-run events”，还需要理解 GitHub Actions 是如何集成的。让我看看完整的问题。实际上让我 ...

强模型会寻找事实来源，弱模型则在相邻路径上猜测。强 CoT 会把源数据定位为事实来源。弱 CoT 表面上理解了可见性和代码路径，却围绕相邻的生成代码路径进行推理，留下了一个边界情况。

弱模型摘录：沿着相邻路径一步步追踪，但主要靠试错。

- 我看到 TypeExtendsReader 收集了所有超类和接口。但我注意到它没有检查可见性 (public/package-private)。让我更仔细地看这个问题。关键在于理解 “package protected class and interface” 的含义。在 Java 中，如果类不是 public，它就是包私有 (package-private) 的（只能在该包内访问）。类似地 - 测试中没有直接引用。让我看看 isAddBeanFor 在 API 中是如何定义的。让我检查 DBuilder 和 Builder 接口。

- 嗯，让我在测试文件中更具体地搜索 isAddBeanFor。

- 我在 API 中看到了 isAddBeanFor。但我需要了解全貌。让我看看 Builder 接口。

- 现在我理解了 API 这边。但生成器那边呢？让我看更多生成代码的示例，以了解 isAddBeanFor 的实际输出模式。我需要看看 TypeAppender 生成了什么。让我查看测试数据，或查看测试用例中实际生成的 DI 类。

- 那些是生成器类。让我在测试目标中寻找实际生成的 DI 文件。

- 如果我能理解 `package-private` 类型何时会引发问题，问题或许会更清晰。让我更仔细地看这个问题。让我搜索相关的 `issue` 或关于“`package protected`”的更多上下文。等等——让我读完整的 `TypeExtendsReader`，以便更好地理解。因为当我们收集要注册的类型（用于 `isAddBeanFor`）时，我们收集：1. The ba...

强模型摘录：将行为与驱动生成的数据源联系起来，并从那里开始追踪。

- 我看到了这些引用。让我看看 `isAddBeanFor` 是如何生成的。我需要了解全貌。

- 我看到它们在检查 `Modifiers`。所以存在一种模式。让我看看具体是怎么做的。让我更全面地搜索。

- 我看到 `isAddBeanFor` 定义在 `Builder` 接口中，并在 `DBuilder` 中实现。让我看看它如何使用。让我读 `DBuilder` 并理解其语义。`isAddBeanFor` 决定一个 `bean` 是否应被添加——我认为它用于“`types`”——即在注册时，这个 `bean` 是哪些类型的候选？所以只有 `public` 类型才是相关的，从 `con...`

- 我看到了 `isAddBeanFor` 在运行时的工作方式——它根据父作用域是否已包含该 `bean` 来决定是否添加/注册。但这无助于我理解这个问题。问题出在生成的代码上。让我看看生成的代码实际长什么样。让我找一些包含 `isAddBeanFor` 的生成 `D` 类示例。让我搜索实际的 `gene...`

- 那些是运行时类。让我看看生成的测试代码。让我在测试资源或构建输出中寻找生成的代码。但这是源码——让我看看黑盒测试或 `inject-test`。让我搜索包含 `isAddBeanFor` 的生成代码。

- 所以 `target` 中还没有生成的代码（尚未构建）。让我在测试资源或现有测试用例中查找。让我在测试源码中搜索‘`isAddBeanFor`’——也许我能从测试中理解预期行为。

- 我看到 `MethodReader` 的 `buildAddFor` 在处理返回类型的工厂方法时，也会使用 `typeReader.getTypesRegister()`。所以决定哪些类型进入 `isAddBeanFor` 的逻辑来自 `TypeReader`。但回到主要问题——“Bug `isAddBeanFor()`”生成的代码包含了包受保护的类和接口。修复方案是只考虑 `public` 类...

12 SWE 智能体工具 Schema

我们以兼容 OpenAI 的函数调用（function calling）Schema 格式，在图 23 和图 24 中描述 SWE 智能体强化学习（RL）所使用的工具。

```

{
  "type": "function",
  "function": {
    "name": "bash",
    "description": "Execute bash commands in the context of the repository.\nDirectory or environment variable changes are not persistent. Every action is executed in a new subshell.\nHowever, you can prefix any action with `MY_ENV_VAR=MY_VALUE cd /path/to/working/dir` or write/load environment variables from files.\nYou have a full Linux shell environment.\nAlways use non-interactive flags (-y, -f) for commands.\nAvoid interactive tools like vi, nano, or any that require user input.\nYou can use bash commands or invoke any tool that is available in the environment.\nYou can also create new tools or scripts to help you with the task.\nIf a tool isn't available, you can also install it.\n",
    "parameters": {
      "type": "object",
      "additionalProperties": false,
      "required": ["command"],
      "properties": {
        "command": {
          "type": "string",
          "description": "The bash command to execute. Can include pipes, redirects, and other shell features. Commands are executed in the repository working directory."
        }
      }
    }
  }
}

```

图 23. Bash 工具 Schema。

{ "type": "function", "function": { "name": "str_replace_editor", "description": "Custom editing tool for viewing, creating and editing files.\n* State is persistent across command calls and discussions with the user\n* If 'path' is a file, 'view' displays the result of applying 'cat -n'. If 'path' is a directory, 'view' lists non-hidden files and directories up to 2 levels deep\n* The 'create' command cannot be used if the specified 'path' already exists as a file\n* If a 'command' generates a long output, it will be truncated and marked with '<response clipped>'\n* The 'undo_edit' command will revert the last edit made to the file at 'path'\n\nNotes for using the 'str_replace' command:\n* The 'old_str' parameter should match EXACTLY one or more consecutive lines from the original file. Be mindful of whitespaces!\n* If the 'old_str' parameter is not unique in the file, the replacement will not be performed. Make sure to include enough context in 'old_str' to make it unique\n* The 'new_str' parameter should contain the edited lines that should replace the 'old_str', "parameters": { "type": "object", "additionalProperties": false, "required": ["command", "path"], "properties": { "command": { "type": "string", "description": "The commands to run. Allowed options are: 'view', 'create', 'str_replace', 'insert', 'undo_edit.'" "enum": ["view", "create", "str_replace", "insert", "undo_edit"] }, "path": { "type": "string", "description": "Absolute path to file or directory, e.g. '/testbed/file.py' or '/testbed:' }, "file_text": { "type": ["string", "null"], "default": null, "description": "Required parameter of 'create' command, with the content of the file to be created." }, "view_range": { "type": ["array", "null"], "default": null, "items": { "type": "integer" }, "minItems": 2, "maxItems": 2, "description": "Optional parameter of 'view' command when 'path' points to a file. If none is given, the full file is shown. If provided, the file will be shown in the indicated line number range, e.g. [11, 12] will show lines 11 and 12. Indexing at 1 to start. Setting [start_line, -1] shows all lines from start_line to the end of the file." }, "old_str": { "type": ["string", "null"], "default": null, "description": "Required parameter of 'str_replace' command containing the string in 'path' to replace." }, "new_str": { "type": ["string", "null"],

```
“default”: null, “description”: “Optional parameter of ‘str_replace’ command containing the new string (if not given, no string will be added). Required parameter of ‘insert’ command containing the string to insert.” }, “insert_line”: { “type”: [“integer”, “null”], “default”: null, “description”: “Required parameter of ‘insert’ command. The ‘new_str’ will be inserted AFTER the line ‘insert_line’ of ‘path.’” } } }
```

图 24. String Replace Editor 工具 Schema。

13 指令遵循数据的约束分类体系

我们构建了一个约束分类体系，用于组织模型可能被要求遵循的指令。它有两个用途：引导数据采集覆盖更广的范围（包括罕见的约束组合），并允许在细粒度层面分析模型性能。我们按约束的检查方式来划分约束，与第 3.4.1 节的奖励类型相对应。

客观（硬性）约束。这类约束基于规则，会触发来自自定义与开源检查器目录的 Python 函数（Pyatkin et al., 2026; Zhou et al., 2023）。示例包括数值约束（如“恰好用 3 个句子回答”）、格式约束（如“以包含 name 和 age 键的 JSON 对象输出”）以及语言约束（如“用法语回答”）。

主观（软性）约束。这类约束需要判断才能评估，由奖励模型或 AI 裁判评分。示例包括语气（如“保持鼓励但不要居高临下”）、人设（如“以历史学者身份作答”）和交互行为（如“如果不清楚，先提出澄清问题再回答”）。表 15 展示了示例类别。

多轮对话场景分类体系。受 Deshpande et al. (2025) 启发，我们将对话用例拆解为多种场景，用于在涉及多轮的复杂对话中测试模型的指令遵循（IF）能力，如表 16 所示。

14 G 构建 SWE 环境的基础设施

该基础设施的设计主要旨在应对规模、稳定性和安全方面的挑战。构建基础设施组织为一个双池（two-pool）Ray 集群，运行在约 30,000 个 CPU 核心上。主池（main pool）负责工作调度、Lance I/O 和流水线状态跟踪。构建池（builder pool）运行实际的容器构建；所有构建节点都标记了 Ray 自定义资源标志（custom resource flag），构建步骤利用 Ray 的逻辑资源调度，确保其 actor 只落在具备构建能力的节点上。

两个池之所以分离，是因为容器构建对资源要求高且容易失败（磁盘耗尽、OOM、构建挂起），而协调工作轻量且无状态；将它们隔离可确保构建 pod 的失败不会干扰编排和状态管理。这种分离还实现了凭据隔离：构建 pod 执行来自开源仓库的不可信 Dockerfile，不持有流水线数据存储或内部服务的凭据。Pod 失败——无论由磁盘压力、硬件故障还是节点驱逐引起——都由 Kubernetes 重启处理；重启后的 pod 重新连接到 Ray 集群，并在下一个主池 worker 调度新任务时被自动接管。

每个构建 pod 包含两个容器，共享基于 NVMe 的本地存储，协同部署以提升构建与评分的效率。主容器运行 Ray worker，并使用 rootless podman 加载构建好的镜像、运行评分容器。sidecar 容器运行 rootless BuildKit 守护进程，通过 buildctl 进行 OCI 镜像构建。两个容器通过共享的 NVMe 卷通信——BuildKit 将 OCI tar 归档写入该卷，podman 再加载这些归档——并通过 localhost 走 BuildKit 控制通道。这种协同部署避免了大型容器镜像在构建与评分之间的网络传输；整个构建-加载-运行循环都在本地 NVMe 上完成。Init 容器负责一次性初始化，包括通过联合身份（federated identity）完成容器镜像仓库的身份验证。

该流水线还利用 BuildKit 分层缓存，减少同一仓库内不同问题之间的冗余工作。问题按仓库分组，并分派给构建节点上同一个持久的 Ray actor。由于该 actor 在同一仓库的问题之间保持存活，共享的基础层——apt-get install、pip install、依赖编译——只构建一次并被复用，因为依赖安装往往主导 Docker 构建

表 15. 来自我们分类体系的高层约束示例，按客观（硬性）和主观（软性）类别划分，并给出相应的子类别与示例。

约束类型	子类别 & 示例
客观（硬性）约束	
数值	显式长度限制、句子/段落数量、条目数量、范围约束和比例。
格式	文档结构、列表、表格、代码格式、数据序列化（JSON/YAML）、标记/样式、模板和引用格式。
内容	必需/禁止的术语和符号、必需的元素或概念、来源/引用要求，以及时间/地域范围。
语言	语言选择、方言、语域、句子结构、词性、形态约束和语法规则。
时间与顺序	时间先后排序、程序步骤和字母顺序。
否定约束	排除特定行为或格式模式的指令
主观（软性）约束	
风格	语气、口吻、词汇水平、受众适配、情感效价和品牌语调。
人设 & 自我呈现	角色扮演、个性特质、价值观和情绪状态。
交互 & 响应策略	对话导航、确定性/模糊表达、澄清策略、征询行为和亲社会行为。
结构 & 推理	推理过程要求、论证结构、条件分支和范围约束。
指令处理	条件性、多步指令链、冲突处理。

大型仓库的构建时间。

主要的吞吐瓶颈是 LLM token 消耗，而非 CPU 或磁盘。该流水线运行在 10,000 个 CPU 核心上，具备每分钟 1200 万 tokens 的 LLM 容量，实现了 83% 的提示词缓存命中率。在该配置下，流水线每分钟大约产出 20 个通过评分（grading）的环境。

15 H STEM 评测设置

对于所有 STEM 基准，我们均在最大输出 token 预算为 256k tokens 的条件下报告 pass@1 性能。除了上文已给出的 256k 结果外，我们还在表 17 中展示了模型在 128k 最大输出 tokens 下的表现。

15.1 数学

美国数学邀请赛（American Invitational Mathematics Examination, AIME）和哈佛—麻省理工数学锦标赛（Harvard-MIT Mathematics Tournament, HMMT）都是著名的高中数学竞赛。这些竞赛数学评测可作为衡量模型推理能力的有用代理指标。我们在 2025 和 2026 年版 AIME 以及 MathArena 的 2026 年 2 月 HMMT 基准（Dekoninck 等，

能力	描述
----	----

上下文切换	模型跟踪用户何时切换话题或返回先前话题的能力。
指令保持	在长对话中跟踪、更新和检索指令与事实的能力。
指令分层	在先前多轮中引入的指令基础上，维护并保留一组有效约束的能力。
记忆 & 回忆	正确检索并使用先前轮次中的事实和陈述。
自身一致性	避免与先前陈述产生无意的矛盾。
版本编辑	能够回退到更早的版本、有选择地撤销编辑，或重现先前轮次的精确文本。

表 16. 针对多轮的 IF 能力示例。

表 17. STEM 基准上的训练后模型评测结果，对比 MAI-Thinking-1 在 128k 与 256k 最大输出 tokens 下的评测表现。其他模型的数字取自各自官方模型卡。以 “—” 列出分数的模型表示未报告相应基准的数字。

基准	MAI-Thinking-1		Sonnet 4.6		Opus 4.6		GPT 5.4		Kimi K2.6		DeepSeek V3.2		DeepSeek V4		GLM	
	128k	256k														
AIME 2025	95.0	97.0	95.6	99.8	—	—	93.1	—	—	—	—	—	—	—	—	—
AIME 2026	93.6	94.5	—	—	—	—	96.4	—	—	—	95.3	—	—	—	—	—
HMMT Feb 2026	84.3	84.9	—	—	—	—	92.7	—	—	95.2	82.6	—	—	—	—	—
GPQA Diamond	84.2	84.2	89.9	91.3	92.8	90.5	82.4	90.1	86.2	—	—	—	—	—	—	—
LCB v6	87.3	87.7	—	—	—	—	89.6	83.3	93.5	—	—	—	—	—	—	—

2026) 上评测了 MAI-Thinking-1。这些竞赛的预期答案是简单的数字或表达式。我们要求模型以 boxed 格式输出最终答案，从而可以通过基于正则表达式的提取配合 SymPy 验证，并以 AI 评判器作为兜底。

15.2 科学

研究生级 Google-Proof 问答 (Q&A) 基准 (GPQA) (Rein 等, 2023) 包含知识密集型、研究生级和研究级问题，主要集中在 STEM 领域。对于 GPQA，我们评测其 Diamond 子集，该子集包含 198 道选择题。我们采用 OpenAI 的 simple-evals 包 (OpenAI, 2025) 中的指令提示词来规定所需的答案格式，并使用自定义的基于正则表达式的提取流程，将模型输出与参考答案进行比较。我们报告 16 次 rollout 的平均 pass@1 性能。我们使用 GPT-5-mini 作为 AI 评分的默认评判器，用图 25 中描述的提示词对 STEM 基准上的模型回答进行评分。GPT-5mini 用作默认评判器。

15.3 竞赛编程

LiveCodeBench (LCB) (Jain 等, 2024) 包含最新的竞赛编程题目。我们在 v6 划分上进行评测，该划分包含 2023 年 5 月至 2025 年 4 月间发布的 1,055 道题目。与下文的多轮智能体编码评测不同，LCB 是单轮 (one-shot) 测试设置。答案在

```

SYSTEM_PROMPT = """Compare two responses, [RESPONSE] and [CORRECT_ANSWER] to a question. Follow the rubric
below in [NOTES]. Then output [FINAL JUDGMENT].

Some general guidelines to follow in determining alignment:
- Units: If the question does NOT explicitly require units/formatting, then if one response provides units (
like km/s) and the other does not, but both have the correct numerical value, you can consider both
responses equally correct. If the question explicitly requires units, missing/incorrect units is NOT
acceptable.
- Formatting: If the question does NOT explicitly require a specific format, then Latex vs plain text or
markdown does not matter as long as the answer is equivalent. If the question explicitly requires a
specific format/structure, deviations are NOT acceptable.
- Rounding: If the question does NOT explicitly require an exact value or specific rounding/precision, small
rounding differences are acceptable when clearly equivalent (e.g., 9.42 vs 9.43 with the same intended
precision). If the question explicitly requires exactness or a specified precision, you must enforce it.

Judgment guidance:
- If [RESPONSE] is exactly identical to [CORRECT_ANSWER] or differs only by harmless formatting, you should
indicate that (alignment) is met in the [NOTES].
- If [RESPONSE] is not exactly identical to but is equivalent to, or can be straightforwardly reduced/
simplified to, [CORRECT_ANSWER] (without adding new assumptions or changing meaning), you should still
indicate that (alignment) is met in the [NOTES].
- If [RESPONSE] clearly does not align with the [CORRECT_ANSWER], you should indicate that (alignment) is NOT
met in the [NOTES] and output [FINAL JUDGMENT] = no.

Examples:
1) [RESPONSE]
Answer: 10
[CORRECT_ANSWER]
10
-> in [NOTES], (alignment) is met, and [FINAL JUDGMENT] = yes.
2) [RESPONSE]
X=\frac{\pi^2}{3}
[CORRECT_ANSWER]
\frac{\pi^2}{3}
-> in [NOTES], (alignment) is met, and [FINAL JUDGMENT] = yes.

[NOTES]
(alignment) How well do the CONTENTS of [RESPONSE] align with the CONTENTS of [CORRECT_ANSWER]? Note any true
mathematical/numerical differences.
(final) Briefly summarize whether [RESPONSE] should be treated as correct overall, based on alignment.

[FINAL JUDGMENT]
Should [RESPONSE] be considered as correct as [CORRECT_ANSWER] (alignment satisfied)? Answer in yes/no."""

USER_PROMPT = """[QUESTION]
{question}

[RESPONSE]
{response}

[CORRECT_ANSWER]
{correct_answer}"""

```

图 25. 面向 STEM 问题的 AI 评判器提示词。我们在对话历史中额外加入少量示例（few-shot examples），作为先前的用户和助手轮次，以提高评分可靠性，并降低受评判器攻击（judge hacking）影响的敏感度。

代码围栏（code fences）中生成以便解析，并在测试框架（test harness）上执行评分。评分设有预定义的内存和运行时间限制，与真实编程竞赛的约束条件类似。

16 I 智能体编码评测

SWE-bench Verified (Chowdhury 等, 2024)、SWE-Bench Pro (Deng 等, 2025) 和 Terminal-Bench 2.0 (Merrill 等, 2026) 的智能体编码基准结果见表 11 的第二部分。与上一节的 STEM 评测不同, 智能体编码评测是多轮交互式评测, 要求模型与环境进行交互。对于这三个基准, 我们都使用一种非常简单的、ReAct 风格 (Yao 等, 2023) 的”始终追加”(always append) 智能体循环来评测我们的模型, 该循环描述见图 18。在每一轮中, 模型可以生成工具调用, 我们会在环境中执行这些调用, 并在模型发起下一次工具调用之前将输出追加到轨迹中。评测时我们使用 256k token 的总上下文长度; SWE-bench Verified 和 SWE-Bench Pro 的最大输出长度为 8k token, Terminal-Bench 2.0 为 32k token, 最大步数为 1,000 步。当模型不再输出工具调用、达到上下文长度上限或达到步数上限时, 智能体循环即告终止。循环终止后, 相应的评分器会在智能体所使用的 SEE 环境中运行。对于 SWE-bench Verified 和 SWE-Bench Pro, 我们使用与爬山过程中相同的两种工具 (bash 和 string replace editor) 进行评测, 如第 3.3.1 节所述。对于 Terminal-Bench 2.0, 我们仅使用 bash 工具进行评测。

SWE-bench Verified。SWE-bench Verified 是 SWE-Bench (Jimenez 等, 2024) 中人工精选的 500 个任务子集, 旨在提高评测的可靠性。每个任务都来自热门 Python 仓库中的真实公开 GitHub issue, 模型必须生成一个能解决该 issue 并通过仓库隐藏测试套件的补丁。所有任务均经过审核, 以确保问题陈述可解且测试能公正地评估正确性。

SWE-Bench Pro。SWE-Bench Pro 是 SWE-bench Verified 更具挑战性的继任者, 包含 731 个任务, 采用相同的基于单元测试的评分机制, 旨在评估 LLM 在来自真实世界开源仓库的专业级软件工程任务上的表现。它强调更困难的多文件修改、更长时域的推理, 以及更贴近工程师实际工作的复杂 bug 修复或功能实现。更重要的是, 与 SWE-bench Verified 不同, SWE-Bench Pro 包含 Python 之外其他语言的仓库。

Terminal-Bench 2.0。Terminal-Bench 2.0 是 Terminal-Bench (Merrill 等, 2026) 经人工验证、难度大幅提升的继任者, 包含覆盖软件工程、调试、数据科学、机器学习、安全和系统管理等领域的 89 个真实任务。每个任务都在沙箱化终端环境中运行, 并根据输出或环境最终状态进行程序化评分。任务设计旨在考查长期规划、错误恢复和真正的命令行熟练度, 而非模式匹配。

17 J 安全评测

17.1 内部评测细节

17.1.1 方法论

评测分两个阶段进行: 先对用户请求进行结构化刻画, 再据此推导出用于给模型行为评分的响应规范。这种分解使我们能够区分真正的安全缺陷与在良性但敏感话题上的过度谨慎行为, 并给出按策略、按模态的细分指标, 而非单一的汇总分数。

表 18. 用户请求分类维度。

维度	描述
危害类别 & 策略违规	识别请求涉及哪个策略领域, 以及是否违反特定规则, 将判断建立在具体策略条款之上。

敏感度等级	基于影响范围（个体 vs. 广泛）和严重程度（最坏情况下的滥用后果），按序数尺度记录主题敏感
用户意图	二值信号：可能存在善意意图 vs. 恶意意图。仅话题危险本身不构成恶意的证据。
用户场景	捕捉用户的实用目标（例如：信息检索、寻求指导、创意生成、情景扮演）。
越狱检测	双重作用：筛查对抗性包装，并为请求是否以对抗方式构造提供信号。

两个评测阶段均由 LLM 评判器执行，其中分类步骤会运行多次并按多数投票聚合，以控制评判器的方差。每个用户请求还会经过越狱检测，该检测利用攻击族与技术变体的分层分类体系筛查对抗性包装。一旦检测到包装，提取步骤会还原底层的原始请求，后续分类基于还原后的请求进行。这确保评测反映的是模型对实际底层意图的行为，而非对表层攻击模板的行为，从而防止针对特定包装的过拟合虚增安全分数。

阶段 1：用户请求分类。每个用户请求都按照表 18 所述的五种互补维度进行刻画。这些维度共同捕捉请求中与策略相关的内容以及请求所处的上下文。对于多轮对话，整个对话历史都被视为上下文输入。

阶段 2：响应策略生成。阶段 1 的分类结果由评分规则评判器（rubric judge）消费，该评判器针对每个请求输出响应规范，涵盖相关的策略约束、适当的拒答姿态（根据意图和严重程度，可以是完全遵从、部分拒答或完全拒答），以及响应所要求的语气和方式。对于被归类为无害或意图不明的请求，该 LLM 生成的评分规则会被一种确定性的默认规则取代，后者会明确惩罚含糊其辞和无端的拒答。

17.1.2 数据收集与采样

设计原则：评测结果的信息量取决于用于评测的提示词本身：如果评测集过于狭窄、权重失当或已经饱和，就会围绕错误的量产生狭窄的置信区间，任何基于该评测集的模型选择决策都会继承这一错误。我们将评测集的构建视为一个方法论问题，评测集必须满足三个性质（Atkinson 等，2007）。

- 覆盖上文介绍的政策领域、敏感度等级、用户场景和对抗性框架，使任何行为模式不同的输入切片都不会在无声无息中代表性不足。
- 在固定评测预算下的统计效率，因为评判成本高昂，且各类别指标的置信区间随样本量增大的收窄速度很慢。
- 与其他前沿模型比较时的区分能力，使模型改进时总分确实会变化，而不会因提示词过于简单或过于困难而饱和。

来源池与标注：候选提示词池由三个互补来源汇集而成：公开安全基准、已过滤 PII（个人身份信息）的消费者 Copilot 日志，以及外部供应商根据我们的策略分类体系收集的提示词。每个候选提示词都会经过上文所述的标注框架，为后续抽样所需的 5 个分层附加标签：危害类别、策略违规、敏感度、用户场景和越狱。

信息量加权实验设计。我们力求从候选提示词池中选出信息量最大的提示词用于评测。我们将此视为一个自动化测试组装问题来处理。这与近期通过心理测量学建模大幅压缩 LLM 评测成本的努力方向一致，例如 tiny-Benchmarks (Polo 等，2024)，它利用项目反应理论 (Item Response Theory) 来识别静态的、具有普遍代表性的评测子集。作为该框架的扩展，我们对模型并非始终失败或始终通过的提示词赋予更高权重。这有助于我们主动发现最有可能改进模型的方向。

阈值设定。上文所述的评测集是安全考量进入 MAI-Thinking-1 爬山过程的主要机制。每个内部发布候选版本都必须在一组固定的安全评测上达到验收阈值，并且不得相对上一版已验收的发布版本发生回退。这组评测既涵盖针对性的策略领域表现，也涵盖在无偏排行榜集上的整体行为，从而避免仅针对单一危害类别的回退被其他方面的改进平均掉。阈值在流程的两个节点发挥作用：其一，作为发布候选签核（sign-off）时的门槛，任何在组评测指标上低于阈值的候选版本都无资格进入发布配方；其二，当实验模型低于较低阈值时触发预警警报。

阈值本身源自对安全、过度拒答和下游模型质量之间可达权衡曲面的帕累托分析：在每个发布周期，我们绘制该周期内所评估配方的前沿面，并识别出拐点，在拐点处边际安全收益开始以不成比例的过度拒答或质量损失为代价。随后，阈值被设定为沿该前沿面的固定百分位数，而非静态的绝对数值，这样门槛就能跟随当前架构与数据配比下可实现的水平，而不是一个过时的历史下限。这使得门槛信号在不同发布周期之间的含义保持稳定——相对于当前前沿面，通过门槛即提供同样的安全质量证据，无论前沿面如何移动——同时为绝对数值随着底层配方的改进而上升留有空间。

18 K 通用能力评测

下文讨论涵盖知识、指令遵循、长上下文、安全、诚实性、健康与工具调用等维度的基准，结果见表 12。

18.1 知识

我们分别在 SimpleQA Verified (Haas et al., 2026) 和 MMLU-Pro (Wang et al., 2024b) 上评测 MAI-Thinking-1，以评估其知识与推理能力。结果见表 12，并与其他模型进行了对比。

MMLU-Pro。MMLU-Pro 在 MMLU (Hendrycks et al., 2021) 的基础上，引入更具挑战性、更侧重推理的题目，并将选项数从 4 个增加到 10 个。它评估模型在一系列广泛的知识与推理任务上的表现。我们采用 OpenAI 的 simple-evals 包 (OpenAI, 2025) 中的指令提示词来规定所需答案格式，并使用自定义的基于正则表达式的提取流程，将模型输出与标准答案进行比对。

SimpleQA Verified。SimpleQA (Wei et al., 2024b) 评估模型在答案简短、无歧义且覆盖多种主题的难题上的短问答事实性知识掌握。我们报告模型在 SimpleQA Verified (Haas et al., 2026) 上的准确率，该子集经过精选，用于解决标签噪声和冗余问题。

18.2 指令遵循

模型可用性的一个关键方面是其对用户指令的响应质量。我们拥有一套内部基准，但下文仅报告三个常见公开基准的结果。

- IFBench (Pyatkin et al., 2026) 评估模型遵循精确指令的能力，涵盖 58 种多样且可验证的约束。
- AdvancedIF (He et al., 2025) 使用评分细则 (rubrics) 和经校准的 LLM 评测模型，衡量复杂的、多轮的以及带系统提示的指令遵循能力。
- MultiChallenge (Deshpande et al., 2025) 针对多轮对话，要求模型具备准确的指令遵循、上下文分配和上

下文内推理能力¹¹。我们已验证，对于公开模型，我们的实现与官方排行榜上报告的分數一致。

18.3 长上下文

MAI-Thinking-1 的长上下文性能通过四个基准进行评测，结果见表 12 和表 19。我们出于下文讨论的原因省略了 MRCR (Vodrahalli et al., 2024)。

GraphWalks。GraphWalks 是一个合成多跳推理基准：模型获得图的边列表表示，并被要求遍历图，以找到给定起始节点的相邻节点（通过广度优先搜索）或父节点。表 12 中的总体得分是模型预测答案与标准答案之间的 F1 分数。我们报告按 o200k_base 分词器测量的 $\leq 128k$ 子集上的 GraphWalks 结果。

LongBenchV2。LongBenchV2 (Bai et al., 2024) 由面向长上下文理解任务的 4 选 1 多选题构成。我们使用与官方实现相同的评测设置，但有一个限制：输入上下文长度限定为 256k tokens，最终得到 408 个唯一问题。

CorpusQA。CorpusQA (Lu et al., 2025) 是一个多文档自由问答数据，模型回答由 AI 评测模型评分。我们没有使用默认的 deepseek-v3 (DeepSeek-AI et al., 2025a) 作为评测模型，而是使用 GPT-5.4 (high)，其精度更高。

表 19. 其他长上下文基准。Sonnet 4.6 的结果由我们自己的独立评测生成。所有模型均在最大 128k 输出 tokens、最大推理强度（如适用）的设置下评测。结果为 4 次独立评测的平均值。

模型	LongBenchV2	CorpusQA
MAI-Thinking-1	61	82
Sonnet 4.6	66	79

MRCR。OpenAI 的 MRCR (Vodrahalli et al., 2024) 是一个人工构造的基准，用于测试模型在长上下文中的计数和复制能力。虽然理想模型应当表现良好，但该任务的性质与真实用户查询有本质差异。我们发现，在未进行针对性训练的情况下，MAI-Thinking-1 在该任务上相比现有模型表现不佳，在 avg_similarity@256K 上仅达到 60%，而目前最先进的结果为 95%。相比之下，仅用 1,000 个合成生成的同分布训练样本，就能将 MAI-Base-1 系列中一个规模小得多的模型在 MRCR 上的表现提升到 90% 以上，这表明该基准容易过拟合。由于我们没有证据表明针对此类数据进行训练能带来更普遍的模型能力提升，因此决定将 MRCR 从优先的长上下文基准集中移除。

18.4 安全

AIR-Bench。AIR-Bench (Zeng et al., 2024b) 基于法规与政策衍生的危害分类体系，对基于策略的拒答行为给出整体评估。评测使用针对每个风险类别定制的分类专属 LLM 评测提示词，评分设计旨在奖励安全参与，而非仅仅惩罚不安全响应。MAI-Thinking-1 在该基准上达到了与 Sonnet 4.6 相同的表现。

CyberSecEval。CyberSecEval 4 (Wan et al., 2024; Meta, 2024) 是一个涵盖多种网络安全相关能力的评测套件。这里我们重点关注不安全代码生成。其中 Instruct 基准提出旨在诱导已知不安全代码模式的编程请求，而 Autocomplete 基准则向模型提供一段已知不安全模式之前的代码上下文，由模型续写完成。两者均通过静态

¹¹我们遵循官方基准的评测模型设置，采用 Gemini 2.5 Pro 作为评测模型。<https://labs.scale.com/leaderboard/multichallenge>

分析规则衡量模型是否生成了存在漏洞的代码。MAI-Thinking-1 在 Autocomplete 基准上优于 Sonnet 4.6, 在 Instruct 上表现相近。

18.5 诚实性

为衡量 MAI-Thinking-1 的诚实性, 我们选取了覆盖两种不同失败模式的基准: 在短问答中重复看似合理的错误陈述, 以及在长文本生成中引入无依据的论断。MAI-Thinking-1 在这两个基准上的表现均与 Sonnet 4.6 相当。

TruthfulQA。TruthfulQA (Lin et al., 2022) 评估模型对常见误解的抵抗能力, 每个问题都旨在诱导出看似合理但错误的答案。为获得可复现的结果, 我们报告模型在推荐的多选题设置下于该数据集上的准确率。

LongFact。LongFact (Wei et al., 2024c) 评估包含多个论断的长文本生成的事实精确性; 我们采用 OpenAI 的简化论断提取与 LLM 评测协议 (Singh et al., 2025a) 替代原有的 SAFE 流水线, 保留 LongFact 提示词集, 并报告论断级别的精确率。

18.6 健康

HealthBench Professional。HealthBench (Arora et al., 2025) 是一套用于评估 LLM 与个人及医疗专业人士进行真实健康对话能力的基准。语言模型被给定一段多轮对话, 并被要求生成回复作为现有对话线程的延续。回复由经校准的基于模型的评测模型, 依据医生小组创建的对话专属评分细则进行评估, 涵盖临床正确性、指南遵从性和安全性。我们报告最新 HealthBench 版本 HealthBench Professional 上的表现, 该版本聚焦于医学专家与语言模型之间的 525 段对话 (Hicks et al., 2026)。HealthBench Professional 对主要指标引入了长度惩罚, 以修正长期观察到的冗长回复与 LLM 评分虚高之间的相关性。所有报告分数均使用 OpenAI 提供的标准 GPT-5.4 评测模型与评分细则。

MedXpertQA。MedXpertQA Zuo et al. (2025) 是一个具有挑战性、尚未饱和的多选题基准, 通过 2,450 道专家精选的多选题评估专科级医学知识。这些题目旨在考察高级临床知识与多步推理能力。每道多选题有 10 个选项 (A-J)。我们遵循 Muse Spark Team (2026) 引入的评测方法, 该方法使用辅助 LLM (GPT-5.4) 从自由文本中解析模型预测的答案选项字母。

18.7 工具调用

BFCL v3 (Patil et al., 2025) 评测 LLM 在单轮、并行和多函数场景以及多轮、多步 agent 交互中的工具使用与函数调用能力。它通过抽象语法树匹配和执行结果匹配来评估函数选择与参数生成, 同时考察长程推理、跨轮状态跟踪以及对噪声或缺失函数的鲁棒性。为获得更好的确定性, 我们按照官方排行榜的建议, 在推理时使用 $T = 0.001$ 、 $p = 0.97$ 。

19 L 集群环境细节

本附录介绍使训练环境在前沿 (frontier) 规模下可用的基础设施机制。核心设计选择是将物理拓扑与硬件健康状态作为一等 (first-class) 调度状态暴露给调度系统。节点与 NVLink 机架域通过拓扑标签 (topology labels)、

预留 (reservation) 对象、机架级服务和认证门控 (certification gates) 来表示。一个 node 并非仅仅因为已被供应 (provisioned) 就视为可用；只有当它健康、拓扑有效、可观测且可恢复时，才算真正可用。

19.1 硬件与计算集群

MAI-Thinking-1 是在一个异构加速器机群上开发的，涵盖 NVIDIA H100、GB200 和 GB300 系统。不过，主预训练运行被放置在单个站点的单一 GB200 集群上。GB200 和 GB300 集群托管在 Microsoft 自营数据中心内，并通过与 Azure 团队联合开发的自定义镜像提供给 MAI。这些系统按机架级 NVL72 单元供应，每个机架提供 72-GPU NVLink 域，用于高带宽的 scale-up (纵向扩展) 通信；机架间 RDMA 使用 InfiniBand。H100 系统也仍保留在实验室环境中，使用 8-GPU node，配备节点内 NVLink/NVSwitch 和 InfiniBand 用于 scale-out (横向扩展) 通信。

逻辑集群供应与管理。每个站点被划分为多个 Kubernetes 集群，通常每个数据中心楼栋一个。逻辑集群包含用于训练的 GPU node 和用于支撑服务的 CPU node。自定义控制器 (custom controllers) 将期望的逻辑集群状态与底层 Azure 资源调和 (reconcile)，维护节点拓扑标签，并将外部健康数据库集成到调度状态中。

拓扑标签在 node 启动时根据物理主机身份分配。这些标签表示机架、主机和位置 (locality) 信息，供调度、预留和机架级服务使用。这使得物理放置对控制平面 (control plane) 可见，让作业能够落在不仅可用、而且拓扑上合适的容量上。

大型作业在需要时被设计为跨越单个 Kubernetes 集群的边界。node 在整个计算环境中均可全局路由，工作负载 pod 使用 host networking，以避免不必要的 overlay 开销。

19.2 集群就绪与认证

在数千 GPU 规模下，故障是预期之中的。因此认证 (certification) 充当第一道可靠性边界。其目的是防止坏节点、退化链路、边缘存储和静默损坏 (silent-corruption) 风险进入生产训练池。认证框架是自动化的、Kubernetes 原生 (Kubernetes-native) 的且分层的。硬件必须先通过逐步更广范围的测试，然后才可调度：单节点诊断、机架级多节点集合通信 (collectives) 以及选定的跨机架 InfiniBand 验证。新节点和修复后的节点遵循相同的路径。从维修返回的 node 必须再次通过认证才能重新进入生产，防止重复出错的节点 (repeat offenders) 循环回到大型训练作业中。

认证。我们的诊断套件对每个节点进行独立验证。与通用健康检查不同，这些测试会对单个组件进行长时间压力测试：GPU、CPU 核、HCA、NVLink 链路和主内存，从而在 node 被接纳进生产之前，隔离出高 ECC 率、异常节流行为、GPU 时钟违规、链路抖动 (link flaps) 及其他边缘状况。

该套件还使用 NCCL 集合通信对每个机架的节点内和节点间通信路径进行压力测试，以识别异常的 NVLink 行为。这些自动化测试在稳定 GB200 和 GB300 机架方面发挥了关键作用：我们识别出多个健康节点少于 16 个的机架，这些机架被排除在大型训练运行之外。

认证的最后阶段包括跨机架 InfiniBand 测试，覆盖单节点和机架级测试无法触及的 fabric 路径。这些测试将跨机架、rail 和 leaf 组的节点分组，以验证 spine 层行为、路径多样性和跨机架 RDMA 性能。由于它们的 blast radius (影响范围) 更大，因此在 bring-up (启动)、事故调查和 fabric 验证期间按需运行。

节点生命周期。认证是闭环节点生命周期中的一个阶段，如图 26 所示。新的或修复后的 node 进入 Init，在那里被打上 taint、完成初始化、标记 label 并配置。随后，认证控制器运行分层测试套件。通过的节点变为 Available，失败的节点变为 Impaired。当检测到 NPD 条件、XID 错误、ECC 阈值、NVLink 退化、InfiniBand 链路抖动或存储故障时，运行时监控也可将 Available 节点移至 Impaired。瞬态问题（如重启或 soft drain（软排空））进入 Auto Remediation（自动修复）。持续性或归因于硬件的问题通过 Guest Health Reporting 路由至厂商或数据中心维护。修复后的节点返回 Init 并重新进入认证流程。无法恢复的节点被退役（decommissioned）。

19.3 调度、编排与控制平面

调度问题是在异构容量上放置异构工作负载，同时不牺牲局部性（locality）、配额隔离或恢复能力。工作负载包括长时间预训练作业、后训练和强化学习运行、推理与评测作业，以及 CPU 密集型数据管线。因此技术栈是分层的：Kubernetes 维护集群状态，Kueue 做出 admission 和配额决策，MAI 控制器维护预留与拓扑就绪，Ray 执行分布式作业，而机群范围（fleet-wide）的控制平面统一了跨集群和调度后端的可见性。

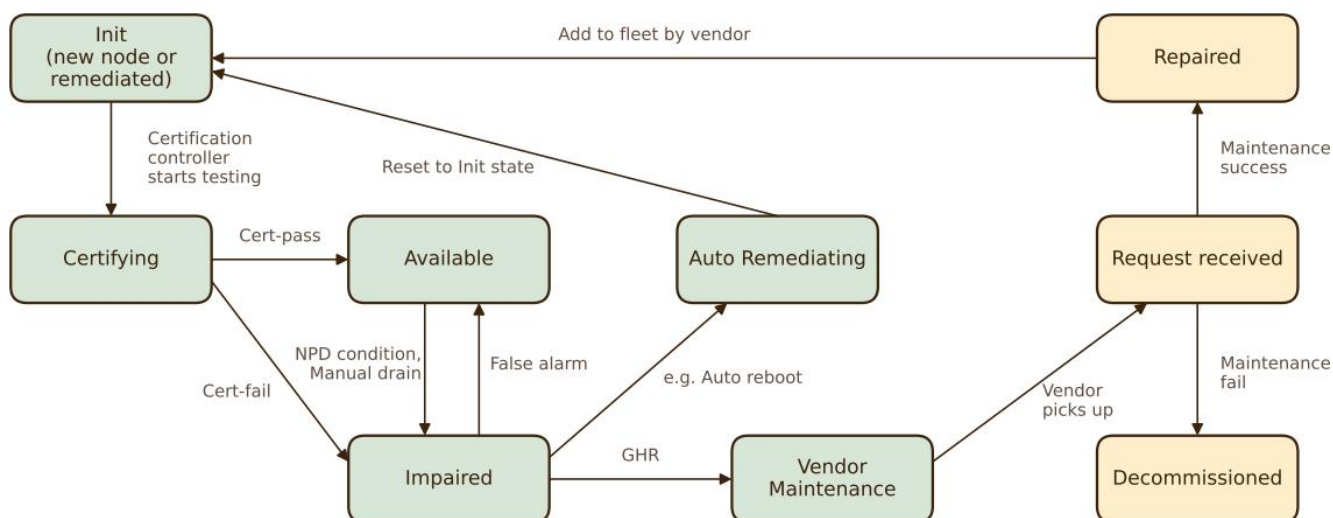


图 26. 节点生命周期：从初始化到认证、生产使用、修复、维护与退役。

集群本地控制平面。集群本地的 MAI 控制平面管理预留、机架拓扑、配额一致性和调度就绪门控。它不替代调度器，而是维护调度器所需的状态。Kueue 处理优先级、配额、admission、preemption（抢占）和拓扑感知放置。当一个工作负载被接纳（admitted）时，Kueue 会尽量保持高带宽域内的局部性，使大型作业落在紧凑的拓扑区域，而不是零散的容量上。机架碎片化（rack fragmentation）是反复出现的调度风险。来自许多队列的小作业可能会填满机架，从而阻止后来的大型作业获取连续容量。集群本地控制平面通过软机架预留（soft rack reservations）解决这一问题。队列被分配优先机架，工作负载会优先落在这些机架上。当容量空闲时仍可借用（borrowing），但 reclaimWithinCohort 允许队列在需要时收回其预留机架。这提高了负载下的装箱（bin packing）效率，同时在低需求时保持利用率。

Ray 运行时。Ray 在被接纳的作业内部提供分布式运行时。MAI 驱动程序将已接纳的拓扑转换为 actor 放置、通信组和 NCCL clique 配置。预训练作业要求严格的 learner 可用性，而 RL 作业管理多种 actor 类型，包括 learner、推理服务器、rollout worker 和路由器，每种都有不同的放置和容错需求。驱动程序监控 actor 存活状态、协调训练循环，并在异步组件之间保持 checkpoint 一致性。

19.4 可观测性、遥测与集群监控

我们将底层硬件遥测、调度器状态、作业元数据和机群效率（fleet-efficiency）信号整合到统一的运维视图中。GPU 健康状态通过 XID、ECC、热状态、功耗、时钟节流、NVLink 状态、NVLink 误码率、chip-to-chip 链路、InfiniBand 设备状态、本地 NVMe 健康、PCIe 错误和驱动状态进行跟踪。这些信号被转换为 Kubernetes node conditions（节点条件），再通过自定义 triage 和 drain 控制器转化为调度与修复动作。这使得健康状态不仅对运维人员可观测，对调度器也可观测。

作业可观测性聚合自 Kueue、Kubernetes、Ray、训练日志和实验元数据。运维人员可以按 namespace、pod、job 和重启索引（restart index）检查队列、优先级、admission 状态、节点放置、worker 就绪状态、重启次数、训练配置、step 进度以及作用域日志（scoped logs）。这使得无需手动跨系统重建状态，即可区分调度延迟、运行时故障、节点故障、存储退化和应用级停滞（application-level stalls）。

遥测数据存储并查询于针对不同时间尺度（time horizons）优化的系统上。Datadog 提供近实时指标和日志搜索。Azure Managed Prometheus 提供集群内和跨集群（cross-cluster）的时间序列采集。Azure Data Explorer 提供长期保留的分析存储，用于日志、指标、存储遥测和集群状态。告警同样采用分层模型：本地 Prometheus 规则用于低延迟组件故障，Datadog monitor 用于服务和日志衍生信号，基于 KQL 的检查用于长期保留分析，Azure Monitor 用于资源和 Prometheus 告警。关键告警被路由到事件管理（incident management），而较低严重程度的信号则流入仪表盘和运维评审。

独特的设计之处在于，可观测性是控制回路的一部分。硬件遥测、fabric 健康、存储行为、调度状态和作业进度并不是被动的仪表盘；它们决定容量是被接纳、排空、修复，还是重新投入服务。这使得整个机群可以按可用训练容量和有效吞吐（goodput）来管理，而不仅仅是按已供应的 GPU 数量。